

NESHBOY SOC PROFESSIONAL LIBRARY

SPLUNK SECURITY OPERATIONS

Platform Architecture, the Common Information Model, and
Enterprise Security — Running Splunk Itself as a Security
Platform, Once You Already Know the Query

Build date: 2026-09-15

Table of Contents

CHAPTERS

Section A — The Platform Underneath the Query Language

- Part 1 — Why This Book Exists: From SPL Syntax to Platform Operations
- Part 2 — Splunk Deployment Architecture
- Part 3 — Indexes, Sourcetypes, and the Bucket Lifecycle
- Part 4 — Licensing and Ingest Economics

Section B — Normalizing Data Into a Common Model

- Part 5 — The Common Information Model: What It Actually Standardizes
- Part 6 — Data Models and Pivot
- Part 7 — Building CIM-Compliant Data Onboarding

Section C — Reusable Detection Infrastructure

- Part 8 — Lookup Tables as Detection Infrastructure
- Part 9 — Search Macros and Reusable Logic

Section D — Performance at Scale

- Part 10 — Data Model and Report Acceleration
- Part 11 — Summary Indexing and Other Pre-Computation Patterns
- Part 12 — Search Performance and Tuning at Scale

Section E — Splunk Enterprise Security

- Part 13 — Enterprise Security Architecture, Editions, and the CIM Dependency
- Part 14 — Correlation Searches: From Analytic to Notable Event, and Their Lifecycle
- Part 15 — Risk-Based Alerting: Risk Objects, Risk Scores, and the Risk Index
- Part 16 — Notable Events and the Incident Review Workflow
- Part 17 — Asset and Identity Correlation

Section F — Dashboards, Investigation, and the Extended Ecosystem

- Part 18 — Dashboards and Visualizations for Security Operations
- Part 19 — Splunk-Specific Investigation Workflows
- Part 20 — What This Book Doesn't Know Yet: Validation Gaps and the Path to Real Evidence

CHAPTERS

Section A — The Platform Underneath the Query Language

neshboy

Part 1 — Why This Book Exists: From SPL Syntax to Platform Operations

Why this part exists

[CONCEPT] *The Detection Engineering Handbook* (DEH) V2 spends six parts — 24 through 29 — teaching one query language each: Sigma, KQL, SPL, AQL, YARA-L, and Elastic's three query surfaces. Part 26 is the SPL installment, and it closes with a scope note this book takes as its own founding brief rather than paraphrasing at a distance: it states plainly that it "does not cover Splunk administration, index/bucket lifecycle management, or `tstats`/data-model acceleration in depth... a full accelerated-search treatment belongs in a dedicated performance-engineering appendix, not here" (DEH Part 26, closing paragraph of "Why this part exists"). *Splunk Security Operations* is that appendix, expanded into a full platform-operations title. This part exists to state that relationship precisely, once, so that no later part in this book has to reconstruct it from scratch — the same problem DEH's own Part 23 solves for its six language parts, applied here to this book's relationship with DEH as a whole.

Three things follow from that relationship, and this part states each as a design fact rather than a disclaimer buried in a preface:

1. **This book assumes you can already write the query.** If you have not read DEH Part 23 (query language strategy) and DEH Part 26 (SPL syntax), read those first — §1 below tells you exactly what they cover and exactly where this book picks up instead of repeating them.
2. **This book runs a platform-layer/operations-layer split through every part that follows,** adapted from the telemetry-layer/analytic-layer split DEH runs through its own domain parts. §2 states that split and previews how the book's twenty parts sit on either side of it.
3. **This book is written without a real Splunk deployment in the author's lab.** Every platform claim in it is sourced from Splunk's own public documentation, product pages, and Splunkbase listings, or marked as an explicitly labeled conceptual illustration. §3 states why that's a structural fact about this book's evidence, not an apology for it, and what would change it.

1. What the Detection Engineering Handbook already taught you

[CONCEPT] This section is deliberately short, for the same reason DEH Part 23 is deliberately short: its job is to name a boundary precisely enough that the parts on either side of it can each do their own job without duplicating the other's.

1.1 DEH Part 23 — the analytic, and the detection rule it becomes

[CONCEPT] DEH Part 23 §1 formalizes one canonical detection — **DET-23-01**, a process other than a small, known set of legitimate system components opening a handle to `lsass.exe` (the Local Security Authority Subsystem Service, the Windows process that holds credential material in memory) with access rights sufficient to read that memory — **MITRE T1003.001 (OS Credential Dumping: LSASS Memory)**. DET-23-01 is stated at the analytic layer, independent of any query language, precisely so

that Parts 24 through 29 can each re-implement it file-for-file and let a reader compare the translation loss between backends directly. DEH Part 23 §3 then names that translation loss explicitly: a field a Sigma rule assumes exists (`GrantedAccess`, `TargetImage`) may not exist under the same name, or with the same semantics, in a given backend's actual schema, and that gap is Detection Debt's translation-layer failure mode, not a cosmetic authoring inconvenience.

This book does not re-teach the Analytic-vs-Detection-Rule distinction or Sigma-vs-native-authoring tradeoffs — DEH Part 23 owns both, in general. What this book does, starting in earnest at Part 14, is pick up DET-23-01 exactly where DEH Part 26 leaves it (§1.2 below) and carry it one layer further: from a working SPL search into a scheduled, throttled, notable-generating correlation search inside Splunk Enterprise Security — the platform-specific realization DEH's own detection ledger explicitly leaves for a sibling volume to complete, per this book's own ID discipline (no new `DET-##-##` numbers minted here; see `STYLE-GUIDE.md` §0).

1.2 DEH Part 26 — SPL syntax, and where it stops on purpose

[CONCEPT] DEH Part 26 teaches the SPL execution model this book assumes throughout: the pipeline shape of a search, and — per DEH Part 26 §1.2 — the search-time/index-time field split, the one distinction Part 26 calls out as separating SPL from most of the other five languages in DEH's own comparison. This book does not restate either term's definition; a reader who needs it should go read DEH Part 26 §1.2 directly rather than rely on a summary here. DEH Part 26 §2 through §4 then teach `stats`, `transaction`, and subsearch cost — culminating in DEH Part 26 §5's own worked implementation of DET-23-01 as **DET-26-01**, a real SPL search against Sysmon Event ID 10 (ProcessAccess) telemetry.

DEH Part 26 §1.3's syntax table names `tstats` — "aggregates directly over indexed/accelerated fields without reading raw events" — and immediately flags it as "named where scale matters; not covered in depth here." That single row is the specific, literal forward reference this book's Part 10 resolves. This book does not re-teach `search`, `eval`, `where`, `rex`, `stats`, or `transaction` syntax anywhere in its twenty parts; where a correlation search or RBA rule body in Parts 14–15 needs one of those commands, the framing sentence states what the command does in one clause and points back to the relevant DEH Part 26 subsection rather than re-deriving it. A reader who hasn't read DEH Part 26 §1.2 in particular should stop and read it before Part 3 of this book — the search-time/index-time split is the single piece of DEH Part 26 this book's index-design and CIM content (Parts 3, 5–7) depends on most directly, and neither restates it.

1.3 The scope note that created this book

[CONCEPT] DEH Part 26's own closing scope note — quoted in full at the top of this part — names three things it declines to cover: Splunk administration, index/bucket lifecycle management, and `tstats`/data-model acceleration in depth. Those three are not a random remainder; they are, almost exactly, this book's Sections A and D (platform deployment and indexing in Parts 2–4, acceleration and performance in Parts 10–12). The rest of this book's twenty parts — CIM and data models (Section B), lookups and macros as maintained infrastructure (Section C), and the whole of Enterprise Security's operations layer (Sections E and F) — go further than DEH Part 26's scope note asks for, because a platform-operations title that stopped exactly where DEH stopped would be a pamphlet, not a

handbook. §2 lays out where each of those sections sits relative to the platform layer/operations layer split this book uses to organize all of it.

2. Two layers, one platform: how this book is organized

[PLATFORM ENGINEER] DEH runs a telemetry-layer/analytic-layer distinction through its own domain parts: telemetry is what a system actually logs; an analytic is the logical statement of what pattern in that telemetry indicates a targeted behavior. This book adapts that same shape to a single-platform scope, because "telemetry" and "analytic" both undersell what Splunk specifically adds on top of raw log data once it's indexed. The two layers this book uses instead:

- **The platform layer** — what Splunk itself *is* and how it's built: indexers, search heads, forwarders, indexes, sourcetypes, buckets, licensing, the Common Information Model, data models, acceleration, and search performance under load. Nothing in this layer knows or cares what a detection is; it's the plumbing every detection and every dashboard in the book runs on top of.
- **The operations layer** — what a SOC actually *does* with that platform once it's built: correlation searches, risk-based alerting, notable-event triage inside Incident Review, asset/identity enrichment, dashboards, and the specific pivot paths an analyst or hunter follows through Splunk's own tooling during an investigation.

Every part in this book states, in its own opening scope paragraph, which of the two layers it occupies and which neighboring part owns the layer it isn't covering — the same mechanical discipline DEH uses against its own V1 draft's silent-duplication problem, applied here one level up, at the layer boundary rather than the part boundary. Table 1.1 previews the full twenty-part map against that split; use it to decide where to go next rather than reading linearly if you already know which layer's problem you have.

Table 1.1 — This book's twenty parts, mapped to the platform/operations split. Use this table to find the part that owns a specific problem rather than guessing from a title alone; the "Scope note" column names each part's own stated boundary against its neighbors.

Part(s)	Section	Layer	Scope note
2	A	Platform	Deployment topology — indexers, search heads, forwarders, clustering, on-prem vs. Splunk Cloud
3	A	Platform	Index/sourcetype design, bucket lifecycle, retention enforced at the bucket, not the query
4	A	Platform → SOC management	Licensing/ingest economics; the cost levers before "reduce license" becomes "reduce visibility"
5–7	B	Platform	CIM as a field-naming/tagging contract; data models and Pivot; the maintained work of onboarding a source into CIM compliance
8–9	C	Operations	Lookup tables and search macros as maintained infrastructure, not one-off files
10–11	D	Platform	<code>tsidx</code> summaries, <code>tstats</code> , and summary indexing — the pre-computation decisions made before a query runs
12	D	Platform	Search concurrency, the Job Inspector, workload management — cost paid <i>after</i> a query runs
13–17	E	Operations	Enterprise Security: architecture/editions, correlation searches and RBA, notable-event triage, asset/identity enrichment
18–19	F	Operations	Dashboards and the actual investigation pivot path through Splunk's own tooling
20	—	Both (capstone)	Honest inventory of every claim in Parts 1–19 resting on OFFICIAL REFERENCE / CONCEPTUAL evidence

Figure 1.1 draws the same map as a flow rather than a table, following one event roughly the way it actually moves through a real deployment: from raw ingestion, through the platform-layer machinery that makes it queryable at scale, into the operations-layer machinery that turns a matching pattern into something a human looks at.

flowchart LR

```

subgraph PLATFORM["Platform layer – Sections A-D"]
    P2["Part 2\nDeployment architecture"] --> P3["Part 3\nIndexes, sourcetypes,\nbucket lifecycle"]
    P3 --> P4["Part 4\nLicensing and ingest\neconomics"]
    P4 --> P57["Parts 5-7\nCIM, data models,\nonboarding"]
    P57 --> P89["Parts 8-9\nLookups and macros"]
    P89 --> P1012["Parts 10-12\nAcceleration and\nperformance"]
end
subgraph OPERATIONS["Operations layer – Sections E-F"]
    P1317["Parts 13-17\nEnterprise Security:\ncorrelation, RBA,\nnotables, A&I"]
    P1819["Parts 18-19\nDashboards and\ninvestigation workflow"]
end
P1012 --> P1317 --> P1819
P1819 --> P20["Part 20\nValidation gaps and\npath to real evidence"]

```



Figure 1.1 — This book's part sequence as a platform-to-operations data flow. *CONCEPTUAL.* Illustrates how this book's twenty parts sit on either side of the platform-layer/operations-layer split

described above, loosely following the direction one indexed event actually travels through a real Splunk deployment. This is a map of the book's own structure, not a capture of a running system's dataflow — no Splunk instance backs it (see §3).

Product Version Note *Splunk Enterprise Security is currently sold in two editions — Essentials and Premier — with SOAR integration, User and Entity Behavior Analytics (UEBA), and "Automated Threat Analysis" gated to Premier; AI capabilities, SIEM, Threat Intelligence, Detection Studio, and Exposure Analytics ship in both. As of 2026-09-15, verified against Splunk's own Enterprise Security product page (Enterprise Security 8.7.0, released 2026-09-02; REFERENCES.md entry [SPLUNK-ES-PRODUCT-PAGE]). Part 13 covers this split in the depth it deserves; it's previewed here only because it's the sharpest example of why the operations layer in Sections E–F needs its own Product Version Notes far more often than the platform layer in Sections A–D does — an indexer's job hasn't been restructured into a two-tier product line, and ES's has. Re-verify this split before treating it as still accurate more than a few release cycles out.*

3. The evidentiary-honesty constraint, stated as a design fact

[SOC MANAGEMENT] This book's author maintains a real, actively-operated home lab — Proxmox virtualization, a honeynet with live indicator enrichment, a vulnerability-scanner platform, general infrastructure documented in that lab's own operational notes. DEH draws on that lab directly: its Part 26, for instance, cites a real `lastb` capture and a real honeynet HTTP-events table as evidence for specific claims. None of that lab runs Splunk, and as of this part's `last_validated` date, no plan exists to change that. That's not a gap this book is quietly working around; it's a structural fact about the evidence base every one of the following nineteen parts has to work inside, and STYLE-GUIDE.md §9 turns it into a hard rule rather than a soft caveat: this book's figures are, and are expected to remain, almost exclusively **OFFICIAL REFERENCE** (sourced from Splunk's own public documentation, product pages, or Splunkbase listings, cited in REFERENCES.md) or **CONCEPTUAL** (this book's own labeled illustrative diagrams and mockups) — with **zero** **CONTROLLED LAB EXAMPLE** or **REAL LAB EXAMPLE** tags anywhere in the book, unless that changes and the change is disclosed as a dated policy update, not a silent upgrade of old figures.

Engineering Reality *The platform-layer/operations-layer split in §2 is analytically clean and organizationally false more often than not. A well-resourced SOC might have a dedicated Splunk platform team running Sections A and D as their whole job and a separate detection-engineering team owning Sections C, E, and F; a smaller team has one or two people doing all of it, context-switching between a bucket-retention setting and a correlation-search threshold in the same afternoon. This book's part boundaries track the kind of decision being made, not a staffing model — don't read Section headers as an org chart, and don't use this book's structure to justify splitting a two-person Splunk practice into roles that need five.*

That constraint has a direct, checkable consequence for how you should read every part that follows: a claim that names a specific Splunk version, edition, default setting, or documented percentage carries a Product Version Note (§6.9 of `STYLE-GUIDE.md`, and see the worked example in §2 above) stating what source it was checked against and what would make it stale. A claim with no such note and no citation, describing something this book can't verify against its own evidence base, gets marked `CONCEPTUAL` or dropped outright — never asserted on the strength of general familiarity with the product, the same discipline DEH applies to MITRE ATT&CK IDs ("never invent or guess an ID," per DEH `STYLE-GUIDE.md` §5, extended here to platform facts generally). Part 20 closes the book by inventorying every one of those notes and marks in one place, organized by what specific observation would upgrade each claim — most often, in this book's case, "a real Splunk deployment, instrumented and observed over a real retention window," which is a legitimate answer to give twenty times over, not a deflection repeated for lack of a better one.

What Would Change My Mind Every platform-behavior claim in this book that isn't a direct quote from Splunk's own documentation is, at best, a documented expectation rather than a measured result. What would move a specific claim in this book from "documented" to "verified": standing up a Splunk instance in the author's own lab — even a single-instance, non-clustered deployment ingesting the honeynet's and vulnerability-scanner's own real log traffic would let Parts 3, 5–7, and 10–12 cite an actual `tstats` result count, an actual CIM-compliance check output, and an actual bucket-rollover event instead of Splunk's documentation's description of what those should look like. That instance does not exist as of this part's `Last_validated` date. If it's stood up later, that's a dated, disclosed change to `STYLE-GUIDE.md` §9.2, not a quiet retroactive upgrade of this part's own `CONCEPTUAL` figure.

4. Where DEH's concepts land in this book

[CONCEPT] Table 1.2 is the crosswalk this part exists to make explicit: for each DEH concept this book's own content depends on, where DEH states it, and where this book takes it further as a Splunk-specific, platform-operations extension. Use it the way you'd use an index — to jump directly to the part that extends a DEH concept you already understand, rather than reading this book's twenty parts in strict numeric order if your actual need is narrower.

Table 1.2 — DEH concepts this book extends, and where. Read the "This book's extension" column as "once you know the DEH concept in the left two columns, here's where this book turns it into something you configure inside Splunk specifically."

DEH concept	DEH location	This book's extension	Where
Analytic vs. Detection Rule distinction	DEH Part 1; formalized as DET-23-01 in DEH Part 23 §1	Correlation-search anatomy (<code>savedsearches.conf</code>) and RBA rule design as the Splunk-specific realization of a detection rule	Part 14
SPL execution model, search-time/index-time field split	DEH Part 26 §1, §1.2	Index/sourcetype design decisions that determine which fields even can be index-time, and CIM field aliasing built on search-time extraction	Parts 3, 5–7
<code>tstats</code> , named but not taught	DEH Part 26 §1.3 (forward reference, explicitly undeveloped)	<code>tsidx</code> summary mechanics, accelerated data models, and <code>tstats</code> itself as a command that reads them directly	Part 10
<code>stats</code> / <code>transaction</code> / subsearch cost	DEH Part 26 §2–4	Lookup tables as the maintained alternative to a costly subsearch, at platform scale	Part 8
Sigma-vs-native authoring tradeoff, translation loss	DEH Part 23 §2–5	ES Content Update model and correlation-search versioning as Splunk's own content-packaging answer to the same tradeoff	Part 14
Detection Debt's translation-layer failure mode	DEH Part 23 §3	A stale CIM field mapping or a decommissioned-host lookup entry as this book's own version of the identical failure shape	Parts 7, 8, 17
Normalization framing (ECS/OCSF/UDM/ASIM/CIM in general)	DEH Part 6	What CIM specifically standardizes for Splunk — <code>tags.conf</code> , <code>eventtypes.conf</code> , field aliases — as a testable, falsifiable compliance claim	Part 5
DET-23-01 → DET-26-01 (SPL implementation)	DEH Part 23 §1; DEH Part 26 §5	The same analytic deployed as a scheduled, throttled correlation search inside Enterprise Security	Part 14

Two rows in that table repeat "Part 14" because Part 14 is this book's flagship-depth part for exactly the reason DEH's own DET-23-01 walkthrough is: a single, carried-forward example lets a reader who's already followed it through DEH Part 26 recognize it immediately when it becomes a Splunk correlation search, rather than learning a new illustrative scenario from a cold start.

5. How to read a tag, a callout, and a figure in this book

[CONCEPT] This book uses DEH's own content-tagging and callout system with two tag renames and one added callout, all defined in full in `STYLE-GUIDE.md` §§6–7 — this part previews the renames only, because they're the two places a reader who already knows DEH's system could misread this book by pattern-matching too fast:

- **[SOC ANALYST]**, not DEH's **[ANALYST]** — because this book talks about Splunk's own Incident Review workflow often enough that the triage-facing tag needs to be unambiguous against the informal sense of "Splunk analyst" that also means a detection-content developer. If a paragraph is about working a notable event, it's **[SOC ANALYST]**; if it's about building the correlation search

that generated that notable, it's [DETECTION ENGINEER] even though a person with the job title "Splunk analyst" might be the one doing it.

- [PLATFORM ENGINEER], not DEH's [ENGINEERING] — because DEH's tag spans onboarding, parsing, normalization, and pipeline cost across any log source in general, and this book's engineering content is narrower and more specific: the Splunk platform itself — indexers, search heads, buckets, licensing, acceleration. A paragraph about a parser silently mis-normalizing a timestamp before it ever reaches Splunk is DEH's [ENGINEERING] territory (Parts 5–7 there); a paragraph about what happens to that same event once it's inside Splunk's index is this book's [PLATFORM ENGINEER].

[CONCEPT], [DETECTION ENGINEER], [THREAT HUNTER], and [SOC MANAGEMENT] keep their DEH names and scope unchanged. The one callout this book adds beyond DEH's eight — **Product Version Note** — is not cosmetic: §2 and §3 above each use one, and the reviewer checklist in `STYLE-GUIDE.md` §11 treats a version/edition/default claim with no Product Version Note attached as a hard rejection, not a style suggestion. Expect it constantly in Sections E and F, where Enterprise Security's own feature surface has visibly reorganized within the current release cycle (the Essentials/Premier split covered above is one instance of that, not the only one this book will flag).

6. What to read next

[CONCEPT] If you arrived here without having read DEH Part 23 and DEH Part 26, stop and read those two parts first — this book's Parts 3 and 5 in particular assume DEH Part 26 §1.2's search-time/index-time split as settled background, not something this book re-derives for you. If you've read both, Part 2 picks up immediately where this part leaves off: the actual deployment topology — indexers, search heads, forwarders — that everything else in this book eventually runs on top of. If your problem is narrower than "read the whole book" — a specific `tstats` question, a specific lookup-staleness incident, a specific ES edition question — Table 1.1 and Table 1.2 above are built to send you directly to the part that owns it.

One last framing point worth stating plainly rather than leaving implicit: this book does not treat "we don't run Splunk in our lab, so we can't write about it accurately" as a reasonable inference. The inference this book makes instead is narrower and, we think, correct — a platform's own public documentation, product pages, and Splunkbase listings are a legitimate source for platform-behavior claims, provided every one of them is cited, dated, and honestly labeled **OFFICIAL REFERENCE** rather than dressed up as something a live deployment confirmed. Where that's not enough — where a claim genuinely needs a running instance to verify, not just a documentation page to cite — this book says so, in a **What Would Change My Mind** box, rather than quietly asserting the claim anyway. Part 20 collects every one of those admissions in one place at the end. The nineteen parts between here and there are where the actual platform-operations content lives.

Cross-references: DEH Part 23 §§1, 3 (DET-23-01, translation loss); DEH Part 26 §§1, 1.2, 1.3, 2–5 (SPL execution model, search-time/index-time fields, `stats/transaction/subsearch`, DET-26-01, `tstats` forward reference); this book's Part 2 (deployment architecture), Part 5 (CIM), Part 10 (acceleration and

tstats), Part 13 (Enterprise Security editions), Part 14 (correlation searches and DET-26-01's realization as a scheduled search), Part 20 (validation-gap capstone).

neshboy

Part 2 — Splunk Deployment Architecture

Why this part exists

[CONCEPT] DEH Part 26 states its own scope limit plainly: it "does not cover Splunk administration, index/bucket lifecycle management, or `tstats`/data-model acceleration in depth... a full accelerated-search treatment belongs in a dedicated performance-engineering appendix, not here." This book is that appendix, and this part is where it starts, because everything else in the book assumes a topology already exists underneath it. Part 3's bucket lifecycle, Part 4's licensing math, Part 10's acceleration mechanics, Part 12's search-concurrency limits, and every Enterprise Security part from 13 onward all describe things that happen *on* indexers and search heads — none of them make sense until the reader knows what an indexer and a search head actually are, how data gets from an endpoint to one, and how a single search fans out across a fleet of them.

This is a **platform-layer** part, in this book's own platform/operations vocabulary: it covers what Splunk *is*, structurally, not what a SOC does with it. It does not cover index and bucket design (Part 3 owns that), ingest licensing (Part 4), the Common Information Model (Parts 5–7), or how a correlation search actually gets scheduled and throttled (Part 14) — this part hands off to each of those the moment it would need to go deeper than "here is where that thing runs."

DEH Part 26 also owns SPL syntax itself — `stats`, `transaction`, the search-time/index-time field distinction — and DEH Part 23 owns the general query-language-strategy vocabulary (analytic vs. detection rule, Sigma-vs-native tradeoffs) this book's later correlation-search content assumes. Neither is re-taught here. Where this part needs a syntax concept to explain *why* a search behaves differently depending on topology, it names the relevant DEH Part 26 section and moves on.

One more thing follows this book everywhere, including here: no real Splunk deployment exists in this book's evidence base. Every claim below about how a component behaves is sourced from Splunk's own public documentation, product pages, or Splunkbase listings — cited where a specific version or edition fact is involved — or stated as an explicitly labeled conceptual illustration. `STYLE-GUIDE.md` §9.2 makes this the structural default for every figure and code sample in the book; this part is not an exception.

1. The core components: indexers, search heads, and forwarders

[PLATFORM ENGINEER] A Splunk deployment is built from a small number of role types, and almost every operational question in this book eventually reduces to "which of these roles is doing the work, and where is it running."

An **indexer** is the Splunk instance that parses incoming data into events, writes those events into indexed buckets on disk, and — when a search head asks it to — runs a search against its own locally stored buckets and returns results. Indexing and searching that indexer's own data are both jobs the indexer does; nothing else in the deployment touches those buckets directly.

A **search head** is the Splunk instance an analyst or a scheduled job actually talks to. It doesn't hold the primary copy of indexed data. Instead, it takes a search, works out which indexers ("search peers," in Splunk's own terminology) actually hold data relevant to that search, dispatches the search to each of them, and merges their partial results into one final answer. It also hosts the knowledge objects everything else in this book depends on — saved searches, correlation searches, dashboards, lookups, macros, and apps like Enterprise Security (ES) — Splunk's premium security-analytics app, covered in depth starting at Part 13, which installs on a search head the same way any other app does.

A **universal forwarder** (UF) is a minimal, dedicated build of Splunk whose only job is getting data off an endpoint and onto the wire toward an indexer or an intermediate forwarder. A **heavy forwarder** (HF) is a full Splunk Enterprise instance running in a forwarding role — it can do everything an indexer's parsing pipeline does before the data ever reaches an indexer, and it can index locally as well as forward, though most security deployments configure it purely as a routing/parsing hop. Section 2 works through the practical difference.

A **deployment server** distributes apps and configuration files to a fleet of forwarders (and, less commonly, other components) grouped into named "server classes." It is not itself part of the search or indexing path — it is fleet-management plumbing, covered in Section 3.

Engineering Reality A single-instance ("all-in-one") Splunk deployment — one process acting as indexer, search head, and often the only forwarder — is the default shape of an evaluation install, and it stays that way in a lot of small production environments longer than it should. It works fine until either the ingest volume outgrows one indexer's disk/CPU budget or the scheduled-plus-ad-hoc search load outgrows one search head's concurrency limit (Part 12 covers that limit directly). Splitting indexer and search head roles apart — and only then considering clustering either — is the first scaling decision every deployment eventually faces, and doing it reactively, after a Sunday-night correlation search backlog, is worse than doing it on a sizing exercise (Section 8).

The table below compares the roles this section introduces, since a single-instance evaluation install is exactly the case where it's easy to lose track of which role is doing which job.

Table 2.1 — Splunk deployment component roles at a glance.

Component	Primary Job	Holds Indexed Data	Hosts Knowledge Objects	Typical Scale Unit
Indexer	Parses, indexes, stores events; executes searches against its own buckets	Yes	No	Indexing volume (GB/day) per peer
Search head	Dispatches distributed searches, merges peer results, renders output	No (transient job artifacts only)	Yes — dashboards, saved/correlation searches, apps	Concurrent and scheduled search load
Universal forwarder	Tags and forwards data toward an indexer or heavy forwarder	No	No	Endpoint count
Heavy forwarder	Parses, conditionally routes, optionally indexes locally, before forwarding onward	Optional	Limited (mainly routing/parsing apps)	Aggregation-point throughput
Deployment server	Distributes app/config bundles to registered forwarders	No	No — distributes objects, doesn't run them	Managed client count

Product Version Note Splunk's current release line is in the 10.x series. As of 2026-09-15, Splunkbase listings for Enterprise Security (version 8.7.0) and the Common Information Model Add-on (version 8.7.0), both last updated 2026-09-02, list compatibility with Splunk platform versions 10.5 down through 10.2 — the CIM Add-on's listing additionally extends compatibility back to 9.4 (REFERENCES.md entries [ES-SPLUNKBASE] and [SPLUNKBASE-CIM-ADDON]). What would make this stale: any new Splunk Enterprise major/minor release, or either app publishing a new version with a different compatibility floor — re-check the current Splunkbase listing before assuming a given environment's version is still inside the supported range for whichever app you're deploying.

2. Universal forwarders and heavy forwarders: what actually crosses the wire

[PLATFORM ENGINEER] A universal forwarder tags each event with `host`, `source`, and `sourcetype` metadata and ships it toward an indexer (or an intermediate heavy forwarder) over Splunk's own forwarder-to-indexer protocol, conventionally on TCP port 9997. It deliberately does not run the heavier parts of the event-processing pipeline — the line-breaking heuristics, `SEDCMD` masking, and structured-data extraction that a full Splunk instance applies. Splunk's own longstanding operational guidance is to keep that heavier parsing work on the indexing tier (or an intermediate heavy forwarder) rather than the forwarder, precisely because a universal forwarder's whole value proposition is a small, stable, low-privilege footprint running on tens of thousands of endpoints you'd rather not have to individually tune.

A heavy forwarder exists for the cases where that light-touch model doesn't fit. Two concrete examples: routing decisions that need to happen before data lands in an index — sending one `sourcetype` to a restricted, RBAC-scoped index and another to a general one, based on content rather than just source — need a full parsing pipeline to make that call, and syslog aggregation for network and security appliances that speak UDP/TCP syslog rather than writing to a local file needs a listening service a universal forwarder isn't built to run. Splunk DB Connect is a sharper example still: it requires a full

Splunk Enterprise instance (a heavy forwarder, a search head, or an indexer) because its modular input needs a Java runtime and app framework a universal forwarder's minimal build doesn't include.

Engineering Reality "Forwarder" is not one tier of cost and one tier of behavior — a heavy forwarder is a full Splunk Enterprise instance, licensed and administered like one, running in a mode where forwarding happens to be its main job. Treat it in your capacity planning like the indexer-adjacent component it actually is, not like a slightly heavier universal forwarder. Part 4 covers how ingest volume actually gets metered through a topology like this; this part is only naming where the parsing decision gets made, not what it costs on your license.

3. Deployment servers and app distribution

[PLATFORM ENGINEER] Once a fleet of forwarders exists, someone has to push app updates — a new Technology Add-on (TA) version, a `props.conf` fix, a new `outputs.conf` target — to all of them without touching each one by hand. That's the deployment server's entire job, and it's a push-on-checkin model, not a live push: a forwarder configured as a deployment client polls its deployment server periodically and pulls whatever app bundles are currently assigned to it.

On the forwarder side, `deploymentclient.conf` points the instance at its deployment server:

CONCEPTUAL SAMPLE — illustrative stanza shape, not captured from a running deployment.

```
[deployment-client]

[target-broker:deploymentServer]
targetUri = deploymentserver.internal.example:8089
```

serverclass.conf — grouping forwarders for app distribution

On the deployment server side, `serverclass.conf` defines named server classes — groups of clients matched by hostname pattern, DNS name, or an explicit `clientName` the forwarder sets in its own `deploymentclient.conf` — and maps each class to the apps it should receive:

CONCEPTUAL SAMPLE — illustrative stanza shape, not captured from a running deployment.

```
[serverClass:security_endpoint_forwarders]
whitelist.0 = *

[serverClass:security_endpoint_forwarders:app:TA-windows]
```

This is a fan-out mechanism, not a search-path component — a forwarder that never checks in still forwards whatever data it's already configured to forward with whatever app version it already has; it just stops receiving updates. That failure mode is quiet rather than loud, which is exactly why it's worth naming: a TA that silently stops matching a renamed field after an upstream log-format change (the

failure mode Part 7 covers directly) can just as easily be explained by a forwarder that stopped checking in months ago and never picked up the fix.

Engineering Reality *The deployment server manages forwarders, not search heads or indexers as a general configuration-management tool — trying to use it as your only mechanism for pushing knowledge-object changes to a search head cluster is fighting the tool. Search head clusters have their own distribution mechanism, the deployer, covered in Section 5, and it is a different piece of infrastructure with a different job, not a renamed version of the same one.*

4. Indexer clustering: replication, search factor, and what "highly available" actually buys you

[PLATFORM ENGINEER] An indexer cluster is a group of indexers ("peer nodes") that replicate each other's data, coordinated by a dedicated manager node.

Product Version Note *This book uses Splunk's current terminology, the manager node, for the indexer-cluster coordinator role. Splunk renamed this role from its earlier name, the cluster master; some older Splunk documentation, third-party material, and community discussion still use that older term. This book's research tooling could not reach docs.splunk.com directly to pin the specific version at which the rename took effect ([STYLE-GUIDE.md](#) §9.4 records this as a standing constraint on that tooling), so treat the rename itself as real but its exact version boundary as unverified here. What would make this stale: either term becoming exclusively current, or a further rename — confirm which term the documentation set for your own deployed version actually uses before assuming either name is current for it.*

Two numbers define what a cluster actually protects against: the **replication factor** (RF) is how many raw copies of each bucket the cluster keeps across peers, and the **search factor** (SF) is how many of those copies are additionally made fully searchable — with their own index-time metadata files built, not just the raw journal — where SF is always less than or equal to RF. A peer holding a bucket copy that counts toward RF but not SF has the raw data for replication/disaster-recovery purposes but can't serve a search against it until it's promoted. Multisite clustering extends the same RF/SF model per site, so a deployment can guarantee, for example, that losing an entire data center still leaves a searchable copy of every bucket somewhere.

server.conf — declaring a node's clustering role

Each peer and the manager node declare their clustering role in `server.conf`'s `[clustering]` stanza:

CONCEPTUAL SAMPLE — illustrative stanza shape, not captured from a running deployment.

```
# On the manager node
[clustering]
mode = manager
replication_factor = 3
search_factor = 2
pass4SymmKey = <shared-secret>
```

CONCEPTUAL SAMPLE — illustrative stanza shape, not captured from a running deployment.

```
# On a peer node
[clustering]
mode = peer
manager_uri = https://cm.internal.example:8089
pass4SymmKey = <shared-secret>
```

The specific `replication_factor`/`search_factor` values above are illustrative, not a recommended default — there is no factory-set value that fits every environment; both are sizing decisions made against retention requirements (Part 3) and available disk, not values to copy from a worked example.

Engineering Reality Replication is not a free insurance policy — it is disk, multiplied. A replication factor of 3 means three full copies of every bucket's raw data somewhere in the cluster, and a search factor of 2 means two of those three additionally carry the searchable index files, which themselves consume meaningful additional disk on top of the raw data they index (the same accounting problem Part 10 covers for data-model acceleration's `tsidx` summaries). Budget storage for RF copies from the start of a sizing exercise, not as a surprise after the first peer fills its disk.

5. Search head clustering: the deployer, the captain, and what actually replicates

[PLATFORM ENGINEER] A search head cluster (SHC) is a group of search heads that share configuration and knowledge objects and present a single logical search-head identity to the rest of the deployment, even though any member can serve a given search. Two mechanisms make that work, and they are easy to conflate with similarly-named things elsewhere in the topology.

The **deployer** is a separate Splunk instance — not itself a cluster member — whose job is pushing apps and configuration bundles to every member of the search head cluster. Despite the name similarity, it is not the deployment server from Section 3; the deployment server manages forwarders, the deployer manages search head cluster members, and they are configured, and administered, independently.

The **captain** is one cluster member, dynamically elected among the members, that takes on a small number of cluster-coordination responsibilities beyond just serving searches — most relevantly for this book, it's the captain that ensures a given scheduled search runs exactly once across the cluster on its configured schedule, rather than once per member. Captain status can move to a different member (an election, whether from a clean failover or the member holding captaincy becoming unreachable).

[DETECTION ENGINEER]

This matters directly for correlation-search troubleshooting, not just cluster administration: a correlation search scheduled on a search head cluster is coordinated by whichever member currently holds captaincy, not by every member independently. A captain election happening mid-cycle — a rolling upgrade, a network blip, a manual failover during maintenance — can delay or, depending on the search's own skip-if-still-running and scheduling-priority settings (Part 12 covers both), cause that cycle to be skipped outright. "Why didn't the notable fire this hour" is sometimes a captain-transition question before it's a correlation-search-logic question.

Engineering Reality *A search head cluster buys continuity of the search-head role, not zero scheduling disruption during a transition. Treat a captain election as a brief, expected gap in coordinated scheduling, not a bug — and treat repeated, frequent elections (rather than occasional ones tied to actual maintenance or failure events) as the actual signal worth investigating, since that pattern usually points at a network or resource problem on one or more members rather than anything about the scheduled content itself.*

6. Distributed search: how a search actually gets planned and executed

[PLATFORM ENGINEER]

A search head doesn't run a search itself in the way an indexer does — it plans one, dispatches pieces of it, and assembles the answer. This is the mechanism that makes every correlation search, dashboard panel, and ad hoc investigation in later parts of this book actually execute against real data, and it's worth being explicit about the two phases involved, since they behave differently and fail differently.

In the dispatch phase, the search head determines which indexers are relevant search peers for the query (an indexer cluster's manager node maintains the peer list the search head consults) and sends each of them the parts of the search that can run locally against that peer's own buckets. In the execution phase, each peer runs its portion in parallel against only the data it actually holds, streaming partial results back as it goes rather than waiting to finish before responding. In the merge phase, the search head combines every peer's partial results, runs whatever commands in the search can't be split across peers this way, and finalizes the result set the analyst or the correlation search actually sees.

DEH Part 26 §1.2 already covers the search-time/index-time field split at the language level; the distributed-search consequence of that split is that field extraction happening at search time runs on whichever machine is actually evaluating the search — the peer, for anything pushed down to it, or the search head, for anything that has to wait for the merge. Which SPL commands can be pushed down to peers and which force a wait for the full merged result set is exactly DEH Part 26 §1.1's "cost is paid left to right" principle; this part only introduces the two-phase shape that principle operates inside, and Part 12 applies it at fleet scale rather than single-query scale.

The following illustrates the shape without re-deriving `stats` syntax, which DEH Part 26 §2 already owns:

CONCEPTUAL SAMPLE — illustrative query shape, not a captured production search.

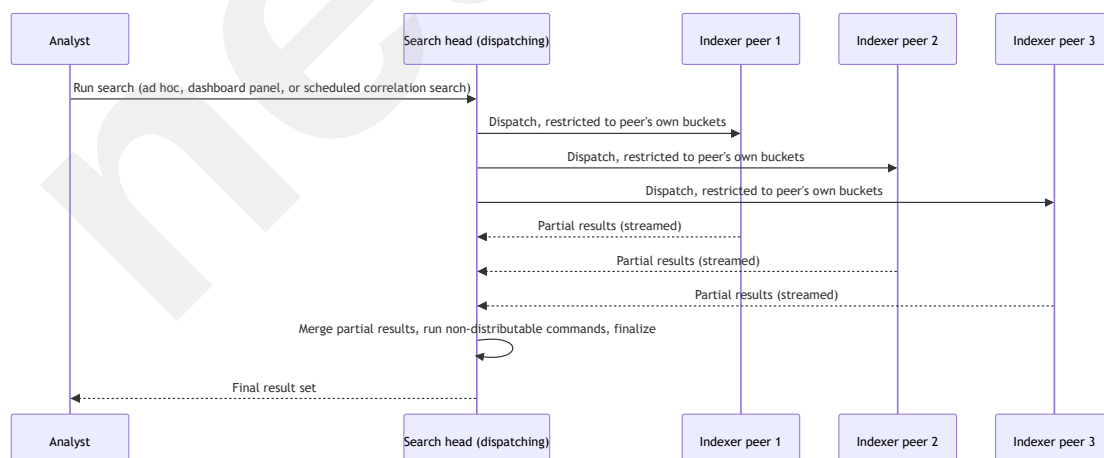
```
index=network_traffic sourcetype=cisco:asa
| stats count by src_ip, dest_ip
| sort - count
```

The `search` and the `stats` immediately after it can both run independently on every peer holding `index=network_traffic` buckets, each peer contributing its own partial count table for the fields it actually has. The `sort` cannot run that way — it has to wait until the search head has merged every peer's partial `stats` output into one table, because sorting requires seeing the complete result set at once. A search built almost entirely of commands like the first two is cheap at scale; a search that front-loads a command with `sort`'s requirement is not, regardless of how small the query looks on the page.

Figure 2.2 — Distributed search dispatch and merge. *CONCEPTUAL.* Illustrates the expected sequence from a search head's dispatch of a search across three indexer peers through partial-result streaming and final merge. This is a sequence diagram of documented expected behavior, not a capture from a real Splunk job inspector or a live deployment — none exists in this book's evidence base (see `STYLE-GUIDE.md §9.2`).

```
sequenceDiagram
    participant Analyst
    participant SH as Search head (dispatching)
    participant P1 as Indexer peer 1
    participant P2 as Indexer peer 2
    participant P3 as Indexer peer 3

    Analyst->>SH: Run search (ad hoc, dashboard panel, or scheduled correlation search)
    SH->>P1: Dispatch, restricted to peer's own buckets
    SH->>P2: Dispatch, restricted to peer's own buckets
    SH->>P3: Dispatch, restricted to peer's own buckets
    P1-->>SH: Partial results (streamed)
    P2-->>SH: Partial results (streamed)
    P3-->>SH: Partial results (streamed)
    SH->>SH: Merge partial results, run non-distributable commands, finalize
    SH-->>Analyst: Final result set
```



A search built against an accelerated data model instead of raw buckets — using `tstats`, named but deliberately not taught at the syntax level in DEH Part 26 §1.3 — changes this picture by having each

peer read from a pre-built summary rather than raw events. Part 10 of this book is where that mechanism, and its own cost and coverage tradeoffs, gets covered in depth; this part's job was only to establish the dispatch/merge shape that acceleration modifies rather than replaces.

7. On-premises Splunk Enterprise vs. Splunk Cloud Platform: what changes, what doesn't

[PLATFORM ENGINEER] Everything in Sections 1–6 describes the search/indexing model itself, and that model doesn't change between running Splunk Enterprise on infrastructure you operate and running Splunk Cloud Platform, Splunk's managed offering. Splunk's own framing of Splunk Cloud Platform is explicit that the operational burden, not the architecture, is what moves: Splunk states that its own personnel "manage your IT backend so you can focus on acting on your data, while our platform scales to your analytics needs" (REFERENCES.md entry [SPLUNK-CLOUD-PRODUCT-PAGE]). Concretely, that means Splunk operates the indexing and much of the search-head tier's underlying infrastructure, handles its scaling, and applies platform patching — work that, on-premises, is the customer's own responsibility end to end.

What doesn't change is the authoring surface: correlation searches, dashboards, lookups, and macros are still built the same way, in the same Splunk Web, on either. What does change beyond infrastructure ownership is *administrative reach* — direct filesystem and `server.conf`-level access that's routine on a self-managed Splunk Enterprise instance is either unavailable or replaced by a narrower, API-mediated path (Splunk's Admin Config Service, or a support case for anything that API doesn't cover) on Splunk Cloud Platform. Data still arrives the same two ways either deployment supports — forwarders (universal or heavy) or the HTTP Event Collector (HEC) — just pointed at a cloud-hosted indexing tier instead of one you run yourself.

Table 2.2 — On-premises Splunk Enterprise vs. Splunk Cloud Platform: who controls what.

Choosing between the two is a decision about who operates the infrastructure this part describes, not a difference in the underlying search/indexing model — the table states what actually moves.

Responsibility	On-Premises Splunk Enterprise	Splunk Cloud Platform
Indexer/search-head OS and infrastructure	Customer	Splunk
Indexer/search-head clustering configuration	Customer, direct <code>server.conf</code> access	Splunk-managed; topology is visible, not customer-edited at the <code>server.conf</code> level
App installation	Direct install of any Splunkbase or custom app	Self-service for vetted apps; others require Splunk review before install
Configuration below the knowledge-object layer (<code>limits.conf</code> , most <code>server.conf</code> stanzas)	Direct file access	Admin Config Service API/CLI where exposed, a support case otherwise
Correlation searches, dashboards, lookups, macros	Customer, authored in Splunk Web	Customer, authored in Splunk Web — same surface
Data ingestion path	Forwarders or HEC, pointed at self-hosted indexers	Forwarders or HEC, pointed at the cloud-hosted indexing tier

Product Version Note *Splunk Cloud Platform's own internal architecture — how elastically its indexing and search tiers scale independently, and exactly which administrative settings are self-service through the Admin Config Service versus support-case-only — has changed over the product's history and should be expected to keep changing. This book could not reach Splunk's own documentation domain to verify current specifics ([STYLE-GUIDE.md](#) §9.4 records this as a standing constraint on this book's research tooling); the managed-service framing above is confirmed only at the level Splunk's own product page states it, as of 2026-09-15 ([REFERENCES.md](#) entry [SPLUNK-CLOUD-PRODUCT-PAGE]). Confirm the current administrative boundary against your own Splunk Cloud Platform tenant's documentation before assuming any specific setting is or isn't self-service.*

[SOC MANAGEMENT]

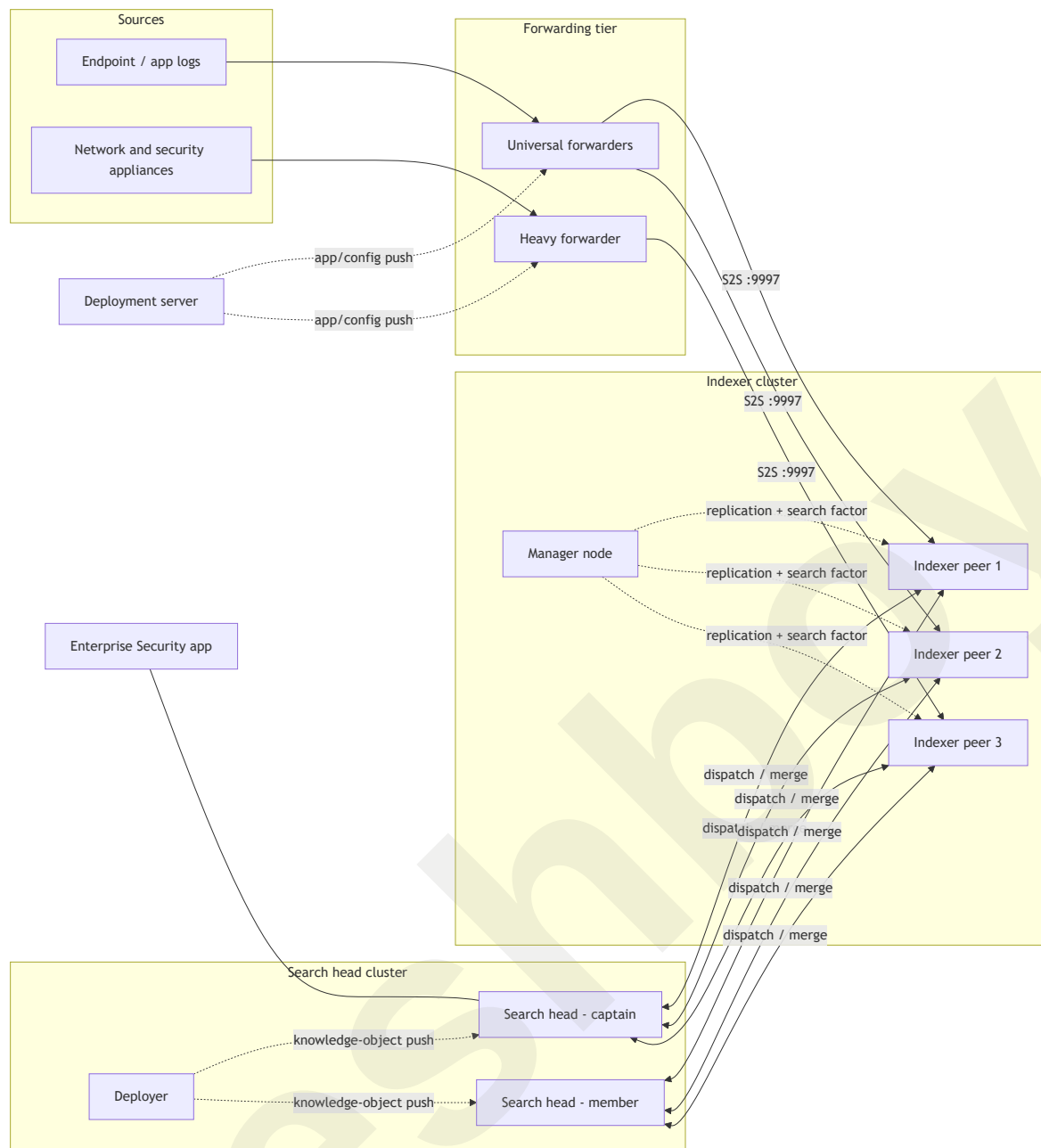
The on-prem-versus-Cloud choice is a real infrastructure build-versus-buy decision with staffing and cost consequences on both sides — a self-managed cluster needs someone who can run one; a managed one trades that staffing cost for a recurring service cost and less direct control. Part 4 covers the licensing and ingest-economics side of that tradeoff in the depth it deserves; this part is only naming that the decision exists and pointing at where the cost analysis actually lives.

Figure 2.1 — Splunk distributed deployment topology, sources through Enterprise Security.

CONCEPTUAL. Illustrates the components covered in Sections 1–5 and how they connect: forwarders feeding an indexer cluster coordinated by a manager node, a deployment server managing the forwarder fleet, and a search head cluster (managed by a deployer, with one member acting as captain) dispatching searches against the indexer tier and hosting the Enterprise Security app. This is an architecture sketch of documented component relationships, not a capture from a real deployment — see [STYLE-GUIDE.md](#) §9.2.

```
graph LR
  subgraph Sources
    A1[Endpoint / app logs]
    A2[Network and security appliances]
  end
  subgraph Forwarding_tier [Forwarding tier]
    UF[Universal forwarders]
    HF[Heavy forwarder]
  end
  DS[Deployment server]
  subgraph Indexer_cluster [Indexer cluster]
    CM[Manager node]
    IDX1[Indexer peer 1]
    IDX2[Indexer peer 2]
    IDX3[Indexer peer 3]
  end
  subgraph Search_head_cluster [Search head cluster]
    DEP[Deployer]
    SH1[Search head - captain]
    SH2[Search head - member]
  end
  ES[Enterprise Security app]

  A1 --> UF
  A2 --> HF
  UF -->|S2S :9997| IDX1
  UF -->|S2S :9997| IDX2
  HF -->|S2S :9997| IDX3
  DS -.app/config push.-> UF
  DS -.app/config push.-> HF
  CM -.replication + search factor.-> IDX1
  CM -.replication + search factor.-> IDX2
  CM -.replication + search factor.-> IDX3
  DEP -.knowledge-object push.-> SH1
  DEP -.knowledge-object push.-> SH2
  SH1 <-->|dispatch / merge| IDX1
  SH1 <-->|dispatch / merge| IDX2
  SH1 <-->|dispatch / merge| IDX3
  SH2 <-->|dispatch / merge| IDX1
  SH2 <-->|dispatch / merge| IDX2
  SH2 <-->|dispatch / merge| IDX3
  ES --- SH1
```



8. Sizing a deployment for a security workload

[PLATFORM ENGINEER] Sizing a topology like the one this part describes comes down to four inputs feeding back into the components already covered: daily ingest volume (indexer disk and CPU, and Part 4's license), retention requirements by index (Part 3's bucket lifecycle), how much of that data will sit behind accelerated data models (Part 10's storage and maintenance cost), and — the input most often underestimated for a security-specific deployment — the scheduled-search load Enterprise Security itself adds once it's installed.

[SOC MANAGEMENT] That last input is the one worth naming explicitly, because it's easy to size a search head tier for the ad hoc investigation load a team of analysts generates and then discover that ES's own scheduled correlation searches — potentially dozens to low hundreds of them, each on its own cadence, once a reasonably complete detection content set is enabled — are competing for the same

concurrent-search budget. Sizing "for search," not just "for ingest," is the mistake that under-sizes a SOC's search head tier; Part 12 covers the concurrency limits and scheduling-priority mechanics that make this concrete.

Engineering Reality *An Enterprise Security deployment's baseline scheduled-search load exists from the moment a reasonable set of correlation searches is enabled, independent of how much ad hoc searching anyone does on top of it. Treat that baseline as a fixed cost to size against from day one, not a variable that only shows up once analysts start actively hunting.*

What Would Change My Mind *This section stays deliberately qualitative — it names the inputs to a sizing decision without giving a specific "GB per core" or "searches per search head" formula, because this book has no real Splunk deployment to benchmark those numbers against, and inventing a specific figure without evidence behind it would be exactly the kind of unsupported claim `STYLE-GUIDE.md` §9.4 rules out. A real Splunk lab, ingesting a representative security-telemetry mix and instrumented over an actual retention window with ES's own scheduled content running against it, is what would let this section move from naming inputs to giving numbers — Part 20 names this same gap as one of the book's largest open items.*

Cross-references

- DEH Part 26 §1.1 ("cost is paid left to right") and §1.2 (search-time/index-time field split) — the language-level foundation for Section 6's distributed-search dispatch/merge model; Part 12 of this book applies §1.1's principle at fleet scale.
- DEH Part 26 §1.3 — the named-but-undeveloped `tstats` forward reference this book resolves in Part 10.
- DEH Part 23 §1 — the analytic-vs-detection-rule vocabulary this book's correlation-search content (Part 14) assumes; not used directly in this part but load-bearing for what runs on the search heads this part describes.
- This book's Part 1 — states the general non-duplication boundary against DEH Parts 23 and 26 that this part applies specifically to deployment topology.
- This book's Part 3 (indexes, sourcetypes, bucket lifecycle), Part 4 (licensing and ingest economics), Part 10 (data model and report acceleration), Part 12 (search performance and tuning at scale), and Part 13 (Enterprise Security architecture and editions) — each owns depth this part deliberately deferred to.

Part 3 — Indexes, Sourcetypes, and the Bucket Lifecycle

Why this part exists

DEH Part 26's own introductory scope note is explicit about what it chose not to cover: it "does not cover Splunk administration, index/bucket lifecycle management, or `tstats`/data-model acceleration in depth... a full accelerated-search treatment belongs in a dedicated performance-engineering appendix, not here." This part is the first half of that appendix, expanded into full chapter depth. It assumes you already know how to write a search — DEH Part 26 owns `stats`, `transaction`, subsearch cost, and the search-time-vs-index-time field split at the language level, and DEH Part 23 owns the Analytic-vs-Detection-Rule vocabulary this book's own correlation-search content (Part 14 onward) will lean on. Neither is re-taught here.

What this part owns instead: the storage layer every one of those searches runs against. An index is not a cosmetic label you type after `index=` — it is a named, access-controlled, independently-retained collection of data on disk, and the decisions you make about index design determine who can see what, how long it survives, and whether a retention or compliance requirement is actually enforced or just aspirational. A sourcetype is a different, easily confused thing: a parsing contract, not a storage boundary, and one that can silently drift out from under a running detection without anyone touching the detection itself. Underneath both sits the bucket — the actual physical unit Splunk ages, compresses, moves, and eventually deletes or archives, on a clock that runs whether or not anyone is searching.

This part explicitly defers three things to their own parts rather than covering them here: the cost and licensing side of index sizing (Part 4), how a sourcetype's fields become a queryable CIM data model (Parts 5–7), and `tstats`/data-model acceleration performance (Part 10, resolving the forward reference DEH Part 26 §1.3 names but doesn't develop). What follows is scoped to the platform mechanics underneath all of those: index and sourcetype architecture, and the bucket lifecycle that makes retention a real, physically-enforced thing rather than a setting you configure once and trust.

1. The index as a security boundary

[CONCEPT] An index is the largest unit of data organization Splunk exposes to an administrator, and it is defined by where its data physically lives and who is allowed to search it — not by what the data is about. Two log sources describing completely unrelated systems can share an index; two log sources describing the same system cannot safely share an index if they need different retention or different access control, because both of those are properties of the index, not the event. This is the single fact this part keeps returning to: an index is a security and retention boundary first, and a convenient query filter only as a side effect of that.

1.1 What an index actually is

[CONCEPT] Mechanically, an index is a named directory tree containing a set of buckets (\$5), each bucket holding a contiguous range of indexed events plus the compressed raw data and index

structures (`.tsidx` files) Splunk uses to search it. Every event lands in exactly one index, assigned either by whatever `index=` value a forwarder's outputs configuration specifies or by an explicit `props.conf`/`transforms.conf` routing rule at the parsing tier. Once an event is written into a bucket in an index, moving it to a different index is not a metadata edit — it requires re-ingesting the underlying data, because the index assignment, like the sourcetype assignment covered in §3, is baked into the bucket at write time.

1.2 Default indexes, and why `main` is not a security boundary

[PLATFORM ENGINEER] A stock Splunk installation ships several indexes out of the box, most visibly `main` (the default destination for any data source that isn't explicitly routed elsewhere), `_internal` (Splunk's own operational logging — the platform monitoring itself), `_audit` (search and configuration-change auditing), and `_introspection` (host resource metrics). `main` is the one that causes the most avoidable damage: it is the path of least resistance for a new data source's onboarding, and because it's the default, it accumulates whatever nobody made an explicit decision about. A security team that lets Windows authentication logs, a vulnerability scanner's findings feed, and a noisy IoT device's syslog all land in `main` by default inheritance has, without anyone deciding it on purpose, given every role with access to `main` visibility into all three, and locked all three into whatever single retention setting `main` carries. Treat `main` as a place things end up, not a place you designed.

1.3 Sourcetype vs. index: two different axes

[CONCEPT] It's worth being precise about this distinction early, because the rest of this part depends on readers not conflating the two: an index is a storage and access-control boundary, and a sourcetype (`sourcetype=WinEventLog:Security`, `sourcetype=aws:cloudtrail`) is a parsing and field-extraction label describing what format an event is in. Two events with the same sourcetype can live in different indexes (Windows Security events from a PCI-scoped domain controller vs. a general one); two events in the same index can carry different sourcetypes (a SOC's `security` index routinely holds dozens of distinct sourcetypes side by side). Confusing the two leads to two different design mistakes: trying to enforce access control with sourcetype filters in a search (which a user with broad index access can simply omit), or trying to isolate retention by sourcetype within a shared index (which doesn't work, because retention settings apply to the whole index — see §2.1).

2. Designing indexes around retention and access control

[PLATFORM ENGINEER] The two properties that actually make index design a security decision — not just a storage-tidiness one — are retention and role-based access control. Both are configured per index, both are enforced by the platform independently of any individual search, and both are the reason "which index does this data go in" deserves a deliberate answer before ingestion starts, not an after-the-fact cleanup project.

2.1 Retention is index-scoped, not source-scoped

[PLATFORM ENGINEER] Every retention-relevant setting this part discusses in §4 — `frozenTimePeriodInSecs`, `maxTotalDataSizeMB`, `coldToFrozenDir` — lives in an `indexes.conf` stanza for one

named index and applies to everything stored under it. There is no per-sourcetype or per-source override sitting underneath that; if two data sources with genuinely different legal or operational retention requirements share an index, the index's single retention clock governs both, and it will be either too short for the one that needed to be kept longer, or too long (with the storage and license cost that implies — Part 4) for the one that didn't need to be. This isn't a limitation you work around with a clever search-time filter; a search's time range only decides what a given query looks at among data that still physically exists (§6.2 has the sharp version of this).

Engineering Reality *The team that puts "temporary" debug-level application logs and a year-mandated compliance log into the same index because "it's just easier to manage one index" is not choosing a compromise between two retention needs — they're choosing the debug logs' short retention requirement or the compliance logs' long one, and whichever one loses stays lost even if a later audit needs it. Splitting a shared index after the fact doesn't retroactively fix already-indexed data: the events already written stay under the old index's settings until they age out or get manually migrated, which for a compliance requirement means the exposure already happened by the time anyone notices the mistake.*

2.2 `authorize.conf` — the file that actually enforces index-level RBAC

[PLATFORM ENGINEER] Splunk's role-based access control for search scope is configured in `authorize.conf`, through two settings on a role stanza: `srchIndexesAllowed`, the list of indexes (wildcards permitted) a role is permitted to search at all, and `srchIndexesDefault`, the indexes a search scans automatically when a user doesn't type an explicit `index=` clause. A role with `srchIndexesAllowed = security_*` cannot see events in an index outside that pattern no matter what SPL it runs — this is enforced below the query, at the same layer that decides which buckets a search is even allowed to open, not by filtering results after the fact.

CONCEPTUAL SAMPLE — illustrative role stanza; field values are representative, not reproduced from a running deployment.

```
[role_soc_tier1]
srchIndexesAllowed = security_endpoint;security_network;security_auth
srchIndexesDefault = security_endpoint;security_network;security_auth
importRoles        = user

[role_pci_auditor]
srchIndexesAllowed = pci_cardholder_data
srchIndexesDefault = pci_cardholder_data
importRoles        = user
```

[PLATFORM ENGINEER] The failure mode to watch for is `importRoles` inheritance stacking `srchIndexesAllowed` values across roles a user holds simultaneously: Splunk unions the allowed-index lists of every role a user is assigned, so a tier-1 analyst who also picks up an unrelated role granted broad access (a shared "read-only-everything" role created for a one-off investigation and never revoked) inherits that role's index scope permanently, silently, and outside whatever review process

governs the tier-1 role itself. Index-level RBAC is only as tight as the least-reviewed role a user has ever been assigned.

2.3 Multi-tenancy and sensitivity segregation

[PLATFORM ENGINEER] The same mechanism that isolates one compliance domain from another scales directly into multi-tenant separation — an MSSP SOC running detection content across multiple customers, or a single enterprise segregating a regulated business unit's data from the rest of the company, both use index-per-tenant (or index-per-sensitivity-tier) as the enforcement boundary, paired with `srchIndexesAllowed` scoped per tenant's role. The table below is the decision matrix this part uses to answer "does this new data source need its own index," and it's worth working through explicitly rather than defaulting to `main` or to whatever index the last onboarding used.

The table below supports the index-vs-sourcetype design decision a platform engineer makes at onboarding time, before the first event from a new source is written.

Decision Driver	Same Index OK?	Why
Retention requirement differs materially (e.g., 30 days vs. multi-year)	No	<code>frozenTimePeriodInSecs</code> and size caps (\$4.3) apply per index; the whole index inherits one retention clock
Access-control boundary differs (e.g., regulated cardholder data vs. general infrastructure logs)	No	<code>srchIndexesAllowed</code> grants at index granularity; there is no sub-index search scope
Multi-tenant or MSSP customer separation	No	Tenant isolation is a hard requirement here, not a preference, and index-level RBAC is the mechanism that actually enforces it
Same retention, same RBAC, different <code>sourcetype</code>	Yes	Sourcetype and index are independent axes (\$1.3); splitting on sourcetype alone adds administrative overhead with no isolation benefit
Very high-volume source vs. low-volume source, same retention/RBAC, little query overlap	Consider	Isolating the high-volume source limits how many small, low-value buckets a broad search across the low-volume index has to open

3. Sourcetype assignment and drift

[PLATFORM ENGINEER] A sourcetype is Splunk's label for "what format is this data in," and it drives which field extractions, event-type tags, and (eventually) CIM mappings (Parts 5–7) apply to an event. Getting sourcetype assignment right matters as much as getting index assignment right — the difference is that a sourcetype mistake tends to fail quietly, months after everyone stopped thinking about it, rather than immediately and visibly the way a wrong `index=` in an RBAC-restricted search does.

3.1 `props.conf` and `transforms.conf` — where sourcetype actually gets assigned

[PLATFORM ENGINEER] Sourcetype is normally assigned by a forwarder's monitor stanza (a fixed value per file or input) or, for data that needs conditional assignment based on content, by a `transforms.conf` stanza with `DEST_KEY = MetaData:Sourcetype` referenced from a `props.conf` `TRANSFORMS-<class>` line.

Assignment happens at the parsing tier — a heavy forwarder or the indexer's own parsing queue — and once it's written into a bucket, it's part of that event's fixed metadata alongside `_time`, `host`, and `source` (the same index-time-vs-search-time boundary DEH Part 26 §1.2 covers for individual fields applies to sourcetype itself: it is fixed at index time, and changing the assignment logic going forward does nothing to events already on disk). Retroactively correcting a misassigned sourcetype for historical data means re-ingesting that data under the corrected rule, not editing a config and waiting.

3.2 Sourcetype drift: when a TA upgrade breaks a running detection

[PLATFORM ENGINEER] Most sourcetype assignment in a real deployment doesn't come from hand-written `props.conf`/`transforms.conf` — it comes from a Splunk-supported or community Technology Add-on (TA) that ships its own parsing rules for a specific vendor product. That's a maintenance dependency, not a one-time install: a TA upgrade can rename a sourcetype, split one sourcetype into several by log-channel, or change a field extraction's regex to match a vendor's new log format, and none of those changes announce themselves — the upgrade completes cleanly, new data keeps flowing, and it just stops matching whatever `sourcetype=` literal is hardcoded into any correlation search, macro, or dashboard panel written against the old name.

Detection Autopsy — "the notable that stopped firing after a TA version bump"

This illustrates a failure mode that follows mechanically from how TA-versioned sourcetype naming works (§3.2), not a single cited incident — no specific conference talk or TA changelog is cited here, so per STYLE-GUIDE.md §9.4 this box should be read as a CONCEPTUAL composite pattern rather than a verified, publicly-documented case study, pending a citable source in REFERENCES.md.

The rule: A correlation search filters on a literal `sourcetype=cisco:asa` (or an equivalent vendor-specific value) as its primary scoping term.

Why it shipped: The value was correct at the time the rule was written, validated against the TA version installed then, and nobody expected a minor-version TA update to touch sourcetype naming.

How it failed: A TA update reorganizes its sourcetype scheme — commonly splitting one broad sourcetype into several more specific ones, or renaming it to match a new vendor product line — and every saved search hardcoded against the old literal value returns zero results from that point forward. The search still runs on schedule, still completes without error, and reports nothing wrong, because an empty result set and "no matching events occurred" are indistinguishable to a scheduled search with no independent health check.

The fix: A parser/sourcetype-health monitor — a scheduled search using `| metadata type=sourcetypes index=<index>` to track each index's live sourcetype list and each one's `recentTime`, alerting when an expected sourcetype's `recentTime` goes stale or an unexpected new one appears — catches this within a monitoring cycle instead of at the next incident review, the same principle DEH Part 26's own Detection Autopsy applies to a broken field extraction rather than a renamed sourcetype.

[DETECTION ENGINEER]

The consequence for a correlation search built on top of a CIM data model (Parts 5–7) rather than a raw sourcetype literal is narrower but not zero: CIM compliance is enforced through `tags.conf / eventtypes.conf` mappings tied to the sourcetype, so a renamed sourcetype has to be re-tagged into the relevant data model before it contributes to that model's search results again — a step a TA upgrade does not do for you automatically unless the TA itself ships updated CIM mappings for its new sourcetype names. A correlation search written directly against a data model rather than a hardcoded sourcetype degrades to "silently missing one data source's contribution" instead of "returns zero results," which is a smaller failure but not a self-announcing one either.

4. `indexes.conf` — where storage tiers and retention actually live

[PLATFORM ENGINEER]

Every retention and storage-tiering decision described conceptually in §1–§2 is configured in one place: an `indexes.conf` stanza named for the index it governs. This is also where the bucket-lifecycle mechanics in §5 get their actual trigger values.

CONCEPTUAL SAMPLE — illustrative stanza combining commonly-used settings; numeric values shown are one admin's chosen configuration, not Splunk's factory defaults for any particular version.

```
[pci_cardholder_data]
homePath              = $SPLUNK_DB/pci_cardholder_data/db
coldPath              = $SPLUNK_DB/pci_cardholder_data/colddb
thawedPath            = $SPLUNK_DB/pci_cardholder_data/thaweddb
maxDataSize           = auto_high_volume
maxHotBuckets         = 3
maxWarmDBCount        = 300
frozenTimePeriodInSecs = 94608000
maxTotalDataSizeMB    = 512000
coldToFrozenDir       = $SPLUNK_DB/pci_cardholder_data/frozendb
```

4.1 Storage-tier paths: `homePath`, `coldPath`, `thawedPath`

[PLATFORM ENGINEER] `homePath` is where an index's hot and warm buckets live — typically the fastest storage available, since hot buckets are being actively written and both hot and warm buckets are fully searchable and queried the most. `coldPath` is where buckets roll to once they age out of the warm tier; it's common (and a legitimate cost lever — Part 4) to point `coldPath` at slower, cheaper storage, since cold buckets are still searchable but statistically queried far less often than recent data. `thawedPath` is not part of the automatic lifecycle at all — it's the destination an administrator manually restores an archived frozen bucket into when older data needs to become searchable again for a specific investigation, and Splunk does not manage anything placed there under the index's ordinary retention clock.

4.2 Sizing and rolling: `maxDataSize`, `maxHotBuckets`, `maxWarmDBCount`

[PLATFORM ENGINEER] `maxDataSize` bounds how large a single hot bucket is allowed to grow before Splunk rolls it to warm; `maxHotBuckets` bounds how many hot buckets an index may have open for writing at once; `maxWarmDBCount` bounds how many warm buckets accumulate before the oldest ones roll to cold. The exact numeric defaults for these settings have shifted across Splunk's release history — confirm the literal current values against the `indexes.conf.spec` shipped with your installed version rather than trusting a number quoted from a training deck, a blog post, or, for that matter, this chapter.

4.3 Aging out: `frozenTimePeriodInSecs`, `maxTotalDataSizeMB`, `coldToFrozenDir`

[PLATFORM ENGINEER] `frozenTimePeriodInSecs` is the age threshold (in seconds) past which a bucket becomes eligible to roll from cold to frozen; `maxTotalDataSizeMB` is an independent, size-based cap on the index's total on-disk footprint that can force the oldest buckets to freeze early even if they haven't yet reached the age threshold. `coldToFrozenDir` (or, for more complex handling, `coldToFrozenScript`) determines what "frozen" actually means for that index: if set, frozen buckets are archived to the named location instead of deleted; if unset, Splunk's default behavior on freezing is deletion. An index with neither setting configured is not retaining its old data somewhere safe by default — it is deleting it once the age or size threshold is crossed, permanently, with no separate confirmation step.

Blind Spot Splunk's shipped default retention settings favor keeping data over deleting it, which sounds safe until you notice what that actually means operationally: an administrator who never touches `frozenTimePeriodInSecs` has not chosen "no retention policy" — they've chosen whatever multi-year default ships with their installed version, silently accumulating storage cost against the license (Part 4) with no one having made that a deliberate decision. The opposite failure is just as real and less obvious: `maxTotalDataSizeMB` (or a volume-level size cap) is a hard ceiling that freezes the oldest buckets first regardless of whether that violates a compliance mandate to keep a minimum retention window. A legal or regulatory requirement to retain twelve months of authentication logs is not satisfied by "we never set a retention limit" if the index quietly hit its size cap at month seven and started freezing the oldest data anyway — disk pressure enforces itself even when nobody configured it to.

5. Bucket states: hot, warm, cold, frozen, thawed

[CONCEPT] The bucket is the actual unit Splunk manages — a directory holding a contiguous range of events' raw data and search indexes for one time span within one index. Every index-level setting in \$4 exists to control when and how a bucket moves between five states.

```
stateDiagram-v2
    [*] --> Hot: new events written; index has one\nor more open, writable hot buckets
    Hot --> Warm: maxSize reached, maxHotBuckets\nexceeded, or splunkd restarts and rolls the
    bucket
    Warm --> Cold: maxWarmDBCount exceeded for the index
    Cold --> Frozen: bucket age passes frozenTimePeriodInSecs,\nor an index/volume size cap is
    exceeded
    Frozen --> Deleted: no coldToFrozenDir / coldToFrozenScript configured
    Frozen --> Archived: coldToFrozenDir or coldToFrozenScript configured
    Archived --> Thawed: administrator manually restores\ninto thawedPath for a specific search need
    Thawed --> [*]: searchable again; not managed by\nthe ordinary retention clock
    Deleted --> [*]
```

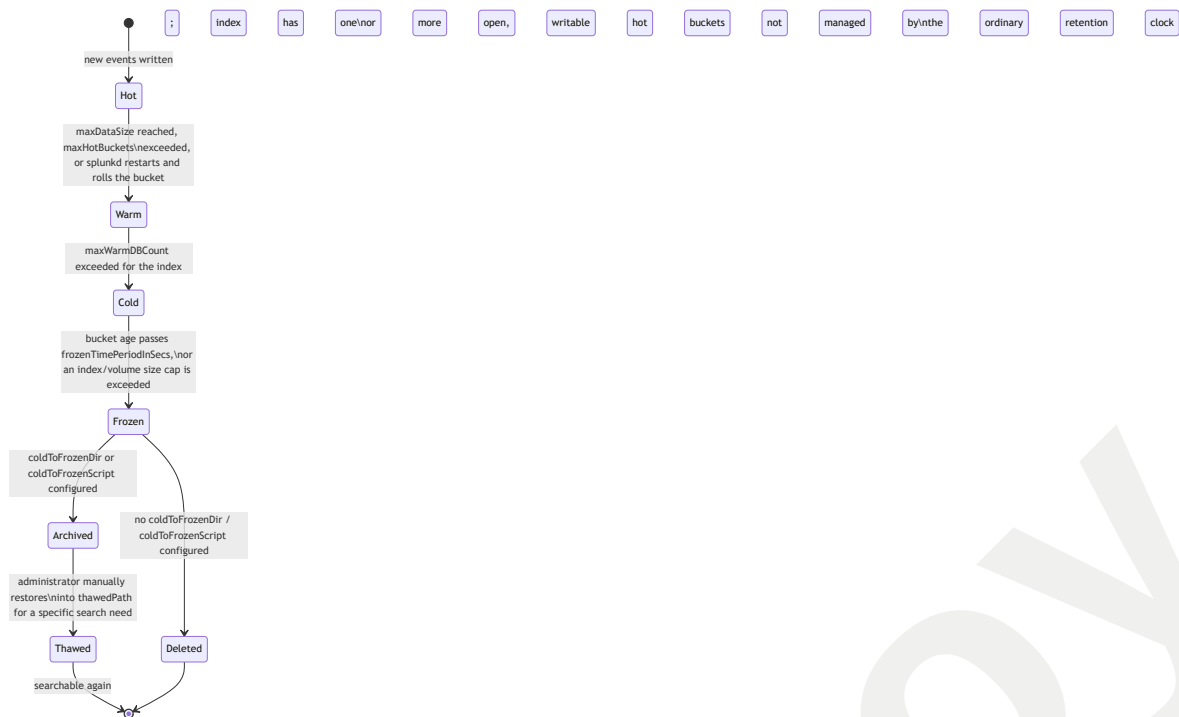


Figure 3.1 — Bucket lifecycle: hot to frozen, and back if thawed. *CONCEPTUAL*. Illustrates the documented, expected sequence of bucket-state transitions and the `indexes.conf` settings that trigger each one. This is a diagram of expected platform behavior, not a capture from a live indexer's bucket directory listing — no Splunk deployment exists in this book's evidence base (see `STYLE-GUIDE.md §9.2`).

The table below supports deciding what to check when a search seems to be missing data that should still be within retention — which state a bucket is in determines both its searchability and where its bytes physically live.

Bucket State	Typical Location	Searchable	Moves In When	Moves Onward When
Hot	<code>homePath</code>	Yes, always	An index opens a new bucket to receive incoming events	<code>maxDataSize</code> reached, <code>maxHotBuckets</code> exceeded, or a <code>splunkd</code> restart rolls it
Warm	<code>homePath</code>	Yes	Rolled from hot	<code>maxWarmDBCount</code> exceeded for the index, oldest-first
Cold	<code>coldPath</code>	Yes	Rolled from warm, oldest-first	Bucket age exceeds <code>frozenTimePeriodInSecs</code> , or a size cap (<code>maxTotalDataSizeMB</code>) is exceeded
Frozen	Deleted, or the <code>coldToFrozenDir</code> / <code>coldToFrozenScript</code> target	No	Rolled from cold	Nothing automatic — frozen is a terminal state unless manually thawed
Thawed	<code>thawedPath</code>	Yes, once restored	An administrator manually restores an archived frozen bucket	Nothing automatic; not re-subject to the same retention clock

`dbinspect` — checking what state your buckets are actually in

[PLATFORM ENGINEER] `dbinspect` is the SPL command purpose-built for this: run against an index, it returns one row per bucket with its state (`hot` / `warm` / `cold`), path, size on disk, event count, and the earliest and latest event times it covers. That last pair matters more than it looks: a bucket's age, for freezing purposes, is a function of the event-time range it holds, not of when the bucket was created or how recently it was last written to — which is exactly why a delayed forwarder, a bulk historical import, or a backfill job can produce a bucket whose *latest* event is recent even though most of its contents are old, keeping the whole bucket (and everything in it) alive well past what its oldest events' age alone would suggest. Treat this specific bucket-aging mechanic as documented Splunk platform behavior, not independently re-verified against docs.splunk.com for this book — that domain returned an access error to this book's research tooling as of 2026-09-15 (see STYLE-GUIDE.md §9.4) — and confirm it against your own `dbinspect` output before relying on it operationally.

```
| dbinspect index=security_auth
| table bucketId, state, path, sizeOnDiskMB, eventCount, startEpoch, endEpoch
| sort - endEpoch
```

[THREAT HUNTER] The same command is worth running before trusting a negative hunting result: a hypothesis that comes back empty because the relevant bucket has already frozen (and the hunter's search window quietly excludes it) looks identical, from the search results alone, to a hypothesis that came back empty because the activity genuinely never happened. Checking `dbinspect`'s coverage for the index and time range in question before writing up a "no evidence found" finding is a cheap, five-second discipline that avoids exactly that confusion.

6. How retention is actually enforced: buckets, not events, not queries

[PLATFORM ENGINEER] This is the point everything in §4 and §5 has been building toward, and it's the one most likely to surprise someone who thinks about retention the way they think about a database's row-level TTL: Splunk does not evaluate retention event by event, and it does not evaluate it in response to any search. It evaluates it bucket by bucket, on its own schedule, using the bucket's own time boundaries — completely independent of whether a human ever runs a query against that data at all.

6.1 Why a bucket ages out as a whole, not event by event

[PLATFORM ENGINEER] A single bucket typically spans a range of event times, not one instant — and freezing eligibility is evaluated against that bucket's boundary (§5's `dbinspect` discussion), not against each event inside it individually. The practical result: events that are, considered individually, still well within a retention window can be deleted or frozen anyway, dragged along by a bucket whose overall time range has aged out; conversely, older events can survive longer than their own age alone would suggest if they share a bucket with a much more recent event that keeps the whole bucket's latest boundary young. Retention is a property Splunk applies to a storage unit, and the storage unit's boundary — not any individual event's timestamp — is what gets checked.

6.2 The query-time-range trap

[SOC ANALYST] This has a direct, concrete consequence for anyone investigating an incident: restricting a search's time-range picker to "last 90 days" tells Splunk what to look at, not what to keep. If the underlying buckets for data older than 90 days have already been frozen and deleted, narrowing your search window doesn't protect that data — it was never protected by the search in the first place, because the search and the retention clock are two completely independent systems. An analyst who assumes "we haven't needed to look past our normal search window, so the older data must still be there if we ever need it" is trusting a mechanism (the search) that has no relationship at all to the mechanism (bucket aging) that actually decides whether the data still exists. If preserving specific data for an active investigation matters, that has to be a deliberate action against the retention configuration or the bucket itself (§6.3), not an assumption based on search habits.

6.3 Legal hold and evidence preservation against a live retention clock

[SOC MANAGEMENT] The retention clock described in §4.3 keeps running during an active investigation exactly the same way it runs on an ordinary Tuesday — Splunk has no built-in awareness that a legal hold or evidence-preservation requirement applies to a specific event, a specific host, or a specific time range, because retention is configured per index, not per case. Satisfying a hold on data that's approaching its `frozenTimePeriodInSecs` threshold, or that lives in an index whose `maxTotalDataSizeMB` cap is close to forcing an early freeze, means intervening at the platform level: exporting the relevant buckets, copying them out of the ordinary lifecycle, or temporarily adjusting the index's retention settings — not assuming that "the investigation is still open" pauses anything on its own, because it doesn't.

What Would Change My Mind The claim in §6.1–§6.3 — that bucket-level aging can silently outrun an individual event's own retention expectation, and that nothing about an open investigation pauses it — is standard, documented Splunk storage-architecture behavior, not something observed running against a real deployment here. What would move this from "documented mechanism" to "verified operational risk" is a real Splunk environment with a real legal-hold procedure, tested by actually letting a bucket approach its freeze threshold mid-investigation and confirming whether the organization's hold process catches it in time. No lab in this book's evidence base runs Splunk, so that test hasn't happened and can't be reported as a result here.

7. Index sprawl, and the cost of getting index design wrong in the other direction

[SOC MANAGEMENT] Everything above argues for splitting indexes along retention and RBAC boundaries. That argument has a real cost on the other side of it, and a design that over-corrects into one index per data source, one per team, or one per minor sensitivity distinction pays for that granularity in ways that show up as a performance and administration problem rather than a security one.

SOC Management View Index-per-tenant is the right call when tenant isolation is a genuine, auditable requirement — but "genuine requirement" and "seemed like good hygiene at the time" are different justifications, and only the first one is worth the ongoing cost. Every additional index is a stanza someone has to maintain in `indexes.conf`, a value someone has to remember to add to the right roles' `srchIndexesAllowed`, and — at scale — more individual small buckets a broad cross-index search has to open and check, even when most of them contain nothing relevant to that search. A design with one index per customer works cleanly at ten customers and becomes a real search-performance and administrative-overhead problem at five hundred, at which point the fix (consolidating low-volume tenants behind a shared index with field-based logical separation instead of physical separation) is itself a migration project, not a config change. Decide the granularity you actually need against real projected scale before index count becomes something nobody wants to touch.

Product Version Note Splunk Enterprise Security 8.7.0 (released 2026-09-02) lists compatibility with Splunk Enterprise and Splunk Cloud Platform versions 10.5, 10.4, 10.3, and 10.2 on its own Splunkbase listing. As of 2026-09-15, verified against that listing (REFERENCES.md entry [ES-SPLUNKBASE]) — that listing speaks only to ES-to-platform version compatibility, not to storage-layer internals. Separately, and not verified against that same listing or any other dated source: this book's own reading of Splunk's current index/bucket documentation is that the bucket-state names and `indexes.conf` stanza structure described in this part have gone unchanged across that 10.x line and well before it. Treat that specific continuity claim as this book's own unverified assessment of core platform architecture, not as a sourced fact, and confirm it against the `indexes.conf.spec` for your own installed version before relying on it. What would make this part stale: a major version introducing a materially different storage model (Splunk's SmartStore remote/object-storage tiering already changes how the cold tier in particular is physically managed for indexer-clustered deployments, and this chapter's `coldPath`-to-local-disk framing assumes the traditional model rather than SmartStore specifically), or a future release changing a shipped default value this chapter deliberately declined to state as fact (§4.2).

Cross-references: DEH Part 26 §1.2 (search-time vs. index-time fields, extended here to sourcetype and index assignment) and DEH Part 26's introductory scope note (which names index/bucket lifecycle management as explicitly out of its scope) for the non-duplication boundary this part fills; DEH Part 23 (Analytic-vs-Detection-Rule vocabulary) for the framing Part 14's correlation-search content later builds on top of the index/sourcetype foundation laid here. Within this book: Part 4 (licensing and ingest economics) for the cost side of index sizing and retention; Parts 5–7 (the Common Information Model) for how a sourcetype's fields become a CIM-compliant data model input, and what breaks when the mapping in §3.2 goes stale; Part 10 (data model and report acceleration) for `tstats` and the performance payoff of the bucket architecture described in §5; Part 13 (Enterprise Security architecture) for why CIM compliance and index-level RBAC become hard ES prerequisites rather than optional best practice.

Part 4 — Licensing and Ingest Economics

Why this part exists

Part 2 covered where indexers, search heads, and forwarders sit relative to each other. Part 3 covered what happens to an event after it lands on an indexer — sourcetype assignment, and the hot/warm/cold/frozen bucket lifecycle that enforces retention. This part covers the question that sits underneath both: what does any of that actually cost, who pays for it, and which levers does a security team control before "reduce cost" quietly becomes "reduce visibility"? That question is platform-layer in mechanism (a license is a technical enforcement construct, metered against real ingest and compute numbers) and SOC-management-facing in consequence (a licensing decision is a budget line and a coverage decision at the same time), which is why this part sits at the boundary of Section A rather than fully inside either neighbor.

This part does not re-litigate index design or bucket retention mechanics — Part 3 owns those, and this part cites them wherever a cost lever depends on them. It also does not touch SPL syntax: nothing here needs `stats` or `transaction` beyond what's already covered in DEH Part 26 §2–3, and the one place a query would help illustrate a cost decision, this part points at Part 26 rather than re-deriving pipeline syntax.

Evidentiary note, stated once and not repeated at every section: no Splunk deployment exists in this book's author's lab, and licensing behavior specifically is not the kind of thing a lab of any size would surface convincingly — license economics show up at fleet scale, over months, against a real bill. Every platform-behavior claim below is either sourced from Splunk's own public pricing and documentation pages (cited in `REFERENCES.md`, retrieved 2026-09-15) or explicitly flagged as unverified against current documentation, per STYLE-GUIDE.md §9.4's disclosed constraint that `docs.splunk.com` itself was not reliably fetchable during this book's research.

1. What a Splunk license actually meters

[CONCEPT] For most of Splunk's history, a Splunk Enterprise license metered exactly one thing: the volume of data indexed per day, in gigabytes, against a fixed daily ceiling. That model — call it ingest-based licensing — is still the default most administrators picture when they hear "Splunk license," and it's still one of the models Splunk sells today. It is no longer the only one.

Splunk's own current pricing materials describe the platform as licensed under a small set of named models rather than one universal metric. The two that apply directly to the core Splunk Platform (Splunk Enterprise and Splunk Cloud Platform) are **ingest pricing** — data volume brought into the platform — and **workload pricing** — compute consumption metered in a Splunk-specific unit. A third model, **entity-based pricing**, exists in Splunk's portfolio but is scoped to adjacent products rather than core SIEM ingest — Splunk On-Call bills per seat, for instance — and is mentioned here only so a reader comparing a quote across Splunk's product line doesn't mistake a Splunk On-Call line item for a Splunk Platform ingest or workload charge.

Product Version Note *Splunk currently prices the Splunk Platform (Splunk Enterprise / Splunk Cloud Platform) under two models — ingest pricing and workload pricing — with a separate entity-based model used elsewhere in Splunk's broader portfolio (Splunk On-Call's per-seat pricing, Splunk Observability Cloud's per-host tiers) rather than for core platform ingest. As of 2026-09-15, verified against Splunk's own pricing overview and pricing FAQ pages ([REFERENCES.md](#) entries [\[SPLUNK-PRICING-OVERVIEW\]](#) and [\[SPLUNK-PRICING-FAQ\]](#)). What would make this stale: Splunk restructuring its published pricing page, retiring one of the two core-platform models, or extending entity-based pricing to core ingest in a way it doesn't currently cover.*

The rest of this part treats these two models — ingest and workload — as the two real levers a SOC actually chooses between, and spends the bulk of its length on the cost decisions that apply regardless of which one a given deployment runs under.

1.1 Why this is a security-operations concern, not just a finance one

[SOC MANAGEMENT] A licensing model determines what a detection engineer is implicitly optimizing for every time they onboard a new log source or write a correlation search against a wide time window. Under ingest pricing, every gigabyte a new data source adds is a direct, visible cost line — a team under budget pressure develops a reflex to ask "do we need this feed" before onboarding it, which is exactly the right question, but only if it's asked deliberately rather than answered by whoever happens to be annoyed by the current month's usage report. Under workload pricing, the same reflex points somewhere else: a wide-lookback scheduled search run every five minutes against a high-cardinality field costs real money even if the data it searches was already "free" to ingest under that model. Neither model makes cost disappear; each one moves the pressure point to a different part of the platform, and a team that doesn't know which model it's under is optimizing blind.

2. Ingest-based licensing: volume measured at the index, not at the source

[PLATFORM ENGINEER] Under ingest pricing, the number that counts is the volume of data actually written to an index per day — measured on the uncompressed size of the events as indexed, not the size of the original log file on the source host, and not the size after Splunk's own on-disk compression shrinks it in a warm or cold bucket (Part 3 §3 covers that compression as a storage question, distinct from this licensing question). Two consequences follow directly from "measured at the index":

- **Volume that never reaches an indexer never counts.** Anything filtered, dropped, or routed away before the indexing stage — at a heavy forwarder, or via ingest-time filtering — is invisible to ingest-based license metering, because the metering point is the index itself, not the source. §6.2 below builds on exactly this fact.
- **Reformatting doesn't reduce the number by itself.** Stripping whitespace or reformatting a JSON blob before it's indexed can reduce the byte count that lands in the index, but the meaningful reduction almost always comes from *not indexing an event at all*, not from indexing a slightly smaller version of the same event. A field extraction happening at search time (DEH Part

26 \$1.2 covers the search-time/index-time split this depends on) costs nothing against ingest volume either way, since it never touches what was already written to the index.

[PLATFORM ENGINEER] What ingest-based licensing does *not* meter is equally worth stating plainly, because it's the part administrators moving from another billing-by-seat product get wrong first: the number of users querying the data, the number of dashboards built against it, the number of correlation searches scheduled against it, and the number of concurrent searches running are not part of the ingest license ceiling at all. A SOC that hires five more analysts and triples its correlation-search count changes nothing about ingest-license consumption — it changes search concurrency and compute load instead, which Part 12 covers as a performance question and which is exactly the number workload-based licensing meters directly instead.

3. Workload-based licensing: SVC and what actually drives it

[PLATFORM ENGINEER] Splunk's workload-pricing model for Splunk Cloud Platform is metered in a unit Splunk calls **Splunk Virtual Compute (SVC)**. Splunk's own workload-pricing page defines an SVC as a unit of cloud compute, memory, and I/O resources — not a data-volume unit at all — and states that SVC consumption is driven primarily by search quantity and complexity *and* daily indexing volume, not by search activity in isolation.

Product Version Note *Workload pricing for Splunk Cloud Platform is metered in Splunk Virtual Compute (SVC) units, defined by Splunk as "a unit of cloud compute, memory and I/O resources." Splunk's own pricing documentation states SVC requirements are driven primarily by search quantity and complexity as well as daily indexing volume — indexing volume still matters under this model, it's just no longer the only number that matters. As of 2026-09-15, verified against Splunk's own workload-pricing page ([REFERENCES.md](#) entry [\[SPLUNK-WORKLOAD-PRICING\]](#)). What would make this stale: a change to what counts toward SVC consumption, a renamed unit, or workload pricing's terms diverging between Splunk Cloud Platform and self-managed Splunk Enterprise in a way that isn't reflected here.*

Blind Spot *"Workload pricing decouples cost from ingest volume" is the pitch a team under ingest-license pressure wants to hear, and it's only half true. Splunk's own documentation names daily indexing volume as one of the two named drivers of SVC consumption, alongside search quantity and complexity — a team that migrates to workload pricing expecting ingest volume to stop mattering at all is optimizing against a number the new model still partly cares about, just no longer exclusively. The actual decision variable (\$4) is the ratio of search activity to ingest volume for a given deployment, not which of the two numbers can supposedly be ignored.*

[PLATFORM ENGINEER] The practical implication for a security team specifically: retaining a large volume of low-search-frequency data — the kind of raw telemetry kept mostly for retrospective investigation

and compliance rather than daily correlation-search consumption — behaves differently under workload pricing than under ingest pricing, because the same data sitting mostly idle costs proportionally less against a compute-weighted meter than against a pure volume meter. Conversely, a small, high-value dataset queried constantly by many scheduled correlation searches and ad hoc analyst pivots (Part 19 covers that investigation pattern directly) can cost more under workload pricing than the same dataset's ingest volume alone would suggest. Part 12's search-concurrency and scheduling-priority material is the direct technical continuation of this cost driver — under workload pricing, a scheduling decision covered there is also, without anyone necessarily framing it that way, a cost decision.

4. Choosing between models — the decision variables that actually matter

[SOC MANAGEMENT] The table below states the comparison a SOC manager or platform owner is actually running when a renewal or a new deployment forces the ingest-vs-workload choice.

Factor	Favors ingest pricing	Favors workload pricing
Search-to-ingest ratio	High-volume data, searched rarely (compliance retention, cold forensic archives)	Smaller, high-value datasets queried constantly by scheduled correlation searches and analysts
Growth pattern	Predictable, slow-growing ingest volume	Ingest volume growing faster than headcount/search load, or vice versa
Retention posture	Long raw retention with little acceleration or summarization	Retention leaning on data model acceleration (Part 10) and summary indexing (Part 11) to keep compute bounded
Budgeting preference	Cost as a flat, predictable function of data volume	Cost as a function of platform activity, harder to forecast without a workload baseline first
Contractual term	Comfortable with GB/day as the negotiated unit	Comfortable negotiating and tracking SVC as the billed unit — a less familiar unit to most procurement teams

[SOC MANAGEMENT] None of these factors can be assessed honestly without a real usage baseline — a team switching pricing models on a guess about its own search-to-ingest ratio is choosing blind. Splunk's workload-pricing materials describe a calculator that estimates SVC needs from expected daily ingest and workload type; treat that estimate as a starting point for a negotiation, not as a substitute for the deployment's own Monitoring Console usage data (§9) once one exists.

SOC Management View *A licensing-model decision is not a one-time platform-team call, even though it's usually made in a platform-team conversation. It sets the shape of every future "should we onboard this feed" and "should we schedule this correlation search hourly or every five minutes" conversation a detection engineer has for the life of the contract term — usually one to three years. Involve whoever owns the detection-content roadmap (Part 14's correlation-search lifecycle, Part 15's risk-based alerting scoring cadence) before signing a multi-year term under either model; a model chosen purely on this year's ingest volume can become the wrong model the year RBA-driven search frequency triples without a matching ingest change.*

5. License warnings and violations: what's stable, and what this book can't confirm

[PLATFORM ENGINEER] Splunk's license enforcement, going back many versions, has tracked usage against a rolling window rather than a single day's overage in isolation — the practical effect being that a single one-off spike (a misconfigured forwarder replaying an overnight backlog, a log source briefly misconfigured to double-ship) behaves differently under enforcement than the same volume sustained day after day. That much is long-standing, widely-documented platform behavior and unlikely to have changed in shape even if specific numbers have moved.

What this book will not restate as verified fact is the *exact* current threshold (how many exceedances within the window trigger a formal violation) or the *exact* current consequence of a violation once triggered — historically a search-blocking enforcement action on some license types, and a subject Splunk has adjusted its own policy on more than once. This book's own research tooling could not fetch Splunk's current licensing-mechanics documentation directly (docs.splunk.com; see STYLE-GUIDE.md §9.4's disclosed constraint), so neither number is given here dressed up as a Product Version Note with a citation this book doesn't actually have.

Engineering Reality *Whatever the current threshold and consequence are, they are visible in your own instance without guessing: [Settings > Licensing](#) on a self-managed deployment, or the equivalent capacity/usage notification in Splunk Cloud Platform, shows current usage against the rolling window directly. Confirm both the threshold and the consequence there — or against your current support contract's licensing terms — before assuming either matches what an older deployment, a training course from several versions back, or a colleague's memory suggests. This is exactly the category of default Splunk has changed before without restructuring anything else around it.*

What Would Change My Mind A real Splunk deployment's own `Settings > Licensing > Usage Report`, observed across at least one full rolling window, would let this section state the current threshold and consequence as a verified, dated fact instead of an acknowledged gap. Until this book's evidence base includes that (see Part 20), treat this section's hedge as the honest state of the claim, not an oversight to route around by trusting an unsourced number instead.

6. The cost levers a security team actually controls

[PLATFORM ENGINEER] "Reduce license usage" has three concrete levers underneath it — beyond the model choice itself, already covered in §3–4 — that don't simply mean "ingest less security-relevant data and hope nothing important was in what got dropped." Each is a real design decision with a real tradeoff, not a free win.

6.1 Index design and retention tiering

[PLATFORM ENGINEER] Part 3 §1 already frames index design as a security-relevant decision for retention isolation and index-level RBAC; it's also the first and cheapest cost lever, because retention length is set per index, not globally. Splitting a single monolithic `security` index into separate indexes by data value and required retention — a short-retention index for high-volume, low-investigative-value telemetry (verbose firewall accept logs, for instance) alongside a long-retention index for what actually drives investigations (authentication, process execution, EDR telemetry) — doesn't reduce today's ingest volume at all, but it directly reduces the storage a bucket lifecycle (Part 3 §3) has to carry going forward, and it stops a compliance-driven long-retention requirement on one data type from silently forcing the same retention length, and the same storage cost, onto everything else sharing its index.

6.2 Filtering at the heavy forwarder or at ingest time

[PLATFORM ENGINEER] This is the lever that acts directly on ingest-based license consumption, because — as §2 states — volume that never reaches an indexer never counts against an ingest license. The long-standing mechanism for this is a `transforms.conf` stanza that routes matching events to Splunk's null queue at a heavy forwarder, before indexing:

CONCEPTUAL SAMPLE — illustrative filtering pattern; stanza names and the target event code are representative, not a literal recommendation to drop 4634 specifically without checking what depends on it first (see §7).

```
# props.conf (on the heavy forwarder)
[wineventlog:security]
TRANSFORMS-drop_noisy_events = drop_4634_logoff

# transforms.conf (on the heavy forwarder)
[drop_4634_logoff]
REGEX = EventCode=4634
DEST_KEY = queue
FORMAT = nullQueue
```

A newer, GUI-driven alternative to hand-writing `props.conf` / `transforms.conf` stanzas exists for filtering, masking, and routing data at ingest without deploying a dedicated heavy forwarder tier — Splunk has shipped this class of capability under the Ingest Actions name in recent Splunk Cloud Platform and Splunk Enterprise releases.

Product Version Note *Splunk offers a GUI-driven ingest-time filtering, masking, and routing capability under the name Ingest Actions, as an alternative to hand-written `transforms.conf` stanzas. This book's research tooling could not independently re-verify Ingest Actions' current feature scope or edition availability against a fetchable source as of 2026-09-15 (docs.splunk.com was not reliably fetchable during this book's research; see `STYLE-GUIDE.md §9.4`) — no dated citation backs the claim beyond the feature's existence. What would make this stale, or what would confirm it: a direct check of your own instance's current documentation for Ingest Actions' exact behavior, licensing tier, and deployment-type availability before designing a filtering strategy around it specifically, rather than the underlying `transforms.conf` mechanism, which is stable and has been for many major versions.*

6.3 Summary indexing and acceleration as alternatives to keeping everything raw

[PLATFORM ENGINEER] Where §6.2 removes data before it's ever indexed, this lever changes what has to stay in *raw*, full-fidelity form once it is. Data model acceleration (Part 10) and summary indexing (Part 11) both let a detection keep working against a compact, pre-computed representation of a dataset without every downstream correlation search needing the full raw event volume retained at long retention just in case a query needs it later. This doesn't reduce today's ingest number either — the raw events still get indexed once — but it changes the retention-length decision in §6.1 from "keep everything raw for three years because some detection might need it" to "keep the accelerated summary for three years and the raw events for a shorter, cheaper window," which is a meaningfully different storage-cost shape at scale.

7. Detection Autopsy — the ingest filter that quietly starved a correlation search

[PLATFORM ENGINEER] This box describes a composite, commonly-reported operational pattern from Splunk practitioner discussion of license-cost reduction, not a single captured incident — no real Splunk

deployment exists in this book's evidence base to draw one from (STYLE-GUIDE.md §9.2), and it's disclosed as composite rather than dressed up as a specific named case this book can't actually cite.

Detection Autopsy — "the filter that looked like noise reduction"

The rule: Under license pressure, a platform team adds a `transforms.conf` null-queue route (§6.2) dropping Windows Security Event ID 4662 ("An operation was performed on an object") at the heavy forwarder, well before it reaches the indexer. 4662 fires constantly in a typical Active Directory environment and, read in isolation as a raw event stream, looks like exactly the kind of low-value verbosity ingest-cost reviews target first.

Why it shipped: Nobody on the platform team was actively using 4662 in a dashboard or a correlation search at the time of the change, and the immediate ingest-volume savings were real and visible the same week.

How it failed: A detection engineer had, months earlier, built a correlation search against the CIM's Change Analysis data model — the CIM data model covering object, permission, and configuration change events — extending directory-object and ACL-change detection that depends specifically on 4662 for its Active-Directory coverage. That search stops firing. Nothing about its own execution looks wrong: it runs on schedule, returns zero results, and a correlation search returning zero results looks identical to "nothing suspicious happened" right up until an incident retrospective asks why it didn't catch something it was supposedly built to catch.

The fix: Route every ingest-time filtering decision through whoever owns the correlation searches and data models built against that sourcetype — Part 7's CIM-onboarding review is the natural place for this check — before the `transforms.conf` change ships, not after a gap gets discovered the hard way. A filtering decision and a detection-coverage decision are the same decision here; treating them as two separate approvals (platform team signs off on cost, detection team finds out later) is the actual root cause, not the specific choice of 4662.

Blind Spot An ingest-time filter produces no error, no warning, and no visible signal anywhere in Splunk that a class of event has stopped arriving — it simply stops arriving, and every downstream search built against it keeps running successfully, returning fewer or zero results, exactly as designed for the actual absence of the underlying activity. The only reliable way to catch this class of gap is a source-health or volume-anomaly check on the sourcetype itself (the same null-rate/parser-health pattern DEH Part 26 §1.2 names for a broken field extraction, applied here to a broken or newly-added ingest filter instead) — not trusting that a correlation search's own continued execution proves its inputs are still intact.

8. Where ingest is metered, and where it can still be reduced

[PLATFORM ENGINEER]

The diagram below traces one event's path from source host to search results, marking the two distinct places cost actually gets measured — the ingest-license metering point at the indexer, and the workload/SVC metering point at the search head — against the one place upstream of both where a filtered event can be removed from the ingest count entirely before it's ever metered.

flowchart LR

```
UF["Universal Forwarder(s)<br/>on source hosts"] --> HF["Heavy Forwarder /<br/>Ingest Actions<br/>parse, filter, mask, route"]
HF -->|"matched by nullQueue route<br/>(section 6.2) never indexed"| DROP["Discarded before indexing<br/>zero ingest-license cost"]
HF -->|"events that survive filtering"| IDX["Indexer -<br/>ingest-license metering point<br/>(daily indexed volume, section 2)"]
IDX --> BUCKETS["Hot / Warm / Cold / Frozen buckets<br/>(Part 3 retention lifecycle)"]
IDX --> SH["Search Head -<br/>workload/SVC metering point<br/>(search quantity and complexity, section 3)"]
SH --> MC["Monitoring Console -<br/>license usage reporting (section 9)"]
SH --> ANALYST["Correlation search / analyst query<br/>consuming search results"]
```

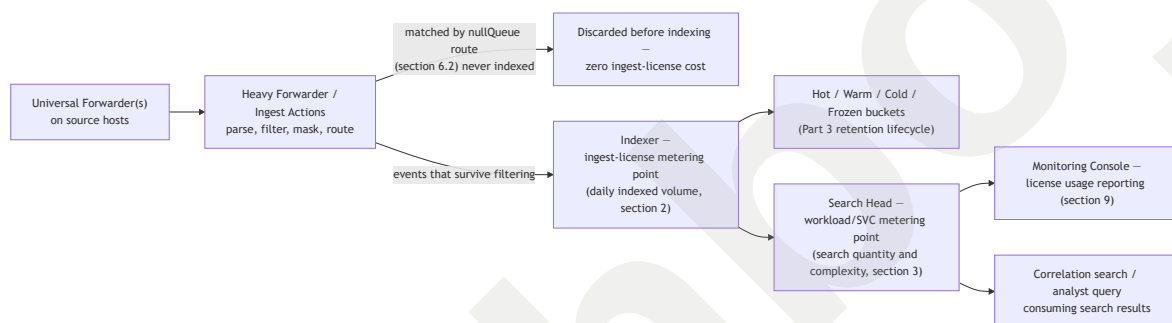


Figure 4.1 — Ingest-license and workload/SVC metering points relative to the filtering decision.

CONCEPTUAL. Illustrates where ingest-based license volume is actually measured (at the indexer, after any heavy-forwarder-side filtering) versus where workload/SVC consumption is measured (at search execution), and where in the pipeline a filtering decision removes an event from ingest-license accounting entirely. This is a sequence/architecture sketch of documented expected data flow, not a capture from a live Splunk deployment or Monitoring Console — none exists in this book's evidence base (STYLE-GUIDE.md §9.2).

9. Treating license headroom as an operational metric

[SOC MANAGEMENT]

Splunk ships a built-in app — the Monitoring Console — for observing the platform's own health, and license usage against the current model's ceiling (daily GB under ingest pricing, SVC consumption under workload pricing) is one of the standard things it reports on. The management discipline worth stating plainly: license headroom belongs on the same operational dashboard as analyst staffing levels and storage capacity, reviewed on a schedule, not discovered when a usage-warning banner appears in Splunk Web. A SOC that only looks at license usage reactively, after a warning, has already lost the ability to make a considered, prioritized decision among the levers in §6 — it makes whichever cut is fastest to implement instead, which is exactly how a decision like §7's 4662 filter gets made without the review it needed.

The table below summarizes the levers this part covered, for exactly that kind of prioritized, non-reactive review.

Lever	Acts on	What it saves	Main risk if done carelessly	Cross-reference
Index design and retention tiering	Storage cost, going forward	Long-term bucket storage, not today's ingest volume	Splitting indexes without updating role-based access controls that assumed one index	Part 3 §1, §3
Ingest-time filtering (<code>transforms.conf</code> null queue / Ingest Actions)	Ingest-license volume directly	Daily indexed GB, immediately	Silently starving a correlation search or data model that depended on the filtered event (§7)	Part 7 (CIM onboarding review)
Summary indexing / data model acceleration	Compute and long-term raw retention need	Query cost and how long raw fidelity must be kept	Summary/accelerated data going stale relative to raw if the underlying search or acceleration range isn't maintained	Part 10, Part 11
Workload-vs-ingest model choice	Which number the SOC is optimizing against	Predictability, if the model matches the actual search-to-ingest ratio	Choosing a model on a guess instead of a Monitoring Console baseline (§4, §9)	Part 12 (search performance, the workload-side driver)

Cross-references: DEH Part 26 §1.2 (search-time vs. index-time fields — the split underlying what "indexed volume" means for ingest-license metering, §2); this book's Part 2 (deployment architecture — where forwarders, heavy forwarders, and indexers sit in the topology this part's filtering lever depends on); Part 3 §1 and §3 (index design and bucket retention — the storage-cost side of §6.1); Part 7 (CIM-compliant onboarding — the review this part's §7 Detection Autopsy argues must gate any ingest-time filtering change); Part 10 and Part 11 (data model acceleration and summary indexing — the pre-computation alternatives in §6.3); Part 12 (search performance and workload management — the direct technical continuation of workload/SVC cost, §3); Part 13 (Enterprise Security architecture and editions — the separately licensed application layer sitting on top of the core platform license this part covers); Part 20 (validation gaps — where this part's §5 hedge and §9's Monitoring Console gap are tracked as open evidence items).

Section B — Normalizing Data Into a Common Model

neshbooy

Part 5 — The Common Information Model: What It Actually Standardizes

Why this part exists

Parts 2–4 covered the platform Splunk gives you before a single security event has been categorized: indexers, buckets, licensing. This part covers the first thing that happens to a security event once it lands — not parsing (DEH Part 5, in the sibling volume, owns that layer generally), but the question of whether Splunk's own security tooling can recognize what kind of event it is at all.

That recognition problem is not new to Splunk, and this book doesn't pretend it is. DEH Part 6 §2.5 already introduces Splunk's Common Information Model (CIM) in one paragraph, as one of five vendor- or ecosystem-specific answers to the same translation problem ECS, OCSF, UDM, and ASIM each solve differently: a shared field vocabulary that lets a detection written once run against data from more than one source. DEH Part 6 deliberately doesn't go field-by-field into CIM, because CIM is Splunk-internal rather than cross-platform, and that part's job is the general shape of the translation problem, not any one vendor's specific implementation of it. This part is that implementation, in depth: what CIM actually standardizes, the specific configuration mechanism that does the standardizing, and — because the book index for this title asks the question directly — why "this sourcetype is CIM-compliant" is a claim you can test and fail, not a checkbox a vendor's add-on ticks for you.

[CONCEPT] The single idea this part keeps returning to: CIM is a naming and tagging *contract*, expressed entirely in ordinary Splunk configuration files, not a normalization engine that touches raw data. Nothing about a raw event changes when CIM is "applied" to it. What changes is which search-time knowledge objects exist to describe that event in CIM's vocabulary — and every one of those knowledge objects can be missing, wrong, or stale, exactly like any other configuration.

1. A naming-and-tagging contract, not a normalizer

[CONCEPT] The Splunk Common Information Model ships as the Splunk Common Information Model Add-on — a free Splunk-authored app distributed on Splunkbase, listed there as app ID 1621. Installing it does three things, and only three things: it adds a set of data model JSON definitions to the search head, it adds a default set of `eventtypes.conf` and `tags.conf` entries for the sourcetypes Splunk already expects to see in a typical deployment, and it adds documentation. It does not add a parser, a normalization pipeline, or anything that runs at index time. Every mechanism CIM actually uses — field aliases, calculated fields, eventtypes, tags, data model constraints — is a search-time knowledge object that already exists in Splunk with or without CIM installed. CIM's contribution is a *convention* for how to use those existing mechanisms consistently: agree that "the address that initiated the connection" is always called `src`, agree that a successful login event always carries the tag `authentication` and the tag `success`, and any search or dashboard built against that convention works against any sourcetype that follows it.

Product Version Note *The Splunk Common Information Model Add-on's current default version is 8.7.0, released 2026-09-02, with listed compatibility against Splunk platform versions 9.4 through 10.5. As of 2026-09-15, verified against the add-on's own Splunkbase listing (REFERENCES.md entry [SPLUNKBASE-CIM-ADDON]). docs.splunk.com's CIM documentation pages returned an access error on every attempt made while researching this part, so the Splunkbase listing is the only source behind this specific version and compatibility claim — see REFERENCES.md for the full retrieval note. What would make this stale: any new CIM Add-on release, or a change to the supported-platform-version floor or ceiling on a future Splunkbase update.*

[PLATFORM ENGINEER] This is the point where a reasonable assumption goes wrong, so it's worth stating the failure mode directly rather than gesturing at it: installing the add-on does not make an existing sourcetype CIM-compliant. It supplies default mappings for the sourcetypes Splunk itself anticipated — mostly its own products and a handful of long-standing, widely-deployed technology add-ons (TAs) that ship their own CIM-aligned configuration already. A custom log source, an internally built application's audit log, or a vendor's TA that predates or never bothered with CIM compliance gets none of that for free. Someone still has to write the field aliases, author the eventtype, and apply the tag — by hand, or by installing a TA that already did it. Part 7 covers that implementation work in depth; this part's job is making sure the mechanism it's implementing is understood first.

Engineering Reality *"We installed the CIM Add-on" is not the same claim as "our authentication data is CIM-compliant," and a reviewer or auditor who accepts the first as evidence for the second is accepting a checkbox instead of a test. The add-on is infrastructure the mapping work runs on top of, not a substitute for the mapping work. Budget CIM onboarding for a new source as real implementation effort — the field-aliasing and tagging work in Part 7, plus the compliance testing in §4 below — not as configuration that arrives free with an app install.*

2. Where the contract actually lives on disk

[PLATFORM ENGINEER] CIM's naming convention has to attach to a real event somehow, and it does that through exactly two Splunk configuration files working together: `eventtypes.conf`, which turns a saved search filter into a named category, and `tags.conf`, which attaches one or more tags to that category. Everything a CIM data model search actually filters on — `tag=authentication`, `tag=network`, `tag=communicate` — resolves back to these two files. There is no CIM-specific configuration file; CIM is built entirely out of generic Splunk knowledge objects used according to a shared naming agreement.

`eventtypes.conf` — **turning a raw search into a reusable category**

[PLATFORM ENGINEER] An eventtype stanza is nothing more than a saved search string given a name.

When an event matches that search string at search time, Splunk considers it a member of that eventtype — no data is copied or moved, and the eventtype's name becomes a searchable value in the built-in `eventtype` field for any event that matches.

```
# CONCEPTUAL SAMPLE – an illustrative eventtype for a hypothetical VPN concentrator
# sourcetype, not a real vendor's default configuration and not validated against a
# live Splunk instance.

# eventtypes.conf
[acme_vpn_authentication]
search = sourcetype="acme:vpn:concentrator" (result="success" OR result="failure")
```

This eventtype targets one invented sourcetype and the two raw values its login events actually carry. Its limitation is the same as any saved-search-based category: if the raw event's field names or values drift (a firmware update renames `result` to `status`, say), the eventtype stops matching silently, exactly the way a `FIELDALIAS` stops resolving in the failure pattern described in §4.

`tags.conf` — where a CIM tag actually gets created

[PLATFORM ENGINEER] A tag is applied to an eventtype (or directly to a field-value pair) in `tags.conf`, and it is this tag — not the eventtype name — that CIM's own data model constraint searches actually filter on.

```
# CONCEPTUAL SAMPLE – continues the illustrative VPN example above; the eventtype
# name is invented, but the stanza format and the tag name (`authentication`) match
# the tag CIM's own Authentication data model constraint search filters on.

# tags.conf
[eventtype=acme_vpn_authentication]
authentication = enabled
```

[DETECTION ENGINEER] This is the entire mechanism. There is no separate "CIM engine" evaluating an event against a schema at search time; there is a search string in `eventtypes.conf` and a tag assignment in `tags.conf`, evaluated the same way any other saved search and tag would be. That matters for how you debug a CIM problem: if a data model isn't returning events you expect, the fault is in one of exactly two places — the eventtype's search string isn't matching the raw event, or the tag isn't attached to the eventtype that is matching. There's no third layer to suspect.

Getting the tag right isn't sufficient by itself, though — the data model's constraint search also expects specific *field* names and, for several fields, specific *values*, which is where field aliasing and calculated fields (`FIELDALIAS` and `EVAL` stanzas in `props.conf`) do the rest of the mapping work. The distinction between the two matters and is a common source of the failure mode below: `FIELDALIAS` only renames a field — `result AS action` makes the raw value visible under a new name, unchanged. It does not translate the *value*. A raw field whose values already match CIM's controlled vocabulary needs nothing more than a rename. A raw field whose values don't (a VPN log reporting `permit` / `deny` instead of `success` / `failure`) needs an `EVAL` calculated field instead, one that maps each raw value onto the

vocabulary the data model actually constrains on. Part 7 covers authoring that field-level mapping end to end; the point to carry forward here is that a correctly tagged event with the wrong field values is still not CIM-compliant, a distinction §4 makes testable.

False Positive Trap *The Authentication data model's `action` field expects exactly two controlled values: `success` or `failure`. A VPN log that reports `result=permit` or `result=deny` isn't malformed data — it just isn't in CIM's controlled vocabulary yet. Alias `result` to `action` without also translating the values (`permit` → `success`, `deny` → `failure`), and every correlation search or dashboard panel filtering on `action=failure` silently returns zero rows against that source, forever, while the raw events sit in the index looking completely normal to anyone who searches them directly. The tag fired, the field exists, and the data model still lies about that source having no failed logins.*

```
# CONCEPTUAL SAMPLE – the corrected mapping for the False Positive Trap above: an
# EVAL- calculated field replacing the plain FIELDALIAS, translating raw vendor
# values into CIM's controlled vocabulary rather than just renaming the field.
# Illustrative only, not validated against a live Splunk instance.

# props.conf
[acme:vpn:concentrator]
EVAL-action = case(result="permit", "success", result="deny", "failure")
```

This fixes the specific vocabulary mismatch above, but it doesn't make the mapping self-maintaining: a firmware update that adds a third raw value (`result=timeout`, say) leaves that value unmapped, `action` blank for those events, and the same silent zero-match failure recurring for a new reason. Value-mapping `EVAL-` stanzas need the same periodic review as the lookup tables Part 8 covers — a mapping that was complete on the day it was written can go stale the same way a lookup does.

3. From tagged events to a queryable data model

[CONCEPT] A CIM data model is a JSON definition describing one or more datasets — the Authentication data model (the CIM data model covering login and logoff events across log sources) is the simplest shape: a single root dataset whose *constraint* is a search string built on the `authentication` tag, plus a list of fields (some directly present after aliasing, some calculated) that the dataset promises to expose for any event matching that constraint. More elaborate data models nest child datasets under a root — the Endpoint data model (the CIM data model covering OS- and EDR-reported process, filesystem, registry, service, and port activity) splits its root into `Processes`, `Filesystem`, `Registry`, `Services`, and `Ports` child objects, each with its own narrower constraint layered on top of the parent's.

The table below lists the CIM data models most relevant to security operations content in this book. CIM ships roughly two dozen data models in total, covering domains (performance, inventory, JVM metrics) that fall outside this book's security scope; the table is a working subset, not the full manifest.

Table 5.1 — CIM data models most relevant to security operations. Use this table to decide which data model a new onboarding effort (Part 7) or correlation search (Part 14) should target.

Data model	Root tag(s)	Typical source events
Authentication	authentication	Login/logoff success and failure across OS, VPN, and application auth logs
Network Traffic	network, communicate	Firewall and flow-log allow/deny decisions
Web	web	Proxy and web-server access logs
Malware	malware, attack	AV/EDR detection and remediation events
Endpoint (Processes, Filesystem, Registry, Services, Ports)	process, report (varies by child dataset)	EDR and OS-native process, file, and registry telemetry
Change (Change Analysis)	change	Config, account, and endpoint change events
Email	email	Mail gateway delivery, filtering, and header data
Data Loss Prevention	dlp	DLP-engine policy match events
Vulnerabilities	vulnerability	Scanner findings
Certificates	certificate	TLS certificate observation events
Network Resolution	dns	DNS query/response logs
Alerts	alert	Third-party alerting-tool output ingested for correlation
Intrusion Detection	ids, attack	IDS/IPS signature hits
Ticket Management	ticketing	Case/ticket lifecycle events from an external ITSM tool

[CONCEPT] A data model becomes queryable two ways: through Pivot, a knowledge-object-driven query builder that lets an analyst work against a data model's fields without writing SPL directly, or through a direct search against the data model's generated dataset. Part 6 (this book) covers Pivot and data-model query mechanics in full; this part's concern stops at the boundary Part 6 picks up — what has to already be true about the underlying tags and fields before Pivot, or any other consumer, can return anything meaningful. An accelerated data model additionally gets its own `tsidx` summary that `tstats` can read directly without touching raw events at all; Part 10 owns acceleration mechanics and cost in full, and DEH Part 26 §1.3 names `tstats` as a working syntax entry without teaching it for exactly the reason DEH Part 26's own scope note gives — this book, and specifically Part 10, is where that forward reference resolves.

Blind Spot `tag=authentication` and the Authentication data model itself only ever see events from a sourcetype someone has actually mapped into CIM, per §2. A perfectly healthy, fully indexed sourcetype that nobody has aliased and tagged is invisible to every tag-based search and every CIM-driven dashboard panel — not filtered out, simply never a candidate.

[THREAT HUNTER] A hunter running a tag-based sweep across "all authentication sources" is really sweeping "all authentication sources someone remembered to map into CIM," and in a large, organically grown environment those two sets are rarely the same size. A raw `index=*sourcetype=*login*` search will surface sources the tag-based sweep never will. Part 19 covers what to do when a pivot from a notable event or a data model hits exactly this gap mid-investigation.

[SOC ANALYST] The consequence for a triage-facing user is worth stating in one clause even though Part 16 owns Incident Review in depth: a blank field on a notable's event detail is frequently a CIM mapping gap upstream, not an absence of data at the source — worth checking before concluding a log source simply "doesn't have" the information a dashboard implies it should.

4. Testing a CIM-compliance claim instead of trusting it

[DETECTION ENGINEER] "This sourcetype is CIM-compliant" decomposes into three separately checkable claims, and each can be true or false independently of the other two:

1. **Tagging is correct.** The right events carry the right tag, and — just as important — no unrelated events carry it too (an over-broad eventtype search string tags things that aren't actually authentication events).
2. **Field coverage is complete.** Every field the target data model's dataset declares has either a raw field, a `FIELDALIAS`, or an `EVAL-` calculated field resolving to it for the tagged events — not just present as a column header when the field list is inspected, but populated with a non-null value across a representative sample of real events.
3. **Values are conformant.** Where the data model constrains a field to a controlled vocabulary (`action` restricted to `success/failure/allowed/blocked`, depending on the model), the mapped value is actually inside that vocabulary, not a raw vendor string that happens to occupy the right field.

None of these three requires a specialized validation tool — each is answerable by running the data model's own constraint search restricted to the target sourcetype and inspecting the results table for the fields in question. That's what makes the claim falsifiable rather than a vendor checkbox: a stakeholder asserting compliance can be asked to produce that table, for that sourcetype, and the answer is either present or it isn't.

Detection Test Setup: In a real Splunk instance, open the target data model's dataset (Authentication, for a login source) restricted to one specific `sourcetype` under test, either through the Data Model Editor's preview or by running the underlying constraint search directly. **Action:** Table every field the dataset declares for that object, across a time range wide enough to include both success and failure events for that source. **Expected result:** Every declared field has a value, and every value-constrained field (`action`, in particular) shows only the vocabulary the model defines — not blank columns, and not raw vendor strings sitting in a field that looks populated but isn't conformant. This has not been run against a live Splunk instance for this book — no deployment exists in the author's lab (see *STYLE-GUIDE.md* §9.2). Treat it as the specific check to run against your own environment before accepting a compliance claim, not as a result already confirmed here.

Detection Autopsy A widely reported failure class among Splunk practitioners — documented across TA upgrade release notes and Splunk community forum threads rather than any single incident this book can cite by name or capture from its own lab — is a technology-add-on upgrade that renames or restructures a field an existing `FIELDALIAS` depends on. The alias now points at a field that no longer exists in the raw event. Every correlation search and dashboard panel built on the affected data model keeps running on its schedule, returns zero matches, and raises no error, because a search head has no way to distinguish "this field legitimately has no matches right now" from "this alias silently stopped resolving after an upgrade." The fix is structural, not clever: version the site's own CIM-mapping `props.conf` / `tags.conf` alongside the TA release it depends on, and pair every field-mapping change with the §4 compliance check above rather than trusting a green dashboard as proof the mapping survived the upgrade.

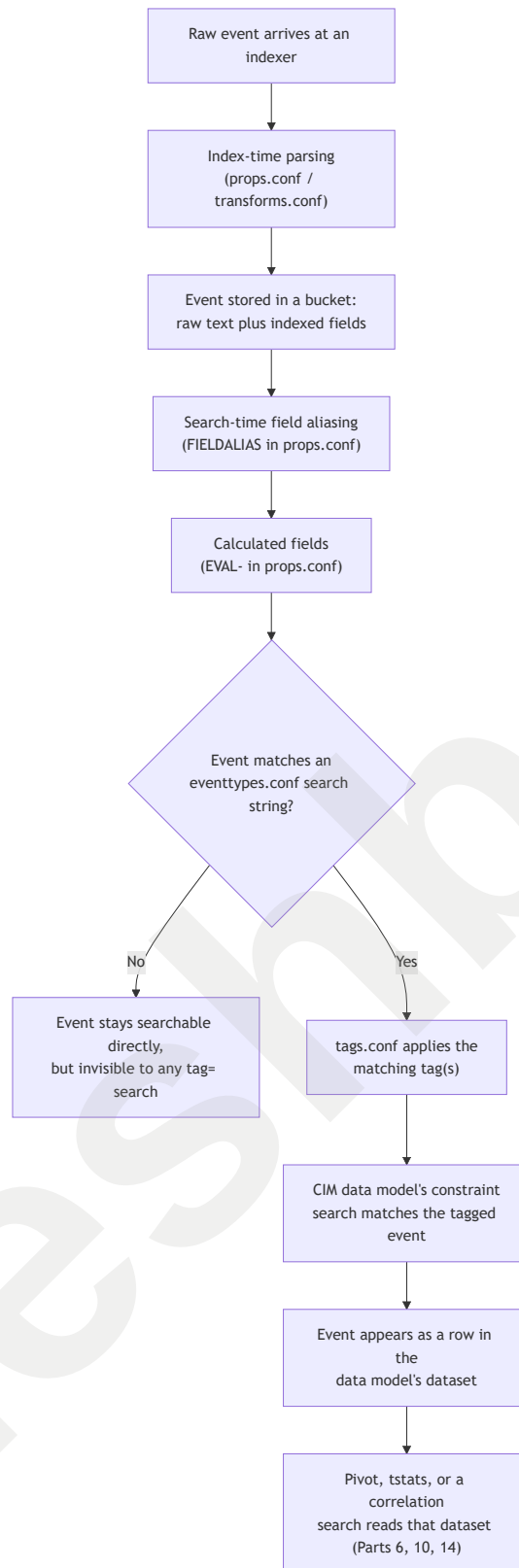
[PLATFORM ENGINEER]

One more distinction worth naming precisely: CIM compliance and data model *acceleration* are independent decisions. A sourcetype can be fully CIM-compliant and never accelerated — every search against it just runs slower over raw events, which Part 10 quantifies — and a data model can be accelerated while several of its constituent sourcetypes remain non-compliant, in which case acceleration simply builds a fast summary of an incomplete picture. Fixing compliance gaps first, then deciding whether the resulting dataset is large or frequently-queried enough to justify acceleration's storage cost, is the correct order of operations; Part 10 assumes that order and doesn't re-argue it.

Figure 5.1 — From raw event to a CIM data model row. CONCEPTUAL. Illustrates the sequence of index-time parsing and search-time knowledge objects — field aliases, calculated fields, eventtypes, tags — that has to succeed, in order, before a raw event becomes visible to a CIM data model search. This is a sequence diagram of documented expected behavior, not a capture from a live Splunk instance or job inspector; no such deployment exists in this book's evidence base (*STYLE-GUIDE.md* §9.2).

flowchart TD

```
A["Raw event arrives at an indexer"] --> B["Index-time parsing\n(props.conf / transforms.conf)"]
B --> C["Event stored in a bucket:\nraw text plus indexed fields"]
C --> D["Search-time field aliasing\n(FIELDALIAS in props.conf)"]
D --> E["Calculated fields\n(EVAL- in props.conf)"]
E --> F{"Event matches an\neventtypes.conf search string?"}
F -- "No" --> G["Event stays searchable directly,\nbut invisible to any tag= search"]
F -- "Yes" --> H["tags.conf applies the\nmatching tag(s)"]
H --> I["CIM data model's constraint\nsearch matches the tagged event"]
I --> J["Event appears as a row in the\ndata model's dataset"]
J --> K["Pivot, tstats, or a correlation\nsearch reads that dataset (Parts 6, 10, 14)"]
```



5. CIM as Enterprise Security's hard dependency

[SOC MANAGEMENT] Every part in Section E of this book (Enterprise Security, Parts 13–17) inherits the mapping work described here as a precondition, not an optional best practice. Enterprise Security's

correlation searches, risk-based-alerting rules, and out-of-the-box dashboards are written against CIM data models, not against any specific vendor's raw field names — which is precisely what lets one correlation search cover a firewall, a VPN concentrator, and a cloud gateway simultaneously, but only for sources that actually cleared the compliance bar in §4. An ES deployment layered on top of poorly mapped data doesn't fail loudly; it produces the same silent zero-match behavior described in this part's Detection Autopsy box, just wearing Enterprise Security's UI instead of a bare dashboard panel. Staffing a CIM-onboarding effort (Part 7) is therefore not separable from staffing an ES rollout — the second doesn't function without the first having already happened for every source ES is expected to correlate.

Product Version Note *Splunk Enterprise Security is currently sold in two editions, Essentials and Premier, with User and Entity Behavior Analytics (UEBA), SOAR integration, and "Automated Threat Analysis" gated to Premier; Detection Studio and the AI Assistant for Security are listed as included in both editions. As of 2026-09-15, verified against Splunk's own Enterprise Security product page (REFERENCES.md entry [\[SPLUNK-ES-PRODUCT-PAGE\]](#)). This is recent edition restructuring, not a long-standing stable boundary — confirm which edition a given environment is actually licensed for before assuming any Premier-gated capability is available, and re-verify this split before treating it as still current more than a few release cycles out. The point that matters for this part: CIM compliance is a dependency of Enterprise Security under either edition — nothing about the Essentials/Premier split changes what §4 requires of the underlying data. Part 13 covers the edition split itself in full.*

What Would Change My Mind *The largest unverified assumption in this part is that the mapping mechanism described here — aliases, eventtypes, tags, data model constraints — behaves exactly as documented once it's running against a real multi-terabyte-per-day index with a genuinely messy mix of vendor TAs, hand-built aliases, and years of accumulated [props.conf](#) cruft, rather than the clean two-file example in §2. A real Splunk deployment, instrumented over a real retention window and carrying that kind of accumulated mapping debt, is the specific evidence that could confirm or overturn any claim in this part about how CIM mapping actually behaves at scale. Nothing short of that observation would move this part's claims from "documented and internally consistent" to "observed" — see Part 20 for the full inventory of where this book's claims stand on that spectrum.*

Cross-references: DEH Part 6 §2.5 (Splunk CIM as one of five vendor/ecosystem field-vocabulary schemas); DEH Part 26 §1.3 ([tstats](#) named as a working syntax entry, not taught); this book's Part 6 (data models and Pivot query mechanics), Part 7 (CIM-compliant data onboarding implementation), Part 10 (data model and report acceleration), Part 13 (Enterprise Security architecture and editions), Part 14 (correlation searches against CIM data models), Part 16 (Incident Review and notable-event field display), Part 19 (investigation-workflow pivots), Part 20 (validation-gap inventory).

Part 6 — Data Models and Pivot

Why this part exists

Part 5 defined the Common Information Model (CIM) as a field-naming and tagging contract — `tags.conf` and `eventtypes.conf` stanzas, field aliases, and the discipline of making "CIM-compliant" a testable claim rather than a checkbox. A contract by itself doesn't answer a question. Something has to turn "every event tagged `authentication`" into a table you can filter, split, and count. That something is a **data model**: a hierarchical, named object that Splunk's knowledge-object layer builds on top of CIM's tags and field aliases, and that a separate feature — **Pivot** — lets an analyst query by pointing and clicking instead of typing SPL. This part covers both: what a data model actually is structurally, how the CIM Add-on's prebuilt models turn a tagging contract into a queryable dataset, and how Pivot generates a real search from a set of UI choices.

This part does not re-teach the query language underneath any of this. DEH Part 26 §1.1–1.3 already covers the SPL pipeline model, the `search / eval / where` commands, and the search-time-versus-index-time field split that determines whether a data model attribute is cheap or expensive to compute; where this part shows a generated search, it points to those sections rather than re-deriving them. DEH Part 23 already covers the general tradeoff between authoring speed and query control across languages — §5 below applies that same tradeoff inside a single tool, Pivot versus hand-written SPL against the same dataset, rather than restating DEH Part 23's cross-language argument. And this part does not cover **acceleration**: DEH Part 26 §1.3's own syntax table names `tstats` as the command that reads an accelerated data model's `tsidx` summary directly, then explicitly scopes it out ("not covered in depth here"). That forward reference is this book's Part 10's job to resolve, not this part's — Part 6 is about the dataset `tstats` would eventually read; Part 10 is about the summary-building mechanism and the command that reads it. Where acceleration matters to something said here, this part names it and points forward.

1. From CIM tag to queryable object

[CONCEPT] A Splunk **data model** is a knowledge object — stored as JSON, editable through Splunk's Data Model Editor under Settings, and, like a saved search or a lookup definition, something Splunk ships, an app ships, or an admin builds by hand. Structurally, a data model is a tree of **datasets**: one or more root datasets, each optionally carrying child datasets underneath it. Every dataset in the tree defines two things — a **constraint** (which events belong to this dataset) and a set of **fields** (what you can filter, split, or aggregate on once you're in it). A root dataset's constraint is usually a base search; the constraint that matters most for a CIM data model is almost always a `tag` value, because that's exactly the contract Part 5 described `tags.conf` as building. The root dataset for the Authentication data model (the CIM data model covering login, logoff, and authentication-decision events across log sources) constrains on `tag=authentication`, and every event that Part 7's onboarding work correctly tags that way becomes a row in that root dataset the moment the tag applies — no separate copy of the data, no reindexing, no new storage. A data model doesn't hold data; it holds a definition of how to find data that's already indexed.

That distinction matters because it's the same one DEH Part 26 §1.2 draws for search-time fields generally: a data model's fields are computed at query time from the same `_raw` text and the same field extractions every other search in the environment uses, unless a field is explicitly built as an index-time extraction. A data model doesn't get its own copy of the truth — it gets a named, reusable lens on the truth that already exists, and that lens is only as good as the tagging and field-aliasing work underneath it.

Figure 6.1 below traces the path from a raw indexed event to a Pivot table, showing where Part 5's CIM contract, this part's data model, and Part 10's optional acceleration layer each sit.

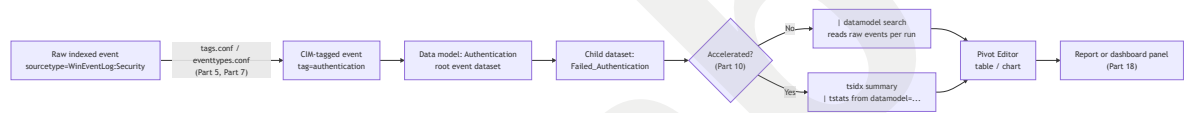
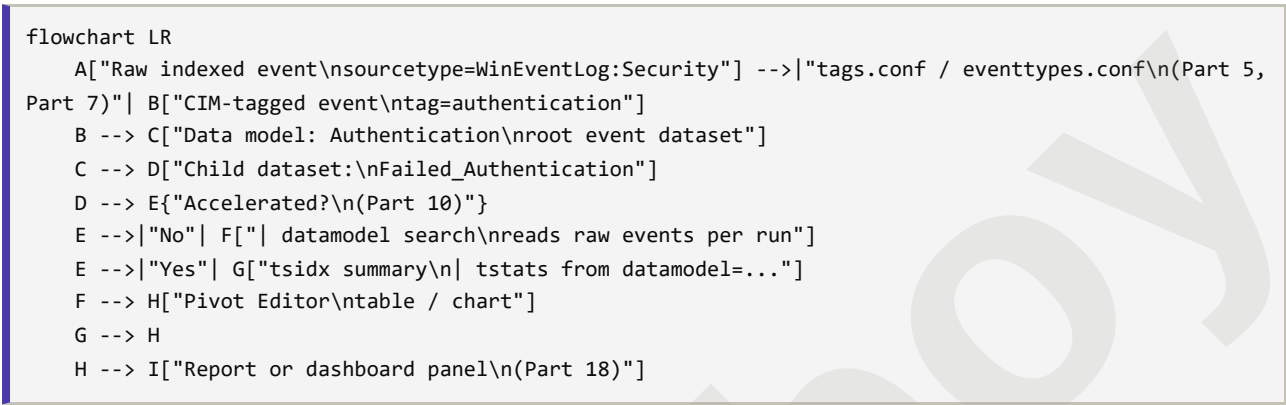


Figure 6.1 — From CIM tag to Pivot table. CONCEPTUAL. Illustrates the expected path from a tagged raw event through a data model dataset to a Pivot-built table or dashboard panel, including the point where an accelerated data model's `tstats` path diverges from an unaccelerated `datamodel` search. This is a sequence diagram of documented expected behavior, not a capture from a live Splunk instance or job inspector — none exists in this book's evidence base (see `STYLE-GUIDE.md` §9.2).

2. Structure and inheritance

2.1 Object types: event, search, and transaction datasets

[PLATFORM ENGINEER] A root dataset is typed when it's created, and the type determines what kind of constraint it accepts and how child datasets under it behave. The table below orients the three root types plus the child-dataset relationship every CIM data model actually uses.

Object type	Constraint shape	Typical use in a CIM data model
Event dataset	A base search (tag=authentication, an index / sourcetype filter, or both)	Root of nearly every CIM data model — Authentication, Web, Malware, Network Traffic all root on an event dataset
Search dataset	An arbitrary saved search, including one with transforming commands already applied	Less common in CIM's own shipped models; more often used in a custom, purpose-built model
Transaction dataset	Groups events from a child event dataset into sessions on a shared field and time window	Used where CIM itself needs to stitch related events — e.g., correlating a network session's start and end records
Child dataset	Inherits its parent's constraint and fields, then narrows the constraint further and/or adds fields	Failed_Authentication under Authentication; Successful_Authentication under the same root

[PLATFORM ENGINEER] Inheritance is additive and one-directional: a child dataset automatically has every field its parent has, plus whatever fields it defines itself, and its constraint is always the parent's constraint **and** something narrower — a child dataset can never see an event its parent's constraint already excluded. This is the practical meaning of "hierarchical" here: Failed_Authentication doesn't redefine what counts as an authentication event, it just adds action=failure on top of the root's tag=authentication and inherits every field (user, dest, src, app, authentication_method) the root already exposes.

2.2 Attributes: how a dataset gets its fields

[PLATFORM ENGINEER] Within a dataset, each field is backed by one of a small set of attribute types, chosen when the field is added in the Data Model Editor.

Attribute type	How the field's value is produced	Cost characteristic
Auto-Extracted	Whatever search-time field extraction already applies to the sourcetype (DEH Part 26 §1.2)	Cheapest — reuses extraction work that already exists
Eval Expression	An eval -style expression computed per event when the dataset is queried	Paid on every search unless the model is accelerated
Lookup	A field pulled in from a lookup table join, same mechanics Part 8 covers for lookups generally	Adds a lookup's own cost on top of the base search
Regular Expression	A rex -equivalent extraction run against another field at query time	Same drift risk DEH Part 26 §1.2's Detection Autopsy box describes for any search-time regex extraction
Geo IP	A built-in enrichment mapping an IP field to geographic attributes	Adds Splunk's bundled Geo IP lookup cost per event

Engineering Reality *A data model is one shared object with many downstream dependents: every Pivot report built against it, every dashboard panel sourced from one of those reports, every hand-written `| datamodel` or `| tstats` search, and — per Part 14 — every correlation search written against the same dataset all read the same field and constraint definitions. Renaming a field, tightening a root constraint, or deleting a child dataset doesn't just change what the next person sees when they open the Data Model Editor. It silently changes the result of every saved artifact built on top of it, the next time each one runs, with no single place that lists everything downstream. Treat a change to a shared data model the way you'd treat a schema change to a table something else in production reads from: find out who depends on it before you touch it, not after a dashboard goes blank or a correlation search stops firing.*

Authentication — one CIM data model, walked from root to children

[CONCEPT] The Authentication data model's root event dataset constrains on `tag=authentication` and exposes fields including `user`, `dest`, `src`, `app`, `action`, and `authentication_method`. Two child datasets are the ones nearly every Authentication-based report actually uses: `Successful_Authentication` (root constraint plus `action=success`) and `Failed_Authentication` (root constraint plus `action=failure`). CIM's documentation describes several additional child datasets under the same root, following the same pattern of narrowing by a specific `action`, privilege level, or credential-security property — this part names the two above with confidence because they're the pair every Authentication-model example, including this book's own, is built around; treat any longer list of every child dataset's exact name as something to confirm against the CIM Add-on actually installed in a given environment rather than as a fixed roster this part can hand you. The **What Would Change My Mind** box below says exactly what would let this part upgrade that roster from "described from familiarity" to "observed."

Product Version Note *The Splunk Common Information Model Add-on — the downloadable app that ships the data model definitions this part describes — is versioned and released independently of Splunk Enterprise, Splunk Cloud Platform, and the Enterprise Security app. As of 2026-09-15, verified against the add-on's own Splunkbase listing (CIM Add-on `8.7.0`, released 2026-09-02; `REFERENCES.md` entry `[SPLUNKBASE-CIM-ADDON]`), the add-on states compatibility with Splunk Enterprise and Splunk Cloud Platform versions `9.4` through `10.5`. What would make this stale: any subsequent CIM Add-on release, a new minimum-platform-version floor, or a data model added, removed, or renamed in a later CIM major version — re-check the Splunkbase listing directly rather than trusting this note past a release cycle or two.*

What Would Change My Mind This part's platform-compatibility figures are verified against the CIM Add-on's own Splunkbase listing as of 2026-09-15 ([REFERENCES.md](#) entry [SPLUNKBASE-CIM-ADDON]) — but that listing doesn't enumerate individual data models or child datasets, so §2.3's Authentication child-dataset description beyond [Successful_Authentication](#)/[Failed_Authentication](#) rests on general familiarity with CIM's long-standing structure, not on a source this book could check line by line. A real Splunk instance with the current CIM Add-on installed would let this part cite an observed roster instead: running `| datamodel` with no arguments lists every installed model, and opening Authentication in the Data Model Editor lists its actual child datasets directly. That's the single most direct upgrade path available for this section specifically — not a general "get a lab" answer, a named command and a named UI screen.

3. Who edits a data model, and what it costs to get wrong

[PLATFORM ENGINEER] Editing a data model — adding a field, adjusting a constraint, adding a child dataset — happens in the Data Model Editor (Settings → Data Models), and creating or editing one requires the `admin` role or a role granted the relevant knowledge-object-management capabilities; viewing and running Pivot against an existing model requires far less. That asymmetry is deliberate: the model is meant to be a small number of maintained objects that a large number of analysts query safely, not something every analyst edits ad hoc. The same asymmetry is why the CIM Add-on ships its own models pre-built rather than expecting every environment to define Authentication from scratch — Part 7's onboarding work is about getting a new source's events tagged so they fall inside a model that already exists, not about building a new model per source.

[PLATFORM ENGINEER] A custom, non-CIM data model is a legitimate thing to build — a model scoped to one application's own log format, with fields that will never be part of CIM's shared vocabulary — but a data model built to extend or shadow a CIM model without going through Part 7's tagging discipline first produces the same downstream confusion a bad field alias does: two things that look interchangeable in the Pivot Editor's dataset picker but return different rows for the same question, because one inherits CIM's tag-based constraint and the other doesn't.

4. Pivot: querying a dataset without writing SPL

4.1 The Pivot Editor's building blocks

[SOC ANALYST] Pivot is the UI that turns a data model dataset into a report without the person building it typing a search. Opening Pivot against a specific dataset — say, `Failed_Authentication` — presents four kinds of choice: a **filter** (narrow further than the dataset's own constraint, e.g., `dest=domain-controller-*`), a **split** (a row or column grouping field, e.g., `src`), a **column value** (an aggregation — count, distinct count, average, sum, list, first/last value — applied to a chosen field), and a **format** (table, single value, or one of Splunk's standard chart types). None of this requires knowing

`stats` syntax; DEH Part 26 §2 teaches what `stats` actually does under the hood, and Pivot's column-value choices map directly onto the same aggregation functions that section covers — Pivot doesn't introduce new aggregation semantics, it just presents `stats`'s functions as a dropdown instead of a clause you type.

[SOC ANALYST] The one hard boundary worth knowing before relying on Pivot for anything beyond a quick look: **a single Pivot report queries exactly one dataset from exactly one data model.** There is no join, in the Pivot Editor itself, between the Authentication data model and the Network Traffic data model (the CIM data model covering flow and session-level network events) to answer something like "which failed logins came from an IP that also appeared in a network-traffic block event." That kind of cross-model question is a hand-written SPL problem — a `join` or a shared-field correlation across two separate searches — not a Pivot problem, and reaching for Pivot on a question it structurally can't answer wastes time discovering that limit through the UI instead of recognizing it up front.

4.2 What Pivot actually runs

[PLATFORM ENGINEER] Pivot's "Open in Search" option, and the equivalent way to reach the same dataset from a hand-written search, is the `datamodel` command: it opens a named dataset and returns its rows exactly as if you'd run the dataset's own constraint as a search. The example below targets the same `Failed_Authentication` child dataset used above.

CONCEPTUAL SAMPLE — illustrative `datamodel` query shape, not a captured production search.

```
| datamodel Authentication Failed_Authentication search  
| stats count by user, dest
```

This search opens `Failed_Authentication` directly — every event already satisfying the root's `tag=authentication` constraint and the child's `action=failure` constraint — then aggregates it with `stats`, exactly the way DEH Part 26 §2 describes `stats` collapsing events into summary rows by a group-by field list. Its main limitation is performance, not correctness: unless the Authentication data model is accelerated, `| datamodel ... search` reads and re-filters raw events on every run the same as any other unaccelerated search would, over whatever time range is scoped — Part 10 covers what changes once the model is accelerated and a `tstats` search reads its `tsidx` summary instead.

[PLATFORM ENGINEER] A dedicated `pivot` command also exists, accepting a dataset name plus the same aggregation and split-by choices the Pivot Editor exposes, and it's what the Editor generates under the hood for most report types. This part doesn't reproduce its full argument syntax here, because the Pivot Editor is how the overwhelming majority of Pivot users — including the analysts §4.1 is written for — actually interact with a data model; hand-typing an equivalent `pivot` command is rare enough in practice that showing its syntax without a live instance to verify the current argument order against would trade a real, checkable `datamodel` example for a lower-confidence one.

The table below summarizes when reaching for the Editor versus writing SPL by hand against the same dataset actually pays off.

Situation	Pivot Editor	Hand-written <code>datamodel / stats search</code>
One dataset, standard aggregation (count, distinct count, average)	Fastest path — no syntax to recall	Equivalent result, more typing for no benefit
Question spans two data models	Not possible in Pivot itself	Required — separate searches joined or correlated by hand
Result needs post-aggregation filtering (<code>where count > 20</code>)	Supported via Pivot's own filter-after-aggregation option, more limited than <code>where</code>	Full <code>where / eval</code> expressiveness (DEH Part 26 §1.1)
Output feeds a scheduled correlation search or RBA rule (Part 14, Part 15)	Not the target use case	Required — a correlation search is SPL in <code>savedsearches.conf</code> , not a saved Pivot report
Analyst has no SPL background	Only realistic option	Not usable without someone who writes the search

5. Pivot as a hunting tool

[THREAT HUNTER] Pivot's value to a hunter isn't the finished chart — it's the speed of reshaping a hypothesis without retyping a `stats` clause from scratch every time the question changes shape. Point the Pivot Editor at `Failed_Authentication`, split rows by `src`, add a distinct count of `user` as the column value, and "which sources tried the widest spread of distinct usernames" is answered in under a minute with no SPL typed. Swap the split-by field to `app` and rerun the same shape of question against a different angle on the same dataset.

Hunter's Note The tradeoff DEH Part 23 names for query-language choice in general — authoring speed versus control over exactly what's being asked — shows up inside a single tool here. Pivot is fast precisely because it only lets you ask the questions its dataset's fields and its own filter/split/aggregate model can express; the moment a hunt needs a field the data model doesn't expose, a join across two models, or a conditional that Pivot's filter UI can't represent, the fast path stops being available and the honest move is to open the generated search and finish the thought in SPL — not to force the question into a shape Pivot happens to support.

6. Blind spots: what never reaches the Pivot table

[PLATFORM ENGINEER] A child dataset only ever returns events that already satisfied every constraint above it in the tree, and a CIM root dataset's constraint is almost always a `tag` value. An event whose CIM mapping never applied that tag — a newly onboarded log source missing its `eventtypes.conf` stanza, or a tag that stopped applying after the source's log format changed (the exact failure mode Part 7 names for CIM mapping drift) — doesn't appear in the Pivot Editor, doesn't produce an error, and looks identical to "this source genuinely produced zero matching events."

Blind Spot A Pivot table showing zero failed-authentication events from a given host reads the same whether that host genuinely had zero failures or whether its logs never made it into the Authentication data model at all. Pivot has no built-in signal that distinguishes "the population is empty" from "the population never reached this dataset," because both look like an empty result set to the same query. The parser-health-monitor pattern DEH Part 26 §1.2's own Detection Autopsy box uses for a broken field extraction — watching a specific field's null rate independently of whether the detection built on top of it ever fires — is the same pattern that catches this failure mode for a data model: watch whether a source that should be tagging into the model still is, not just whether the Pivot report built on top of it still returns rows.

7. Licensing, editions, and where this sits relative to Enterprise Security

[SOC MANAGEMENT] Data models and Pivot are core Splunk platform features — they ship with Splunk Enterprise and Splunk Cloud Platform and work whether or not Enterprise Security is installed at all. The CIM Add-on that supplies the prebuilt Authentication, Network Traffic, Web, and other models is itself free content on Splunkbase, not something gated to a specific Splunk or Enterprise Security edition. What Enterprise Security adds on top is its own dependency on those same CIM models — its correlation searches, notable-event enrichment (Part 14, Part 17), and much of its own prebuilt dashboarding (Part 18) assume the CIM data models this part describes are populated and current, which is exactly why Part 13 treats CIM compliance as Enterprise Security's hard dependency rather than an optional best practice.

Product Version Note Splunk Enterprise Security is currently sold in two editions — Essentials and Premier — with UEBA, SOAR integration, and "Automated Threat Analysis" gated to Premier; Essentials includes SIEM functionality, AI-assisted features, threat intelligence, Detection Studio, and Exposure Analytics in both editions. Data models and the Pivot Editor described in this part sit outside that gate entirely: both are core-platform features, not Enterprise Security features, and neither edition restricts access to either. As of 2026-09-15, verified against Splunk's own Enterprise Security product page ([REFERENCES.md](#) entry [SPLUNK-ES-PRODUCT-PAGE]). What would make this stale: Splunk folding data-model authoring or Pivot itself behind a licensing tier in a future restructuring, or a further edition split beyond Essentials/Premier — re-verify before assuming this boundary is permanent, the same caution the Part 13 Product Version Note gives the edition split itself.

8. What carries forward

[CONCEPT] Everything in this part depends on Part 5's CIM contract already being applied correctly and Part 7's onboarding work keeping it that way — a data model is a lens on tagged data, not a

substitute for the tagging. Everything after this part builds on the dataset, not the tagging: Part 10 accelerates it and teaches `tstats`; Part 14 writes correlation searches against it; Part 18 builds dashboard panels on top of the reports this part's Pivot Editor produces. None of those parts re-explains what a root or child dataset is — that explanation lives here, once.

Cross-references: DEH V2 Part 26 §1.1–1.3, §2 (SPL pipeline model, search-time/index-time fields, `stats`, and the `tstats` forward reference this book's Part 10 resolves); DEH V2 Part 23 (query-language authoring-speed-versus-control tradeoff, applied in §5 to Pivot versus hand-written SPL); this book's Part 5 (Common Information Model), Part 7 (CIM-compliant data onboarding and mapping drift), Part 8 (lookup tables), Part 10 (data model and report acceleration, `tstats`), Part 13 (Enterprise Security's CIM dependency), Part 14 (correlation searches), Part 17 (asset and identity correlation), and Part 18 (dashboards and visualizations).

Part 7 — Building CIM-Compliant Data Onboarding

Why this part exists

[CONCEPT] Part 5 defined the Common Information Model as a field-naming and tagging contract, not a magic normalizer, and Part 6 showed how a CIM-compliant data model becomes a queryable dataset through Pivot and, eventually, `tstats`. Neither part had to answer the question this one does: given one new, real log source that has never touched Splunk before, what is the actual sequence of configuration work — file by file, stanza by stanza — that turns its raw events into something the Authentication, Network Traffic, or Endpoint data model will recognize as a member? This part is implementation-focused because "map the fields onto CIM" is where a design conversation about data models turns into a `props.conf` diff someone has to write, test, and maintain for as long as that source keeps sending data.

This part does not re-teach the search-time/index-time field distinction DEH Part 26 §1.2 already covers, or the general translation-problem framing DEH Part 6 (Normalisation) applies across ECS, OCSF, UDM, and ASIM — CIM is Splunk's specific answer to the same general problem, and Part 5 already positioned it that way. What this part adds is the mechanics: field extraction and aliasing, `tags.conf` and `eventtypes.conf` authoring, and validating a mapping against the CIM Add-on's own compliance tooling — and, just as importantly, what silently breaks when that mapping goes stale after a source-format change. That failure mode is Detection Debt's translation-layer problem, per DEH Part 23 §3, wearing Splunk's own clothes: a broken `FIELDALIAS` compiles cleanly, runs on every search, and returns a field that is quietly null forever, which looks identical to "this data source stopped producing the event" to anyone not specifically watching for it.

Evidentiary note: every configuration excerpt in this part is a hand-built illustration built around a fictitious `corp_vpn` source, marked `CONCEPTUAL SAMPLE` per `STYLE-GUIDE.md` §3 — none of it has run against a live Splunk instance, because none exists in this book's evidence base (`STYLE-GUIDE.md` §9.2). Where the underlying platform mechanism (a `props.conf` stanza key, a `tags.conf` format, the CIM's own tag vocabulary) is stated as fact rather than illustration, it reflects Splunk's own published `.conf` file reference and Common Information Model Add-on documentation, not this book's own testing.

1. The onboarding pipeline, end to end

[CONCEPT] "CIM-compliant" is not one configuration action — it is the cumulative result of five separate, independently-failable steps, each living in its own `.conf` file or add-on component, each maintained by whoever touches that source next. Skipping straight from "raw event" to "field aliasing" without stopping at sourcetype assignment, and skipping from aliasing straight to "it shows up in Pivot" without stopping at tagging, are two of the most common places a new source gets partially onboarded and then quietly abandoned once the most visible field looks right.

Figure 7.1 lays out the sequence this part covers, in order.

flowchart LR

```

A[Raw event arrives] --> B[Sourcetype assignment]
B --> C["Field extraction (props.conf / transforms.conf)"]
C --> D["Field aliasing onto CIM names (FIELDALIAS)"]
D --> E["Calculated CIM fields (EVAL-fieldname)"]
E --> F["Eventtype classification (eventtypes.conf)"]
F --> G["Tag assignment (tags.conf)"]
G --> H["CIM data model membership"]
H --> I{"CIM Add-on compliance check"}
I -->|pass| J["Queryable via Pivot / tstats"]
I -->|fail| C

```

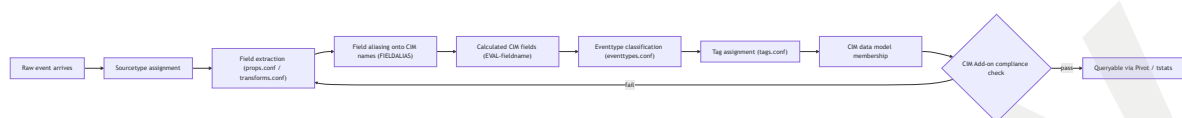


Figure 7.1 — The CIM onboarding pipeline, from raw event to data-model membership.

CONCEPTUAL. Illustrates the expected sequence of configuration stages a new log source passes through before a correlation search or dashboard built against a CIM data model can see it; this is a sequence diagram of documented, expected configuration behavior, not a capture from a live Splunk instance or job inspector — none exists in this book's evidence base (STYLE-GUIDE.md §9.2).

[PLATFORM ENGINEER]

Table 7.1 gives the same sequence as a working reference — which file each step lives in, and which role typically owns it in a mid-sized SOC. Use it to orient before the detailed sections below, not as a substitute for them.

Step	Configuration mechanism	Typical owner
Sourcetype assignment	<code>props.conf</code> (<code>[sourcetype]</code> stanza), input configuration on the forwarder	Platform/onboarding engineer
Field extraction	<code>props.conf</code> (<code>EXTRACT-</code> , <code>REPORT-</code>) and <code>transforms.conf</code> , or a vendor-supplied Technology Add-on	Platform engineer, or the TA's maintainer
Field aliasing onto CIM names	<code>props.conf</code> (<code>FIELDALIAS-<class></code>)	Platform/detection engineer, jointly
Calculated CIM fields	<code>props.conf</code> (<code>EVAL-<cim_field></code>)	Detection engineer
Eventtype classification	<code>eventtypes.conf</code>	Detection engineer
Tag assignment	<code>tags.conf</code>	Detection engineer
Compliance validation	CIM Add-on's own validation views, or a <code>tstats</code> / <code>datamodel</code> spot check	Whoever owns the onboarding ticket

This part covers extraction briefly (§2), since DEH Part 26 already owns the general field-extraction concept, and spends most of its depth on aliasing (§3), tagging (§4), and validation (§5) — the three steps that are specific to CIM and that Parts 5–6 named but did not walk through mechanically.

2. Field extraction: where this part's job begins

[PLATFORM ENGINEER] DEH Part 26 §1.2 establishes the distinction that matters here: an index-time field is written into an event's metadata once, permanently, at ingestion, while a search-time field is re-extracted from `_raw` on every search that needs it, using whatever extraction configuration is active *at the moment the search runs* — not whatever was active when the event was originally indexed. Everything this part does — aliasing, tagging, calculated fields — is a search-time construct built on top of whatever raw extraction already produced, which means every one of them inherits that same retroactive-but-fragile property: change a `FIELDALIAS` today and it applies to three years of already-indexed history the next time anyone searches it, and a regex-based extraction that breaks after a vendor format change breaks that same three years of history's ability to populate the field, going forward, from the moment it breaks.

Most onboarding work never touches raw extraction directly, because most sources arrive with a Splunkbase Technology Add-on (TA) that ships its own `props.conf`/`transforms.conf` extractions for its own sourcetypes — the Splunk Add-on for Unix and Linux, the Splunk Add-on for Microsoft Windows, and equivalents for major firewall and cloud platforms all extract their own raw fields before CIM compliance enters the picture at all. The onboarding engineer's actual job, for a TA-covered source, starts one step later: confirming which raw fields the TA already extracts, and aliasing *those* onto CIM names rather than re-deriving extraction that already exists. For a source with no TA — an internal application, a niche appliance, a home-built log shipper — extraction has to be built first, and it is built exactly the way DEH Part 26 describes a `rex`-equivalent working: a `props.conf` `EXTRACT-` stanza (or a `transforms.conf` regex referenced by `REPORT-`) pulling named capture groups out of `_raw`. This part assumes that step is either already handled by a TA or already covered by DEH Part 26's extraction mechanics, and picks up at what happens to those fields once they exist.

3. Field aliasing onto CIM field names

[PLATFORM ENGINEER] A raw field almost never arrives already named `src`, `user`, or `action` — the three Authentication-data-model fields nearly every correlation search in Part 14 and Part 15 filters or groups on. It arrives named whatever the source calls it: `remote_ip`, `login`, `result`, `SourceAddress`, `usr`. Field aliasing is the step that gives that field a second name, the CIM name, without touching the original.

`FIELDALIAS` — renaming a field without duplicating it

[PLATFORM ENGINEER] `FIELDALIAS-<class>` is a `props.conf` key, scoped to a `[sourcetype]`, `[source]`, or `[host]` stanza, that maps one or more existing field names onto one or more alternate names at search time. Both names resolve to the same underlying value; nothing is copied or duplicated in the index. The block below hand-onboards a fictitious VPN concentrator's JSON access log — no CIM-aware TA exists for it, which is the ordinary case for an internally built or lower-volume vendor source:

```
# CONCEPTUAL SAMPLE – illustrative props.conf stanza for a hand-onboarded VPN sourcetype;
# not copied from any real Technology Add-on or observed in a live deployment.
[corp_vpn]
SHOULD_LINEMERGE = false
KV_MODE = json
FIELDALIAS-corp_vpn_auth = remote_ip AS src login AS user vpn_gateway AS dest
```

This one line does the entire renaming job for three fields in a single pass. `remote_ip`, `login`, and `vpn_gateway` still exist under their original names — a report built before onboarding that still refers to `login` keeps working — and `src`, `user`, and `dest` now exist alongside them, resolvable by any search, dashboard, or correlation search written against the Authentication data model's own field vocabulary.

`EVAL-<cim_field>` — deriving a CIM field that doesn't already exist

[PLATFORM ENGINEER] Not every CIM field has a one-to-one raw equivalent to alias. The Authentication data model's `action` field takes, for a login/logoff event, one of two documented values — `success` or `failure` — and this VPN log's own `result` field uses vendor-specific codes (`ACCEPT`, `REJECT`, `ACCEPT_MFA_PENDING`) that don't map onto those two values by simple renaming. A calculated field, defined with `EVAL-<cim_field>` in the same `props.conf` stanza, derives the CIM value with an expression instead:

```
# CONCEPTUAL SAMPLE – continues the corp_vpn stanza above; illustrative only.
EVAL-action = case(result=="ACCEPT","success", result=="REJECT","failure", true(),null())
EVAL-app = "corp_vpn"
```

The `EVAL-app` line is worth noting on its own: not every CIM field needs source data to derive from at all. `app`, in the Authentication data model, is expected to identify which application or service produced the event, and for a single-purpose sourcetype like this one, a literal constant is a completely legitimate calculated field. DEH Part 26 §1.2's SPL-level `eval` and this `props.conf`-level `EVAL-` share expression syntax but not execution context: this one runs implicitly, once, on every event of this sourcetype at search time, rather than being written explicitly into a query.

Table 7.2 gives the full mapping for this example source, aliasing and calculated fields together, as a reference for what a completed field-mapping deliverable typically documents for a reviewer or a future maintainer.

Raw Field (corp_vpn)	CIM Field (Authentication)	Mechanism
<code>remote_ip</code>	<code>src</code>	<code>FIELDALIAS</code>
<code>login</code>	<code>user</code>	<code>FIELDALIAS</code>
<code>vpn_gateway</code>	<code>dest</code>	<code>FIELDALIAS</code>
<code>result</code> (vendor code)	<code>action</code> (<code>success</code> / <code>failure</code>)	<code>EVAL-action</code>
— (no raw equivalent)	<code>app</code> (constant <code>"corp_vpn"</code>)	<code>EVAL-app</code>

Blind Spot A `FIELDALIAS` and an `EVAL`-derived field both apply retroactively to already-indexed events the next time anyone searches them — but they are not identical in cost or fragility. An alias is a cheap rename with no expression to re-evaluate; a calculated field re-runs its `case()/if()` logic against every matching event on every search that needs it. A `case()` expression enumerating vendor codes, as above, silently stops classifying anything the day the vendor adds a new code (`ACCEPT_MFA_PENDING` in the example) that isn't in any branch — it falls through to the final `null()` and produces a record with no `action` at all, not a wrong one. Nothing errors. A dashboard counting `action=success` against `action=failure` simply undercounts both, forever, until someone notices the totals don't add up to the event count and goes looking for why.

4. `tags.conf` and `eventtypes.conf`: how an event earns data-model membership

[PLATFORM ENGINEER] Renaming fields onto CIM names is necessary but not sufficient. A CIM data model's datasets are not defined by field names alone — the Authentication data model's root dataset constrains itself, roughly, to events carrying `tag=authentication`, and its `Failed_Authentication` and `Successful_Authentication` child datasets narrow that further to `tag=authentication` combined with `tag=failure` or `tag=success` respectively, on top of the field set. An event with a perfectly aliased `src/user/action` but no matching tag is invisible to `| datamodel Authentication search` and to any `tstats` search run against it — it has the right fields and the wrong membership.

Two `.conf` files do this work together, and they are almost always authored as a pair.

`eventtypes.conf` — giving a raw search a name to tag

[PLATFORM ENGINEER] An eventtype is nothing more than a saved search string with a name. It classifies events; it does not, by itself, connect that classification to CIM at all:

```
# CONCEPTUAL SAMPLE – illustrative eventtypes.conf entries for the corp_vpn sourcetype above.
[corp_vpn_auth_failed]
search = sourcetype=corp_vpn action=failure

[corp_vpn_auth_success]
search = sourcetype=corp_vpn action=success
```

Note that this search string filters on `action`, the CIM field aliased in §3 — an eventtype definition can reference either the raw or the CIM-aliased field name once aliasing is in place, and writing it against the CIM name rather than the vendor's own result code means the eventtype definition doesn't need to change again if the vendor renames or adds new result codes later; only the `EVAL-action case()` expression that produces `action` in the first place would need updating.

`tags.conf` — where a named eventtype earns data-model membership

[PLATFORM ENGINEER]

`tags.conf` attaches one or more tags to an eventtype (or, less commonly, directly to a specific field=value combination). This is the step that actually makes the CIM's constraint logic match:

```
# CONCEPTUAL SAMPLE – illustrative tags.conf entries tagging the eventtypes above so they
# satisfy the Authentication data model's Failed_Authentication and Successful_Authentication
# dataset constraints.
[eventtype=corp_vpn_auth_failed]
authentication = enabled
failure = enabled

[eventtype=corp_vpn_auth_success]
authentication = enabled
success = enabled
```

Once both files are in place, an event that used to be an opaque line of VPN concentrator JSON is, at search time: sourcetyped as `corp_vpn`; carrying `src`, `user`, `dest`, `action`, and `app` under CIM names; classified as one of two named eventtypes depending on its `action` value; and tagged such that it satisfies the Authentication data model's `Failed_Authentication` or `Successful_Authentication` dataset constraint. That last step is what makes it appear in Pivot against Authentication, in a `tstats` sweep against the accelerated data model (Part 10), and in the search body of any correlation search in Part 14 written against `datamodel=Authentication`.

Product Version Note *The Authentication data model's tag-based dataset constraints — a base dataset keyed on `tag=authentication`, with `Failed_Authentication` and `Successful_Authentication` child datasets adding `tag=failure` and `tag=success` respectively — are documented in Splunk's own Common Information Model Add-on reference. As of 2026-09-15, verified against the CIM Add-on's Splunkbase listing (default version `8.7.0`, released 2026-09-02; REFERENCES.md entry `[SPLUNKBASE-CIM-ADDON]`). `docs.splunk.com`'s own CIM reference pages were not reachable while researching this part (see REFERENCES.md's disclosed fallback note), so no claim is made here about how long this constraint shape has held across earlier CIM major versions — only that it holds as of the cited version. What would make this stale: a future CIM major-version release restructuring the Authentication data model's child datasets, or retiring or renaming a tag this mapping relies on — confirm the tag vocabulary against the CIM Add-on version actually installed in a given environment before trusting a mapping like this one as complete.*

5. Validating against the CIM Add-on's own compliance checks

[PLATFORM ENGINEER]

Writing the aliasing, eventtype, and tag configuration above is not the same claim as "this source is CIM-compliant" — that claim needs to be checked, not assumed, and checking it two different ways catches two different failure classes.

The Common Information Model Add-on (distributed on Splunkbase as `Splunk_SA_CIM`) ships its own validation tooling, built specifically to answer "does this sourcetype satisfy this data model's required fields" without hand-writing a search for every field on every dataset. Pointed at a chosen data model and a chosen sourcetype or eventtype, it reports field coverage — which required and recommended fields are populated, and at what rate — rather than a single pass/fail bit, which matters because a source can satisfy every *required* field and still be missing enough *recommended* fields that a downstream dashboard panel built against them comes up empty.

Product Version Note *The CIM Add-on ships built-in validation views for checking a source's field coverage against a chosen data model, distributed through its Splunkbase listing. As of 2026-09-15, verified against the CIM Add-on's Splunkbase listing (default version `8.7.0`, released 2026-09-02; REFERENCES.md entry `[SPLUNKBASE-CIM-ADDON]`). This book's research could not reach `docs.splunk.com` to confirm how many prior releases have shipped this capability, so no claim is made about its history — only that the current listed version has it. What would make this stale: the exact menu label and navigation path for this view are the kind of interface detail that has moved between past CIM Add-on releases and is reasonably likely to move again — confirm the current path against the specific installed version's own bundled documentation before writing an onboarding runbook that names one.*

The second, complementary check doesn't depend on the CIM Add-on's UI at all: running a `tstats` count against the accelerated data model, scoped to the new sourcetype, and comparing it to the raw event count for the same time range and sourcetype. Part 6 covers what a data model actually is as a queryable object, and Part 10 covers `tstats` and acceleration mechanics in depth — this part only needs the outcome, not the syntax. If the raw count and the `tstats`-against-the-data-model count for the same window diverge by more than rounding, some fraction of this sourcetype's events are failing the tag or field constraint silently, and the gap is the thing to chase, not the two counts individually.

Detection Test Setup: *A sourcetype with field aliasing, calculated fields, eventtypes, and tags already configured per §3–§4, and read access to both the raw index and the target CIM data model. **Action:** Run a raw count of events for the sourcetype over a fixed time window, then run the CIM Add-on's own field-coverage validation for that sourcetype against the target data model, and separately compare a `tstats` count against the accelerated data model for the same sourcetype and window. **Expected result:** Field coverage at or near 100% for every field this part's mapping claims to populate, and the `tstats` count matching the raw count within a small margin explainable by acceleration lag rather than missing tag/field constraints. This has not been run against a live instance for this book — there is none — so treat the expected result as what the documented mechanism should produce, not a measured outcome.*

6. What breaks when the mapping goes stale

[PLATFORM ENGINEER]

Every mechanism in §3–§4 is a search-time construct anchored to specific raw field names and specific value strings — `remote_ip`, `result=="ACCEPT"` — and every one of them shares the exact fragility DEH Part 23 §3 names as Detection Debt's defining translation-layer failure mode: a broken mapping produces zero errors and zero alerts, which looks identical to "the technique isn't happening," or, in this part's own case, "the vendor stopped sending this field," when what actually happened is a config change on the vendor's side that nobody on the Splunk side knew to check for.

The pattern below is a well-documented CIM onboarding failure repeatedly reported in Splunk practitioner community discussion of TA and vendor firmware upgrades — not a captured incident from this book's own deployment, since none exists here.

Detection Autopsy — "the VPN authentication panel that went quietly blank after a firmware upgrade"

The rule: A VPN concentrator's JSON access log, onboarded per §3–§4 above, with `FIELDALIAS-corp_vpn_auth` renaming `remote_ip/Login/vpn_gateway` onto `src/user/dest`, and `EVAL-action` deriving `success/failure` from a `result` field carrying a small, enumerated set of vendor codes.

Why it shipped: The mapping was validated once, at onboarding time, against a live sample of the vendor's log output, and every code observed in that sample had a matching branch in the `case()` expression.

How it failed: A vendor firmware upgrade added a new authentication-outcome code — for MFA-pending logins, say — that didn't exist when the mapping was written. Events carrying the new code fell through `case()`'s final branch to `EVAL-action`'s `null()` default. `action` came back empty for every one of them; `src/user/dest` kept populating normally because those aliases never depended on `result` at all; and any dashboard panel or correlation search filtering specifically on `action=success` or `action=failure` simply stopped counting that subset of logins — with no ingest error, no extraction error, and, critically, no drop in the sourcetype's overall event count, since the events were still arriving and still populating most fields correctly.

The fix: the same pattern DEH Part 26 §1.1 already names for a broken regex extraction — a scheduled null-rate check on `action` for this specific sourcetype, alerting independently of whether any correlation search built on top of it ever fires, so a new vendor code is caught by a parser-health monitor within a day rather than discovered when someone finally asks why the VPN authentication panel's numbers look low.

[DETECTION ENGINEER]

The blast radius of a broken mapping like this is never contained to the one panel or search someone happened to notice. Every correlation search in Part 14 and every risk-based-alerting rule in Part 15 written against the Authentication data model inherits whatever the mapping actually produces, not what the mapping was designed to produce — a correlation search counting failed logins per user across the whole Authentication data model undercounts silently for every source whose `action` field has quietly gone null, in exactly the same "zero errors, zero alerts, looks identical to

nothing happening" shape DEH Part 23 §3 describes for a Sigma backend's stale field-mapping pipeline. The platform-layer failure and the operations-layer consequence are the same failure, one level apart.

Engineering Reality *"We onboarded this source, it's CIM-compliant" is a claim about a moment in time, not a durable property. A field mapping is validated against whatever the vendor's log format looked like on the day someone wrote it, and it stays correct only as long as that format doesn't change underneath it — which a TA upgrade, a vendor firmware update, or a source application's own new release can all do without any coordination with whoever owns the Splunk-side mapping. Treat every hand-built EVAL- / FIELDALIAS mapping the same way DEH treats a regex-based extraction: something that needs a monitored null-rate or field-coverage check running indefinitely, not a one-time validation step closed out on an onboarding ticket and never revisited.*

7. Onboarding as maintained infrastructure, not a project

[SOC MANAGEMENT] The failure pattern in §6 is a staffing and ownership problem before it is a technical one. An onboarding project has a start date and an end date; a field mapping has neither — it needs an owner for as long as the source keeps sending data, the same way Part 8's lookup tables and Part 9's macros need a maintained owner rather than a one-time author. Budgeting onboarding as "N hours to map this source" and closing the ticket when the CIM Add-on's validation view shows green is the exact gap the Detection Autopsy above walks through: the validation passed on the day it was checked, and nobody was assigned to re-check it after that.

[PLATFORM ENGINEER] The practical version of that ownership, at minimum: a per-sourcetype null-rate or field-coverage check on every CIM field this part's mechanisms populate, alerting on its own schedule independent of any downstream correlation search; and a review trigger tied to any known upgrade event on the source side — a TA version bump, a vendor firmware release, an application deployment — that prompts a re-check of the mapping rather than waiting for someone to notice a dashboard looks wrong. Neither of these is exotic engineering; both require someone to own them past the onboarding ticket's close date.

What Would Change My Mind *Every claim in this part about what a stale mapping looks like in practice — the null-rate signature, the "still arriving, still mostly correct, one field quietly empty" shape of the failure — is drawn from Splunk's own documented .conf mechanics and DEH's own documented translation-layer failure pattern, not from watching it happen against a real Splunk instance ingesting a real, evolving VPN log over months. A real deployment, instrumented with the null-rate monitor this part recommends and observed across at least one vendor firmware upgrade or TA version bump, would either confirm this failure shape exactly as described or reveal a different, messier failure mode this book's evidence base has no way to have anticipated. That is the specific, concrete gap Part 20 names again alongside every other unverified claim in this book.*

Cross-references: DEH V2 Part 26 §1.1–§1.2 (search-time vs. index-time fields, extraction cost); DEH V2 Part 23 §3 (Detection Debt's translation-layer failure mode, field-mapping completeness); this book's Part 5 (the Common Information Model as a field-naming and tagging contract), Part 6 (Data Models and Pivot), Part 8 (lookup tables as maintained infrastructure), Part 10 (Data Model and Report Acceleration, `tstats`), Part 14 (correlation searches built against CIM data models), Part 20 (validation gaps and the path to real evidence).

Section C — Reusable Detection Infrastructure

neshbooy

Part 8 — Lookup Tables as Detection Infrastructure

Why this part exists

[CONCEPT] DEH Part 26 §4.2 dissects a lateral-movement correlation search that excluded known vulnerability-scanner hosts with a subsearch against a CMDB index — `NOT [ search index=cmdb tag=scanner | fields host ]` — and shows exactly how that subsearch silently truncated past its result-count cap, suppressing the exclusion for some scanner hosts and not others with no error surfaced anywhere. Part 26 §4.3 states the fix in one sentence: replace the subsearch with a `lookup` against a CSV export of the same data, "which has no comparable result-count cap and makes the full exclusion list an inspectable, versioned artifact instead of a live query nobody thought to check the size of." That sentence is where Part 26 stops, on purpose — its job is the SPL language, and `lookup` is a command whose syntax it has now taught. This part starts exactly where that sentence ends. Deciding a lookup beats a subsearch is a five-minute call once you know the truncation mechanism Part 26 §4.1 documents. Building the table that `lookup` command actually reads, wiring it into `transforms.conf` and `props.conf` so a search head resolves it correctly, choosing whether it lives as a flat CSV file or a KV Store collection, and keeping its contents from drifting out of date six weeks after the SOC stops thinking about it — that is a maintained infrastructure problem, not a one-line syntax choice, and it is this part's whole subject.

This part does not re-teach the `lookup`, `inputlookup`, or `outputlookup` commands themselves — Part 26 §4.1 and §4.3 already cover what each does and when a `lookup` beats a `join` or a subsearch; a framing sentence pointing back there stands in for re-deriving that syntax anywhere below. What follows instead: what a lookup table actually *is* as a platform object (§1); the two `.conf` files that turn a file or collection into something SPL can query (§2); CSV lookups as files with a real replication and editing story (§3); KV Store collections as the alternative built for partial writes at scale (§4); the match-type and case-sensitivity decisions that determine whether either one actually matches what you think it matches (§5); where a lookup sits in the path from raw event to notable event (§6); and the ownership and review discipline that keeps a lookup table from becoming exactly the kind of stale, silently-wrong reference data DEH Part 23 §3 already named as Detection Debt's translation-layer failure mode (§7–8).

1. What a lookup table actually is, and the three shapes it comes in

[CONCEPT] A lookup table is a named, external mapping from one or more key fields to additional output fields, resolved against a search's result set rather than stored inside the indexed event stream itself. That single idea splits into three distinct platform objects that are easy to conflate because they share a name:

- **The lookup table** — the actual data: a CSV file's rows and columns, or a KV Store collection's documents. This is the thing that goes stale, gets reviewed, and needs an owner.
- **The lookup definition** — the `transforms.conf` (and, for automatic lookups, `props.conf`) stanza that names the table, states its match behavior, and makes it addressable by a short name SPL can reference. This is configuration, not data.

- **The lookup invocation** — the `lookup`, `inputlookup`, or `outputlookup` command in an actual SPL search that reads from or writes to the table through its definition. DEH Part 26 §4 owns this layer; this part never re-teaches it.

Splunk supports three backing shapes for the table itself, and picking the wrong one for a given use case is the single most common source of the problems §3 through §5 walk through in detail.

Table 8.1 — The three lookup-table shapes and what each is actually for. Use this table to decide which shape a new reference dataset belongs in before writing a single `transforms.conf` stanza; §3–§4 go into the CSV and KV Store rows in depth, and the third row is included for completeness rather than as something this book's own examples build on further.

Shape	Update pattern	Typical security use	Practical limit
CSV lookup file	Whole-file replace	Static or slow-changing allowlists, asset-criticality tables, MITRE technique-name references	Fine into the low tens of thousands of rows; whole-file replace cost grows with size
KV Store collection	Per-document create/update/delete via REST or SPL	Frequently-updated reference data — a risk-score cache, a dynamic asset-criticality table an external script maintains	Scales further than a CSV for write-heavy tables; adds a database dependency (§4)
External/scripted lookup	Computed on demand by an external script at search time	Enriching against a live external source (a threat-intel API) without pre-loading its full dataset	Adds per-search latency and an external-service failure mode; used sparingly in practice

The rest of this part is organized around the first two rows, because they cover the overwhelming majority of lookup tables an actual Splunk security deployment maintains. External/scripted lookups exist and are documented, but they trade a maintained-artifact problem for a live-dependency problem — the same tradeoff Part 26 §4.1 describes for a subsearch against a live index, just moved outside Splunk entirely — and this book does not develop them further.

2. Defining a lookup: from file to searchable object

[PLATFORM ENGINEER] A CSV file sitting in an app's `lookups/` directory is not, by itself, something SPL can query by name. Two `.conf` files turn it into one: `transforms.conf` names the table and states how it matches, and — only if the lookup should apply automatically, without an explicit `lookup` command — `props.conf` attaches it to a sourcetype.

`transforms.conf` — the stanza that turns a file or collection into a lookup

[PLATFORM ENGINEER] The stanza below defines the same allowlist DEH Part 26 §5.2 reads from in its worked SPL implementation of DET-26-01 (a process other than a known-good source opening a memory-reading handle to `lsass.exe`, the Windows process holding credential material): a two-column table of source-image paths and an allowlist flag.

```
# transforms.conf
[lsass_access_allowlist]
filename = lsass_access_allowlist.csv
case_sensitive_match = false
max_matches = 1
```

This is the whole of what makes ``lsass_access_allowlist.csv`` — the CSV lookup table file DEH Part 26 §5.2's search reads via ``lookup lsass_access_allowlist.csv SourceImage OUTPUT is_allowlisted`` — resolvable by name at all. `case_sensitive_match = false` and `max_matches = 1` are the two settings that most often cause a lookup to behave differently than whoever wrote the CSV expected; §5 covers both directly, because getting either wrong produces a silent mismatch, not an error.

`props.conf` — automatic lookups that run without being asked

[PLATFORM ENGINEER] A `LOOKUP-<class>` stanza in `props.conf` attaches a lookup definition to a sourcetype so that every matching event gets the lookup applied automatically at search time — no explicit `lookup` command required anywhere in the search that later reads the enriched field.

```
# props.conf
[XmlWinEventLog:Microsoft-Windows-Sysmon/Operational]
LOOKUP-lsass_allowlist = lsass_access_allowlist SourceImage OUTPUT is_allowlisted
```

With this stanza in place, any search against the ``XmlWinEventLog:Microsoft-Windows-Sysmon/Operational`` sourcetype (the sourcetype Part 26 §5.1 names as the common landing spot for Sysmon telemetry ingested through the Splunk Add-on for Sysmon) already has `is_allowlisted` populated before the search body ever runs, and DET-26-01's explicit `lookup` line becomes redundant rather than necessary — the automatic version and the explicit version produce the same enriched field, and a detection engineer choosing between them is choosing where the cost is paid, not what the result looks like.

Engineering Reality *An automatic lookup runs against every event of the matching sourcetype at search time, for every search that touches that sourcetype — including searches that never reference the looked-up field and would never have noticed it was missing. That cost is invisible in the SPL text itself; nothing in a search that doesn't mention `is_allowlisted` tells you the search is still paying to resolve it. A sourcetype carrying several automatic lookups, each one a moderately expensive match, adds real per-search overhead to a high-volume sourcetype that no single search's own text discloses — Part 12 covers diagnosing this kind of hidden cost with the Search Job Inspector. Attach an automatic lookup only to a sourcetype where the enrichment is genuinely needed by most searches against it; otherwise, an explicit `lookup` command in the specific searches that need it keeps the cost visible where a reviewer can actually see it.*

3. CSV lookups: file semantics, replication, and editing at platform scale

[PLATFORM ENGINEER] A CSV lookup table file lives under an app's `lookups/` directory and is distributed to every indexer or search peer that needs it as part of the **knowledge bundle** — the full set of an app's knowledge objects (lookups, macros, saved searches, field extractions) that a search head pushes out before dispatching a distributed search. That distribution mechanism has a consequence worth stating plainly: a knowledge-bundle push replicates the *entire* lookup file, not a diff against whatever version the peer already has. A ten-row allowlist and a two-hundred-thousand row asset-criticality table cost the same *kind* of push — a full-file copy — just at very different sizes, and a CSV lookup that's grown well past what it was originally sized for turns every bundle replication cycle into a larger, slower push than the table's original design accounted for.

[PLATFORM ENGINEER] Editing the contents of an existing CSV lookup at the platform level is, by default, a whole-file operation too: Splunk's own **Settings** → **Lookups** → **Lookup table files** page lets you upload a new file that replaces the existing one, but it has no built-in mechanism for editing a single row or cell in place. The free **Lookup Editor** app, widely used across Splunk security deployments and available on Splunkbase, adds exactly that — inline cell editing, per-row history, and lookup-level permissions — on top of the same underlying CSV file and the same knowledge-bundle replication story described above; it changes how the file gets edited, not what happens once it's saved. SPL's own `outputlookup` command (Part 26's territory, not re-taught here) can also write a CSV lookup's contents directly from a search's result set, which is how a scheduled search maintaining a dynamic table — a rolling list of hosts seen in the last 24 hours, say — keeps a CSV lookup current without a human editing it by hand at all.

[PLATFORM ENGINEER] Because a CSV lookup file is, mechanically, just a file, it belongs in the same version-controlled deployment pipeline as `savedsearches.conf` and `macros.conf` — Part 9 covers this discipline for macros in depth, and it applies to a lookup file with no meaningful difference: a diff-able history of who changed which row and when, promoted across environments the same way a correlation search's own `.conf` stanza is, rather than edited directly in a production search head's filesystem with no record of the change.

3.1 What a truncation fix doesn't fix: staleness

[DETECTION ENGINEER] A CSV lookup solves Part 26 §4.2's truncation problem cleanly — but a file that no longer truncates can still go stale, and staleness is a different failure mode with a different fix.

Detection Autopsy — "the scanner allowlist that outlived the scanner"

The rule: A lateral-movement correlation search excludes source hosts listed in ``scanner_allowlist.csv`` (columns: `host`, `added_date`, `added_by`) from firing, so the vulnerability-scanner platform's own nightly broad-host-touching behavior doesn't page the SOC every night — exactly the fix Part 26 §4.3 recommends over a subsearch against a live CMDB index.

Why it shipped: It's cheap, reviewable, and versioned — a static exclusion list checked into the same deployment pipeline as the correlation search itself, with no live-query truncation risk at all. It looked like a solved problem.

How it failed: The scanner host was decommissioned during an infrastructure refresh, and its hostname and IP were reassigned to a new database server six weeks later. Nobody removed the old entry from `scanner_allowlist.csv` — no scheduled review owned that file, and its own presence in version control gave everyone false confidence that "it's tracked" meant "it's current." The correlation search kept silently suppressing anything from that host identifier, including the new database server's own genuine lateral-movement pattern, for three months after the reassignment.

The fix: Two changes, not one. First, add an `added_date` column (already present above) and a companion scheduled search that flags any allowlist row past a fixed review window, rather than trusting version-control history alone to surface staleness. Second, tie removal from the allowlist to the same decommission workflow that removes a host from asset inventory — Part 17's Asset and Identity framework is exactly the system that already knows a host was decommissioned — instead of a manual "someone remembers to also update the lookup" step that has no owner and no trigger.

A staleness-review search of the kind the fix above calls for is a small, direct application of `inputlookup`, `eval`, and `where` — all already taught in DEH's own SPL and query-language parts — not new syntax this part is introducing:

```
| inputlookup scanner_allowlist.csv
| eval age_days=round((now()-strptime(added_date,"%Y-%m-%d"))/86400)
| where age_days > 180
```

This search's whole job is to surface any allowlist row older than roughly six months so a human looks at it on a schedule; its main limitation is that it depends entirely on `added_date` having been populated accurately when the row was added — a lookup table built without a date column in the first place gives this search nothing to work with, which is itself an argument for building that column in from the start rather than retrofitting it after the first staleness incident.

4. KV Store collections: partial writes, larger scale, and a separate backup problem

[PLATFORM ENGINEER] A **KV Store collection** is a MongoDB-backed set of documents, defined in `collections.conf` and exposed to SPL through the same `lookup/inputlookup/outputlookup` mechanism a CSV lookup uses — from a search's point of view, a KV-Store-backed lookup and a CSV-backed lookup look nearly identical to invoke. What differs is underneath: a KV Store collection supports **per-document create, update, and delete** through Splunk's REST API, without replacing the whole table the way a CSV lookup's default editing model requires.

```
# collections.conf
[asset_criticality]
field.host = string
field.criticality = number
accelerated_fields.host_idx = {"host": 1}
```

```
curl -k -u admin:changeme \
  https://localhost:8089/servicesNS/nobody/search/storage/collections/data/asset_criticality/<_key> \
  -X POST -d '{"criticality": 90}'
```

[PLATFORM ENGINEER] That REST call updates one document's `criticality` field without touching any other row in the collection — the exact operation a CSV lookup's whole-file-replace model cannot do without rewriting the entire file. This is what makes KV Store the right shape for a frequently-updated reference table: an external asset-criticality feed correcting one host's score several times a day, or a risk-score cache an automation script updates continuously, both fit KV Store's per-document write model far better than a CSV lookup's file-replace model. `accelerated_fields` adds an index on a named field inside the collection, the KV Store equivalent of a database index, which matters once a collection holds enough documents that an unindexed lookup against it becomes its own performance problem — a concern Part 12 covers at the platform-performance level more generally.

Product Version Note Splunk Enterprise Security is currently sold in two editions — Essentials and Premier — with User and Entity Behavior Analytics (UEBA), SOAR integration, and "Automated Threat Analysis" gated to Premier; both of those Premier-only capabilities lean on KV-Store-backed state (behavioral baselines for UEBA, playbook and case state for SOAR) rather than CSV lookups, because both need the partial-write pattern described above at a scale a flat file doesn't comfortably reach. As of 2026-09-15, verified against Splunk's own Enterprise Security product page (Enterprise Security 8.7.0, released 2026-09-02; [REFERENCES.md](#) entry [\[SPLUNK-ES-PRODUCT-PAGE\]](#)). Part 13 covers this edition split in full; it's named here only because it's the clearest example of why a security team would reach for KV Store instead of a CSV lookup in the first place. Confirm which edition a given environment is actually licensed for before assuming either Premier capability — or the KV-Store volume it implies — is present.

Blind Spot KV Store data is not part of the knowledge bundle a search head replicates to its search peers, and it is not backed up by the same configuration-backup process that protects `.conf` files and CSV lookups. A KV Store collection needs its own, separately-scheduled backup — Splunk exposes this through its own KV Store backup tooling, not through whatever process already backs up `$SPLUNK_HOME/etc`. A team that treats "we back up our Splunk config" as covering everything a lookup table might live in will discover the gap exactly once: after a KV Store collection is lost or corrupted and the asset-criticality table, risk-score cache, or ES Asset and Identity data it held turns out not to have been backed up at all.

Exact per-document and per-collection size caps for KV Store are set in `limits.conf`'s `[kvstore]` stanza, and the specific default numbers have changed across Splunk releases; `docs.splunk.com`'s own KV Store sizing page was not reliably fetchable by this book's research tooling (per `STYLE-GUIDE.md` §9.4), so this part deliberately does not restate a specific byte figure it cannot verify. Check the `[kvstore]` stanza's current documented defaults for the version actually installed before sizing a new collection against a number carried over from a different release or a different book.

5. Choosing between CSV and KV Store, and the match-type gotchas that bite either one

[DETECTION ENGINEER] §1's Table 8.1 named the update-pattern distinction; the table below turns that into a decision a detection engineer can actually make when a new reference dataset needs a home.

Table 8.2 — CSV lookup vs. KV Store collection: which one fits a given reference table. Read each row as a question to ask about the specific dataset in front of you, not as a rule that applies uniformly to every lookup in an environment.

Question	Favors CSV lookup	Favors KV Store collection
How often does the data change?	Rarely — reviewed and edited on a schedule	Continuously — updated by automation or a live feed
Does an update touch one row or the whole table?	Whole-table edits are acceptable	Needs per-document partial updates
Does it need to be git-diffable, human-readable text?	Yes — plain CSV diffs cleanly in a pull request	No — inspected via SPL or the REST API, not a text diff
Does another Splunk app or ES capability expect a specific storage type?	—	Yes — e.g., ES's own Asset and Identity framework (Part 17) is KV-Store-backed
Realistic table size and write volume	Comfortably under tens of thousands of rows, low write rate	Larger tables, or high write rate, where whole-file replace becomes the bottleneck

[DETECTION ENGINEER] Neither shape protects against a mismatch between how a lookup is defined to match and how the field it's matching against is actually formatted, and that gap produces a result that

looks like a working lookup — no error, a clean search — that silently matches nothing, or matches the wrong thing, for a specific subset of events. Three settings in `transforms.conf` control this directly:

- `case_sensitive_match` — `true` by default for most match types; confirm this against the `transforms.conf.spec` for the Splunk version actually installed rather than assuming it carries over unchanged from an earlier release — `docs.splunk.com` was not reliably fetchable by this book's research tooling (`STYLE-GUIDE.md` §9.4), so this specific default is stated from general product familiarity, not a cited source, and should be treated accordingly. A field extraction upstream that starts emitting `C:\Windows\System32\lsass.exe` in a different case than the allowlist file was originally written in — a common side effect of a TA (Technology Add-on) upgrade that changes how a path gets normalized — produces exactly zero matches against rows that used to match, with no error anywhere in the search.
- `match_type = WILDCARD(<field>)` — enables `*`-style wildcard matching for a specific field instead of the default exact-string match; useful for a process-path allowlist that needs to match any version-numbered subdirectory without a row per version.
- `match_type = CIDR(<field>)` — enables CIDR-range matching for a field holding IP addresses, so a single row (`10.0.0.0/8`) can match every address in that range instead of needing one row per address. Without this setting, a lookup table's `10.0.0.0/8`-formatted column is compared as a literal string against an event's IP field and will not match any actual address in that range — the lookup runs cleanly, returns no match, and gives no indication that the CIDR notation was never being interpreted as a range in the first place.

False Positive Trap *An allowlist lookup that stops matching because of a case or match-type mismatch doesn't produce false positives directly — it produces the opposite failure, a detection that should have been suppressed and wasn't, which then gets triaged, closed as a false positive by an analyst who never sees the underlying cause, and quietly re-tuned by widening a threshold instead of fixing the lookup. The fix is not "raise the threshold until the noise stops" — that just makes a real positive matching the same pattern harder to catch too. Confirm the actual case and format of the field being matched against, in real search results, before assuming a `transforms.conf` stanza written months earlier still matches what an upstream TA or field extraction now produces.*

6. Where a lookup sits in the enrichment path

[CONCEPT] Every lookup discussed above — CSV or KV Store, automatic or explicit — resolves at the same point in a search's execution: after the event is already in the result set, before whatever consumes the enriched field runs. Figure 8.1 draws that single convergence point across both invocation paths and both storage shapes, because it is the one structural fact this part's sections all depend on and none of them state as a picture until now.

flowchart LR

```
A["Raw event\nindexed"] --> B{"Sourcetype has an\nautomatic lookup?"}
B -- "Yes" --> C["props.conf LOOKUP-*\nfires at search time"]
B -- "No" --> D["Explicit `lookup` command\nin the search itself"]
C --> E["transforms.conf\nlookup definition"]
D --> E
E --> F{"Backing store"}
F -- "CSV file" --> G["CSV lookup table\nknowledge-bundle replicated"]
F -- "KV Store" --> H["KV Store collection\nMongoDB-backed, REST-writable"]
G --> I["Enriched field added\nto the result set"]
H --> I
I --> J["Correlation search / RBA rule\nconsumes the enriched field"]
J --> K["Notable event or\nrisk-score contribution"]
```

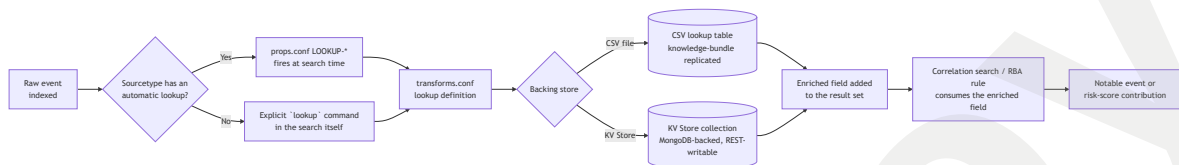


Figure 8.1 — Lookup resolution path from raw event to notable event. *CONCEPTUAL.* Illustrates the two lookup-invocation paths (automatic via `props.conf`, explicit via a `lookup` command) and the two backing-store options (§3, §4) converging on the same enrichment point before a correlation search or risk-based alerting rule (Part 14, Part 15) consumes the result. This is a diagram of documented, expected SPL and platform behavior, not a capture from a live Splunk job inspector — no Splunk deployment exists in this book's evidence base (`STYLE-GUIDE.md` §9.2).

7. Lookup tables as maintained infrastructure: ownership, review cadence, and versioning as code

[SOC MANAGEMENT]

Everything in §3 through §5 solves a technical problem — replication cost, partial writes, match-type correctness. None of it solves the organizational problem the Detection Autopsy in §3.1 actually turned on: a lookup table with no assigned owner and no scheduled review drifts silently, the same way any allowlist drifts, and Splunk's platform gives that drift no warning of its own. A CSV file that matches nothing because of a case mismatch and a CSV file that matches the wrong host because of a stale row both look, from the platform's point of view, like a lookup working exactly as configured — because it is. The staleness is a content problem the platform was never going to catch, and treating "it's in version control" as equivalent to "someone reviews it" is the specific mistake the §3.1 Autopsy shows the cost of.

SOC Management View *A lookup table with no named owner is a liability the same way an unowned firewall rule or an unowned service account is: nobody notices it's wrong until it's already caused a gap, and by then the gap has usually existed for a while. Assign every detection-relevant lookup table an owner and a review cadence as a matter of policy — the same governance a correlation search itself gets under Part 14's content-lifecycle discipline — rather than treating a lookup file as a one-time setup artifact that doesn't need the same ongoing attention as the detection logic that reads from it. The cost of a scheduled quarterly review of a handful of allowlist lookups is trivial against the cost of a three-month blind spot on a reassigned host, and that comparison is the one to put in front of a team that treats lookup maintenance as optional cleanup work.*

[DETECTION ENGINEER] In practice, this means applying the exact discipline Part 9 describes for search macros to lookup table files as well: check them into the same version-controlled repository as `savedsearches.conf` and `macros.conf`, promote changes across environments through the same pipeline, and require a reviewed pull request for a change to a production allowlist the same way a change to a correlation search's threshold would require one. A lookup file is not exempt from code-review discipline just because its contents are data rather than SPL — the scanner-allowlist failure in §3.1 was, in the end, a change-management failure (a decommission that never propagated) wearing a technical costume.

What Would Change My Mind *This part recommends a scheduled review cadence and an ownership assignment for every detection-relevant lookup table, on the reasoning that unreviewed reference data drifts the same way any allowlist drifts. No measured incident rate from a real SOC backs the specific cadence recommended here — that would require observing actual lookup-staleness incidents against actual review intervals over a real retention window, which this book's evidence base (`STYLE-GUIDE.md` §9.2) cannot supply. If a real deployment's incident history showed staleness-driven gaps clustering well outside whatever cadence this part implies is sufficient, that's a direct reason to tighten it, not a reason to abandon scheduled review altogether.*

8. Where lookups feed the rest of the platform

[CONCEPT] A lookup table built and maintained the way §1 through §7 describe doesn't stop at enriching a single search's results. Part 14 covers a correlation search's adaptive response actions writing back to a lookup — flagging a host as already investigated, for instance, so a second correlation search's exclusion logic can read that flag on its own next run. Part 15 covers risk objects consuming lookup-backed asset-criticality data directly, so a risk score for a given host reflects that host's actual business importance rather than treating every asset identically. Part 17 covers the Asset and Identity framework specifically — Enterprise Security's own KV-Store-backed system for exactly the kind of asset and identity enrichment §4's `asset_criticality` example illustrates in miniature, built at a scale and with

a maintenance workflow this part only sketches. None of those three parts re-derives what a lookup table is or how `transforms.conf` and `props.conf` wire one up — that grounding is this part's job, done once, so Parts 14, 15, and 17 can each build on it without restating it.

Cross-references: DEH Part 26 §4.1–4.3 (subsearch truncation, the scanner-exclusion Detection Autopsy, `lookup`/`join`/subsearch tradeoffs); DEH Part 23 §3 (Detection Debt's translation-layer failure mode); this book's Part 9 (search macros — the sibling maintained-infrastructure discipline this part's §7 applies to lookup files directly); Part 12 (search performance and the Job Inspector, for diagnosing automatic-lookup and KV-Store-accelerated-field cost at scale); Part 13 (Enterprise Security editions and the Essentials/Premier split referenced in §4's Product Version Note); Part 14 (correlation searches and adaptive response actions writing to lookups); Part 15 (risk-based alerting consuming lookup-backed asset criticality); Part 17 (the Asset and Identity framework as KV Store's canonical Enterprise Security application).

Part 9 — Search Macros and Reusable Logic

Why this part exists

This is an operations-layer part. Part 8 covered lookup tables — reusable *data* joined into a search. This part covers the other half of reusable detection infrastructure: reusable *search logic* itself, in the form of a search macro. Where a lookup answers "what do I know about this value," a macro answers "what expression, filter, or computation do I not want to retype in every correlation search that needs it." The two are frequently used together, and §5 below is explicit about when to reach for which.

This part assumes the reader already knows what `eval` computes per event and what a `lookup` does to a result set — DEH Part 26 §1.3 owns that syntax-level foundation, and this part does not re-teach it. What this part adds is Splunk-specific and platform-scoped: how a macro is actually defined on disk, how it differs from plain text substitution once `iseval` enters the picture, how it's shared and versioned as a knowledge object, and — the part's real payload — how an unowned, untested macro becomes one of the quietest and most expensive forms of detection debt a Splunk environment can accumulate, because nothing in Splunk's own UI shows you, by default, which correlation searches would break if you changed it. Part 10 picks up the other named-but-undeveloped thread from DEH Part 26 — `tstats` and data model acceleration — and this part's §6 hands off directly to it, since one of the two real macros examined here exists specifically to control accelerated-search behavior.

1. What a macro actually is

[CONCEPT] A Splunk search macro is a named, reusable fragment of search-language text, expanded by substitution before the search ever runs. That's the entire mechanism: Splunk stores a macro's definition, and when a search string contains a backtick-delimited invocation of that macro's name, Splunk replaces the invocation with the definition's text, then parses and dispatches the resulting, fully-expanded search — the same way a C preprocessor macro or a shell alias expands before the compiler or shell ever sees the substituted form. Nothing about a macro is evaluated per event by default; it's a text-substitution step that happens once, at search-parse time, before dispatch.

That single mechanism supports two genuinely different use patterns, and conflating them is the single most common source of confusion for someone new to macros:

- **A macro that expands to a piece of SPL** — a `search` clause, a `lookup` invocation, a `stats` fragment — dropped into the pipeline at the point where it's invoked. This is the common case, and it's what most hand-authored macros in a Splunk environment actually are.
- **A macro that expands to a value, computed from an eval expression** — a number, a string, a boolean — for use inside another command like `eval` or `where`. This is the `iseval` case, covered in §3, and it behaves differently enough from plain substitution that it's worth treating as its own topic rather than a variant of the first.

[PLATFORM ENGINEER] Macros live in `macros.conf`, are knowledge objects like lookups, saved searches, and eventtypes, and inherit the same app-scoping and permission model — private to the user who

created it, shared within an app, or exported globally, controlled the same way any other knowledge object's sharing is controlled (via `default.meta`/`local.meta` `export` settings or the equivalent Splunk Web permissions dialog). A macro defined in one app is invisible to a search running in another app's context unless it's explicitly shared — a routine cause of "this correlation search worked in my sandbox app and throws an unknown-macro error once promoted," and one more reason Part 14's content-promotion discussion and this part's §7 on ownership are related concerns, not independent ones.

2. `macros.conf` — where a macro definition actually lives on disk

[PLATFORM ENGINEER]

A macro's stanza name is the macro's own name, with one wrinkle: if the macro takes arguments, the stanza name carries the argument count in parentheses, because Splunk allows two macros with the same base name and different argument counts to coexist as distinct definitions.

CONCEPTUAL SAMPLE — `get_asset_criticality(1)` is an invented macro built as this part's running example, not a macro shipped by Splunk or captured from a real deployment; every later code block in this part that invokes it inherits the same status.

```
[get_asset_criticality(1)]
args = dest
definition = lookup asset_criticality.csv asset_id AS $dest$ OUTPUT criticality_tier, asset_owner
validation = isnotnull($dest$)
errorMsg = get_asset_criticality requires a non-empty dest field or literal value as its argument
```

This macro takes one argument (`dest`), performs a lookup against `asset_criticality.csv` — the same lookup table this book's Part 8 uses as its running example — and outputs a criticality tier and an owner. Its main limitation is the one every lookup-backed macro shares: it enriches silently and fails silently. A lookup with no matching row does not error the search; it simply leaves `criticality_tier` and `asset_owner` unset for that event, and a downstream `where criticality_tier>=3` on an unset field drops the event instead of flagging a data problem. §7's Detection Autopsy walks through exactly this failure mode.

The table below summarizes the `macros.conf` keys this part actually uses — not an exhaustive spec, an orientation before the worked sections.

Key	Purpose	Required	Example
<code>definition</code>	The text (or eval expression, if <code>iseval = true</code>) the macro expands to	Yes	<code>lookup asset_criticality.csv asset_id AS \$dest\$ OUTPUT criticality_tier</code>
<code>args</code>	Comma-separated argument names, referenced in <code>definition</code> as <code>\$argname\$</code>	Only if the stanza name has an arg count	<code>dest</code>
<code>validation</code>	An eval expression checked against the supplied arguments before expansion	No	<code>isnotnull(\$dest\$)</code>
<code>errmsg</code>	The error text shown if <code>validation</code> evaluates to false	Only alongside <code>validation</code>	<code>get_asset_criticality requires a non-empty dest field...</code>
<code>iseval</code>	If <code>true</code> , treat <code>definition</code> as an eval expression evaluated once at parse time, not as literal SPL text	No — default is <code>false</code>	<code>iseval = 1</code>
<code>disabled</code>	Disables the macro without deleting its definition	No	<code>disabled = 1</code>

Invocation in SPL is the mirror image of the stanza: backticks around the macro name, arguments as a comma-separated list matching the order in `args`.

CONCEPTUAL SAMPLE — invoking the invented `get_asset_criticality(1)` macro above; not captured from a real correlation search.

```
| `get_asset_criticality(dest)`  
| where criticality_tier>=3
```

[DETECTION ENGINEER] Nothing here re-derives `lookup` or `where` syntax — DEH Part 26 §1.3 already covers both — this part's contribution is that the macro invocation itself is a first-class piece of the search string, expanded before anything else about the query is evaluated. That ordering matters: a macro's `validation` expression runs at parse time against whatever literal or field token was passed as an argument, before a single event has been read.

Engineering Reality A macro invoked with no matching definition — wrong argument count, typo'd name, or a macro that exists but isn't shared into the calling app's context — fails the search at parse time with an "unknown macro" error, not a silent no-op. That's the good case: it's loud, and it's the first thing a broken promotion shows you. The bad case, covered in §7, is a macro that does expand successfully but against data that's changed underneath it — that failure mode is quiet, and it's the one worth actually planning for.

3. Eval-based macros: computing a value once, not per event

[DETECTION ENGINEER]

Setting `iseval = true` changes what Splunk does with `definition`: instead of treating it as literal SPL text to splice in, Splunk evaluates it once, as an eval expression, at parse time, and substitutes the *result* into the search string. This is the detail that trips people up — an `iseval` macro's expression does not run per event the way an `eval` command in the pipeline does. It runs exactly once, before the search dispatches, using only its own arguments and any values available at parse time — never a per-event field pulled from the data the search hasn't read yet.

The practical use case that follows from that constraint is a parameterized constant, not a per-event classification:

CONCEPTUAL SAMPLE — `lookback_seconds(1)` is an invented `iseval` macro built to illustrate parse-time constant computation, not a macro shipped by Splunk or captured from a real deployment.

```
[lookback_seconds(1)]
args = hours
definition = $hours$ * 3600
iseval = 1
```

CONCEPTUAL SAMPLE — invoking the invented `lookback_seconds(1)` macro above; the `tstats` command itself is real syntax (Part 10 covers it in depth), but this specific search line is illustrative, not a capture from a real deployment.

```
| tstats count from datamodel=Authentication where earliest=-`lookback_seconds(4)`s
```

The macro computes `4 * 3600` once and substitutes the literal `14400`, so the dispatched search reads `earliest=-14400s` — a small convenience, but a real one, since it lets a correlation search's schedule window be expressed in hours in one place instead of forcing every author to do unit arithmetic by hand and get it wrong at 3 a.m.

A Blind Spot callout would be the wrong shape for what actually goes wrong with `iseval` macros in practice — the real trap is the opposite mistake: writing an `iseval` macro that *tries* to reference a per-event field like `date_hour` or a source-specific field, expecting per-event evaluation, and getting a parse-time error or a nonsensical constant instead, because that field doesn't exist yet at the point the macro expands. If you want per-event business-hours logic, that's a plain `eval` command in the pipeline, computed after the events are read — not an `iseval` macro, which by construction runs before there are any events to compute against.

[PLATFORM ENGINEER]

Splunk's own published detection content gives a real, verified example of a non-`iseval` macro used for exactly the kind of small, repeated transform this section is about. The `security_content_ctime` macro, shipped as part of Splunk's Enterprise Security Content Update (ESCU) detections in the public `splunk/security_content` repository, is defined as:

```
[security_content_ctime(1)]
args = field
definition = convert timeformat="%Y-%m-%dT%H:%M:%S" ctime($field$)
```

One argument, one `convert` clause, invoked as ``security_content_ctime(_time)`` anywhere a detection needs a consistent ISO-8601 timestamp string instead of repeating the same `convert` clause in every rule. It is plain text substitution, not `iseval` — and it illustrates the more common of the two patterns from §1 well enough that it's worth recognizing on sight if it shows up in a vendor-supplied detection your environment has installed.

4. Nesting: a macro that expands to another macro's invocation

[DETECTION ENGINEER] A macro's `definition` can itself contain a backtick-delimited invocation of another macro, and Splunk expands nested invocations before dispatch the same way it expands the outer one — this is how a small number of primitive macros get composed into a more complex, still-reusable unit, and it's also exactly where "who depends on what" stops being visible from any single macro's own definition.

Splunk's own `security_content_summariesonly` macro, from the same `splunk/security_content` repository, is a real, verified example of this composition:

```
[security_content_summariesonly]
definition = summariesonly=`summariesonly_config` allow_old_summaries=`oldsummaries_config`
fillnull_value=`fillnull_config`
```

Its own definition is nothing but three further macro invocations — ``summariesonly_config``, ``oldsummaries_config``, and ``fillnull_config`` — each presumably a small macro somewhere else in the same app that resolves to a single configured value. The composed macro exists to control `tstats`'s `summariesonly` behavior against an accelerated data model in one place across every correlation search that uses it, rather than three raw parameters repeated in every rule. Part 10 covers what `summariesonly` and accelerated-summary coverage actually mean at the `tstats` level; this part's point is narrower: read a macro's own definition and you may still be looking at three more macros you haven't read yet, and Splunk will expand all of them without complaint as long as there's no cycle. A macro that invokes itself, directly or through a chain, is a configuration error Splunk detects and refuses to expand rather than an infinite loop — but a three-hop chain like this one, spread across three separate `macros.conf` stanzas possibly owned by three different teams, is a real, non-pathological case that's already hard to trace by eye.

Product Version Note `security_content_ctime` and `security_content_summariesonly` are real macros published in Splunk's `security_content` GitHub repository as supporting content for Enterprise Security Content Update (ESCU) detections, verified against that repository on 2026-09-15 (see [REFERENCES.md](#) entry `[SECURITY-CONTENT-REPO]`). ESCU's own detection and macro set changes on its own release cadence, independent of the core Splunk Enterprise or Enterprise Security version installed underneath it — treat any specific macro name from that repository as accurate to the retrieval date above, and re-check the repository directly before assuming a given environment's installed ESCU version still ships the exact same definition.

5. Macros, lookups, and subsearches: choosing the right reuse mechanism

[PLATFORM ENGINEER] DEH Part 26 §4.3 already covers the choice between a `lookup` and a subsearch for a filtering problem, and the reasoning there — a subsearch pays an extra, capped, independently-scheduled search's cost every time the outer search runs, where a lookup is a table join with no second search behind it — carries over unchanged. What that section doesn't cover, because it's out of DEH's scope, is where a *macro* fits alongside those two, since a macro isn't a data-reuse mechanism at all — it's a logic-reuse mechanism that can wrap either of the other two.

Mechanism	Reuses	Evaluated	Best for	Main limitation
Search macro (SPL fragment)	A search-string fragment — a filter, a lookup call, a sequence of commands	Once, at parse time, via text substitution	Repeated query <i>shape</i> across many searches — a standard enrichment step, a standard time-window filter	No visibility into which searches use it (§7); breaks silently if the fragment it wraps changes underneath it
Search macro (<code>iseval</code>)	A computed constant derived from its own arguments	Once, at parse time, via eval	Unit conversion, small parameterized formulas — not per-event logic	Easy to mistake for per-event <code>eval</code> ; cannot see fields the search hasn't read yet
Lookup (CSV or KV Store)	External reference data joined to the result set	Per search execution, against current table contents	Allowlists, enrichment tables, anything that changes without a code change	No versioning of its own beyond file/collection history; a lookup rename breaks every macro or query referencing the old name
Subsearch	An independently-run search whose results become a filter for the outer search	Per outer-search execution, as its own capped, scheduled search	Small, genuinely dynamic result sets that can't be pre-computed into a lookup	Result-count cap and extra search cost — DEH Part 26 §4.3

The practical rule: if what's being reused is *data that changes* — an allowlist, an asset inventory, a criticality rating — put it in a lookup and reference the lookup, directly or through a macro. If what's being reused is *query shape* — the same filter, the same enrichment call, the same threshold expression, written identically across a dozen correlation searches — put it in a macro. `get_asset_criticality(1)` from §2 is both at once, deliberately: the data lives in `asset_criticality.csv` (Part 8's concern), and the macro exists so that every correlation search wanting that enrichment writes one backtick-delimited line instead of the full `lookup ... AS ... OUTPUT ...` clause, and so that if the lookup's key field ever needs to change, there's exactly one place — the macro's `definition` — that needs to change to match, instead of every correlation search that inlined the lookup clause directly.

[SOC ANALYST] None of this changes what shows up in Incident Review — a notable event enriched via a macro-wrapped lookup looks identical to one enriched by an inline lookup clause. The distinction in this section is entirely a detection-engineering and platform-maintenance concern; an analyst working a notable has no reason to know or care which mechanism populated `criticality_tier`, only that it's there and current.

6. Sharing, precedence, and where a macro lands in the search-head layer stack

[PLATFORM ENGINEER] A macro is layered configuration like any other `.conf` file: an app's `default/macros.conf` ships the vendor or team default, a `local/macros.conf` in the same app or in a higher-precedence app overrides it, and Splunk's configuration-merging rules (same precedence order as every other `.conf` file — a subject Part 2 covers for deployment architecture generally) decide which definition actually wins when more than one exists for the same stanza name. `splunk btool macros list -debug` shows the effective, merged definition after that precedence resolves, along with which file it actually came from — the right first diagnostic step when a macro behaves differently than its `default/macros.conf` definition suggests it should, because something in `local/` or a higher-precedence app is quietly overriding it.

In a search head cluster, a macro's definition has to reach every cluster member the same way any other knowledge object does — bundled and pushed by the deployer as part of the cluster's configuration bundle, not hand-edited on one member and expected to replicate on its own. A macro that works on the search head where it was authored and throws an unknown-macro error on a scheduled correlation search running on a different cluster member is very often exactly this: a bundle push that hasn't happened yet, not a macro-syntax problem at all. Part 2 covers the deployer and bundle-push mechanics directly; this part's point is narrower — a macro's `.conf` file is subject to the same clustering discipline every other knowledge object is, and it's easy to forget that a macro is a knowledge object at all, since it doesn't show up in Incident Review the way a notable or a risk score does.

Sharing scope follows the same permission model as any other knowledge object: private to its author, shared within the owning app, or exported globally (`export = system` in the relevant `.meta` file, or the equivalent toggle in Splunk Web's permissions dialog). A macro exported only within its authoring app is invisible to a search dispatched from a different app's context — including, commonly, a correlation search that was developed in a sandbox app and then promoted into the Enterprise Security app's context per Part 14's content-lifecycle discussion, without anyone re-checking the macro's sharing setting during that move.

7. Macro sprawl as detection debt

[PLATFORM ENGINEER] Splunk ships no native, always-on dependency graph showing which saved searches, dashboards, or correlation searches invoke a given macro. `btool` and the REST interface for knowledge objects will enumerate every macro that exists; neither one answers "what breaks if I change this macro's definition." That direction of the dependency graph — from the shared macro outward to everything that calls it — has to be built by the team that owns the environment, typically by grepping `savedsearches.conf`, dashboard XML/JSON, and any other macro's definition for the backtick-delimited invocation of the macro in question. This gap in Splunk's own tooling is real and structural; the specific incident walked through below is a conceptual illustration of what that gap allows to happen, not a captured event from a real deployment.

Blind Spot Splunk's own tooling tells you every macro that exists and, via `btool`, exactly which file each one's effective definition came from. It tells you nothing about which correlation searches, dashboards, or other macros actually invoke a given macro. A shared macro's blast radius is invisible from inside Splunk itself — it has to be reconstructed externally, by searching configuration files for the macro's own name, every time someone wants to change it safely.

Detection Autopsy — the asset-criticality macro that went quietly dark

CONCEPTUAL — a scenario built on this part's own invented running-example macro (`get_asset_criticality(1)`), not a documented public incident or a capture from a real deployment; per *STYLE-GUIDE.md* §9.2, no Splunk instance backs this book's evidence base.

The rule: Several correlation searches, including an Enterprise Security realization of DET-26-01 (the LSASS memory-access analytic DEH Part 26 §5 builds as raw SPL), invoke ``get_asset_criticality(dest)`` to enrich every match with a criticality tier, then filter with `where criticality_tier>=3` so a hit against a low-value test host doesn't generate a notable.

Why it shipped: Centralizing the enrichment in one macro, backed by one lookup, looked like exactly the kind of DRY discipline Part 8 recommends — one place to maintain the criticality data, one place to maintain the join logic, both reused everywhere the concept was needed.

How it failed: An unrelated CIM-onboarding cleanup (Part 7's subject) renamed the asset inventory's key field from `asset_id` to `ci_id` as part of a broader identity-normalization pass, and updated most of the lookups that referenced it — but not `asset_criticality.csv`, which nobody flagged as depending on that field name because no artifact in Splunk showed that `get_asset_criticality(1)` was the thing actually consuming it. The macro kept expanding without error. The `Lookup` command inside it kept running without error. It simply stopped matching any row, `criticality_tier` came back unset on every event, and `where criticality_tier>=3` silently dropped everything — including real LSASS-access hits against genuinely critical hosts. Every correlation search invoking the macro went quiet at the same time, with no error, no failed-search notification, and no change to any of those searches' own version history.

The fix: A scheduled sanity search asserting that ``get_asset_criticality(dest)`` returns a non-null `criticality_tier` for a small set of known-good, permanently-criticality-rated hosts, alerting if it ever comes back empty — the macro-testing discipline covered in §8 — plus explicit ownership of the macro and its backing lookup recorded somewhere a field-rename PR would actually surface it, covered next.

[SOC MANAGEMENT] The failure above isn't a testing gap in the abstract — it's a staffing and ownership gap. `get_asset_criticality(1)` had no assigned owner, so a field-rename change elsewhere had no reason to check whether anything depended on the old field name through an indirection nobody was

tracking. Treat every shared macro that more than one correlation search depends on the way you'd treat a shared library in application code: it needs a named owner, a change-review step before its definition or its backing lookup's schema changes, and a place that owner is recorded — a `CODEOWNERS`-style file in the detection-content repository DEH Part 22 already assumes exists, not tribal knowledge held by whoever wrote the macro first and may have since left the team.

8. Testing and versioning macros as code

[DETECTION ENGINEER] DEH Part 22's detection-as-code discipline — version control, pull-request review, lint/syntax validation, and CI-gated promotion before a detection reaches production — applies to a macro at least as strongly as it applies to a correlation search, and arguably more strongly, because a macro has no notable-event history, no urgency field, and no dashboard panel of its own to make a silent behavior change visible after the fact. A correlation search that starts misbehaving eventually shows up as a spike or a drought in Incident Review's own metrics; a macro that starts misbehaving shows up as several *unrelated* correlation searches misbehaving at once, in a way that doesn't obviously point back to the one shared file that actually changed.

Three concrete practices follow directly from treating a macro as maintained infrastructure rather than a one-off convenience:

- **Version-control every `macros.conf` alongside the correlation searches that depend on it**, in the same repository and the same review process DEH Part 22 describes for detection content generally — a macro's `definition` changing is exactly the kind of diff a reviewer should be able to see and reason about, not something buried in a `local/` directory nobody diffs.
- **Write a small canary search per shared macro** — the fix described in the Detection Autopsy above generalizes: a scheduled, low-volume search that invokes the macro against a small set of known-good inputs and asserts the output looks right, alerting on drift rather than waiting for a downstream correlation search's silence to be noticed by an analyst wondering why a normally-noisy rule went quiet.
- **Record, in the macro's own `macros.conf` comments or its entry in the detection-content repository, every correlation search and dashboard known to invoke it** — a manually maintained reverse-dependency list is a poor substitute for tooling Splunk doesn't provide (per the Blind Spot above), but a poor substitute maintained honestly beats no record at all, and it's the artifact that turns "grep everything before you change this" into "check this list first."

What Would Change My Mind This section's claim — that macro ownership and canary testing meaningfully reduce the risk described in the Detection Autopsy above — is argued from mechanism, not measured. What would move it from argued to demonstrated is a real Splunk environment tracked over at least one full schema-change cycle: one cohort of shared macros with assigned owners and canary searches, one cohort without, and a comparison of how long a silent enrichment failure went undetected in each. No deployment in this book's evidence base can supply that comparison; it would need a real SOC's change history to measure, which is exactly the gap Part 20 closes this book out by naming.

```

flowchart LR
    subgraph CS["Correlation searches"]
        CS1["'SSH Brute Force against DB Server'"]
        CS2["ES realization of DET-26-01 (LSASS access)"]
        CS3["'Excessive Privileged Group Changes'"]
    end
    end
    subgraph MC["Shared macros"]
        M1["get_asset_criticality(1)"]
        M2["security_content_summariesonly"]
    end
    end
    subgraph DATA["Backing data"]
        L1["asset_criticality.csv"]
    end
    end

    CS1 --> M1
    CS2 --> M1
    CS3 --> M1
    CS2 --> M2
    M1 --> L1

```

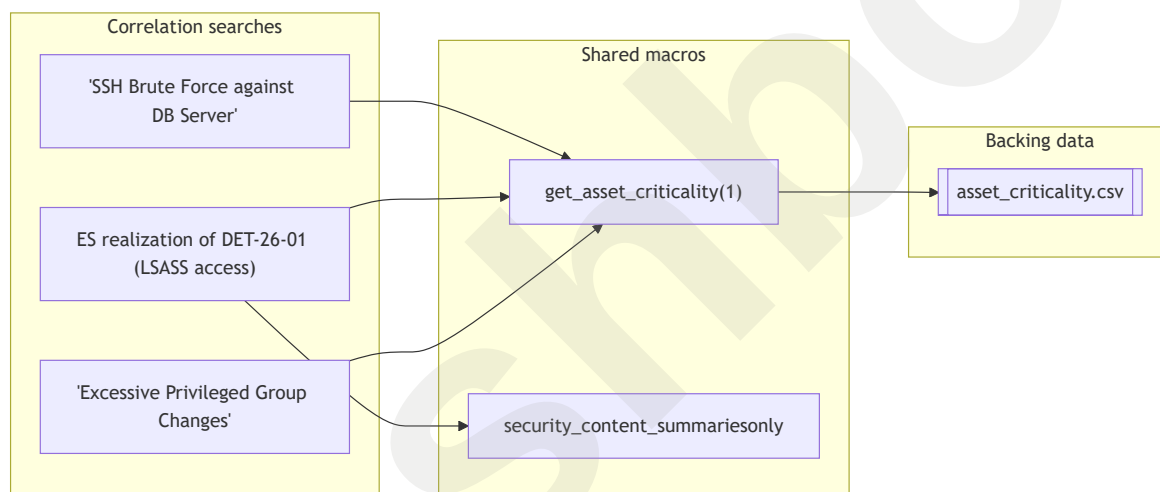


Figure 9.1 — Shared-macro dependency fan-in. *CONCEPTUAL*. Illustrates how three unrelated correlation searches can share one macro, and how that macro in turn depends on one lookup table — so a single edit to `get_asset_criticality(1)`'s definition, or to `asset_criticality.csv`'s key field, changes the behavior of all three correlation searches at once with no diff appearing in any of their own version history. This is an illustrative dependency sketch, not a capture from a real Splunk deployment or a dependency-graph tool — no such tool ships natively with Splunk, which is precisely the Blind Spot named in §7.

9. Worked walkthrough: `get_asset_criticality(1)` end to end

[DETECTION ENGINEER] Pulling the sections above into one sequence, using the macro this part has returned to throughout:

1. **Definition** (§2): `[get_asset_criticality(1)]` in `macros.conf`, one argument (`dest`), a `lookup` clause against `asset_criticality.csv`, a `validation` expression rejecting a null argument before expansion even attempts the lookup.
2. **Choice of mechanism** (§5): the criticality data lives in a lookup because it changes independently of any search; the macro exists purely so the join-and-filter shape doesn't get retyped in every correlation search that wants it.
3. **Invocation**, e.g. inside the Enterprise Security realization of DET-26-01 discussed in Part 14:

CONCEPTUAL SAMPLE — invoking this part's own invented running-example macro; illustrates where the line would sit inside a real correlation search, not a capture of one.

```
| `get_asset_criticality(dest)`  
| where criticality_tier>=3
```

One sentence on what this targets: it's the enrichment-and-filter step that keeps a raw LSASS-access match from generating a notable against a low-criticality test host. Its main limitation is the one the Detection Autopsy in §7 already walked through in full — a silent, non-erroring failure mode if the lookup's key field or the macro's own `$dest$` reference ever drift apart.

4. **Sharing and layering** (§6): exported at the app level so both the sandbox correlation search under development and its promoted counterpart in the Enterprise Security app's context can resolve the same macro name to the same definition, and pushed through the search head cluster's deployer bundle like any other knowledge object rather than hand-edited on one cluster member.
5. **Ownership and testing** (§7–§8): a named owner recorded alongside the macro, and a canary search asserting `criticality_tier` never silently goes null against a fixed set of known-critical hosts — the concrete artifact that would have caught the failure in §7's Detection Autopsy before an analyst noticed a normally-active rule had gone quiet.

[SOC ANALYST] From Incident Review, none of the five steps above are visible — a notable either shows a populated `criticality_tier` field or it doesn't. An analyst who notices a notable missing an expected enrichment field that peer notables usually carry has just found a live instance of exactly the failure mode this part spends most of its length on, and the right escalation is to the macro's owner (step 5), not a manual re-run of the lookup by hand.

Cross-references: DEH Part 26 §1.3 (SPL syntax fundamentals — `eval`, `lookup`, `where` — assumed throughout), DEH Part 26 §4.3 (lookup-vs-subsearch tradeoff, extended in §5 above to include macros), DEH Part 23 §1 and §6 (DET-23-01 and its cross-language carry-forward convention, continued here via DET-26-01), DEH Part 22 (detection-as-code discipline, applied to macros in §8). Within this book: Part 8 (lookup tables as detection infrastructure, backing `asset_criticality.csv`), Part 10 (data model acceleration and `tstats`, the mechanism `security_content_summariesonly` controls), Part 14 (correlation-search anatomy and content-lifecycle promotion), Part 17 (asset and identity correlation, extending the criticality-enrichment pattern), Part 20 (validation gaps, including the untested claim in §8's What Would Change My Mind).

neshbooy

Part 10 — Data Model and Report Acceleration

Why this part exists

This is a platform-layer part. It answers a question DEH Part 26 §1.3 raises by name and deliberately leaves open: that part's SPL command table lists `tstats` as a command that "aggregates directly over indexed/accelerated fields without reading raw events," tags it "named where scale matters; not covered in depth here," and its own opening section states plainly that "a full accelerated-search treatment belongs in a dedicated performance-engineering appendix, not here." This part is that appendix. It also resolves the same forward reference DEH Part 23 leaves standing when it carries the canonical DET-23-01 analytic (MITRE T1003.001, OS Credential Dumping: LSASS Memory) through to DEH Part 26 §5's SPL realization as DET-26-01 — a search this part comes back to below, once there's a concrete reason to.

Two Splunk features do the same underlying trick for different scopes, and this part covers both: **data model acceleration**, which pre-computes a shared summary that any number of searches can read against a whole data model, and **report acceleration**, which pre-computes a summary for exactly one saved report. Both are pre-computation decisions made before a query ever runs, which is why this part sits next to Part 11 (summary indexing, a third pre-computation strategy with different tradeoffs) rather than next to Part 12 (search performance and tuning, which covers what happens when a query runs against whatever pre-computation already exists). This part assumes Part 3's bucket lifecycle (hot/warm/cold/frozen) and Part 6's data model structure as background — it cites both rather than re-deriving them — and it feeds directly into Part 14's correlation searches and Part 18's dashboards, both of which lean on the mechanism built here to stay fast at scale.

1. One mechanism, two features: the accelerated summary

[CONCEPT] Every Splunk index already keeps a `tsidx` file alongside each bucket's compressed raw-data journal — a lightweight, time-bucketed index of a small, fixed set of fields (`_time`, `host`, `source`, `sourcetype`, and anything explicitly configured for index-time extraction) that lets Splunk locate matching events without scanning every raw event first. Data model acceleration and report acceleration both extend that same idea deliberately: instead of relying only on the handful of fields every index already tracks, Splunk builds and maintains a *second*, purpose-built `tsidx` summary that captures a wider, chosen set of fields — the fields a specific data model defines, or the fields a specific report's search computes — so that a search targeting exactly that field set never has to touch `_raw` at all.

The payoff is the same shape in both cases: a search that would otherwise decompress and re-parse every matching raw event, on every run, instead reads a pre-built summary that already has the answer in a compact, indexed form. The cost is the same shape too — disk to store the summary, and scheduled work to keep it current as new data arrives. Neither feature makes data appear that wasn't already indexed; both are strictly an optimization over data Splunk already has, applied selectively to the fields and time range someone decided were worth summarizing in advance.

1.1 What actually gets pre-computed

[CONCEPT] For a data model, the fields that get summarized are exactly the fields the data model's own object hierarchy defines — the auto-extracted, calculated, and hierarchically-inherited fields visible in the data model editor (Part 6 covers that structure in full). A field an analyst adds later with an ad hoc `eval` downstream of a search against that data model was never part of the summary; it still works, but it runs as ordinary per-event computation over whatever (already-reduced) result set the summary handed back, not as something the summary itself can filter or group on.

For a report, what gets summarized is whatever a single saved search's transforming command computes — a `stats`, `chart`, or `timechart` result, keyed however that one search's `by` clause keys it. That's a narrower, more literal scope than a data model's: the summary is good for that report's exact aggregation, and for any other search Splunk can recognize as a subset or "roll-up" of it, not for an arbitrary new question against the same raw data.

flowchart LR

```
A["Raw events land in a bucket\n(hot to warm to cold to frozen,\nPart 3)"] --> B["Index's own\n_tsidx\n_time, host, source, sourcetype,\nplus any index-time fields"]
A --> C{"Data model or report\nacceleration enabled?"}
C -->|No| D["tstats limited to fields\nalready in the index's own\n_tsidx"]
C -->|Yes| E["Scheduled acceleration search\nbuilds a wider summary _tsidx\nfor the model's or\nreport's fields"]
E --> F["Summary _tsidx travels with\nthe same bucket lifecycle as A\n(Part 3) -- frozen buckets\ntake\ntheir summary slice with them"]
B --> G["tstats"]
F --> G
G --> H["Dashboards, Pivot,\ncorrelation searches, hunts\n(Parts 6, 14, 18, 19)"]
F -->|bounded by configured\nsummary range| I["Query window older than\nthe summary range: partial\nor\nzero results, no error\n(Section 4)"]
```

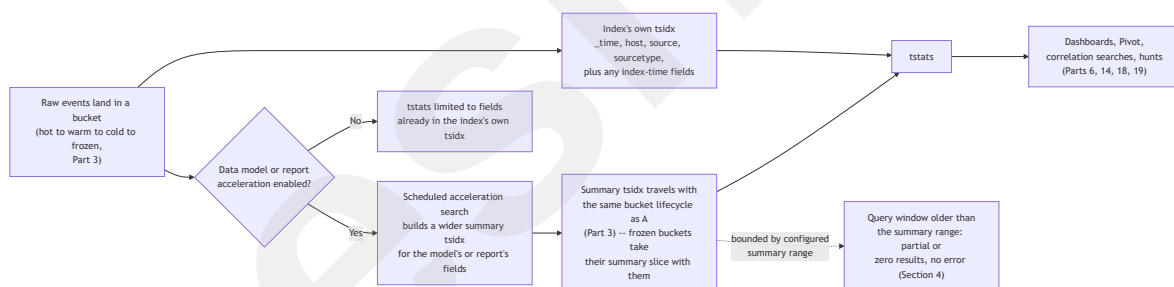


Figure 10.1 — From raw bucket to accelerated summary to `tstats`. *CONCEPTUAL.* Illustrates the expected data flow from an indexed event, through the index's own built-in `_tsidx` and the optional wider acceleration summary, to the point where `tstats` reads either one and where the summary-range blind spot (§4) actually sits in that path. This is a diagram of documented expected mechanism, not a capture from a live Splunk deployment or its Job Inspector — none exists in this book's evidence base (STYLE-GUIDE.md §9.2).

2. Data model acceleration: a summary many searches can share

[PLATFORM ENGINEER] Data model acceleration is enabled per data model — in Splunk Web, from the data model's edit menu, or directly in `datamodels.conf` — and once enabled, it applies to every dataset built on that model's indexed-event root, not to one query. That's the feature's whole value proposition:

a correlation search, a dashboard panel, an analyst's ad hoc Pivot, and a threat hunter's exploratory sweep can all read the same summary, built once on a schedule, instead of each paying its own raw-event scan cost independently. Part 13 will come back to this from the Enterprise Security side — ES's own out-of-the-box correlation searches are written as `tstats`-first content against a fixed set of CIM data models, and enabling acceleration on those specific data models is a step ES's own setup workflow prompts an administrator through directly, not an optional tuning exercise left for later.

2.1 Turning it on: scope, summary range, and the backfill cost

[PLATFORM ENGINEER] Enabling acceleration requires choosing a summary range — how far back the summary covers, from a short recent window out to "all time" — and that choice is a real tradeoff, not a default to leave alone. A wider range means more historical coverage for any `tstats` search built later, at the cost of more disk and, the first time it's enabled or widened, a backfill: Splunk has to build summary data for everything already sitting in matching buckets across the newly-covered range, and that backfill runs as real search work competing for the same scheduler and concurrency slots Part 12 covers for ordinary searches. Turning on a multi-year summary range against a large existing index for the first time is not a quiet background operation — plan it for a maintenance window, not a Tuesday afternoon.

```
# CONCEPTUAL SAMPLE – illustrative datamodels.conf stanza shape, not a captured
# configuration from any real Splunk instance. acceleration.earliest_time is shown
# as an administrator's chosen value here, not a Splunk-shipped default.
[Authentication]
acceleration = true
acceleration.earliest_time = -3mon
```

This stanza targets `datamodels.conf`'s own `[Authentication]` object — the CIM data model covering login/logoff events across log sources, introduced properly in Part 5 and structured in Part 6 — and its main limitation is exactly the tradeoff above: three months of summary range is cheaper to store and faster to backfill than a year, but it also means any `tstats` search relying on this summary is blind to anything older than three months, silently, with no error (\$4 below covers exactly this). `datamodels.conf` also exposes an acceleration schedule and a summary-size ceiling (`acceleration.cron_schedule`, `acceleration.max_summary_size`), and this book's own research pass could not confirm current default values for either against a fetchable source — check your installed version's `datamodels.conf.spec` directly rather than trusting a number repeated from memory.

Engineering Reality *An accelerated data model's summary is additional storage on top of the raw index it summarizes, not instead of it — deleting the raw data doesn't happen, and the summary doesn't replace retention (Part 3), it rides alongside it. That storage doesn't count against your Splunk ingest license, because it isn't newly-ingested external data; it's a derived structure built from data already licensed once at ingest (Part 4 covers licensing mechanics in full). But "doesn't count against the license" is not the same as "free" — budget disk for it explicitly, and budget scheduler capacity for the recurring acceleration searches that keep it current, especially once a handful of ES-shipped CIM data models are all accelerated on the same search head at once.*

[SOC MANAGEMENT] The decision to accelerate a data model is a cost decision before it's a performance one: more summary range and more accelerated data models both mean more disk and more recurring scheduled search load, and neither line item shows up on the ingest-license side of the ledger Part 4 walks through. A SOC that budgets only for license cost and ignores accelerated-summary storage and scheduler headroom will find the gap the first time a search head's disk fills up or its scheduled-search queue backs up, not before.

tstats — searching a summary without touching raw events

[PLATFORM ENGINEER] `tstats` is the command DEH Part 26 §1.3 names and deliberately doesn't teach the syntax of, and this is where that forward reference resolves: `tstats` reads directly from `tsidx` files — either the small set of fields every index already tracks, or, when a `from datamodel=<name>` clause targets an accelerated data model, that data model's own wider summary — and it never decompresses or re-parses `_raw` to do it. That's the entire source of its speed advantage over an equivalent `stats` search: it skips search-time field extraction (DEH Part 26 §1.2's index-time/search-time split) entirely, because every field it can see was already written into a `tsidx` structure ahead of time. The direct consequence is symmetric with that same split: `tstats` can only filter or aggregate on a field that exists in one of those structures. Point it at a field that exists only as a search-time extraction — the overwhelming majority of fields in most Splunk environments, by Part 26 §1.2's own Engineering Reality guidance — and `tstats` simply can't see it, no matter how reliably that field shows up in an ordinary `search | stats` query against the same events.

The query below counts failed authentication events per user and destination host by reading only the `Authentication` data model's accelerated summary, never touching a raw Windows Security Event Log record, a syslog line, or any other source feeding that data model.

```
| tstats count from datamodel=Authentication where Authentication.action="failure" by
Authentication.user, Authentication.dest
| rename Authentication.* as *
```

Its main limitation is the one this whole part is really about: it returns nothing for any time range outside the `Authentication` data model's configured summary range, and if that data model isn't accelerated in a given environment at all, `tstats` still runs, but falls back to whatever subset of its fields

happen to already be indexed at the platform level — a much smaller, much less useful field set than Pivot (Part 6) normally exposes over that same data model.

DET-26-01, DEH Part 26 §5's SPL realization of the canonical LSASS-memory-access analytic, is a concrete illustration of the same tradeoff from the other direction. As written, that rule filters and groups on `SourceImage`, `TargetImage`, and `GrantedAccess` — all three search-time fields, extracted fresh from Sysmon's Event ID 10 payload on every run, exactly as Part 26 §1.2's Engineering Reality box recommends for fields like these. That's the right default. It also means DET-26-01 cannot be rewritten as a `tstats` sweep without first promoting those three fields to index-time extraction and capturing them in an accelerated data model — a deliberate, budgeted storage decision (Part 26 §1.2 again: "treat every one you add as a deliberate storage-cost decision, not a default"), not a free syntax swap. An analyst who wants to run DET-26-01's logic as a fast, long-lookback hunt across months of history rather than a scheduled correlation search over a short recent window (Part 19 covers exactly this kind of pivot) runs into this constraint directly: the fields have to already be in a summary before `tstats` can use them, full stop.

3. Report acceleration: the same trick, scoped to one search

[PLATFORM ENGINEER] Report acceleration solves a narrower problem than data model acceleration, and the two are easy to conflate because they're configured in similarly-named places and both produce a `tsidx`-backed summary under the hood. The difference that matters operationally: report acceleration is transparent to whoever runs the search it accelerates. It's enabled on one specific saved report — from Splunk Web's Searches, Reports, and Alerts settings, or via a report's own `auto_summarize` settings — and it only applies to a search built around a transforming command (`stats`, `chart`, `timechart`, and similar). Once enabled, Splunk builds and maintains a summary for that report's exact aggregation on a schedule, and any later search that Splunk recognizes as an exact match or a subset "roll-up" of the accelerated report's own search automatically reads the summary instead of raw events — the person running that later search doesn't invoke a special command or even necessarily know acceleration is involved. Splunk's own search UI has long prompted an admin to consider accelerating a report it's identified as slow and frequently re-run, which is the practical signal that a report is a good acceleration candidate in the first place: repeated cost against a narrow, stable aggregation.

Data model acceleration is the opposite shape on that same axis: it produces one shared summary that many different, not-yet-written `tstats` queries and Pivot definitions can all draw on, but every one of those queries has to be written to use it deliberately — nothing about a plain `stats` search against raw events automatically redirects to a data model's accelerated summary just because one exists.

3.1 Why it's not a smaller version of data model acceleration

[PLATFORM ENGINEER] Because the two features solve different problems, accelerating both a report and the data model underneath the same events is a real, avoidable failure mode, not a belt-and-suspenders safety margin. Each accelerated object gets its own scheduled maintenance search and its own storage footprint; layering a report's summary on top of a data model summary that already covers the same fields and time range doubles the disk and scheduler cost for a workload that only needed one summary to begin with. Before accelerating a report, check whether the aggregation it computes is

already expressible as a `tstats` search against an existing accelerated data model — if it is, that's usually the cheaper, more reusable choice, since one data model summary can serve the report's need and every other `tstats` query that touches the same fields, where the report's own summary can only ever serve that one report.

4. The summary-range blind spot

Blind Spot A `tstats` search against an accelerated data model, or a search that transparently rolls up into an accelerated report, does not fail, warn, or degrade gracefully when the requested time range extends past the summary's configured coverage — it simply returns whatever the summary has for the portion of the range it does cover, silently, with no indication anywhere in the results that part of the window was never searched at all. A three-month summary range queried over the last six months doesn't error on the missing three months; it returns three months of real counts that look exactly as complete as six months of real counts would, and nothing in the output distinguishes "zero events matched" from "this half of the window was never in scope." A correlation search (Part 14) or an ad hoc hunt (Part 19) built on `tstats` inherits this blind spot automatically, and it inherits it silently — this is the single most consequential operational fact about accelerated search in this entire part, and it is easy to miss precisely because nothing about it looks like a failure.

[PLATFORM ENGINEER] The practical defense is procedural, not technical: whoever builds a `tstats`-based correlation search or dashboard panel has to know the summary range of every data model it touches and consciously bound the search's own time range inside that coverage — or explicitly decide, and document, that the search is only ever meant to run over a shorter recent window than the summary supports. A summary range is configuration, and configuration drifts; a data model's range that was three months when a correlation search was written can be quietly shortened later by someone trying to cut storage cost (\$2.1's Engineering Reality tradeoff, made in the other direction), and the correlation search's own schedule and lookback window don't change to match. Nothing forces the two to stay in sync.

4.1 A second, quieter version: retention outruns the range

[PLATFORM ENGINEER] The summary-range gap has a second form that has nothing to do with the range setting itself. A data model's accelerated summary is stored as `tsidx` files that live alongside the buckets of the indexes it draws from (Part 3's hot/warm/cold/frozen lifecycle), not in some separate, independently-retained location — so a bucket that ages into frozen and is deleted or archived under an index's own retention policy takes its slice of the accelerated summary with it, regardless of what the data model's own `acceleration.earliest_time` setting claims to cover. Configuring a one-year summary range on a data model whose underlying index only retains ninety days of raw data doesn't produce a year of summary; it produces, at most, ninety days, because the summary can never outlive the raw buckets it was built from. A wider summary-range setting than the index's own retention isn't wrong exactly, but it's a number that promises coverage the platform underneath it was never configured to hold — check both settings together, not just the one that lives in the data model editor.

5. Choosing a pre-computation strategy

[PLATFORM ENGINEER] The table below supports a real, recurring decision — which of Splunk's pre-computation mechanisms to reach for when a specific search or dashboard is too slow against raw events, and none of the four options is a strict upgrade over the others.

Mechanism	What it pre-computes	Reusable across queries	Where the summary lives	Best fit
Data model acceleration	Every field defined in one data model's object hierarchy	Yes — any <code>tstats</code> /Pivot query against that model	<code>tsidx</code> alongside the source indexes' own buckets	Shared infrastructure many correlation searches and dashboards draw on (Parts 14, 18)
Report acceleration	One saved report's exact transforming-command aggregation	Only that report and searches Splunk recognizes as a subset of it	<code>tsidx</code> tied to the report definition	A single slow, frequently-rerun report with no matching data model already accelerated
Summary indexing (Part 11)	Whatever a scheduled search's <code>collect / sistsats</code> explicitly writes out	Yes, but only for the exact rows written — no re-aggregation flexibility later	A dedicated summary index, retained on its own schedule	Long-horizon trend reporting where raw-event retention would be prohibitively expensive
Plain <code>tstats</code> on raw indexes	Nothing extra — only the fields every index already tracks (<code>_time</code> , <code>host</code> , <code>source</code> , <code>sourcetype</code> , index-time fields)	Yes, for free	The index's own existing <code>tsidx</code>	A quick count/aggregation on already-indexed fields with zero setup cost

Product Version Note Splunk Enterprise Security is currently sold in two editions — Essentials and Premier — with UEBA, SOAR integration, and Automated Threat Analysis gated to Premier; Essentials includes SIEM, Threat Intelligence, Detection Studio, and Exposure Analytics. As of 2026-09-15, verified against Splunk's own Enterprise Security product page and the Enterprise Security and Common Information Model Add-on Splunkbase listings (ES 8.7.0 and CIM Add-on 8.7.0, both listed with a 2026-09-02 release date; REFERENCES.md entries [SPLUNK-ES-PRODUCT-PAGE], [ES-SPLUNKBASE], [SPLUNKBASE-CIM-ADDON]). This edition split, and which specific CIM data models ES's own setup workflow prompts an administrator to accelerate by default, are both recent, actively-restructured product surface — re-verify both before assuming this book's description still matches the ES release a given environment runs.

6. What would actually confirm this

[PLATFORM ENGINEER] Every mechanism claim in this part — the `tsidx` summary structure, the acceleration schedule and backfill cost, the summary-range blind spot — is reasoned from Splunk's own documented behavior, consistent with this book's evidence-class default (STYLE-GUIDE.md §9.2), not observed against a running Splunk instance.

What Would Change My Mind Every specific number this part avoided stating — the real default acceleration cadence and summary-size ceiling in a current `datamodels.conf.spec`, the actual speed multiplier `tstats` delivers over an equivalent `stats` search on a specific data shape, how long a real multi-year backfill takes against a real multi-terabyte index — has one concrete answer: a real Splunk instance, with a real accelerated data model, observed through its own Job Inspector and `acceleration.log` over a real retention window. That instance doesn't exist in this book's evidence base (STYLE-GUIDE.md §9.2), and no amount of familiarity with the documented mechanism substitutes for watching it actually run. If you have that instance, the check is direct: compare a `tstats` query's Job Inspector timeline against the equivalent raw `stats` search's timeline over the same time range, and read the acceleration summary's actual on-disk size against the raw index it summarizes before trusting any ratio this part might have implied but didn't state.

Cross-references: DEH Part 26 §1.2–§1.3, §5 (search-time/index-time fields, the `tstats` forward reference, DET-26-01); DEH Part 23 (DET-23-01, the canonical analytic DET-26-01 implements); this book's Part 3 (bucket lifecycle), Part 4 (licensing and ingest economics), Part 5–6 (CIM and data model structure), Part 11 (summary indexing), Part 12 (search performance and tuning), Part 13 (Enterprise Security architecture and editions), Part 14 (correlation searches), Part 18 (dashboards), Part 19 (investigation workflows).

Part 11 — Summary Indexing and Other Pre-Computation Patterns

Why this part exists

Part 10 covered data model acceleration: the `tsidx` summary Splunk builds and maintains for you, transparently, behind an accelerated data model, and the `tstats` command that reads it directly — the exact gap DEH Part 26 §1.3 named and stepped around ("`tstats` ... not covered in depth here") and DEH Part 26 §2's own `stats` treatment left for a "dedicated performance-engineering appendix." This part covers the *other* Splunk pre-computation strategy: summary indexing, built with the `collect` command and its `si`-prefixed siblings (`sistats`, `sitimechart`, `sichart`, `sitop`, `sirare`), scheduled through an ordinary saved search rather than maintained by an internal Splunk process. Both strategies solve the same underlying problem — a query that's too expensive to re-run over raw events every time someone needs the answer — and both trade something for the speed they buy. Part 10's tradeoff is disk (the `tsidx` summary sits alongside the raw index) and a bounded acceleration window. This part's tradeoff is different in kind, not just degree: a summary index is a second, ordinary index that exists only because a scheduled search wrote into it, and every property that makes a scheduled search fallible — it can be paused, skipped under concurrency pressure, or silently misconfigured — becomes a property of the summary index too.

This part assumes DEH Part 26's SPL fundamentals throughout — `stats` syntax and aggregation-function semantics (DEH Part 26 §2), and the search-pipeline cost model that makes "run this once, read the result many times" worth doing in the first place (DEH Part 26 §1.1). It does not re-teach either. It also assumes the general query-language vocabulary DEH Part 23 establishes — an analytic's translation into a specific platform's query syntax is a distinct, lossy step (DEH Part 23 §3) — because a summary-indexing search is exactly that kind of platform-specific realization of a query someone already had reasons to run. What this part owns that neither DEH part does, and that Part 10 doesn't either: the mechanics of `collect`/`sistats`/`sitimechart`, how a summary-indexing search actually gets scheduled, and — the scope note that BOOK-INDEX.md gives this part specifically — the staleness and backfill risk a self-maintained summary index carries that Splunk's own accelerated `tsidx` summary does not.

Two pre-computation strategies, one goal

[CONCEPT] Both data model acceleration and summary indexing exist to answer the same question cheaply: "what happened over a time range too large, or a search too expensive, to re-scan raw events every time someone asks." They get there by different routes. Data model acceleration (Part 10) is Splunk-managed — you flip a setting on a data model, and Splunk's own acceleration process builds and refreshes a `tsidx` summary over a configured window, transparently, without you writing or scheduling anything yourself. Summary indexing is search-managed — you write a search that computes something (a daily count, a per-user aggregate, a rare-value list) and explicitly tell Splunk to run that search on a schedule and write its results into a separate index, using `collect` or an `si`-

prefixed command. Nothing happens automatically; the summary index only contains what a scheduled search actually wrote into it, when that search actually ran.

That difference in ownership is the whole story for everything else in this part. An accelerated data model's `tsidx` summary is infrastructure Splunk maintains for you inside a window it tracks; a summary index is infrastructure *you* maintain, using the same scheduling and execution machinery as any other saved search, with every failure mode a saved search can have — a skipped run, a paused schedule, a field that silently stops existing after a source's `sourcetype` changes upstream (Part 7's onboarding-drift problem, recurring here in a different shape).

The table below is the decision reference this part and Part 10 jointly support — use it to pick a strategy, not to relearn either one's mechanics from scratch.

Dimension	Data model acceleration (Part 10)	Summary indexing (this part)
What builds and maintains it	Splunk's internal acceleration process	A saved search you write, own, and schedule
Query-time command	<code>tstats</code>	<code>stats / timechart</code> , run again over the already-summarized data — or finished with <code>stats</code> after <code>sistats</code> (see below)
Where the result lives	A <code>tsidx</code> summary stored alongside the raw index	A separate, ordinary index (its own bucket lifecycle, per Part 3)
Coverage window	Bounded by the data model's configured summary range	Bounded only by the summary index's own retention policy
Behavior on a missed run	Splunk's acceleration process catches up within the summary range on its own schedule	No automatic catch-up — a skipped scheduled search leaves a permanent gap unless someone reruns it explicitly
Best fit	Ad hoc pivoting and dashboards against current, CIM-mapped data	Trend reporting beyond the acceleration window, or an aggregate that must survive the raw data aging out of retention entirely

[PLATFORM ENGINEER] The "best fit" row is doing real work: summary indexing's actual reason to exist, once data model acceleration and report acceleration are on the table as alternatives, is retention outlasting the raw data. An accelerated data model's summary only covers data still inside its configured acceleration range and still inside the raw index's own retention — once a bucket rolls to frozen and is deleted (Part 3), both the raw events and anything the acceleration process built from them are gone. A summary index that was populated *while the raw data still existed* keeps its aggregated rows independently, on its own retention clock, because it's a separate index with separate buckets. If a security team needs a year of daily failed-login counts by user but can only afford ninety days of raw authentication-log retention, summary indexing — not acceleration — is the mechanism that makes that possible, because the aggregate is computed and written while the raw data is still there to compute it from, then kept long after the raw data is gone.

`collect` — writing search results into a summary index

[PLATFORM ENGINEER] `collect` is the primitive summary-indexing command: it takes whatever result set reaches it in the pipeline and writes each row into a target index as a new event, with each field

serialized as a `key=value` pair. Splunk gives that written event a `sourcetype` of `stash` by default (overridable), which exists specifically to mark an event as summary-index output rather than naturally-ingested machine data — a `stash`-sourcetyped event never went through a forwarder or a parsing pipeline; it was written directly by a search. The target index itself is an ordinary index — commonly named `summary` by convention, though nothing forces that name — and needs to exist and be writable before any search tries to `collect` into it, the same as any other index (Part 3).

A minimal illustrative example — a daily count of authentication failures by user, small enough to keep for a year even if the raw `windows` index (the CIM-mapped index holding Windows Security-log events, first introduced in Part 3) only keeps ninety days:

```
index=windows sourcetype=WinEventLog:Security EventCode=4625
| stats count as failure_count by user
| collect index=summary_auth marker="daily_auth_failures"
```

CONCEPTUAL SAMPLE — illustrative index/marker names, not captured from a running system.

[PLATFORM ENGINEER] The `marker` argument is worth calling out explicitly: it tags every event this search writes with a fixed `key=value` pair, which is the mechanism a later search uses to select "just the output of this particular summary-indexing search" out of a summary index that, in any real deployment, accumulates output from more than one scheduled search over time. Skipping the marker is the single most common way a summary index becomes unusable months later — without it, a query against `index=summary_auth` has no reliable way to tell one search's output apart from another's once several summary-indexing searches share the same target index.

Engineering Reality `collect` writes whatever reaches it in the pipeline, including any field the upstream search happens to produce — internal fields like `_raw`, `_time`, and Splunk's own search-metadata fields (`info_min_time`, `info_max_time`, `info_search_time` among them) ride along unless the search explicitly strips them first. A summary-indexing search built by copying a working ad hoc search and appending `| collect` without reviewing the field list first routinely writes several megabytes of metadata noise per run alongside the two or three fields anyone actually wanted — multiply that by a daily schedule over a year and the "small" summary index is not small. Review the field list immediately before the `collect` command, not after the summary index has already grown.

`sistats` and `sitimechart` — pre-aggregating without losing the ability to finish the math

[PLATFORM ENGINEER] `stats` and `timechart` (DEH Part 26 §2 teaches `stats` syntax and function semantics; `timechart` is its time-bucketed sibling) both produce a *final* aggregated answer — a `count`, an `avg`, a `dc()` — computed once, over whatever event population reached the command. That's a problem for summary indexing specifically: if a scheduled search runs `stats avg(bytes) by dest` once a

day and `collect` s the result, and a later search reads three months of those daily summary rows back out and tries to average them again with a second `stats avg(...)`, the answer is wrong for almost every function except `count` and `sum` — an average of daily averages is not the same number as the true average across the full three-month event population, and the error compounds for anything involving distinct counts or percentiles.

`sistats` (and its siblings, `sitimechart`, `sichart`, `sitop`, `sirare`) exist to solve exactly that problem. Rather than writing the *final* computed value, an `si`-prefixed command writes the intermediate calculation state a later `stats/timechart` pass needs to finish the aggregation correctly — the running sum and count behind an average, the partial structure behind a distinct count — carried in a set of internal fields Splunk generates for that purpose. The practical rule that follows: if a summary-indexing search's whole purpose is to let someone re-aggregate later with different grouping or a wider time window, use the `si`-prefixed command and finish the math with a plain `stats/timechart` pass at query time; if the search's output is the final answer and nobody will ever re-aggregate it further, plain `stats` piped into `collect` is simpler and produces a smaller, more directly readable summary index.

```
index=firewall sourcetype=vendor:fw:traffic
| sistats avg(bytes_out) as avg_bytes_out, dc(dest_ip) as distinct_destinations by src_ip
| collect index=summary_fw marker="fw_traffic_by_src"
```

CONCEPTUAL SAMPLE — illustrative index and sourcetype names, not captured from a running system.

A query against the resulting summary index finishes the aggregation the same way it would against the original data, substituting `stats` for `sistats` and reading from the summary index instead of the raw one:

```
index=summary_fw marker="fw_traffic_by_src" earliest=-90d
| stats avg(avg_bytes_out) as avg_bytes_out, dc(distinct_destinations) as distinct_destinations by src_ip
```

CONCEPTUAL SAMPLE — illustrative field and index names, not captured from a running system.

[DETECTION ENGINEER] The distinction matters most for exactly the aggregation functions a detection or a trend report actually leans on — distinct-count and average, per DEH Part 26 §2.1's list of the `stats` functions that come up constantly in detection logic. A rule or report that needs "how many distinct source IPs hit this host per day, summed over ninety days" gets a correct answer from `sistats` re-aggregated with `stats`; the same logic built on plain `stats` piped straight into `collect` gives a plausible-looking but arithmetically wrong number the moment someone re-aggregates across more than one summary-indexing run, and nothing about the query's syntax signals that it's wrong.

savedsearches.conf — the stanza that turns a scheduled search into a summary-indexing job

[PLATFORM ENGINEER] A summary-indexing search is, mechanically, an ordinary scheduled saved search with one extra property: Splunk's "Enable summary indexing" action, which lets a search's SPL

end at a plain aggregation (no explicit `collect`) while the action itself handles writing the result into the target summary index on every scheduled run. That action is stored the same way any other saved-search setting is — as a stanza in `savedsearches.conf` — and it composes with everything Part 12 covers for scheduled searches generally: cron schedule, skip-if-already-running behavior, and priority relative to every other scheduled search competing for the same search-head concurrency slots.

```
[Daily Failed Auth Count by User]
search = index=windows sourcetype=WinEventLog:Security EventCode=4625 | stats count as failure_count
by user
cron_schedule = 5 0 * * *
dispatch.earliest_time = -1d@d
dispatch.latest_time = @d
action.summary_index = 1
action.summary_index._name = summary_auth
action.summary_index.marker = daily_auth_failures
```

CONCEPTUAL SAMPLE — illustrative stanza name and search, not captured from a running system.

[PLATFORM ENGINEER] `dispatch.earliest_time/dispatch.latest_time` deserve deliberate attention here in a way they don't for an ordinary alerting search, because they define the exact time window this run's data covers — get them wrong (an off-by-one boundary, a time zone mismatch between the scheduling cron and the data's own timestamps) and the summary index either double-counts a boundary event on two consecutive runs or silently skips it on both. `dispatch.latest_time = @d` (midnight, snapped to the day) paired with `dispatch.earliest_time = -1d@d` (midnight the day before) is the conventional non-overlapping daily window; any other pairing needs to be checked by hand for a gap or an overlap before it ships, not assumed correct because it looks similar to the last one that worked.

The staleness problem: what a summary index can't tell you that a live accelerated search can

[PLATFORM ENGINEER] Part 10 already establishes that an accelerated data model's `tsidx` summary is maintained continuously by Splunk's own internal process, inside a configured window, and that Splunk itself is responsible for keeping that summary current relative to the raw data arriving underneath it. A summary index has no equivalent guarantee, because nothing but the scheduled search itself is responsible for keeping it current — and a scheduled search is a fallible thing, not a managed platform service.

Engineering Reality *A summary-indexing search that gets skipped — the search head was down for maintenance during its scheduled window, it hit a concurrent-search limit and Splunk deferred it (Part 12 covers the skipped-search mechanics behind this), or someone paused it during an unrelated change and forgot to resume it — leaves a permanent, silent gap in the summary index. Nothing about the summary index itself records that the gap exists; a query against it for that date range simply returns fewer rows, or none, with no error and no obvious signal that data is missing rather than genuinely absent. A dashboard panel built on that summary index shows a trend with an unexplained dip, and unless someone happens to cross-check against the raw data for that specific window, the dip reads as a real finding rather than a missed scheduled run.*

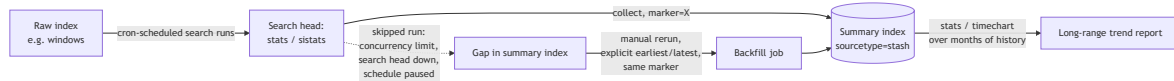
The fix, when a gap is found, is a manual backfill: rerun the same search logic with explicit `earliest/latest` values covering the missed window and `collect` the result into the same summary index with the same marker, so the backfilled rows are indistinguishable from what the scheduled run would have written. That's a deliberate, by-hand operation every time — there is no Splunk process that notices a summary index has a hole and fills it automatically the way the acceleration process behind an accelerated data model can rebuild inside its own summary range. A team that treats a summary index the way it treats an accelerated data model — trusting it to self-heal — finds out it doesn't at the exact moment a gap actually matters, usually while investigating why a trend report disagrees with a raw-data spot check.

What Would Change My Mind *This section treats "a skipped summary-indexing run is a silent, permanent gap with no built-in detection" as the default failure mode. That would need revising if a specific deployment had independently built monitoring for it — a scheduled search auditing `_internal` for skipped-run events tied to summary-indexing searches specifically, alerting when a marker's expected daily row count doesn't appear — because that monitoring, while not something Splunk provides out of the box for this purpose, is buildable with the same platform primitives this part already covers. No such monitoring is verified as deployed anywhere in this book's own evidence base; if a reader has built and can point to one working reliably at scale, that's the concrete observation that would move this from "the default risk" to "a solved problem with a known pattern," and this section would need to say so.*

Figure 11.1 — Summary index build, query, and backfill flow. CONCEPTUAL. Illustrates the expected data flow from a raw index through a scheduled summary-indexing search into a summary index, the later query path that reads it back, and the manual backfill loop a skipped run requires. This is a sequence/flow sketch of documented expected behavior, not a capture from a live Splunk job inspector or a real deployment — none exists in this book's evidence base (STYLE-GUIDE.md §9.2).

flowchart LR

```
A[Raw index<br/>e.g. windows] -->|cron-scheduled search runs| B[Search head:<br/>stats / sistats]
B -->|collect, marker=X| C[(Summary index<br/>sourcetype=stash)]
C -->|stats / timechart<br/>over months of history| D[Long-range trend report]
B -.skipped run:<br/>concurrency limit,<br/>search head down,<br/>schedule paused.-> E[Gap in summary index]
E -->|manual rerun,<br/>explicit earliest/latest,<br/>same marker| F[Backfill job]
F --> C
```



Duplicate data: the other failure mode backfill creates if you're not careful

[PLATFORM ENGINEER] Backfilling a gap fixes one failure mode and, handled carelessly, creates the opposite one: if the backfill's `earliest / latest` window overlaps a window the scheduled search already covered on a run that actually succeeded, the summary index now has two sets of rows for the same period under the same marker, and any later query summing or averaging over that marker double-counts the overlap. This is easy to get right in principle — backfill exactly the confirmed-missing window, no more — and easy to get wrong in practice, because confirming the exact missing window requires checking the search's own execution history (Part 12's Search Job Inspector, or the `_internal` index's scheduler logs) rather than guessing a round-numbered window like "the whole missing day" when only six hours of it were actually missed.

The same overlap risk shows up without any backfill involved, in a subtler form: a summary-indexing search whose `dispatch.earliest_time / dispatch.latest_time` window is set even slightly wider than its own schedule interval — a daily search configured to look back 25 hours "just to be safe" — writes an hour of overlap into the summary index on every single run, every day, indefinitely. Nobody notices immediately because the daily counts still look approximately right; the error accumulates quietly until a long-range report (the exact use case summary indexing exists for) is off by a margin nobody can explain without re-deriving the schedule's own overlap by hand. A summary-indexing search's earliest/latest window should be checked against its cron schedule as a matched pair, on first deployment and again any time either one changes — not treated as two independent settings that happen to be near each other in the same stanza.

When summary indexing still earns its keep

[SOC MANAGEMENT] Given the staleness and overlap risk above, the honest framing for a manager evaluating this pattern is: summary indexing is infrastructure your team now owns and has to operate, not a feature Splunk operates for you the way it operates data model acceleration — and "owns and operates" means someone's on-call rotation is implicitly responsible for noticing a gap, backfilling it correctly, and reviewing scheduled-search health, even if nobody wrote that responsibility down when the search was first deployed. That's a real, ongoing cost, and it's the cost that should be weighed

against report acceleration and data model acceleration (Part 10) before summary indexing is the answer by default.

Product Version Note *Splunk currently prices Splunk Enterprise and Splunk Cloud Platform under three named models — Ingest Pricing (by GB/day ingested), Workload Pricing (by compute capacity consumed running searches), and Entity Pricing (for Observability Cloud, by monitored host count). As of 2026-09-15, verified against Splunk's own pricing FAQ page (REFERENCES.md entry [SPLUNK-PRICING-FAQ]). Which model an environment runs under changes what a summary-indexing search actually costs day to day: under Ingest Pricing, everything a `colLECT` call writes counts against the same daily ingest license as any naturally-arriving data, so a chatty summary-indexing search (see the Engineering Reality box above on stray metadata fields) is a direct license cost, not just a storage inconvenience. Under Workload Pricing, the recurring scheduled search consumes compute capacity instead, and the summary index's own storage becomes the secondary cost. Confirm which model a given environment is licensed under (Part 4 covers this in depth) before assuming either cost story applies. What would make this stale: Splunk restructuring its pricing-model names or eligibility again, which it has done before.*

[PLATFORM ENGINEER] With that cost honestly on the table, summary indexing still wins in one specific, recurring situation that neither data model acceleration nor report acceleration can substitute for: retention that needs to outlast the raw data. An accelerated data model's summary is only ever as old as the raw index still standing behind it (Part 3's bucket lifecycle) and only ever as wide as its own configured acceleration window (Part 10) — once a bucket freezes and is deleted, so is anything built from it. A summary index populated while the raw data still existed keeps its own aggregated rows on its own retention clock, in its own buckets, entirely independent of what happens to the raw index afterward. A security team asked for a year-over-year trend of authentication-failure volume, running on ninety days of raw log retention, has exactly one platform-native way to answer that honestly: a summary-indexing search that ran daily for the last year, whether or not the raw events behind any given day still exist. That's the case worth the operational cost this section has spent most of its length naming — not "summary indexing is generally the right default," but "summary indexing is the specific tool for outliving your own raw-data retention," and every other use case should be checked against Part 10's acceleration options first.

Cross-references

Part 10 (Data Model and Report Acceleration) for the `tsidx`-summary alternative this part is contrasted against throughout; Part 12 (Search Performance and Tuning at Scale) for the scheduled-search concurrency and skipped-search mechanics behind this part's staleness risk, and for the Search Job Inspector referenced above; Part 3 (Indexes, Sourcetypes, and the Bucket Lifecycle) for the summary index's own retention and bucket behavior as an ordinary index; Part 4 (Licensing and Ingest Economics) for the full ingest-vs-workload cost tradeoff this part's Product Version Note only introduces; DEH Part 26 §2 for `stats` syntax and function semantics, and §1.3 for the `tstats` forward reference Part 10

resolves; DEH Part 23 §3 for the general translation-loss framing behind why a summary-indexing search is a platform-specific realization of a query, not a syntax-neutral one.

neshboy

Part 12 — Search Performance and Tuning at Scale

Why this part exists

DEH Part 26 §1.1 makes one claim about SPL that this part exists to scale up: "cost is paid in the order commands appear, left to right" — filter early, and every later pipeline stage inherits a smaller working set. That claim is true and complete at the level of one search running once. It stops being complete the moment a search head is running two hundred scheduled correlation searches, a dozen dashboards' worth of ad hoc queries, and whatever an analyst is typing into the search bar right now, all competing for the same finite pool of CPU, memory, and search-process slots. At fleet scale, the question isn't just "how much does this search cost" — it's "does this search get to run at all before the next one is due," and if it doesn't, what happens to the detection depending on it. Nobody notices a slow search the way they notice a search that silently never ran.

This part is platform layer, in the Section D performance-at-scale group alongside Part 10 (data model and report acceleration) and Part 11 (summary indexing). Part 10 already owns `tstats` and accelerated-search mechanics — the pre-computation decisions made before a query ever runs. This part is about what happens *when* a query runs: concurrency limits, scheduling priority, workload isolation, and the diagnostic tooling that tells you which of those three actually explains a slow or missing search result. It does not re-teach SPL syntax (DEH Part 26 owns that), the analytic-vs-detection-rule vocabulary (DEH Part 23 owns that), or accelerated-search internals (Part 10 owns that) — it assumes a reader who already has a working correlation search or dashboard panel and needs to know why it's slow, why it sometimes doesn't run, and what to check first.

[CONCEPT] Two distinctions run through everything below and are worth stating once, up front, rather than re-deriving in every section. First: a search can fail to produce a result for two entirely different reasons that look identical to an analyst staring at an empty dashboard panel — it ran and found nothing, or it never ran at all. Second: "performance problem" on a Splunk search head is almost never one undifferentiated thing called "slow" — it's a specific, named contention point (CPU-bound concurrency ceiling, memory pressure from a `transaction` or `stats` command holding too much state, disk I/O against cold buckets, or a scheduler queue backed up behind higher-priority work), and the fix for one of those does nothing for the others. The rest of this part is organized around naming which contention point you're actually looking at before reaching for a fix.

1. `limits.conf` — where the platform actually caps concurrent searches

[PLATFORM ENGINEER] Every Splunk search head enforces a ceiling on how many searches can run at once, and that ceiling is not infinite no matter how much hardware you throw at it — it is a deliberately configured number, computed from settings in `limits.conf` (inline-coded per this book's Splunk-object convention — the file that holds most of the platform's resource-behavior tuning knobs), because an unbounded number of concurrent searches would let a handful of runaway queries starve the instance's memory and CPU for everyone else. The mechanism, at a level of detail stable across Splunk Enterprise's `[search]` and `[scheduler]` stanzas: a base concurrency figure is derived from the number of CPU cores

on the search head (a `max_searches_per_cpu`-style multiplier), with an additional fixed floor and, separately, a percentage of that total explicitly reserved for the scheduler (`scheduler_max_searches_perc`-style setting) so that a burst of ad hoc searches from analysts can't starve every scheduled correlation search on the box, and vice versa.

```
# CONCEPTUAL SAMPLE – illustrative limits.conf shape, not a captured or verified current default
[search]
max_searches_per_cpu = 1
base_max_searches = 6

[scheduler]
max_searches_perc = 50
max_rt_search_multiplier = 1
```

Product Version Note `limits.conf` has carried concurrency-governing settings under the `[search]` and `[scheduler]` stanzas across many major Splunk Enterprise releases, and the mechanism described above — a per-CPU multiplier plus a fixed floor, with a scheduler-reserved percentage of the total — reflects that long-standing design. The specific setting names and numeric defaults shown in the example above are illustrative, not verified against current documentation: `docs.splunk.com` returned HTTP 403 on every retrieval attempt made while drafting this part (2026-09-15; see `REFERENCES.md`), and no alternate public source covers `limits.conf` internals at this depth. Confirm the actual stanza names and current defaults for your installed version against the Admin Manual or `btool` output on a real search head before tuning any of them — what would make this stale is exactly a version bump that renames or re-defaults one of these settings, which Splunk has done before across major releases.

[PLATFORM ENGINEER] The practical consequence for a SOC running Enterprise Security is that "add more correlation searches" is not a cost-free operation just because each individual search is cheap — every additional scheduled search is one more claimant on the same fixed concurrency ceiling, and the ceiling doesn't grow just because detection content did. This is DEH Part 26 §1.1's cost principle at fleet scale: the cost of one query is paid left to right within that query; the cost of a fleet of queries is paid against a shared, finite concurrency budget, and a search head that "has room" for fifty correlation searches doesn't necessarily have room for two hundred, no matter how efficiently each one is individually written.

2. Skipped searches: the failure mode that doesn't look like a failure

[PLATFORM ENGINEER] When the scheduler wants to dispatch a scheduled search and the concurrency ceiling from §1 has no free slot, Splunk doesn't queue the search and run it late by default — it skips that scheduled run entirely and waits for the next scheduled time. A skipped search is not an error in the conventional sense: nothing crashes, nothing throws an exception a monitoring tool would catch, and the search's own saved-search object shows no red status. Splunk records the skip as an entry in the `_internal` index (the platform's own index for its internal operational logs, distinct from any

customer data index) under the `scheduler` sourcetype (Splunk's own record of every scheduled search dispatch decision, one entry per attempt, successful or not), with a status field and a reason string identifying the specific limit that blocked the run.

Blind Spot A correlation search that gets skipped produces exactly the same visible result, to an analyst watching Incident Review, as a correlation search that ran cleanly and genuinely found nothing: zero new notables. There is no dashboard panel most SOCs check by default that distinguishes "this detection ran across the last hour and the environment was clean" from "this detection did not run for the last three scheduled intervals because eleven other searches with higher priority got the concurrency slots first." A detection engineer who only monitors notable-event volume, not scheduler health, can watch a correlation search go quiet for entirely the wrong reason and read it as good news.

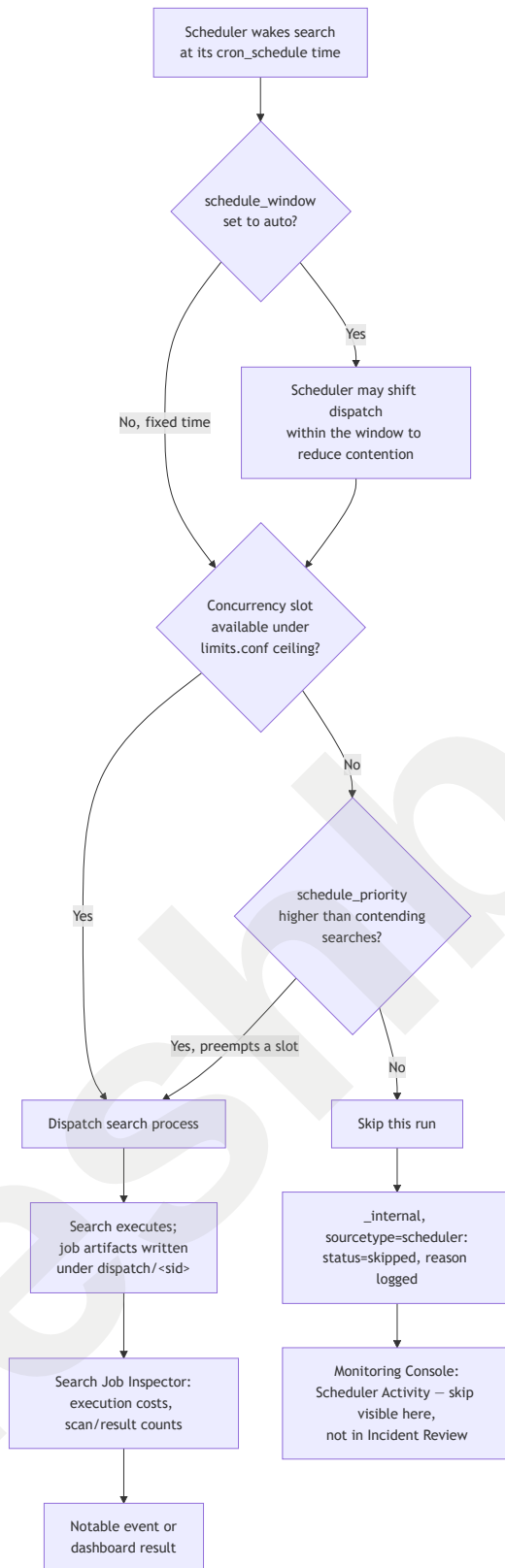
[PLATFORM ENGINEER] The instance's Monitoring Console (Splunk's built-in operational-health app, formerly branded the Distributed Management Console) surfaces scheduler activity and skip counts in its own dashboards rather than requiring an analyst to hand-query `_internal` every time, and checking it on a schedule — not just when someone notices a detection has gone quiet — is the actual fix for the blind spot above. A search skipped once during a genuine load spike is a capacity signal worth watching; a search skipped on a recurring schedule is a detection silently disabled by contention, and the two look identical in Incident Review.

Figure 12.1 — A scheduled search's dispatch decision, from wake-up to result or skip.

CONCEPTUAL. Illustrates the expected sequence a scheduler-dispatched search goes through on a Splunk search head, synthesized from the mechanisms named in §1–§4: a concurrency check against `limits.conf`'s ceiling, a priority/window adjustment where one applies, and a resulting execute-or-skip outcome that either produces Job Inspector artifacts or a `scheduler` sourcetype skip record. This is a diagram of documented expected platform behavior, not a capture from a live scheduler or a real Enterprise Security instance — none exists in this book's evidence base (STYLE-GUIDE.md §9.2).

flowchart TD

```
A[Scheduler wakes search<br/>at its cron_schedule time] --> B[schedule_window<br/>set to auto?]
B -->|Yes| C[Scheduler may shift dispatch<br/>within the window to reduce contention]
B -->|No, fixed time| D[Concurrency slot<br/>available under<br/>limits.conf ceiling?]
C --> D
D -->|Yes| E[Dispatch search process]
D -->|No| F[schedule_priority<br/>higher than contending<br/>searches?]
F -->|Yes, preempts a slot| E
F -->|No| G[Skip this run]
E --> H[Search executes;<br/>job artifacts written<br/>under dispatch/&lt;sid&gt;]
H --> I[Search Job Inspector:<br/>execution costs, scan/result counts]
G --> J[_internal, sourcetype=scheduler:<br/>status=skipped, reason logged]
I --> K[Notable event or<br/>dashboard result]
J --> L[Monitoring Console:<br/>Scheduler Activity – skip visible here,<br/>not in Incident Review]
```



3. `schedule_priority` and `schedule_window` — telling the scheduler which searches matter more

[PLATFORM ENGINEER]

`savedsearches.conf` (the `.conf` file holding every scheduled report's and correlation search's own configuration stanza, one stanza per saved search, inline-coded per this book's convention) exposes two settings that directly shape how a given search competes for the concurrency slots described in §1: `schedule_priority`, which accepts values including a default tier and elevated tiers that let a specific search preempt lower-priority contenders for a slot, and `schedule_window`, which lets the scheduler shift a search's actual dispatch time within a window (including an `auto` setting that hands the decision entirely to the scheduler) instead of insisting on the exact cron-specified minute.

```
# CONCEPTUAL SAMPLE – illustrative savedsearches.conf stanza, not a captured real correlation search
[Excessive Failed Logins by Host - Rule]
cron_schedule = */5 * * * *
schedule_priority = highest
schedule_window = auto
dispatch.earliest_time = -10m
dispatch.latest_time = now
```

[DETECTION ENGINEER]

The reason this matters past pure platform administration: a correlation search that implements a security-critical analytic and a dashboard's background refresh query are not equally important if both are competing for the last open concurrency slot, and the platform has no way to know that on its own — `schedule_priority` is the mechanism by which a detection engineer tells it. Part 14 covers correlation-search anatomy in depth; the point to carry forward from here is narrower: a correlation search's `.conf` stanza is not just "what it searches for," it's also "how hard it fights for a slot when the search head is under load," and leaving every correlation search at default priority means the scheduler breaks ties by whatever internal ordering it uses, not by which detection actually matters most to the SOC running it.

[PLATFORM ENGINEER]

`schedule_window`'s `auto` setting exists specifically to reduce the "everything scheduled at the top of the hour" contention spike that a large content library produces when every correlation search's cron expression independently lands on `0 * * * *` or similar round numbers — letting the scheduler spread dispatch times across a tolerance window instead of forcing every search through the exact same sixty-second slice cuts the peak concurrency demand without changing what any individual search actually does.

4. The Search Job Inspector: reading the diagnostic Splunk already gives you

[PLATFORM ENGINEER]

The Search Job Inspector is a built-in, long-standing diagnostic view attached to every search job Splunk runs — reached from the small inspector icon next to a completed or running search in the Search app — that breaks a search's actual execution down by pipeline component rather than reporting only a final duration. Where DEH Part 26 §1.1's cost model describes, in the abstract, that cost accumulates left to right through a pipeline, the Job Inspector is the tool that shows you the real, measured number for each stage of one specific run: how much wall-clock time was spent in the initial `search/index-scan` phase versus in `eval`, `stats`, `transaction`, or any other command in the pipeline, alongside how many raw events were scanned versus how many results were actually returned.

Job Inspector section	What it shows	What it's diagnostic for
Execution costs	Per-command duration and invocation count across the pipeline	Which specific pipe stage — not "the search" generically — is where time is actually going
Search job properties	<code>scanCount</code> , <code>eventCount</code> , <code>resultCount</code> , whether the job ran as real-time	Whether a slow search is slow because it scanned too much raw data, not because a downstream command is inefficient
<code>search.log</code>	Component-level log lines for the job's own execution, including optimization decisions the search processor made	Whether an expected optimization (index-time filtering, a bloom-filter skip) actually applied, or the search silently fell back to a full scan
Search type / distribution	Whether the job ran as a <code>streaming</code> , <code>reporting</code> , or non-streamable search, and across how many indexers	Whether a search that "should" scale well got demoted to a non-distributable pattern by a specific command choice

[DETECTION ENGINEER] For a correlation search specifically, the Job Inspector is the first place to look before assuming a detection is "just slow" — a search whose execution costs show the overwhelming majority of time in the initial scan phase, with `scanCount` far larger than `resultCount`, is telling you the filter (the part of the search DEH Part 26 §1.1 says should run first and cheapest) isn't narrow enough, not that the aggregation logic downstream needs work. Optimizing a `stats` clause when the real cost is an unfiltered index scan fixes nothing measurable.

Detection Test What to check, against your own instance: open the Job Inspector for a specific scheduled correlation search's most recent completed run (Job Inspector is available from the search's job history, not only from an ad hoc search you run yourself). **What to look at:** the execution-costs breakdown and the `scanCount`-to-`resultCount` ratio. **What would confirm a filtering problem rather than an aggregation problem:** `scanCount` in the millions against a `resultCount` in the dozens, with the largest single execution-cost line item attributed to the base search/scan phase rather than to `stats`, `eval`, or `transaction`. This has not been run against a real instance for this book — no Splunk deployment exists in this book's evidence base (STYLE-GUIDE.md §9.2) — so treat the procedure as a documented starting point to run yourself, not a result reported here.

5. Workload Management: resource pools instead of hope

[PLATFORM ENGINEER] Concurrency limits and scheduling priority both govern *whether* and *when* a search runs; they say nothing about how much CPU and memory it's allowed to consume once it's running, which means a single badly-written search that gets a slot can still starve every other process on the instance for the duration of its run. Splunk Enterprise's Workload Management feature addresses that gap directly: a policy-based mechanism for reserving system resources — CPU and memory — for categories of work (ingestion versus search, or finer-grained pools an administrator defines) rather than letting every process compete for the same undifferentiated resource pool.

Product Version Note *Splunk Enterprise's Workload Management feature — a policy-based mechanism for reserving system resources (CPU and memory) for ingestion versus search workloads — is real, documented platform behavior, but this claim's exact wording could not be re-verified against a reachable, citable source while fixing this section: docs.splunk.com returned no accessible page during drafting (2026-09-15), and no www.splunk.com product page could be located stating it in those terms either, so no [REFERENCES.md](#) entry backs a direct quote here and none is claimed. What this book also could not independently verify, for the same reason: the specific operating-system requirement (Workload Management has historically been documented as a Linux-only capability enforced via Linux cgroups, with no Windows equivalent) and whether it applies identically, partially, or not at all to Splunk Cloud Platform versus self-managed Splunk Enterprise. Confirm the feature's current existence, scope, and OS requirement against current documentation for your deployment before assuming Workload Management is available to you at all.*

[SOC MANAGEMENT] The management-facing framing of this feature: Workload Management is a licensing- and platform-tier decision as much as a technical one. If a SOC's Splunk instance runs on Splunk Cloud Platform, resource-pool administration may sit largely or entirely with the vendor rather than the customer, in which case the actual lever available to a SOC lead frustrated by search contention is content hygiene (fewer, cheaper, better-prioritized searches per \$1–\$3) rather than a workload-pool configuration change nobody on the customer side can make. Knowing which side of that line your deployment sits on — before promising a SOC that "we'll just carve out a resource pool for detections" — is worth confirming against your specific contract and platform, not assumed from Splunk Enterprise's on-prem documentation.

6. Search head clustering: scheduling doesn't stay on one box

[PLATFORM ENGINEER] Part 2 covers indexer/search-head clustering topology in full; the piece relevant here is narrower — on a search head cluster (SHC), the concurrency ceiling and scheduling decisions from §1–§3 aren't necessarily evaluated against one machine's resources in isolation. A SHC has a single elected captain member that coordinates cluster-wide scheduling state, and Splunk's scheduler has, across the product's history, moved toward distributing the actual execution of scheduled searches across multiple cluster members rather than concentrating every scheduled run on the captain alone.

Product Version Note *Distributing scheduled-search execution across SHC members rather than running every scheduled search on the captain is real, documented Splunk behavior, but exactly which release introduced it as default and how much administrative configuration is required to enable or tune it are details this book could not verify against current documentation — docs.splunk.com was unreachable during drafting (2026-09-15), and no alternate source in [REFERENCES.md](#) covers SHC scheduler internals at this depth. If you administer a search head cluster, confirm your specific version's default scheduling-distribution behavior directly against the Distributed Search Manual before assuming load is spread the way a newer cluster's defaults might spread it.*

[PLATFORM ENGINEER] The operational upshot regardless of the exact version boundary: a skipped search on a clustered search head is not necessarily explained by "this one member is busy" — the concurrency ceiling and priority contention described in §1–§3 apply at the scope the scheduler is actually distributing work across, and diagnosing a skip pattern on a SHC without accounting for that scope is a common way to chase the wrong member's resource graphs.

7. Real-time search: the cost that doesn't amortize

[PLATFORM ENGINEER] Every search discussed so far dispatches, runs for a bounded duration, and releases its resources. A real-time search is structurally different: it opens and holds a continuous pipeline against incoming data for as long as the search stays open, which for an "all time" real-time dashboard panel can mean indefinitely. That difference matters for exactly the concurrency and workload concerns this part has built up — a real-time search doesn't compete for one slot and free it; it occupies a slot, and a meaningful share of the resources behind it, for the entire time a dashboard tab with that panel stays open, which is a materially different and larger claim on the same fixed concurrency and workload budget than a five-minute scheduled search that runs, finishes, and gets out of the way.

[PLATFORM ENGINEER] `limits.conf`'s `[realtime]` stanza carries a setting that disables real-time search collection instance-wide, and Splunk's own guidance across multiple product generations has steered administrators toward scheduled searches on a short interval (near-real-time, effectively) rather than genuine real-time search wherever the use case tolerates even a small delay, precisely because of the sustained resource cost above. A SOC dashboard panel that "needs" real-time freshness almost always tolerates a 30-to-60-second staleness window in practice; a panel scheduled every 30 seconds instead of run real-time gets most of the perceived freshness at a fraction of the sustained cost.

Engineering Reality *"We need real-time visibility" is a requirements statement analysts make constantly and rarely mean literally. Before enabling a real-time search or dashboard panel, ask what staleness window the use case actually tolerates — an analyst glancing at a dashboard once every few minutes gets no perceptible benefit from a panel that's technically live to the second, and the platform pays the sustained-resource cost of that liveness the entire time the panel is open regardless of how often anyone is actually looking at it.*

8. Role-based concurrency quotas: the governance angle nobody budgets for

[PLATFORM ENGINEER] Everything above governs instance-wide and search-specific contention.

`authorize.conf` (the `.conf` file defining Splunk's role-based access model, inline-coded per convention) separately caps how many concurrent search jobs a given role's users can hold open at once, through settings scoped to historical searches and, separately, to real-time searches — a per-role quota, distinct from the system-wide ceiling §1 describes, that governs one role's *share* of that ceiling rather than the ceiling itself.

[SOC MANAGEMENT] The failure mode this produces has nothing to do with detection logic and everything to do with role design: a threat hunter running several exploratory ad hoc searches at once, or a dashboard that opens more panels than a role's quota permits, can hit a per-role concurrency quota and get jobs silently queued or rejected — a governance decision made when the role was provisioned, surfacing months later as "search is broken" tickets that have nothing to do with the query itself. A SOC lead who owns role design should treat per-role search quotas as a capacity-planning decision tied to how each role actually uses Splunk, not a security-hardening default left wherever the platform installer set it.

The table below summarizes where to look first for each of this part's named contention points, as a triage starting point rather than an exhaustive diagnostic tree.

Symptom	Likely contention point	Where to check first
Correlation search shows zero notables, but the underlying query returns results when run ad hoc	Skipped scheduled run (§2)	<code>_internal</code> , <code>sourcetype=scheduler</code> , filtered to the search's name; Monitoring Console Scheduler Activity
One correlation search consistently slow, others on the same schedule are fine	Inefficient pipeline stage within that specific search	Search Job Inspector execution costs (§4), <code>scanCount</code> -to- <code>resultCount</code> ratio
Everything scheduled near the top of the hour degrades together, then recovers	Concurrency-ceiling contention from clustered cron times (§1, §3)	<code>schedule_window = auto</code> candidacy; stagger <code>cron_schedule</code> values across affected searches
Dashboard panel is fine at first, then the whole search head degrades the longer it's open	Real-time search holding resources indefinitely (§7)	Whether the panel is genuinely real-time vs. a short-interval scheduled search
A specific team's searches queue or fail, others' don't	Per-role concurrency quota (§8)	<code>authorize.conf</code> role definition for <code>srchJobsQuota</code> /real-time equivalent
Search head cluster member's resource graphs look idle while searches still skip	Scheduling distributed to a different cluster member (§6)	Cluster-wide scheduler state via the captain, not one member's local view

Detection Autopsy — "the correlation-search fleet that ate its own concurrency budget"

The scenario below is a composite, illustrative pattern consistent with widely-discussed Splunk scheduling behavior — cron expressions clustering at round-number times causing a concurrency spike — not a specific cited incident; no single documented case was retrieved and verified against a source in `REFERENCES.md` for this box, and none is presented as one.

The setup: a content team deploys forty correlation searches over several months, each authored independently, each defaulting to an hourly cron schedule with no explicit minute offset — every one of them lands on `0 * * * *`. **Why it shipped this way:** each search was tested individually, ran fine in isolation, and nobody reviewing one new correlation search at a time had visibility into what the other thirty-nine were already scheduled to do at the same minute mark. **How it failed:** at the top of every hour, all forty searches attempt to dispatch simultaneously, exceeding the search head's concurrency ceiling (§1); the lowest-`schedule_priority` searches in the batch get skipped, and because no one search fails loudly, the pattern surfaces only as an intermittent, hard-to-reproduce gap in a handful of detections' notable volume — worse at the top of the hour, invisible the rest of the time. **The fix:** stagger `cron_schedule` offsets across the content library (or set `schedule_window = auto` per §3 and let the scheduler do it), and add a Monitoring Console check for skip-rate spikes tied to the top of the hour specifically, rather than treating "detections are working" and "detections' scheduled runs aren't being skipped" as the same claim.

What Would Change My Mind Everything in this part about how much headroom a given concurrency ceiling, workload pool, or SHC scheduling policy actually provides under real production load is a documented mechanism, not a measured number — this book has no Splunk deployment to instrument. A real Splunk environment, running a realistic correlation-search content library at realistic ingest volume, with Monitoring Console skip-rate and Job Inspector execution-cost data collected over a real retention window (weeks, not a single synthetic burst), would be the specific evidence needed to turn this part's mechanism-level claims into measured guidance — how many correlation searches a given search-head sizing can actually sustain before skip rate crosses some tolerable threshold is exactly the kind of number that only exists once someone runs it and watches.

Cross-references: DEH Part 26 §1.1 (SPL pipeline cost model — "cost is paid left to right" — extended here from single-query to fleet-scale concurrency contention) and §1.3 (scope note naming `tstats`/administration as out of depth for DEH, which this book's Section D fulfills); DEH Part 23 (Query Language Strategy — analytic-vs-detection-rule vocabulary this part assumes without re-deriving); Part 2 (Splunk Deployment Architecture — search head clustering topology behind §6); Part 10 (Data Model and Report Acceleration — `tstats` mechanics and accelerated-search cost, distinct from the runtime concurrency/scheduling concerns owned here); Part 13 (Enterprise Security Architecture, Editions, and the CIM Dependency — edition gating context for platform features like Workload Management); Part 14 (Correlation Searches — `schedule_priority`/`schedule_window` as lifecycle decisions for a specific detection); Part 18 (Dashboards and Visualizations — real-time vs. scheduled panel cost from §7 applied to dashboard design).

neshboy

Part 13 — Enterprise Security Architecture, Editions, and the CIM Dependency

Why this part exists

Parts 1 through 12 covered the Splunk platform itself — deployment topology, index and bucket design, licensing, the Common Information Model (CIM), data models and Pivot, lookups and macros, and acceleration and search performance at scale. None of that is Splunk Enterprise Security (ES). ES is the first premium, security-specific app this book covers, and Section E (Parts 13–17) treats it as five separate audiences rather than one mega-chapter: architecture and editions (here), correlation-search mechanics (Part 14), risk-based alerting (Part 15), analyst-facing triage in Incident Review (Part 16), and asset/identity enrichment (Part 17). This part's job is narrow and foundational: establish what ES actually is as a piece of software sitting on top of the platform, what its current commercial edition structure gates, and — the point every later part in this section leans on — why ES's own correlation searches and dashboards have no data to run against until the CIM-compliance work from Parts 5–7 has actually been done, not just scheduled.

This part does not teach SPL syntax. When a correlation search's search body comes up below, this book points to DEH Part 26 (Splunk SPL) for the pipeline mechanics — the `search / stats / where` syntax, and the search-time-vs-index-time field split DEH Part 26 §1.2 covers in depth — rather than re-deriving it. DEH Part 26's own scope note states plainly that it "does not cover Splunk administration, index/bucket lifecycle management, or `tstats`/data-model acceleration in depth... a full accelerated-search treatment belongs in a dedicated performance-engineering appendix, not here." This book is that appendix, and Section E is where the appendix turns from platform plumbing into the specific security-operations product built on top of it. Where this part needs the Sigma-versus-native-authoring vocabulary DEH Part 23 (Query Language Strategy) establishes — relevant once ES's own shipped content and DEH's canonical DET-23-01 analytic come up together in §6 below — it points there rather than re-explaining the tradeoff.

1. Enterprise Security is an app, not a separate product

[CONCEPT] The single most consequential architectural fact about Splunk Enterprise Security is one a reader coming from a different SIEM's marketing material can easily miss: ES is not a distinct system that happens to share a vendor with Splunk Enterprise. It is a Splunk app — installed onto a search head exactly the way a much smaller app would be, distributed through Splunkbase, versioned independently of Splunk Enterprise itself — that assumes the platform described in Parts 2–12 already exists underneath it and adds a security-specific layer on top. An indexer holding ES-relevant data is, from the indexer's own point of view, an ordinary indexer. It doesn't run ES software; it stores events the way it stores anything else. Everything that makes those events usable as "security operations data" — the data models, the correlation searches, the notable-event and risk-scoring machinery, the Incident Review dashboard — lives at the search layer, in the ES app itself and the several supporting apps it installs alongside it.

That has a direct practical consequence this book returns to repeatedly in Parts 14–17: nothing about installing ES changes how data arrives at an indexer or what fields it has. ES's correlation searches query CIM data models (Part 6), and a CIM data model is only as complete as the field aliasing, tagging, and eventtype work described in Part 7 has actually made it for a given sourcetype. Installing ES does not retroactively CIM-map a single log source. §3 below makes this dependency explicit because it is the one architectural fact in this part every later Section E part assumes the reader already has.

Figure 13.1 sketches the layering this section describes: the platform (Part 2) at the bottom, the CIM-compliance layer (Parts 5–7) as the connective tissue, and the ES app's own components — correlation searches, the notable and risk indexes, the Asset and Identity framework, and Incident Review — sitting on top and depending on everything underneath being in place.

```
flowchart TB
    subgraph PLATFORM["Splunk platform (Part 2)"]
        IDX["Indexers"]
        SH["Search head / search head cluster"]
        IDX --> SH
    end
    subgraph CIMLAYER["CIM compliance (Parts 5-7)"]
        TAGS["tags.conf / eventtypes.conf / field aliases"]
        DM["CIM data models: Authentication, Network Traffic, Malware, ..."]
        TAGS --> DM
    end
    subgraph ESAPP["Enterprise Security app (this part)"]
        CS["Correlation searches (Part 14)"]
        RISK["risk index (Part 15)"]
        NOTABLE["notable index (Part 16)"]
        AI["Asset and Identity framework (Part 17)"]
        IR["Incident Review (Part 16)"]
        CS --> RISK
        CS --> NOTABLE
        RISK --> NOTABLE
        AI --> CS
        NOTABLE --> IR
        RISK --> IR
    end
    SH --> DM
    DM --> CS
```

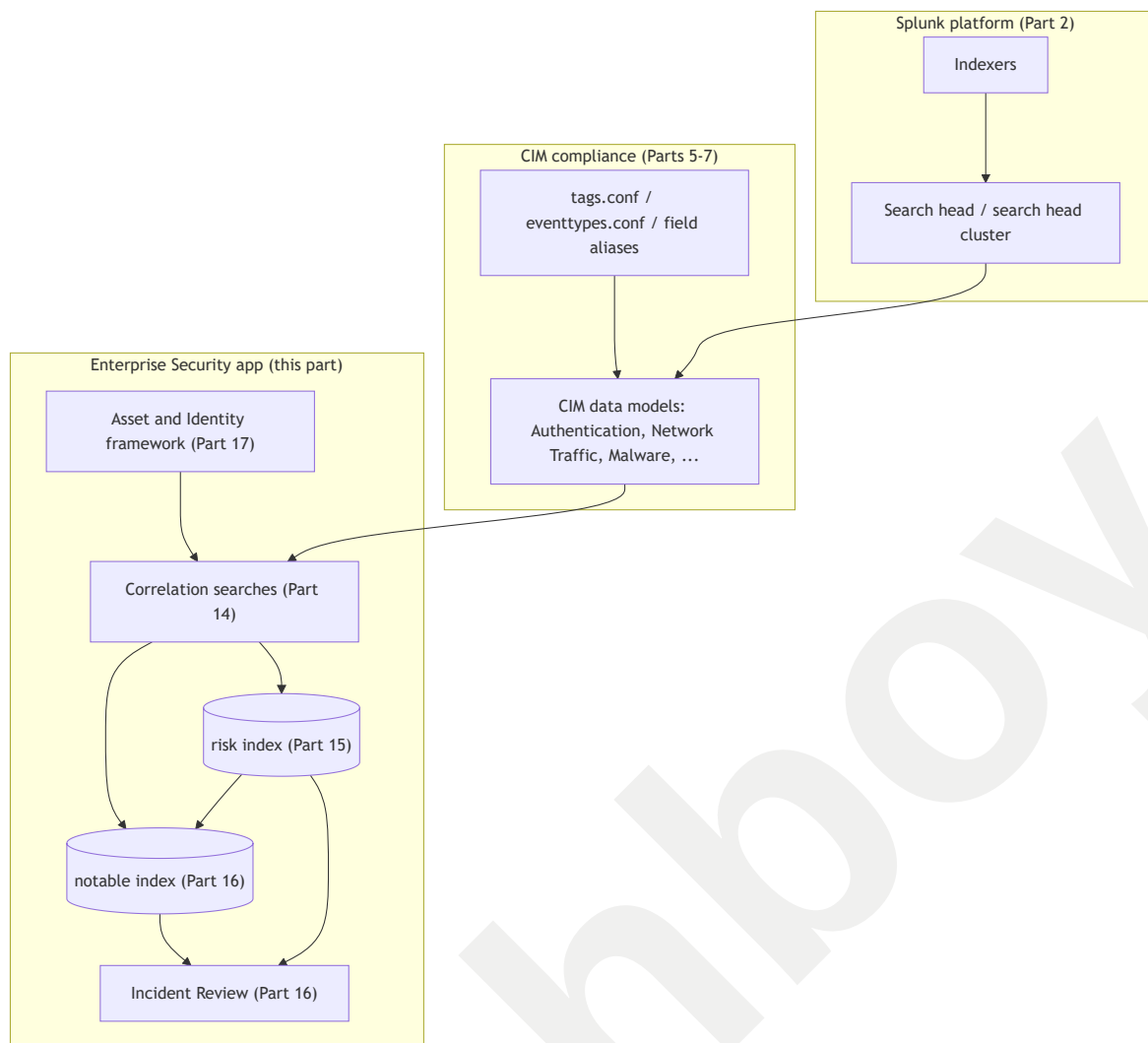


Figure 13.1 — Enterprise Security as a layer on top of the platform and the CIM. CONCEPTUAL.

Illustrates the dependency order this part argues for: the ES app's correlation searches, risk and notable indexes, Asset and Identity framework, and Incident Review view all sit downstream of CIM-compliant data models, which in turn sit downstream of the raw indexed data described in Part 2. This is an architectural sketch of documented expected structure, not a capture from a live ES instance — none exists in this book's evidence base (see STYLE-GUIDE.md §9.2).

[PLATFORM ENGINEER] Mechanically, "installing ES" means installing more than one thing. The ES app itself (distributed on Splunkbase as [Splunk Enterprise Security Suite](#)) depends on a set of supporting add-ons — a Common Information Model add-on providing the CIM's field-alias and tag definitions (Part 5 owns this in depth), and a cluster of domain-specific supporting add-ons covering access protection, endpoint protection, identity management, network protection, and threat intelligence. None of this is optional a la carte installation for a working ES deployment; the domain add-ons ship the base set of correlation searches, lookups, and macros the rest of Section E assumes exist. A reader auditing "what did installing ES actually add to this search head" should expect to find far more than one app in the list — this is one deployment unit from a licensing and support standpoint, not one file.

2. Editions: Essentials and Premier, and what each gates

[SOC MANAGEMENT]

Splunk sells Enterprise Security in more than one commercial edition, and which edition a given environment is licensed for determines which of the capabilities described in Parts 14–17 are actually available to turn on — not a matter of configuration skill, a matter of what the license permits the app to expose. The table below states the current split as this book found it; the Product Version Note immediately after it is not decorative here — this specific fact is exactly the kind of thing this callout exists to flag.

Capability area	Gated to	Notes
Core correlation searches, notable events, Incident Review	Essentials	Base tier; everything Parts 14 and 16 cover as "standard ES" assumes this tier only
Risk-based alerting, risk index, Asset and Identity framework	Essentials	Part 15 and Part 17's subject matter — not Premier-gated as of this writing
User and Entity Behavior Analytics (UEBA)	Premier	Statistical/ML-driven anomaly detection layered on top of correlation-search logic; out of this book's scope beyond naming the gate
SOAR (Security Orchestration, Automation, and Response) integration	Premier	Adaptive-response actions that hand off to a SOAR playbook, not just a notable, require Premier licensing
"Automated Threat Analysis"	Premier	Marketed AI-assisted triage/analysis capability; treat the specific feature name and scope as subject to change per the Product Version Note below

Product Version Note *Splunk Enterprise Security is currently sold in two editions — Essentials and Premier — with UEBA, SOAR integration, and "Automated Threat Analysis" gated to Premier, while core SIEM functionality, AI features, Threat Intelligence, Detection Studio, and Exposure Analytics ship in both editions. As of 2026-09-15, verified against Splunk's own Enterprise Security product page (REFERENCES.md entry [SPLUNK-ES-PRODUCT-PAGE]). This edition split is recent restructuring, not a stable, long-standing feature boundary — confirm which edition a given environment is licensed for before assuming any Premier-gated capability (UEBA, SOAR) is available, and re-verify this split before treating it as still accurate more than a few release cycles out.*

[SOC MANAGEMENT]

The practical reading for a SOC manager scoping a deployment: everything Parts 14 through 17 of this book cover — correlation searches, risk-based alerting, the risk and notable indexes, Incident Review, and the Asset and Identity framework — lives in the Essentials tier as of the date above. Nothing in this book's own Section E content is Premier-gated. That's a deliberate scoping choice on this book's part as much as a description of the license boundary: UEBA's anomaly-detection modeling and SOAR's playbook-orchestration surface are different enough disciplines (statistical baselining, orchestration-platform administration) that they earn separate treatment rather than a rushed mention here, and this book does not cover either in depth. Treat their absence from Parts 14–17 as a scoping decision, not an oversight — if your environment is Premier-licensed and actively using UEBA or SOAR playbooks triggered from ES notables, this book's coverage of your actual operational surface is incomplete by design.

3. The CIM dependency: a hard dependency, not a best practice

[PLATFORM ENGINEER] Part 7 frames CIM compliance as a maintained, testable property of a data source — something that can silently go stale after a TA upgrade the same way any field mapping can. This part's contribution is narrower and more blunt: for Enterprise Security specifically, CIM compliance is not a best practice a team can defer while getting other things working first. ES's own correlation searches, by design, query CIM data models — the Authentication data model (the CIM data model covering login/logoff events across log sources, introduced in Part 6), the Network Traffic data model, the Malware data model, and the rest of the CIM's standard set — not raw sourcetypes directly. A correlation search written against the Authentication data model returns nothing for a log source that hasn't been tagged into that data model, and it returns nothing the same way an empty result set from any other cause looks: no error, no warning badge, just zero matches.

That "not a best practice" framing has a specific consequence for how a new deployment should be sequenced, which is why this part sits ahead of Parts 14–17 rather than after them: standing up ES's correlation searches and dashboards before the underlying sourcetypes are CIM-mapped produces a deployment that looks complete — the app is installed, the searches are scheduled, the dashboards render — and generates almost no notable events, which is very easy to misread as "this environment is unusually quiet" rather than "this environment's data isn't reaching the data models these searches actually query."

Detection Autopsy — the Enterprise Security dashboard that shipped empty

The rule: A newly onboarded firewall log source, ingested and confirmed searchable at `index=network sourcetype=vendor_fw`, with several of ES's out-of-the-box Network Traffic correlation searches enabled against it on the assumption that "ingested and searchable" meant "ready for ES."

Why it shipped: The data looked complete in a raw search — source and destination IPs, ports, action fields were all visible in `_raw` and extracted as search-time fields with vendor-native names. Nobody ran the data model's own field-coverage check before enabling the correlation searches against it.

How it failed: The vendor's native field names (`src_addr`, `dst_addr`, `svc_port`) were never aliased to the CIM's Network Traffic field names (`src`, `dest`, `dest_port`) via `tags.conf` /field aliasing, so the sourcetype was never actually a member of the Network Traffic data model despite being fully ingested and fully searchable on its own terms. Every correlation search and every dashboard panel built against that data model returned zero results for this source — not an error, just silence — for months, read internally as "this firewall generates very little that matters."

The fix: A field-mapping validation pass (Part 7's CIM Add-on compliance check) run against the sourcetype before enabling any correlation search against it, plus a standing dashboard panel showing per-sourcetype event volume actually reaching each data model — so a zero-coverage source is visible as a mapping gap, not silently absorbed into "normal."

This is a commonly reported operational failure pattern among Splunk ES practitioners, not a specific cited incident this book can point to by name — no real ES deployment backs this account in this book's own evidence base (see `STYLE-GUIDE.md §9.2`). Treat it as illustrative of a documented failure class, not a captured postmortem.

[PLATFORM ENGINEER]

The corollary worth stating plainly: onboarding order matters more for ES than it does for a raw Splunk search. A team that's used to "index it, then search it" as the full onboarding lifecycle has to add a third, mandatory step before anything ES-side touches that data — map it into the CIM data model the intended correlation searches actually query, and verify that mapping against real events from that source, not just against the CIM Add-on's own sample data. Parts 5–7 own how to do that work; this part's job is only to make clear that skipping it doesn't produce a smaller, degraded version of ES — it produces an ES deployment that looks fully operational and detects nothing from the unmapped source.

4. What Enterprise Security actually installs

[CONCEPT]

Beyond the correlation-search and CIM machinery described above, ES's installation brings a specific, named set of components that Parts 14 through 17 each pick up and go deep on. This part's

job is to name them once, map each to the part that owns it, and move on — not to explain their internals here.

Component	What it is	Owning part
Correlation searches	Scheduled SPL searches against CIM data models that generate notable events or risk-score contributions	Part 14
<code>risk</code> index	The index Enterprise Security writes every risk-scoring event into when a correlation search contributes to a risk object instead of firing a notable directly	Part 15
<code>notable</code> index	The index Enterprise Security writes each notable event's metadata (status, owner, urgency, severity) into	Part 16
Incident Review	The analyst-facing dashboard for working notable events, filtering by urgency/severity/status/owner	Part 16
Asset and Identity framework	Enrichment of notables and risk objects with asset criticality and identity context from maintained lookup tables	Part 17
Threat intelligence framework	Ingestion and matching of threat-intel indicators (IPs, domains, hashes) against event data	Not covered in depth in this book — named here for completeness

`notable` and `risk` — the two indexes every Enterprise Security install writes to

[PLATFORM ENGINEER] Two of the table's rows deserve a slightly longer first mention because Parts 15 and 16 both assume the reader already knows they're separate, retained datasets rather than an abstraction that only exists inside a dashboard. The `notable` index (Enterprise Security's own index for notable-event metadata) is where a traditional, per-match correlation search's output actually lives once it fires — a notable event is a real, retained, searchable record, not a transient UI state. The `risk` index (Enterprise Security's own index for risk-scoring events) is the newer of the two conceptually, introduced alongside risk-based alerting, and it retains every individual risk-score contribution against a risk object — a user, a host, or an IP — whether or not that contribution ever crosses the threshold that generates a notable. Part 15 covers why that distinction matters operationally: an analyst investigating a notable generated by risk-based alerting is looking at an aggregated threshold crossing with a trail of individual `risk` index events behind it, not a single self-contained record the way a traditional notable is.

Engineering Reality Both indexes are ordinary Splunk indexes from a licensing and retention standpoint — they consume license-metered ingest the same as any other index-writing process, and their retention is governed by the same hot/warm/cold/frozen bucket lifecycle Part 3 covers for any other index, not by some ES-internal exemption. A retention policy set too short on the `notable` or `risk` index ages out investigation history exactly the way an aggressively short retention policy ages out any other index's data — this is a decision worth making deliberately during ES sizing, not one to discover during an audit that needs six-month-old notable history that was never kept.

5. Dedicating a search head to Enterprise Security

[PLATFORM ENGINEER] Splunk's own deployment guidance for ES has been consistent on one point across releases even as other details have shifted: ES is meant to run on a search head, or search head cluster, dedicated to it — not co-located with IT Service Intelligence (ITSI) or other premium apps on the same search head, and not shoehorned onto a general-purpose search head that's also serving unrelated ad hoc search traffic. Two separate, concrete reasons drive this, both of which matter more as an environment scales past a pilot deployment:

- **Resource contention.** ES's own scheduled correlation searches (Part 14) run continuously against the same search-head resources Part 12 covers for general search performance and concurrency limits. A search head also serving ITSI's own scheduled searches, or a large population of unrelated ad hoc analyst queries, competes with ES's correlation-search schedule for the same concurrent-search slots — and a skipped correlation search is a silent detection gap in exactly the way Part 12 already treats a skipped search generally, just with security-specific consequences.
- **App-version compatibility.** ES, ITSI, and other premium apps each certify against specific ranges of the underlying Splunk Enterprise/Cloud Platform version and specific versions of shared dependencies like the CIM Add-on. Running more than one premium app on the same search head means every version bump has to satisfy every app's compatibility matrix simultaneously, not just one — a maintenance-window constraint that compounds with every additional app sharing the box.

[PLATFORM ENGINEER] Neither of these is a hard technical prohibition — nothing stops an administrator from installing ES alongside other apps on a shared search head, and small pilot or lab deployments routinely do exactly that. The distinction is between "will run" and "is Splunk's own recommended production pattern," and a team scoping a real deployment should plan for a dedicated search head or search head cluster as the default, not the exception, budgeting the additional infrastructure into the sizing conversation Part 4's licensing content already frames as a cost lever a security team controls.

6. The content authoring surface: ES Content Update and the correlation-search editor

[DETECTION ENGINEER] ES does not ship as a blank slate expecting every correlation search to be hand-authored from zero. A separate Splunkbase app — commonly referred to by its short name, ESCU (Enterprise Security Content Update) — ships and maintains a large, versioned library of pre-built correlation searches mapped to CIM data models and, where applicable, to MITRE ATT&CK techniques. Part 14 covers the content-lifecycle mechanics this implies in depth: versioning correlation searches as code, promoting content across environments, and treating an ESCU update the way any other maintained detection-content release is treated rather than as a one-time import. This part's job is only to name that ESCU exists as the primary distribution channel for pre-built content, distinct from an organization's own custom correlation searches, and distinct again from Sigma-authored content translated into SPL — DEH Part 23's Sigma-versus-native-authoring framing applies directly here: an ESCU-shipped search is native-authored and Splunk-maintained, while a Sigma rule like DET-23-01 (Part

23 §1's canonical suspicious-LSASS-access analytic, mapped to T1003.001 (OS Credential Dumping: LSASS Memory)) translated into an SPL correlation search is a different provenance entirely, and Part 14 keeps that provenance distinction explicit when it turns DET-23-01 into a scheduled ES correlation search using DEH Part 26's DET-26-01 SPL rendering as the literal search body.

Product Version Note *Splunk has, within the current release cycle, introduced a newer correlation-search and detection-content authoring surface branded "Detection Studio," positioned alongside — and in some released documentation, as a successor to — the classic Content Management page for building and editing correlation searches. As of 2026-09-15, verified against Splunk's Enterprise Security product page (REFERENCES.md entry [SPLUNK-ES-PRODUCT-PAGE]). What would make this stale: a further rename, a full replacement of the classic editor rather than a parallel surface, or a change in which edition (Essentials vs. Premier) either surface ships under — confirm the current authoring surface's name and scope against your own installed ES version before writing internal documentation that assumes either name is permanent.*

[DETECTION ENGINEER] For this part's purposes, the naming churn above is a symptom worth flagging rather than a detail worth chasing: whatever the authoring UI is called in a given release, the underlying artifact it produces is unchanged — a `savedsearches.conf` stanza with a search body, a schedule, throttling settings, and adaptive-response actions, which is exactly what Part 14 teaches from the ground up regardless of which editor produced it. Part 19 picks this same naming-churn theme back up from the investigation-workflow angle, where it matters more, because an analyst's actual pivot path through ES's tooling is more sensitive to a renamed or relocated feature than a detection engineer's `.conf`-level view of the same underlying object.

7. What this part hasn't verified

[SOC MANAGEMENT] Everything in this part is sourced from Splunk's own public product pages and Splunkbase listings, not from a running ES instance — consistent with STYLE-GUIDE.md §9's structural evidence-class default for this book, and worth restating plainly here because Section E's remaining four parts all build on this part's architectural claims without re-verifying them independently. The edition split in §2, the dedicated-search-head guidance in §5, and the Detection Studio naming in §6 are all the kind of claim this book's Product Version Note exists to flag, not settled fact immune to the next release cycle.

What Would Change My Mind *This part's claim that CIM-compliance failure is silent and low-signal (§3) rests on how ES's data-model-driven correlation searches behave when a data model has zero matching events — reasoned from Splunk's own documented search behavior, not observed against a real ES deployment. If a real Enterprise Security instance, actually stood up and instrumented, showed that an unmapped sourcetype produces some visible warning signal this book hasn't found in Splunk's own documentation — a health-check panel, a Monitoring Console alert specific to zero-coverage data models — this part's framing of that failure mode as purely silent would need to soften accordingly. A real Splunk lab, observed over a real onboarding cycle, is the specific missing evidence that would move this from documented-behavior inference to confirmed operational fact.*

Cross-references: DEH Part 23 §1/§6 (Query Language Strategy, DET-23-01) · DEH Part 26 (Splunk SPL, DET-26-01) · Part 4 (licensing and ingest economics) · Parts 5–7 (the Common Information Model) · Part 10 (data model acceleration) · Part 12 (search performance and concurrency limits) · Parts 14–17 (correlation searches, risk-based alerting, notable events and Incident Review, asset and identity correlation) · Part 19 (Splunk-specific investigation workflows) · Part 20 (validation gaps and the path to real evidence).

Part 14 — Correlation Searches: From Analytic to Notable Event, and Their Lifecycle

Why this part exists

[CONCEPT] A correlation search is the object that turns a written-down piece of detection logic into something Enterprise Security acts on: a scheduled search, wrapped in Enterprise-Security-specific configuration, that inspects its own results and — on a match — can create a notable event, annotate a risk object, fire a Splunk SOAR playbook, or all three. Nothing about the SPL inside it is special. What makes it a correlation search rather than an ordinary saved search or alert is entirely the wrapper: the schedule, the throttle, and the adaptive response actions attached to it. This part is about that wrapper, and about what happens to it after someone first saves it.

DEH Part 23 §1 defined this book's running LSASS-access analytic, DET-23-01, at the platform-independent layer: a process outside a maintained allowlist opening a handle to `lsass.exe` (the Local Security Authority Subsystem Service, the Windows process holding credential material in memory) with memory-read rights. DEH Part 26 §5 carried that into Splunk as DET-26-01, a working SPL search against Sysmon Event ID 10 telemetry. Neither part teaches this book anything about what happens once that SPL body needs to run on a schedule, suppress its own noise, and generate something an analyst actually sees — that gap is this part's subject, and §4 below takes DET-26-01's own query, unmodified, and wraps it into exactly that.

This part does not re-teach SPL syntax (DEH Part 26 owns that), does not re-teach how a risk score accumulates toward a notable threshold (this book's own Part 15 owns risk-based alerting mechanics), does not re-teach how an analyst works a notable in Incident Review (Part 16), and does not re-derive the Enterprise Security edition split or CIM dependency in general (Part 13 owns ES's own architecture; Parts 5–7 own CIM). What it does own: the correlation-search object itself — its anatomy, its scheduling and throttling behavior, the adaptive response framework that fires on a match, and the content-lifecycle discipline of building, versioning, and promoting one across environments, extending DEH Part 22's detection-as-code discipline to Splunk's own packaging mechanism.

1. Correlation search, saved search, alert: three objects that look alike

[CONCEPT] Every correlation search is a saved search under the hood — Splunk's own configuration model doesn't have a separate object type for it. What distinguishes the three is which optional pieces of `savedsearches.conf` configuration are turned on, and whether Enterprise Security is installed to interpret them at all.

A plain saved search just stores a query and, optionally, a schedule; it does nothing on its own beyond making results available when someone opens it or a dashboard panel runs it. A standard alert adds an alert action — email, a webhook, a script — that fires when the search's results satisfy a trigger condition, entirely independent of Enterprise Security; alerting is a core Splunk platform feature, not an ES one. A correlation search is a saved search with `action.correlationsearch.enabled` set and, almost

always, at least one Enterprise-Security-specific adaptive response action attached (§3) — and it only behaves as one if Enterprise Security is installed on the search head evaluating it, since the app supplies the code that interprets `action.correlationsearch.*` settings and renders the result in its own Content Management and Incident Review interfaces.

The table below is the decision point this section supports: which object to reach for depends on whether the output needs to become an Enterprise-Security-tracked security event or just needs to notify someone.

Object Type	Where Defined	Fires On	Typical Output	Requires ES?
Saved search / report	<code>savedsearches.conf</code> , no alert action	Manual run or schedule	A results table, a dashboard panel refresh	No
Standard alert	<code>savedsearches.conf</code> , <code>alert.*</code> + a non-ES <code>action.*</code>	Schedule or real-time trigger	Email, webhook, scripted action	No
Correlation search (notable)	<code>savedsearches.conf</code> , <code>action.correlationsearch.enabled=1</code> + <code>action.notable</code>	Schedule, evaluated against a match condition	A notable event in Incident Review	Yes
Correlation search (RBA)	<code>savedsearches.conf</code> , <code>action.correlationsearch.enabled=1</code> + <code>action.risk</code>	Schedule, evaluated per matching entity	A risk-score annotation on a risk object, in the risk index	Yes

[PLATFORM ENGINEER] The last two rows are not mutually exclusive on a single correlation search — a search can carry both `action.notable` and `action.risk` at once, generating a notable and annotating a risk object from the same match in the same run. Part 15 covers what happens to that risk annotation afterward (the risk index, the risk-score accumulation, and threshold-based notable generation for RBA specifically); this part's concern stops at the fact that both adaptive response actions exist and can be attached side by side.

2. The anatomy of a correlation search

`savedsearches.conf` — where a correlation search actually lives on disk

[PLATFORM ENGINEER] A correlation search's title, query, schedule, and every adaptive response action attached to it are all one stanza in `savedsearches.conf` (the Splunk configuration file, part of an app's `default/` or `local/` directory, that stores every saved search the platform knows about). The Enterprise Security web UI's correlation-search editor is a form over this file — every field it exposes is a key in the underlying stanza, and nothing the UI does is reachable any other way than by writing the equivalent key-value pairs directly.

```
# CONCEPTUAL SAMPLE – illustrative stanza shape, not reproduced from a specific ES instance
# (none exists in this book's evidence base). Confirm exact parameter names against your
# own ES version's Content Management > Edit page before treating any key below as final.
[Example Correlation Search]
disabled = 0
cron_schedule = */5 * * * *
enableSched = 1
dispatch.earliest_time = -10m
dispatch.latest_time = now
action.correlationsearch.enabled = 1
action.correlationsearch.label = Example Correlation Search
alert.suppress = 1
alert.suppress.period = 1h
alert.suppress.fields = host
action.notable = 1
```

This shape is what every correlation search in this part builds on — the SPL search string itself sits above the stanza, unchanged from however it was authored, and everything below `[Example Correlation Search]` is Enterprise-Security-specific wrapper. §4 below fills this same stanza shape in with DET-26-01's real query and a real set of parameters.

2.1 Scheduling and the dispatch window

[PLATFORM ENGINEER] `cron_schedule` sets how often the search runs, in standard cron syntax; `dispatch.earliest_time/dispatch.latest_time` set the lookback window each run actually searches, independent of how often it runs. The two are easy to conflate and don't have to match — a search scheduled every five minutes (`*/5 * * * *`) with a ten-minute lookback (`dispatch.earliest_time = -10m`) deliberately overlaps its own previous run's window, which is the standard way to guard against a run that dispatches late or a source that ingests with lag, at the cost of re-evaluating some events the previous run already saw.

Engineering Reality *An overlapping lookback doesn't duplicate a match by itself, because `action.notable` and `action.risk` don't track "have I already alerted on this specific event" across separate runs — that tracking is exactly what `alert.suppress` (below) does, and only within the field combination you throttle on. A ten-minute lookback on a five-minute schedule with no throttle configured re-fires a notable for the same underlying event on every run that still sees it in its window — which, for a five-minute overlap, is one extra notable per match, not an unbounded pile-up, but still one more than most reviewers expect the first time they see it happen. Set a throttle window at least as long as the overlap, or Part 12's scheduling-priority guidance starts fighting a correlation search that's quietly running twice as often as its analyst-facing behavior suggests.*

alert.suppress — throttling a correlation search without touching its logic

[DETECTION ENGINEER] `alert.suppress` (paired with `alert.suppress.period` and `alert.suppress.fields`, all in the same `savedsearches.conf` stanza) suppresses the `alert action` — the notable creation, the risk

annotation, the SOAR playbook trigger — for a configured period, keyed on the field combination named in `alert.suppress.fields`, without changing what the underlying SPL matches on the next run. This is the mechanism most Splunk documentation calls throttling, and it is the direct, platform-specific analog of DEH's general Correlation-Window and Suppression concepts (see TERMINOLOGY.md, cited via DEH Part 22) applied to Enterprise Security's own alerting layer rather than to detection logic itself.

Blind Spot Throttling suppresses the notification, not the search. If the "Suspicious LSASS Memory Access - Non-Allowlisted Process" correlation search (built in full in §4) throttles on `ComputerName` alone for one hour, and a second, genuinely different `SourceImage` matches on the same host forty minutes into that window, no second notable appears — the throttle key matched, so Enterprise Security treats it as "already alerted" regardless of whether the specific process, user, or access pattern that matched is the same one that triggered the first notable. Throttling on too few fields is a silent coverage gap with the same shape as Detection Debt's translation-layer failure (DEH Part 23 §3): the search keeps matching, correctly, on every run — the alert just never reaches anyone a second time. Throttle on the narrowest field combination that still controls the noise you're actually trying to suppress, not the broadest one that's convenient to configure.

3. Adaptive Response Actions: how a match becomes a notable, a risk score, or a SOAR playbook run

[DETECTION ENGINEER] Adaptive response actions are the Enterprise-Security-specific alert actions a correlation search dispatches on a match, configured as `action.<name>` keys in the same stanza (§2) alongside `action.<name>.param.*` keys for that action's own settings. The two this book uses most are `action.notable` (creates a notable event — the row an analyst actually works in Incident Review) and `action.risk` (writes a risk-score annotation to the risk index, the summary index Enterprise Security uses to accumulate scores against a risk object independently of whether any single match produces a notable on its own). Beyond those two, Enterprise Security ships adaptive response actions for sending email, running a script, enriching against threat intelligence, and — where Splunk SOAR is licensed — dispatching a SOAR playbook directly from the match.

```
flowchart TB
    A["DET-23-01\nLSASS-access analytic\n(DEH Part 23 §1)"] --> B["DET-26-01\nSPL search body\n(DEH Part 26 §5)"]
    B --> C["Correlation-search wrapper\nschedule + throttle + adaptive response\n(this part, §2 and §4)"]
    C --> D["action.notable"]
    D --> D1["Notable event\n(Incident Review, Part 16)"]
    C --> E["action.risk"]
    E --> E1["Risk index annotation\n(risk scoring, Part 15)"]
    C --> F["action.<soar playbook>"]
    F --> F1["Splunk SOAR playbook run\n(Premier-gated, see Product Version Note)"]
    D --> G["Asset & Identity enrichment\n(Part 17)"]
    E --> G
    F --> G
```

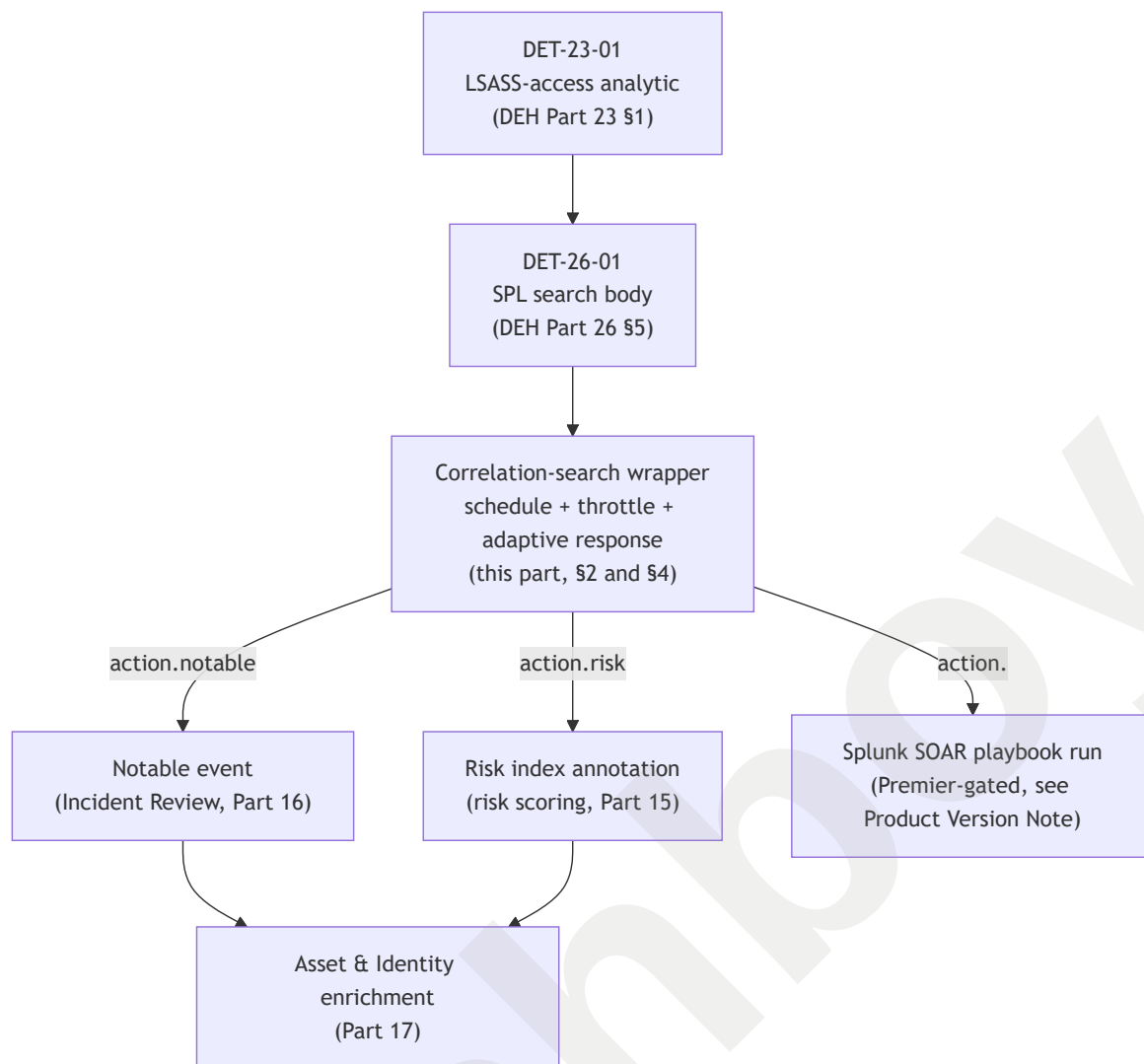


Figure 14.1 — DET-23-01 through DET-26-01 to a deployed correlation search. *CONCEPTUAL.*

Illustrates how the platform-independent LSASS-access analytic and its Splunk SPL realization are wrapped into a scheduled, throttled correlation search whose adaptive response actions fan out into this book's own downstream parts. This is a structural diagram of the book's cross-part design, not a capture of any real deployment.

Product Version Note Splunk Enterprise Security is currently sold in two editions — Essentials and Premier — with SOAR (Security Orchestration, Automation, and Response), UEBA (User and Entity Behavior Analytics), and Automated Threat Analysis gated to Premier; the SIEM core, Threat Intelligence, Detection Studio, and Exposure Analytics ship in both editions. This means a SOAR-playbook adaptive response action, specifically, depends on which edition an environment is licensed for — `action.notable` and `action.risk` do not carry the same gating and are available in both. As of 2026-09-15, verified against Splunk's own Enterprise Security product page (REFERENCES.md entry [ES-PRODUCT-PAGE]). What would make this stale: an edition restructure, a feature moving between tiers, or either edition being renamed — this split is recent enough that re-verifying it before relying on it more than a few release cycles out is the safer default.

[DETECTION ENGINEER] Which adaptive response actions to attach to a given correlation search is itself a design decision, not a default to accept uniformly across every search. A notable-only correlation search puts every match in front of an analyst regardless of how confident the underlying signal is; an RBA-annotating one (Part 15) defers that decision to accumulated risk score instead. Most correlation searches in a mature ES deployment carry `action.risk` and reserve `action.notable` for the subset of analytics where a single match is, on its own, worth an analyst's immediate attention — DET-26-01 is a reasonable candidate for exactly that treatment, which §4 below reflects.

4. DET-26-01, deployed as a scheduled correlation search

[DETECTION ENGINEER] DEH Part 26 §5 built DET-26-01 as a working SPL search against Sysmon Event ID 10 telemetry, targeting a source process outside the `lsass_access_allowlist.csv` lookup (the maintained CSV lookup this book's own Part 8 covers as detection infrastructure) opening `lsass.exe` with memory-read rights. **MITRE:** T1003.001 (OS Credential Dumping: LSASS Memory). This section wraps that same query, unchanged, into a real correlation search — DET-26-01 itself is not renumbered or redefined here; what follows is DET-26-01, deployed as a scheduled correlation search, per this book's own ID discipline (STYLE-GUIDE.md §0).

4.1 The savedsearches.conf stanza

[DETECTION ENGINEER] The query below targets the same telemetry and the same allowlist DET-26-01 already defines; its only dependency beyond what DEH Part 26 §5 already names is that the lookup and the field names resolve identically inside whichever Enterprise-Security-managed app this stanza ships in.

```

index=windows sourcetype="XmlWinEventLog:Microsoft-Windows-Sysmon/Operational" EventCode=10
TargetImage="*\\lsass.exe"
| search GrantedAccess IN ("0x1010", "0x1410", "0x1438", "0x143a", "0x1fffff")
| lookup lsass_access_allowlist.csv SourceImage OUTPUT is_allowlisted
| where isnull(is_allowlisted)
| stats count, values(GrantedAccess) as granted_access_values, earliest(_time) as first_seen,
latest(_time) as last_seen by ComputerName, SourceImage, TargetImage

```

This query's limitation is unchanged from DEH Part 26 §5 — it depends on Sysmon's Event ID 10 rule group actually logging accesses to `lsass.exe`, and an attacker who injects into an allowlisted binary inherits that binary's exemption. The correlation-search wrapper below adds a second limitation on top of it: scheduling and throttling behavior, not the underlying detection logic, decide how often and how visibly a real match actually surfaces.

```

# CONCEPTUAL SAMPLE – illustrative correlation-search stanza wrapping DET-26-01's own SPL body
# verbatim; not validated against a live Enterprise Security instance, since none exists in
# this book's evidence base. Confirm every action.notable/action.risk parameter name against
# your own ES version before deploying.
[Suspicious LSASS Memory Access - Non-Allowlisted Process]
disabled = 0
cron_schedule = */5 * * * *
enableSched = 1
dispatch.earliest_time = -10m
dispatch.latest_time = now
action.correlationsearch.enabled = 1
action.correlationsearch.label = Suspicious LSASS Memory Access - Non-Allowlisted Process
action.correlationsearch.annotations = {"mitre_attack": ["T1003.001"]}
alert.suppress = 1
alert.suppress.period = 1h
alert.suppress.fields = ComputerName,SourceImage
alert.digest_mode = 1
action.notable = 1
action.notable.param.rule_title = Suspicious LSASS Memory Access - Non-Allowlisted Process
action.notable.param.rule_description = A process outside the maintained SourceImage allowlist opened
lsass.exe with access rights consistent with memory reading.
action.notable.param.security_domain = endpoint
action.notable.param.severity = high
action.notable.param.drilldown_search = index=windows sourcetype="XmlWinEventLog:Microsoft-Windows-
Sysmon/Operational" EventCode=10 ComputerName="$ComputerName$" SourceImage="$SourceImage$"
action.risk = 1
action.risk.param._risk_object_1 = ComputerName
action.risk.param._risk_object_type_1 = system
action.risk.param._risk_score_1 = 80

```

The "Suspicious LSASS Memory Access - Non-Allowlisted Process" correlation search — the throttle key above (§4.2) is the choice actually worth scrutinizing, not the schedule or the severity mapping, both of which are ordinary defaults for a high-confidence endpoint analytic.

4.2 What breaks in production: the throttle-field mismatch

[DETECTION ENGINEER] The throttle key chosen in §4.1 — `ComputerName,SourceImage` — is a deliberate choice, not the obvious default a first pass at this stanza tends to reach for. The autopsy below shows

what an under-scoped throttle key costs.

Detection Autopsy — "throttle on the host, not the process"

This box is a conceptual illustration built from the documented mechanics of `alert.suppress.fields` (§2, above) rather than a cited incident from a specific practitioner talk or known-issues report — no real Enterprise Security deployment backs it in this book's evidence base, and it's presented here as a synthesized, plausible failure rather than a captured one.

The rule: *The LSASS-access correlation search above, initially shipped with `alert.suppress.fields = ComputerName` alone — dropping `SourceImage` from the throttle key — to cut down on repeat notables from a single noisy host during initial tuning.*

Why it shipped: *Early testing against a single lab host produced one legitimate match and, twenty minutes later, a second notable from the same host and the same `SourceImage` — an artifact of the ten-minute overlapping lookback (§2.1) re-seeing the same event before its own suppression window had been configured yet. Narrowing the throttle key to `ComputerName` alone looked like the fix, and it did stop the duplicate.*

How it failed: *Three weeks later, a genuinely different `SourceImage` — a second, unrelated tool — accessed `lsass.exe` on the same host thirty-five minutes after the first tool's match, well inside the one-hour suppression window. Because the throttle key was `ComputerName` only, Enterprise Security treated the second access as already alerted-on and suppressed it silently. No notable, no error, no record in Incident Review that a second access had even occurred — the correlation search's own search results (visible only in its ad hoc search history, not in any notable) were the only place the second match existed at all.*

The fix: *Restoring `SourceImage` to the throttle key (`alert.suppress.fields = ComputerName,SourceImage`, as shown in §4.1) so the suppression key matches the actual dimension of duplication the ten-minute-overlap problem was really about — the same event re-seen by the same search run overlap, not a second, distinct source process on the same host. The overlap-duplicate problem (§2.1) gets solved by making the throttle period comfortably longer than the lookback overlap, not by making the throttle key coarser than the entity the analytic is actually trying to distinguish.*

4.3 Confirming this against a real instance

[DETECTION ENGINEER] The autopsy in §4.2 is a synthesized failure, not a measured one — the check below is what actually distinguishes a correctly-scoped throttle key from an over-broad one on a real search head, and it is stated as a test to run, not a result already observed here.

Detection Test This book has no live Enterprise Security deployment to run the check below against — it states what to run against your own instance to confirm the wrapper behaves as designed, not a reproducible result already observed here. **Setup:** The stanza in §4.1 deployed to a search head running Enterprise Security, with Sysmon Event ID 10 telemetry flowing into the target index and `lsass_access_allowlist.csv` populated per DEH Part 26 §5. **Action:** Trigger DET-26-01's underlying match twice from two different, non-allowlisted `SourceImage` values on the same `ComputerName`, roughly ten minutes apart — the same test action DEH Part 26 §5's own Detection Test names, run twice with different tools. **Expected result:** Two notables in Incident Review, one per distinct `SourceImage`, each carrying a risk-index annotation against the same `ComputerName` risk object — and, in the correlation search's own Job Inspector output, a run duration and result count consistent with the SPL query alone, confirming the wrapper's schedule and throttle are the only added latency, not the search logic itself. If only one notable appears, the throttle key is collapsing two distinct entities into one suppression bucket — check `alert.suppress.fields` before assuming the search itself stopped matching.

5. ES Content Update — Splunk's packaged correlation-search content, and why editing it directly is a trap

[PLATFORM ENGINEER] Splunk ships pre-built correlation searches, rather than leaving every organization to author DET-26-01-style rules from a blank stanza, as a packaged Splunkbase app: ES Content Update (commonly abbreviated ESCU), maintained by Splunk's own Threat Research Team and distributed with detections mapped to MITRE ATT&CK technique IDs, with the underlying detection definitions also published on Splunk's public `security_content` GitHub repository.

Product Version Note ES Content Update is currently at version `6.6.0`, released 2026-09-09, and describes itself as delivering "pre-packaged Security Content" that "provide[s] regular Security Content updates to help security practitioners address ongoing time-sensitive threats" — content that can generate notable and risk events directly against an Enterprise Security instance once installed. As of 2026-09-15, verified against the app's own Splunkbase listing (REFERENCES.md entry [ESCU-SPLUNKBASE]). What would make this stale: ESCU ships on a sub-quarterly cadence, faster than most of the version-specific claims in this book — treat any content count or version number named here as provisional the moment it's read, and check Splunkbase directly before citing a specific figure to anyone else.

[PLATFORM ENGINEER] The trap named in this section's own heading is specific and mechanical, not a vague caution about vendor content: ESCU's correlation searches live inside the ESCU app itself, and every ESCU upgrade replaces that app's `default/savedsearches.conf` wholesale. A change made by editing one of ESCU's own correlation searches directly — adjusting a threshold, adding a field to a `where` clause, tightening a throttle — lives in the app's `local/` directory if made through Enterprise

Security's own editor (Splunk's standard default/local layering keeps hand edits out of `default/`), which survives an ESCU upgrade in the sense that the file isn't deleted — but the search that actually runs is whichever the platform resolves after layering `local/` over the new `default/`, and a threshold tuned against last quarter's ESCU logic can silently stop matching, over-match, or point at a field ESCU's own upgrade renamed, with no upgrade-time warning that the local customization and the new default have drifted apart.

Engineering Reality *The fix Splunk's own ESCU documentation and most practitioner guidance converge on is the same one DEH Part 22 §1 names for any vendor console that allows a direct, unreviewed edit: treat ESCU's shipped content as a read-only reference, not an editing surface. Clone the specific correlation search you need to tune into your own app (a locally maintained content app, version-controlled the same way DEH Part 22 describes for any other detection), retarget it under a new title so it doesn't collide with ESCU's own stanza name, and disable the ESCU original rather than editing it in place. An upgrade then only ever touches ESCU's own app; your tuned version is a separate, git-tracked object nothing but your own deployment pipeline can change.*

6. Versioning correlation searches as code, and promoting them across environments

[PLATFORM ENGINEER] DEH Part 22 §1 makes the general case for a git-based rule repository over direct console edits: the console becomes a deployment target, not an editing surface, and the file in version control is the rule's actual source of truth. Applied to a correlation search specifically, that file is the `savedsearches.conf` stanza itself (plus any lookup or macro it depends on — Part 8 and Part 9 cover those as their own maintained artifacts) checked into the same app-based layout Splunk already expects: one custom app per logical content package, `default/savedsearches.conf` holding every correlation search that app ships, deployed to search heads the same way any other Splunk app is deployed — by an indexer/search-head-cluster deployer push, or by exporting the app as a `.tar.gz/.spl` package and installing it, never by an analyst editing the live stanza through Enterprise Security's own Content Management page in production.

[DETECTION ENGINEER] The version-bump discipline DEH Part 22 §8 sets for any Detection Rule applies directly here — a correlation search's own semantic version lives in its metadata (the same `Version` field DEH Part 22 §4's metadata table defines), bumped on every merged change to the stanza, independent of whatever build number the app package itself carries. Promoting a change across environments then follows the same three-role review gate DEH Part 22 §3 requires generically: author opens a pull request against the app repository, a reviewer confirms the changed stanza still matches the analytic's stated logic and that any threshold or throttle change is deliberate, and a merge-approver ships the resulting app package to a staging search head first — one running real (or realistically shaped) telemetry — before production.

[PLATFORM ENGINEER]

What "staging" means concretely for a correlation search is narrower than for most software: there is no unit-test framework that runs SPL against synthetic events the way DEH Part 22 §6 describes for a generic detection-testing pipeline. The realistic staging check is running the search manually against a staging search head's own indexed data, confirming the result count and field shapes look like what the Job Inspector would show in production, and — where a true-positive fixture exists (a captured, known-good matching event, the same discipline DEH Part 22 §6 requires generally) — confirming the search still matches it after the change. A correlation search that "looks right" in a code review but has never actually run against real data before reaching production is the SIEM-specific version of the untested-deploy failure mode DEH Part 22 §1 names for any detection pipeline.

The table below anchors the lifecycle states a correlation search moves through to the concrete Splunk-side action each one implies — the decision this table supports is when a given search is actually allowed to fire against production telemetry, not just what label its metadata carries.

Lifecycle State	What It Means	Concrete Splunk Action
Draft	Stanza written, not yet reviewed	<code>disabled = 1</code> in the dev app; not deployed to any shared search head
In review	Pull request open against the content-app repository	Deployed to a staging search head only, <code>disabled = 0</code> there, production untouched
Released	Reviewed, tested against staging data, merge-approved	Deployed to production via app package or deployer push, <code>disabled = 0</code> , <code>action.correlationsearch.enabled = 1</code>
Tuning	Released, under active threshold/throttle adjustment	Change flows through the same PR gate as a new search — never a live edit through Content Management
Deprecated	Superseded by a newer version or an RBA-based replacement	<code>disabled = 1</code> in production first; stanza removed from the app only after a retention period with no open notables referencing it

[SOC MANAGEMENT]

The deprecated row is where content lifecycle most often gets skipped in practice — disabling a superseded correlation search costs nothing and is easy to remember; actually removing its stanza from the app once nothing references it takes a deliberate cleanup pass nobody schedules unless the search-count-versus-concurrency-budget conversation in §7 below forces it.

7. SOC management view: content sprawl and the concurrency budget

[SOC MANAGEMENT]

§6's lifecycle table treats each correlation search as an individually managed object; at fleet scale, the more consequential decision is how many of them exist at once.

SOC Management View Every enabled correlation search, ESCU's included, competes for the same finite scheduled-search concurrency budget Part 12 covers in depth — installing ESCU and enabling its full content library without first checking which of its correlation searches assume CIM fields your own environment doesn't populate (Parts 5–7's own subject) produces two costs at once: searches that silently never match because their required fields aren't there, and searches that do run on schedule, consuming concurrency slots, for detections that structurally cannot fire in your environment. Treat "install ESCU" and "our correlation-search count went up by several hundred" as the same decision, not two separate ones — each enabled search is a recurring compute cost and a recurring maintenance obligation (§6) whether or not it was authored in-house. A content-adoption review that checks required-field coverage (§5's CIM dependency, made concrete) before enabling a new batch of ESCU content is cheaper than discovering the gap during an incident review, and it is the direct correlation-search-specific instance of the same rule-count-versus-validated-coverage distinction DEH Part 1 warns against in general.

```
sequenceDiagram
    participant Sched as Scheduler
    participant CS as Correlation search (SPL)
    participant AR as Adaptive response dispatch
    participant Notable as Notable index / Incident Review
    participant Risk as Risk index

    Sched->>CS: Dispatch on cron_schedule
    CS->>CS: Run SPL body against dispatch window
    alt Results match
        CS->>AR: Hand off matching result rows
        AR->>AR: Check alert.suppress throttle key
        alt Not suppressed
            AR->>Notable: action.notable creates notable event
            AR->>Risk: action.risk annotates risk object
        else Suppressed
            AR->>AR: Drop silently -- no notable, no error
        end
    else No results
        CS->>Sched: Run completes, nothing to dispatch
    end
    end
```

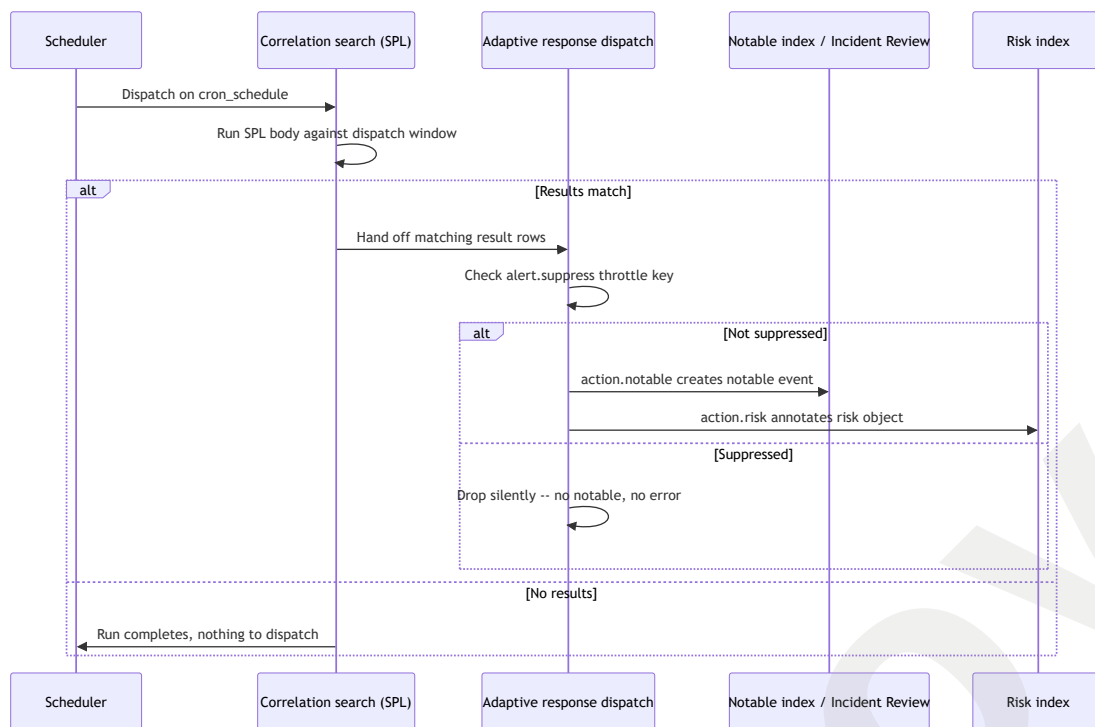


Figure 14.2 — Correlation search to notable event, sequence. *CONCEPTUAL.* Illustrates the expected sequence from a scheduled correlation search's SPL execution through the throttle check, adaptive-response dispatch, and notable-event/risk-index creation, including the silent-suppression path the `alert.suppress` section (§2) and §4.2 both name. This is a sequence diagram of documented expected behavior, not a capture from a live Splunk job inspector or a real Enterprise Security instance — none exists in this book's evidence base (see *STYLE-GUIDE.md* §9.2).

Closing: where this hands off

[CONCEPT] A notable-generating correlation search like the one built in §4 is the simplest, most direct version of turning an analytic into something an analyst sees — one match, one notable, immediately. Part 15 covers the alternative most mature Enterprise Security deployments actually converge on for high-volume analytics: annotate a risk object on every match via `action.risk` instead, and let accumulated score across many correlation searches — not any single one — decide when a notable is actually warranted. Every mechanic this part covers (the stanza, the schedule, the throttle, the ESCU-versus-custom-content lifecycle) applies identically to an RBA-oriented correlation search; only the adaptive response action attached, and what happens after it fires, changes.

Cross-references: DEH Part 22 (Detection as Code) §1, §3, §4, §6, §8; DEH Part 23 (Query Language Strategy) §1; DEH Part 26 (Splunk SPL) §5; this book's Part 5–7 (Common Information Model), Part 8 (Lookup Tables as Detection Infrastructure), Part 12 (Search Performance and Tuning at Scale), Part 13 (Enterprise Security Architecture, Editions, and the CIM Dependency), Part 15 (Risk-Based Alerting), Part 16 (Notable Events and the Incident Review Workflow), Part 17 (Asset and Identity Correlation).

Part 15 — Risk-Based Alerting: Risk Objects, Risk Scores, and the Risk Index

Why this part exists

[CONCEPT] Part 14 covers a correlation search's full anatomy: the `savedsearches.conf` stanza, its schedule, its throttling, and its adaptive response actions — including the oldest and simplest one, `action.notable`, which turns a single search result row into a single notable event. That model has a scaling problem this part exists to name and solve: a real attacker's actual kill chain rarely trips one high-confidence analytic in isolation. It trips several weak ones — an off-hours logon, a new scheduled task, a process touching `lsass.exe` from an unfamiliar path — none of which is confident enough on its own to justify a stand-alone notable without flooding the queue, but which together, on the same host or the same user in the same window, are a real signal a triage queue should surface. Risk-Based Alerting (RBA) is Splunk Enterprise Security's answer: stop generating a notable on every match, and instead let each match add a scored contribution to a **risk object** — a specific user, host, or other entity — then generate a notable only once that object's accumulated risk crosses a threshold.

This part is operations-layer, and it sits directly downstream of Part 14's correlation-search mechanics: everything RBA does still runs as a scheduled search with a `savedsearches.conf` stanza, an adaptive response action, and — for the aggregate step — the same `tstats`-driven aggregation Part 10 covers as this book's realization of the forward reference in DEH Part 26 §1.3. This part does not re-teach `tstats` syntax, `stats` aggregation functions, or the `where` command — Part 26 §§2–4 own those, and Part 10 owns `tstats` mechanics specifically; where this part shows a query, the framing sentence states what it targets and points back rather than re-deriving the command. It also does not re-teach the Analytic-vs-Detection-Rule distinction (DEH Part 23 §1) — it assumes DET-23-01, the canonical LSASS-access analytic carried through DEH Parts 24–29 and this book's own Part 14, and shows what changes when that same analytic is deployed as a risk rule instead of (or alongside) a notable-generating correlation search.

MITRE: T1003.001 (OS Credential Dumping: LSASS Memory) — DET-23-01's technique, reused here without re-derivation.

Every specific mechanism named below — field names, `.conf` stanza keys, the risk index's default name — is sourced from Splunk's own public GitHub repositories for its Enterprise Security Content Update (ESCU) package and its `contentct1` build tooling, since `docs.splunk.com` remains unreachable to this book's own research tooling (`STYLE-GUIDE.md` §9.4; `REFERENCES.md`'s sourcing note). No Splunk instance backs any of it; every artifact below is **OFFICIAL REFERENCE**, cited, or explicitly marked **CONCEPTUAL SAMPLE**.

1. One match, one notable — and why that stops scaling

[CONCEPT] A traditional correlation search (Part 14) with `action.notable = 1` does exactly one thing on a match: it writes one notable event, once, per matching result (subject to throttling). That's the right model for a rule with a low false-positive rate and a clear, single-event story — an audit log clear, a

domain-admin group modification. It's the wrong model for a large class of genuinely useful analytics that are individually weak: a login at 3 a.m. local time, a process that isn't normally seen on that host, a new outbound connection to a rarely-contacted ASN. Each of those, alone, is too common to page anyone on. Enabled as traditional per-event notables, a SOC either drowns in noise or — more commonly — never turns them on at all, which means the actual multi-step behavior they'd have caught together goes completely dark.

Risk-Based Alerting changes the unit of output. A risk-annotating correlation search — Splunk's own terminology calls it a **risk rule** — does not create a notable on a match. It appends one **risk event** to a dedicated, retained dataset (§4 below), scored and attributed to a specific risk object. A separate aggregation search then reads that accumulated history back and decides, on its own schedule, whether any one risk object's accumulated score (or some other threshold condition, §5) has crossed a line worth a human's attention. The weak individual signals never disappear from view — they're still queryable risk history — but none of them, alone, forces a notable.

Product Version Note *Splunk's own Enterprise Security product page states, in its Risk-Based Alerting feature callout, that RBA can "reduce your alert volumes by up to 90%," and its Essentials/Premier edition comparison on that same page does not list Risk-Based Alerting among the capabilities marked Premier-only (UEBA, SOAR integration, and Automated Threat Analysis are the callouts marked Premier-only there). As of 2026-09-15, verified against www.splunk.com/en_us/products/enterprise-security.html (REFERENCES.md entry [SPLUNK-ES-PRODUCT-PAGE]). What would make this stale: a revised edition comparison that moves RBA behind a paywall it isn't behind today, or a marketing refresh that drops or revises the 90% figure without a version bump to flag it — re-verify both before repeating either claim more than a few release cycles out. §6 below treats the figure itself on its own terms.*

Figure 15.1 lays out the full mechanism this part covers piece by piece: risk rules writing scored risk events into a shared dataset, an aggregate search reading that dataset back, and a notable generated only once a threshold trips.

```
flowchart LR
    subgraph RULES[Risk-annotating correlation searches - risk rules]
        R1["Suspicious LSASS access\nrisk_object_type=system\nrisk_score=50"]
        R2["Anomalous off-hours logon\nrisk_object_type=user\nrisk_score=20"]
        R3["New scheduled task on host\nrisk_object_type=system\nrisk_score=30"]
    end
    RULES -->|action.risk adaptive response action| RE["Risk event(s)\nindex=risk\nRisk.All_Risk data model"]
    RE --> AGG["Aggregate/threshold search\ntstats sum(calculated_risk_score),\ndc(source) BY risk_object,\nrisk_object_type"]
    AGG -->|sum below threshold| RE
    AGG -->|sum crosses threshold| NOT["Notable event\ntitle prefixed 'RBA: '"]
    NOT --> IR["Incident Review\nPart 16"]
    RE -->|pivot from risk_object, no threshold crossed| HUNT["Ad hoc hunt against\nthe risk index - Part 19"]
```



Figure 15.1 — Correlation search to risk event to (maybe) a notable. *CONCEPTUAL.* Illustrates the expected sequence from a risk-annotating correlation search's match, through the `action.risk` adaptive response action and the risk index, to an aggregate search's threshold decision. This is a sequence diagram of documented expected behavior, not a capture from a live Enterprise Security instance or job inspector — none exists in this book's evidence base (`STYLE-GUIDE.md §9.2`).

2. Risk objects and risk object types

[DETECTION ENGINEER]

A **risk object** is the entity a risk event is attributed to — the thing whose accumulated score the aggregate search in §5 will eventually threshold on. Every risk event carries two fields naming it: `risk_object` (the actual value — a hostname, a username) and `risk_object_type`, which is not a free-text field but one of a small, fixed set. Splunk's own published ESCU detections — the `security_content` GitHub repository behind the Enterprise Security Content Update app — use exactly three values across the risk rules they ship: `system`, `user`, and `other`.

Table 15.1 — `risk_object_type` values observed in Splunk's own published ESCU content. Use this table to decide which type a new risk rule should assign; guessing a fourth value is not supported by any example this book could verify.

<code>risk_object_type</code>	Typical <code>risk_object</code> Value	Example ESCU Detection	Notes
<code>system</code>	Hostname/asset identifier (<code>dest</code>)	"Access LSASS Memory for Dump Creation"	Host-centric; the default for endpoint-telemetry risk rules
<code>user</code>	Username or identity string	"Okta Risk Threshold Exceeded"	Identity-centric; pairs with Part 17's identity resolution
<code>other</code>	Anything not cleanly a host or a user	"Windows Post Exploitation Risk Behavior"	Catch-all — an aggregate search's own output, a cloud resource ID, or an unresolved IP

Sourced from Splunk's public `security_content` repository (`REFERENCES.md` entry `[SECURITY-CONTENT-REPO]`); §4 below covers what "ESCU" is and how Part 14 treats it as this book's content-lifecycle mechanism.

An IP address is the case worth calling out by name, since this part's own scope names it alongside user and host: it does not get its own `risk_object_type`. An IP that Part 17's asset framework has already resolved to a known host is scored as `system`, using the resolved hostname as `risk_object` rather than the raw address, so that the same physical machine's risk accumulates under one identity regardless of which telemetry source reported which representation of it. An IP with no asset match — a scanner, an unmanaged device, an external address — has nothing more specific to fall back to, and lands in `other`. That fallback is itself a signal worth watching, not a rounding error: a `risk_object_type=other` entry whose `risk_object` is a bare IP is, by construction, an asset your identity/asset framework doesn't

recognize, and a SOC that never looks at its `other`-typed risk events is quietly ignoring exactly the unmanaged-asset population most likely to be unmonitored in every other way too.

3. From a search match to a risk event

[DETECTION ENGINEER] The adaptive response action that turns a match into a risk event is `action.risk`, configured on the risk rule's own `savedsearches.conf` stanza — the same file and the same stanza-per-search model Part 14 covers for `action.notable`. Converting an existing correlation search into a risk rule does not change its search body at all; it changes what happens after a match, by adding the action described below.

`action.risk` — turning a correlation-search match into a risk event, not a notable

[DETECTION ENGINEER] A risk rule sets `action.risk = 1` and supplies two parameters: `action.risk.param._risk_message`, a template string that can reference search fields with `$field$` syntax, and `action.risk.param._risk`, a JSON array of one or more objects, each naming either a risk object (`risk_object_field`, `risk_object_type`, `risk_score`) or a threat object (`threat_object_field`, `threat_object_type` — a non-scored entity worth recording alongside the risk object, such as the specific process image involved, without itself accumulating risk).

The example below adapts this structure to DET-26-01 — the SPL realization of DET-23-01 that DEH Part 26 §5.2 builds against Sysmon Event ID 10 telemetry (one clause, not a re-derivation; see that section for the query itself).

```
# CONCEPTUAL SAMPLE – a composite stanza built from real, individually-cited ESCU parameter
# names and values (REFERENCES.md entries [SECURITY-CONTENT-REPO], [CONTENTCTL-REPO]); no
# single file in Splunk's published content uses this exact stanza name or combination, so
# this is illustrative, not a literal reproduction of one source file.
[DET-26-01 - Suspicious LSASS Access - Risk Rule]
action.risk = 1
action.risk.param._risk = [{"risk_object_field": "dest", "risk_object_type": "system", "risk_score":
50}, {"threat_object_field": "TargetImage", "threat_object_type": "process"}]
action.risk.param._risk_message = Process $SourceImage$ requested memory-read access to $TargetImage$
on $dest$, matching DET-26-01's GrantedAccess/allowlist logic
action.risk.param.verbose = 0
```

The `risk_score = 50` value above is not an invented illustration — it is the actual score Splunk's own "Access LSASS Memory for Dump Creation" ESCU detection assigns its `system`-typed risk object, entity field `dest` (REFERENCES.md entry [SECURITY-CONTENT-REPO]). This stanza's main limitation is the one DEH Part 26 §5.2 already names for DET-26-01 itself: it depends on Sysmon's Event ID 10 rule group actually logging accesses to `lsass.exe`, and on a maintained `SourceImage` allowlist to keep EDR/AV noise from dominating the risk it writes — a risk rule that fires constantly on legitimate AV scanning behavior pollutes every downstream aggregate in §5 exactly as badly as an unfiltered notable would have.

Engineering Reality "Risk-Based Alerting" is not a separate kind of correlation search with its own file format — it's `action.risk` on an ordinary `savedsearches.conf` stanza, and nothing stops the same stanza from also carrying `action.notable = 1` at the same time. Splunk's own ESCU build tooling (`contentctl`) renders both actions onto one stanza when a detection's deployment config enables both, and prefixes the resulting notable's title with `RBA:` specifically when the detection is authored as a risk-based Correlation type (`REFERENCES.md` entry `[CONTENTCTL-REPO]`). A rule carrying both actions generates a notable on every single match and contributes to the aggregate threshold in §5 — a legitimate choice for a genuinely high-confidence analytic, but one that defeats RBA's whole alert-volume argument if applied to a weak one. Decide, per rule, whether it should ever stand alone as a notable; don't assume "it's a risk rule" and "it never pages anyone directly" are the same fact.

Product Version Note The parameter names shown above (`action.risk.param._risk`, `action.risk.param._risk_message`) are drawn from Splunk's own public `contentctl` repository — the tooling that compiles ESCU's YAML-authored detections into the `.conf` stanzas Enterprise Security actually loads — not from a fetched `docs.splunk.com` admin-manual page, which returned HTTP 403 on every retrieval attempt for this book (`STYLE-GUIDE.md` §9.4). As of 2026-09-15, verified against `github.com/splunk/contentctl` (`REFERENCES.md` entry `[CONTENTCTL-REPO]`). What would make this stale: Enterprise Security renaming these underlying `.conf` keys in a future release (unlikely on the timescale of a single release, since a rename breaks every existing stanza using the old key, but not impossible), or this book confirming the exact current label Splunk Web's adaptive-response-action picker shows for the same action — informally referred to as "Risk Analysis" in Splunk's own community and partner materials, but not verified against a fetchable source here.

4. The risk index and the Risk data model

[PLATFORM ENGINEER] Every risk event `action.risk` writes lands in a dedicated index — by default, and in every one of Splunk's own published ESCU detections this book could verify, the risk index (`index=risk`). Splunk ships this as a macro rather than a hardcoded literal in its own content: the `risk_index` macro in the `security_content` repository resolves, by default, to exactly `index=risk` (`REFERENCES.md` entry `[SECURITY-CONTENT-REPO]`), and ESCU's own guidance is to override that macro's definition, not the individual searches, if an environment routes risk events somewhere else. That's the correct pattern for the same reason a hardcoded index name anywhere else in a detection is a maintenance liability — Part 8's lookup-versioning argument and Part 9's macro-sprawl argument both apply here directly. Treat `risk_index` as exactly the kind of shared macro Part 9 covers, not a one-off convenience.

The risk index is a retained dataset in its own right, subject to the same bucket-lifecycle and retention decisions Part 3 covers for any other index — this part does not re-teach hot/warm/cold/ frozen

mechanics, only names the one consequence specific to risk: retention here is a detection-coverage decision, not just a storage-cost one.

Blind Spot *The risk index's retention window and the raw source data's retention window are independent settings, and nothing forces them to match. If the risk index is retained for 90 days while the raw Sysmon/Windows Event Log data behind it is retained for a full year, an aggregate search with a 180-day lookback (§5) will silently stop seeing risk events older than 90 days even though the underlying evidence that generated them is still fully queryable — the aggregate's threshold decision quietly narrows to a shorter effective window than its own `earliest_time` implies, with no error and no obviously missing data, since the raw source events are still there for anyone who thinks to check them directly. Size the risk index's retention to at least the longest lookback window any aggregate search built on top of it actually uses, not to whatever the platform's default retention happens to be.*

Splunk builds a CIM-adjacent dataset over the risk index the same way it builds any other CIM data model (Part 6) over a CIM-tagged source — the Risk data model, referenced in SPL as `Risk.ALL_Risk` (that dataset's own path within the model). Every field named in §2 and §3 above — `risk_object`, `risk_object_type`, `calculated_risk_score` (the risk index's own name for the per-event score `action.risk.param._risk`'s `risk_score` value becomes once written), `risk_message`, and MITRE annotation fields (`annotations.mitre_attack.mitre_technique_id`, `annotations.mitre_attack.mitre_tactic_id`) — is queryable through that data model, which means the aggregate searches in §5 can use `tstats` against it exactly the way Part 10 covers for any other accelerated data model, rather than a slower `datamodel` search or a raw `index=risk` scan. Whether that acceleration is actually turned on for the Risk data model in a given environment is Part 10's question, not this part's; confirm it rather than assuming it, since an unaccelerated Risk data model still works, just at raw-search cost on every aggregate run.

Product Version Note *The risk index defaults to `index=risk`, and the data model it feeds is queryable as `Risk.ALL_Risk`, with per-event fields including `calculated_risk_score`, `risk_object`, `risk_object_type`, `risk_message`, and the `annotations.mitre_attack.*` MITRE fields. All of these names are confirmed from Splunk's own published ESCU/ `security_content` macros and detections (the `risk_index` macro's default definition and the `datamodel=Risk.ALL_Risk` pattern used in shipped risk-threshold detections), not from a fetched `docs.splunk.com` reference page (`STYLE-GUIDE.md` §9.4). As of 2026-09-15, verified against `github.com/splunk/security_content` (`REFERENCES.md` entry `[SECURITY-CONTENT-REPO]`). What would make this stale: Splunk renaming the index, the data model, or any of these fields in a future Enterprise Security release, or a given environment's own ESCU deployment overriding the `risk_index` macro to point elsewhere per this section's own guidance — confirm all of these against the actual environment before writing an aggregate search against any of them.*

5. The aggregate search: from accumulated risk to a notable

[DETECTION ENGINEER]

The second half of RBA is a search that reads the risk index back, groups by risk object, and decides whether the accumulated picture for any one object is now worth a notable. The query below adapts Splunk's own published "Windows Post Exploitation Risk Behavior" ESCU detection, trimmed for clarity ([REFERENCES.md](#) entry [\[SECURITY-CONTENT-REPO\]](#)); it targets the Risk data model directly, using the `tstats` aggregation Part 10 covers in depth rather than re-deriving it here.

```
| tstats `security_content_summariesonly`  
    sum(All_Risk.calculated_risk_score) as risk_score,  
    dc(source) as source_count,  
    values(All_Risk.annotations.mitre_attack.mitre_technique_id) as technique_ids  
    FROM datamodel=Risk.All_Risk  
    BY All_Risk.risk_object, All_Risk.risk_object_type  
| `drop_dm_object_name("All_Risk")`  
| where source_count >= 4
```

This search's main limitation is the same tradeoff every threshold carries: `source_count >= 4` rewards *diversity* of contributing analytics on one object over any single analytic's raw match count, which is a deliberate design choice, not an arbitrary one — see Table 15.2 for the alternatives and what each one actually rewards. `security_content_summariesonly` is Splunk's own convention macro restricting the `tstats` search to the accelerated summary only, skipping any risk events written since the data model's last acceleration pass; a risk event that landed in the last few minutes and hasn't yet been folded into the `tsidx` summary won't count toward the threshold on this particular run, which is a real, small, and usually acceptable staleness window rather than a bug.

Table 15.2 — Threshold strategies for an aggregate search, and what each one actually rewards.

Use this table when deciding what condition should trip a notable, not just what number to set it to.

Threshold Strategy	What It Rewards	Failure Mode to Watch For
Sum of <code>calculated_risk_score</code>	Total accumulated confidence, regardless of source diversity	A single noisy, high-volume rule can cross the threshold alone — see the False Positive Trap below
Distinct contributing analytic count (<code>dc(source)</code>)	Breadth — many <i>different</i> weak signals agreeing on one object	A genuinely severe but narrowly-observed technique may never cross a count-based bar
Distinct MITRE technique/tactic count	Kill-chain diversity specifically, independent of raw score	Two sub-techniques of one parent technique can double-count as "diverse" when they aren't

False Positive Trap A sum-of-score threshold treats ten matches from one noisy, low-confidence rule as identical to ten matches from ten different rules — repetition and diversity both add up to the same number. A single business-critical server that a low-confidence "unusual process" rule fires on dozens of times a day (a legitimate, noisy backup agent behaving in a way that rule's author didn't anticipate) can single-handedly cross a sum-based threshold with zero real diversity of evidence behind it, generating a notable that looks like a multi-signal escalation but is actually one rule complaining about itself repeatedly. The fix is either a distinct-source-count condition (Table 15.2's second row) instead of or alongside a raw score sum, or a per-source-search cap on how much any single rule can contribute to one object's total in a given window — not simply raising the sum threshold, which just delays the same failure to a slightly higher number.

6. Splunk's own alert-volume-reduction claim, examined

[SOC MANAGEMENT] §1's Product Version Note already states the number and its source plainly:

Splunk's own Enterprise Security product page claims RBA can reduce alert volume by up to 90%. It's worth being explicit about what that figure is and isn't, because it's the single most quoted RBA statistic and also the one furthest from anything this book can independently check.

It is a vendor's own marketing claim, published on a page whose purpose is to sell the product, with no methodology, no baseline-environment description, and no independent replication cited alongside it. It is plausible on its face — moving from one notable per weak match to one notable per crossed threshold mechanically reduces notable *count* by construction, for any environment running enough weak analytics to make the comparison meaningful — but "plausible" and "measured, in your environment, under your own threshold choices" are different claims, and only the second one should drive a staffing or tooling decision.

What Would Change My Mind This book cannot measure a before/after notable count for RBA adoption, because no Enterprise Security deployment exists in its evidence base ([STYLE-GUIDE.md §9.2](#)). What would move the 90% figure from "vendor claim, plausible on its face" to something this book could treat as verified: a real Enterprise Security instance, run for a comparable retention window with a fixed, unchanged set of underlying analytics, first with every analytic wired to `action.notable` directly and then re-wired to `action.risk` plus one or more aggregate threshold searches — comparing total notable count across the two configurations against the same underlying event population. Absent that, the honest statement is narrower than the marketing page's: RBA's mechanism for reducing notable count is sound and explainable (§1), and Splunk's own number for the size of that reduction is unverified here, not false — those are two different confidence levels, and collapsing them into one would overstate what this part actually knows.

Threshold design is also a management decision, not only an engineering one, in one specific way worth naming directly: every threshold in Table 15.2 trades detection latency against notable volume. A higher sum threshold or a stricter distinct-count requirement produces fewer, higher-confidence notables at the cost of taking longer (more contributing risk events) to surface a genuinely fast-moving attacker — and nobody but the SOC that owns the triage queue can decide where that tradeoff should sit for a given risk appetite. Splunk's own documentation and marketing can supply the mechanism; it cannot supply the org-specific answer to "how much detection latency is this SOC willing to accept in exchange for how much less noise."

7. Where risk-based alerting hands off

[CONCEPT] This part covered the mechanism — risk objects and their types, `action.risk` as the annotation step, the risk index and Risk data model as the retained dataset underneath it, and the aggregate search's threshold logic as the step that finally produces (or withholds) a notable. Three parts pick up directly from here. Part 14 is where a risk rule's own scheduling, throttling, and content-lifecycle management live, exactly as they would for any other correlation search — this part deliberately didn't repeat that. Part 16 is where the resulting notable actually gets worked: an RBA-generated notable arrives in Incident Review with multiple contributing risk events behind it rather than one, and triaging that shape of notable is a genuinely different task from triaging a traditional single-event one. Part 17 is where `risk_object` values that are raw usernames or unresolved IPs get enriched with asset criticality and identity context before a human ever sees them — the exact gap §2 named for `other`-typed risk objects that are really just unmanaged assets waiting to be resolved.

Cross-references: DEH Part 23 §1 (DET-23-01); DEH Part 26 §1.3 (`tstats` forward reference), §5.2 (DET-26-01's SPL body); this book's Part 3 (bucket lifecycle and retention), Part 6 (CIM data models), Part 8 (lookup tables as maintained infrastructure), Part 9 (search macros and macro sprawl), Part 10 (`tstats` and data model acceleration), Part 13 (Enterprise Security editions), Part 14 (correlation-search anatomy and content lifecycle), Part 16 (notable events and Incident Review), Part 17 (asset and identity correlation), Part 19 (investigation workflows).

Part 16 — Notable Events and the Incident Review Workflow

Why this part exists

[CONCEPT] Part 14 built a correlation search and watched it fire `action.notable`. Part 15 (Risk-Based Alerting) covers the alternative path — `action.risk` annotating a risk object until an accumulated score crosses a threshold and Enterprise Security generates a notable on its own. Both paths land in the same place: a row in the `notable` index (the Enterprise Security index holding notable-event metadata, introduced in Part 13 §4) that an analyst has to actually work. This part is about that row — what fields it carries, how Enterprise Security decides how urgently to present it, and what an analyst does with it inside Incident Review (Enterprise Security's own analyst-facing dashboard for working notable events, named in Part 13 §4's component table). It is the first part in Section E written from the triage seat rather than the content-authoring seat.

This part does not re-teach how a correlation search is built (Part 14), does not re-teach risk-score accumulation or threshold mechanics (Part 15), and does not re-teach SPL syntax — where a drilldown search's own query surfaces below, DEH Part 26 §1 (Splunk SPL, the search-time/index-time field split and pipeline mechanics) is the pointer, not a re-derivation. What it does own: notable-event fields and computed urgency, the status/ownership/disposition workflow an analyst actually clicks through, and — the section this part exists to write — the concrete difference between triaging a traditional, one-match-one-notable event and triaging an RBA-generated notable that represents a threshold crossing with several contributing events behind it, not one.

Evidentiary note, restated per STYLE-GUIDE.md §9.2: no Enterprise Security instance exists in this book's evidence base. Every mechanic below is sourced from Splunk's own public product materials (cited in `REFERENCES.md`) or marked **CONCEPTUAL** as an illustrative reconstruction; nothing here is a screenshot or a captured Job Inspector output from a running deployment.

1. What a notable event actually is

[CONCEPT] A notable event is a retained, searchable record — one row, with its own fields — that Enterprise Security writes to the `notable` index the moment a correlation search's `action.notable` adaptive response action fires (Part 14 §3) or an RBA threshold crossing generates one (Part 15). It is not a transient UI state that disappears if nobody looks at it; closing a browser tab open to Incident Review does not delete or resolve anything, because the notable already exists as an indexed event independent of whoever is currently viewing it. That distinction matters operationally: a notable an analyst never opens still consumes license-metered ingest and retention exactly as Part 13 §4's Engineering Reality box describes, and it still counts toward whatever backlog metric a SOC manager tracks (§7 below) whether or not a human has touched it.

The table below is the reference this part leans on for the rest of its sections — it states what each core notable field is and, just as importantly, which layer actually sets its value, since that determines whether

an analyst can change it directly or has to go fix something upstream instead.

Field	What it holds	Set by
<code>rule_name</code>	The firing correlation search's title (e.g., "Suspicious LSASS Memory Access - Non-Allowlisted Process", Part 14 §4.1)	The correlation search's own definition — fixed at author time
<code>security_domain</code>	A coarse category (<code>access</code> , <code>endpoint</code> , <code>network</code> , <code>identity</code> , <code>audit</code> , <code>threat</code>) used for Incident Review filtering	<code>action.notable.param.security_domain</code> in the stanza
<code>severity</code>	The correlation search author's own confidence/impact rating (<code>informational</code> / <code>low</code> / <code>medium</code> / <code>high</code> / <code>critical</code>)	<code>action.notable.param.severity</code> in the stanza
<code>urgency</code>	Enterprise Security's own computed field, combining <code>severity</code> with the affected asset's or identity's <code>priority</code> rating	Computed by Enterprise Security; not directly settable — see §2
<code>status</code>	The workflow state an analyst moves the notable through	Analyst action inside Incident Review (§3)
<code>owner</code>	Which analyst (or <code>unassigned</code>) is currently responsible for the notable	Analyst action, or an assignment rule
<code>disposition</code>	The analyst's own closing categorization — true positive, benign positive, false positive, undetermined	Analyst action, usually set at or near closure
<code>drilldown_search</code>	A parameterized SPL query, defined at authoring time, that pivots from the notable's own fields back into raw or data-model search	<code>action.notable.param.drilldown_search</code> in the stanza (Part 14 §4.1)

[SOC ANALYST] Two of these rows are worth flagging before the sections below go deeper: `urgency` is computed, not authored — no correlation search sets it directly — and `drilldown_search` is the field that turns a notable from a static record into an actual investigation starting point. An analyst who only ever reads the notable's own summary fields and never runs its drilldown search is triaging on a paraphrase of the underlying event, not the event itself.

[PLATFORM ENGINEER] A notable's `rule_name` is not a unique identifier for the notable itself — every run of the same correlation search that matches produces its own separate notable record, each with its own `event_id`, even against the same underlying host or user. Nothing about writing a new notable checks whether an earlier, still-open notable already covers the same condition; that's exactly what `alert.suppress` throttling (Part 14 §2.2) exists to prevent at the correlation-search layer, before a duplicate notable is ever created. A notable that has already been written is a permanent record from that point forward — an analyst closing it, or an administrator later disabling the correlation search that created it, does not retroactively remove it from the `notable` index, which is the direct implication of §6's retention discussion below.

2. Severity, priority, and urgency: the field that decides sort order

[SOC ANALYST] Incident Review's default view sorts by `urgency`, not `severity` — a deliberate design choice, because `severity` alone only reflects how bad the correlation search author thought the pattern was in the abstract, with no knowledge of which specific host, user, or IP actually matched on a given run. Enterprise Security corrects for that by combining `severity` with `priority` — a rating (`critical/high/medium/low/unknown`) attached to the specific asset or identity the notable's risk object refers to, maintained by the Asset and Identity framework this book's Part 17 owns in depth — into the single `urgency` value Incident Review actually sorts on.

Blind Spot *The exact default mapping from a given `severity/priority` pair to a resulting `urgency` band is Enterprise Security's own shipped lookup, and this book cannot reproduce its specific cell values with confidence — docs.splunk.com was not reachable during this book's research (per `REFERENCES.md`'s sourcing note), and asserting a precise matrix from general familiarity with the product rather than a checked source is exactly what `STYLE-GUIDE.md §9.4` rules out. What is safe to state, because it follows from the fields' own definitions rather than a specific lookup value: `urgency` is a function of both fields, so a `critical`-severity notable against an asset the Asset and Identity framework has never rated (`priority = unknown`) does not automatically rank above a `high`-severity notable against a rating of `critical`. If Part 17's asset-criticality data is stale or incomplete — the exact failure mode Part 17 itself names — the `urgency` an analyst is sorting by is wrong in a way that looks like a ranking decision, not a data-quality one, and the wrong notable gets picked up first.*

[SOC ANALYST] The practical consequence for an analyst working a queue: `urgency` is only as trustworthy as the Asset and Identity data behind it. A queue sorted by `urgency` with a large fraction of notables carrying `priority = unknown` is a queue where the sort order isn't actually doing its job — treat a spike in `unknown`-priority notables as an Asset and Identity coverage gap to escalate (Part 17), not as a property of the underlying detections that fired.

[SOC ANALYST] A related habit worth building deliberately rather than discovering the need for it during an incident: filter Incident Review by `security_domain` and `status` together before trusting `urgency` alone to decide what to work next. Two notables tied on `urgency` are not necessarily equally worth an analyst's next fifteen minutes — one might sit in the `identity` domain against a service account already known to be mid-rotation, the other in `endpoint` against a workstation with no other open notables in the last quarter. Urgency collapses severity and priority into a single number precisely so a queue can be sorted at all, and any single-number sort loses information the underlying fields still carry; an analyst who only ever reads the sorted number back is discarding context the notable's own record still has.

3. Status, ownership, and disposition: the fields an analyst actually changes

[SOC ANALYST] `status` tracks where a notable is in an analyst's workflow, and `owner` tracks who's responsible for moving it there. Both are ordinary editable fields on the notable record, changed from within Incident Review, and — like every other field change on a notable — the change itself is retained

in the notable's own audit history rather than overwriting the prior value silently, so a later reviewer can see who touched a given notable, when, and what they changed it to.

CONCEPTUAL SAMPLE -- an illustrative status set, not independently re-verified against current Splunk documentation for this book (docs.splunk.com was unreachable; see REFERENCES.md). Treat this as a plausible starting point, not a guaranteed default -- confirm the actual status list against your own instance's Incident Review Settings before training analysts against it.

Unassigned -> New -> In Progress -> Pending -> Resolved -> Closed

[SOC ANALYST] Whatever the exact shipped default list is, the workflow shape it represents is stable and worth stating plainly: a notable starts with no owner and moves through an active-investigation state, an optional waiting state (for a notable blocked on something outside the analyst's control — a pending user confirmation, a scheduled patch), and a terminal state. Enterprise Security lets an administrator edit this status list per deployment, so a SOC that has customized it should treat its own Incident Review Settings page as authoritative over any generic list, including the one above.

[SOC ANALYST] `disposition` is a separate field from `status`, and conflating the two is a common new-analyst mistake: `status = Closed` says the workflow is finished; `disposition` says what the notable actually turned out to be — a true positive, a benign positive (real activity, but expected and not a concern), or a false positive, and if false, why. Splunk markets Enterprise Security's own risk-based alerting specifically on reducing false-positive volume relative to per-event notables (§5 below revisits this claim directly), which makes disposition data the actual measurement surface for whether that's happening in a given deployment — a notable closed with `status = Closed` and no disposition set tells a SOC manager nothing about whether the underlying detection is worth keeping.

Product Version Note Splunk's own current Enterprise Security product page lists an "AI Assistant for Security," marketed for triage-support tasks including alert explanation and prioritization, as included in both the Essentials and Premier editions — not gated to either tier specifically — positioned alongside, not replacing, the classic Incident Review status/owner/disposition workflow described above. As of 2026-09-15, verified against Splunk's own Enterprise Security product page (REFERENCES.md entry [SPLUNK-ES-PRODUCT-PAGE]; consistent with this same source's treatment in Part 5 §5 and Part 13 §2). What would make this stale: a rename of this capability, a move of automated triage output into a new field this part doesn't cover, or a future edition-gating change moving it behind Premier specifically — confirm which triage surface (the fields described in this part, an AI-generated explanation layered on top of them, or both) your own licensed edition actually exposes before writing internal runbooks that assume one or the other.

4. Working a notable in Incident Review

[SOC ANALYST] Incident Review presents notables as a filterable, sortable table — by `status`, `urgency`, `owner`, `security_domain`, time range, and free-text search against the notable's own fields — with each

row expanding into the notable's full field set, its `drilldown_search` result, and, where the firing correlation search attached one, a MITRE ATT&CK annotation (`action.correlationsearch.annotations`, Part 14 §4.1) rendered as a technique reference rather than a bare ID buried in a field value. Opening the "Suspicious LSASS Memory Access - Non-Allowlisted Process" notable built in Part 14 §4 shows exactly this shape: `severity = high`, `security_domain = endpoint`, a MITRE annotation resolving to T1003.001 (OS Credential Dumping: LSASS Memory), and a `drilldown_search` that reruns Part 14 §4.1's SPL body scoped to that specific `ComputerName` and `SourceImage` — the same query DEH Part 26 §1 covers the pipeline mechanics of, not re-derived here.

[SOC ANALYST] Because that same correlation search carries both `action.notable` and `action.risk` (Part 14 §3's point that the two are not mutually exclusive), opening this particular notable and pivoting sideways to the affected `ComputerName`'s risk-object view shows the same match contributing an 80-point risk-score annotation at the same time it produced this notable. That is the exception, not the rule, for how most correlation searches in a mature deployment are configured (Part 14 §3), and it is exactly the seam §5 below walks through in the other direction: what an analyst sees when a notable exists *only* because accumulated risk crossed a threshold, with no single correlation search run responsible for it directly.

flowchart TB

```
A["Incident Review list\nfiltered by status / urgency / owner"] --> B["Analyst opens a notable"]
B --> C{"Traditional notable\nor RBA-generated?"}
C -->|"Traditional\n(single action.notable match)"| D["Read the notable's own fields\nrun drilldown_search (DEH Part 26 SPL)"]
C -->|"RBA-generated\n(threshold crossing, Part 15)"| E["Open the risk object's\ncontributing risk-index events"]
D --> F["Pivot to Asset & Identity context\n(Part 17: priority, ownership)"]
E --> F
F --> G["Set disposition + status"]
G --> H{"Escalate?"}
H -->|"Yes"| I["SOAR playbook / manual escalation\n(Premier-gated, Part 13 §2)"]
H -->|"No"| J["Close notable\n(audit history retained)"]
```

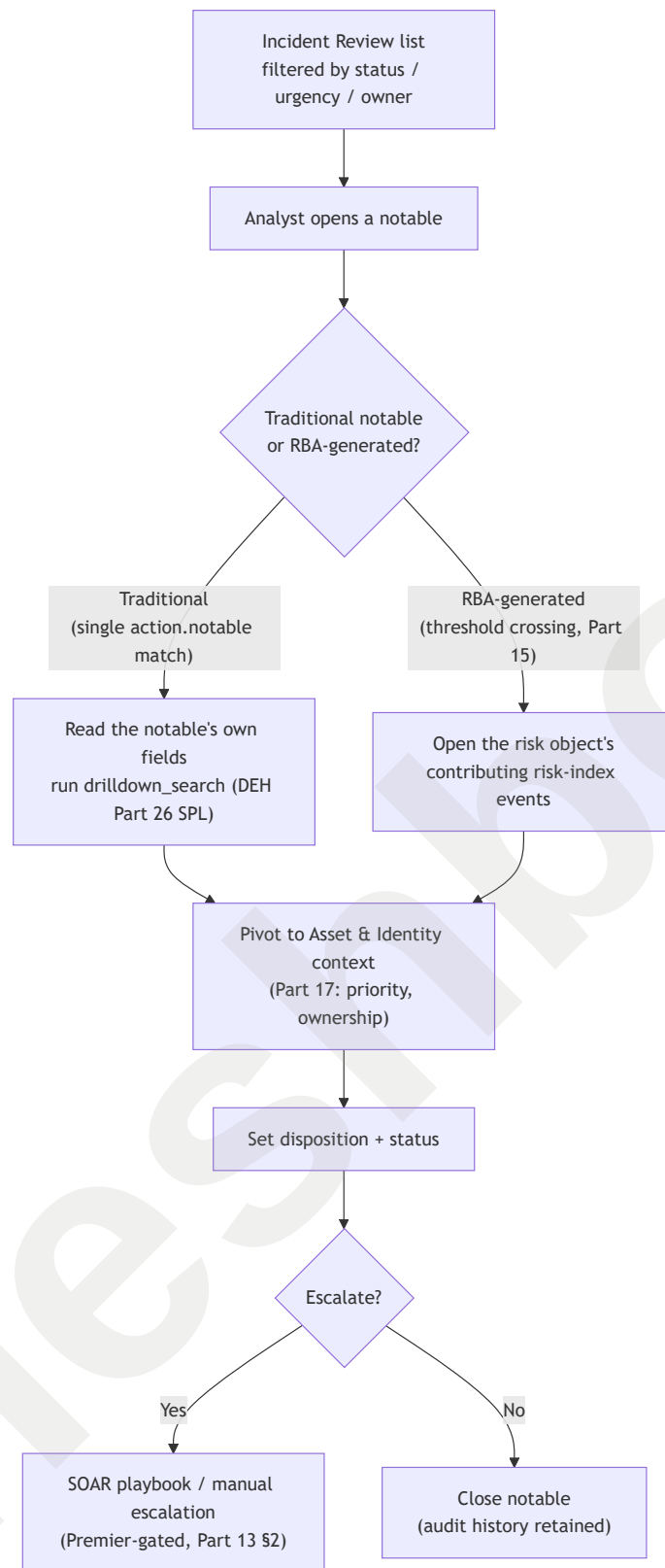


Figure 16.1 — An analyst's pivot path through Incident Review. *CONCEPTUAL.* Illustrates the branch between working a traditional, single-match notable and working an RBA-generated notable's contributing events, both converging on Asset and Identity context before disposition and closure. This is a structural diagram of documented expected workflow, not a capture from a live Incident Review instance — none exists in this book's evidence base (see STYLE-GUIDE.md §9.2).

[SOC ANALYST] Bulk actions matter operationally the way they don't in a single-notable walkthrough: Incident Review supports selecting multiple notables and applying a status change, an owner assignment, or a comment across all of them at once, which is the only realistic way a queue in the hundreds gets worked rather than clicked through one row at a time. A comment added to a notable — free text, timestamped, attributed to the commenting analyst — is itself part of that notable's retained record, and is frequently the only place a shift-handoff note ("confirmed benign — scheduled patch job, see change ticket 4471") survives for the next analyst who opens the same notable after a status of **Pending**.

5. Traditional notable vs. RBA-generated notable: what's actually different at triage time

[SOC ANALYST] This is the distinction the rest of this part has been building toward, and it is a real difference in what an analyst is looking at, not a cosmetic one. A traditional notable corresponds to one correlation search run matching once; everything the analyst needs to evaluate that match — the query, the matched fields, the drilldown search — is attached directly to that one notable. An RBA-generated notable corresponds to a *risk object's* accumulated score crossing a configured threshold (Part 15's subject); the notable itself doesn't represent one underlying event, it represents the moment the threshold was crossed, and the actual evidence behind it — the individual risk-index events, potentially written by several different correlation searches over some window of time — has to be pulled up separately.

The table below is the one this section exists to support: it states, side by side, what an analyst is actually holding in each case.

	Traditional notable	RBA-generated notable
Trigger	One correlation search run matches once (action.notable)	A risk object's accumulated score crosses a threshold (Part 15)
Underlying evidence	One matched event, described directly in the notable's own fields	Multiple risk-index events, possibly from different correlation searches, over a scoring window
Where the evidence lives	The notable's drilldown_search result	The risk object's contributing risk-index events — a separate pivot, not the notable's own fields
Confidence per component	Usually high — the search that fired is often deliberately narrow (Part 14 §3)	Individually often low — RBA is designed to let several weak signals jointly justify one notable
False-positive review	Confirm or refute the one matched event	Confirm or refute the <i>combination</i> — one contributing event being benign doesn't mean the others are
Typical severity source	Set once, by the firing search's author	No single severity — urgency reflects the risk object's accumulated score and priority, not one search's rating

False Positive Trap *An analyst opens an RBA-generated notable, clicks into the first contributing risk-index event listed, recognizes it as a known-benign pattern — a scheduled vulnerability scanner, a routine admin script — and disposes the entire notable as a false positive on the strength of that one event. If four other contributing events pushed the same risk object over threshold, none of them individually benign, closing the notable on the first event checked closes all of them at once, with no record that the other four were ever actually reviewed. This is the RBA-specific version of a per-event false-positive review habit that works fine against a traditional notable — where there genuinely is only one event to check — and fails silently against an RBA one, where "check the notable" and "check every contributing event" are not the same action. The fix is procedural, not technical: an RBA-notable triage checklist that requires opening the full contributing-events list before setting disposition, not just the first entry Incident Review happens to render.*

[SOC MANAGEMENT] Splunk's own Enterprise Security materials describe risk-based alerting's promise in exactly the terms Part 15 covers in depth — high-fidelity alerting intended to reduce false-positive notable volume relative to a per-event model. That is a vendor claim about *volume*, and this part's own contribution is narrower: even where RBA genuinely does reduce how many notables an analyst sees, each surviving RBA notable can take *longer* to triage correctly than a traditional one, because confirming or refuting a threshold crossing means reviewing every contributing event, not one. A mean-time-to-triage metric that doesn't distinguish traditional notables from RBA-generated ones will show a confusing mix of effects after an RBA rollout — fewer notables, but not proportionally less analyst time per notable — and a SOC manager reading only the aggregate count risks concluding RBA "isn't saving as much time as promised" when the real story is that the remaining notables are each doing more work.

6. Closing a notable, and what the audit trail is actually for

[SOC ANALYST] Closing a notable means setting `status` to its terminal value and `disposition` to whatever the investigation concluded — not just the former. A comment recorded at closure (§4) is the cheapest artifact a SOC produces that pays off later: when the same `rule_name` fires again against the same asset three months later, the prior notable's own closing comment, retrievable by searching the `notable` index directly, is often the fastest way to confirm whether this is the same known-benign pattern recurring or something that actually needs fresh eyes.

[SOC ANALYST] Reopening a closed notable is a normal, supported action, not an exception path — a status of `Closed` is not a lock. A notable dispositioned as benign positive last month, revisited after a related notable surfaces new context, can have its status moved back to `In Progress` with a fresh comment explaining why, and the prior disposition and comment stay attached rather than being overwritten. That reversibility is precisely why the audit trail matters more than the current-state fields alone: two analysts looking at the same notable a week apart should be able to reconstruct not just what it is now, but what it was believed to be at each point in between, and why the belief changed.

[PLATFORM ENGINEER]

None of this history is free. Part 13 §4's Engineering Reality box already makes the point generally — the `notable` index is an ordinary Splunk index from a licensing and retention standpoint, governed by the same hot/warm/cold/frozen bucket lifecycle Part 3 covers for any other index. A retention policy tuned for raw event data's compliance window, applied unthinkingly to the `notable` index, can age out six months of disposition history — exactly the history this section just argued is operationally valuable — well before a compliance or insurance-audit request asks for it. Size `notable`-index retention as its own decision, not an inherited default from whatever retention policy the raw telemetry indexes use.

7. SOC management view: measuring the queue, not just the notable

SOC Management View *The metrics worth tracking against a notable queue — backlog by urgency band, mean time to disposition, per-analyst throughput — are only meaningful if they're read alongside the traditional/RBA split §5 establishes. A rising mean-time-to-disposition figure immediately after an RBA rollout is not, on its own, evidence that analysts have gotten slower or that the new content is worse; it may simply reflect that a larger share of the queue now requires reviewing several contributing events per notable instead of one. Track the two notable types' triage time separately for at least one full reporting cycle before drawing a conclusion from either the aggregate number or a naive before/after comparison — the alternative is discovering, well into a staffing conversation, that the metric everyone has been managing against was never measuring one consistent thing.*

Closing: where this hands off

[CONCEPT]

This part covered the notable-event record itself and the workflow an analyst runs it through — urgency's dependency on Asset and Identity data, the status/owner/disposition fields, and the structural difference between a traditional and an RBA-generated notable at triage time. Three parts pick up threads this one deliberately left for them: Part 15 covers how a risk score actually accumulates toward the threshold that generates the RBA notables §5 describes; Part 17 covers the Asset and Identity framework whose priority data §2's urgency computation depends on, and what happens when that data goes stale; and Part 19 covers the broader Splunk-specific investigation pivot — from a notable, into raw search, into asset and identity context, and beyond — including the same authoring-surface naming churn Part 13 §6 already flagged, extended here to whatever an analyst's own triage surface is called in a given release.

Cross-references: DEH Part 26 (Splunk SPL) §1; Part 13 (Enterprise Security Architecture, Editions, and the CIM Dependency) §4, §5; Part 14 (Correlation Searches) §3, §4; Part 15 (Risk-Based Alerting); Part 17 (Asset and Identity Correlation); Part 19 (Splunk-Specific Investigation Workflows); Part 3 (Indexes, Sourcetypes, and the Bucket Lifecycle).

Part 17 — Asset and Identity Correlation

Why this part exists

[CONCEPT] A correlation search (Part 14) matches on raw or CIM-normalized fields — a hostname, an IP, a username — and neither the search nor the risk object (Part 15) it annotates knows, on its own, whether that hostname is a domain controller or a decommissioned test box, or whether that username belongs to a database administrator or a contractor whose access expired last quarter. The Asset and Identity framework is the Enterprise Security component that closes that gap: it maintains a maintained, queryable record of what an asset or identity actually *is* — its criticality, its owner, its business unit — and merges that context onto a notable event or a risk object at the point of enrichment, so that "a match happened" and "how much this match should matter" become two separate, separately-maintained facts rather than one search trying to encode both.

This part does not re-teach what a risk object or a risk score is — Part 15 owns that mechanism, and this part assumes the reader already knows a risk object is a user, a host, or an IP that accumulates score across matches. It does not re-teach how a notable event's urgency field is computed from a severity-by-priority matrix — Part 16 owns that computation in depth; this part's concern is narrower and sits one layer upstream: where the *priority* input to that matrix actually comes from, how it gets there, and what happens when the data feeding it is wrong. It does not re-teach the Common Information Model's field-naming contract either — Parts 5–7 own CIM in general, and §1 below states the one clause of connection this part needs from that material before moving past it. What this part owns: the Asset and Identity framework itself as a maintained data source (§1–§2), how its output actually reaches a notable event or risk object (§3), the expiration mechanics meant to keep it current (§4), the specific staleness failure mode this book's own Part 8 already named as the framework's job to prevent for *other* lookups — and that the framework itself is not immune to (§5) — and the hunting and management practices built on top of it (§6–§7).

1. What the framework actually is, and what CIM already promised

[CONCEPT] Splunk's Common Information Model defines a standard field set not only for events (Authentication, Network Traffic, and the rest of the data models Part 6 covers) but also for two non-event, lookup-backed entities: an **asset** (a host, device, or system) and an **identity** (a person or account). CIM's own documentation specifies the expected field names — `priority`, `category`, `bunit` (business unit), `owner` for an asset; `identity`, `priority`, `category`, `bunit`, `watchlist` for an identity — the same way it specifies `src/dest/action` for the Network Traffic data model. Parts 5–7 own that field-naming contract in general; what's specific to this part is that Enterprise Security ships a whole operational framework — configuration UI, storage, a merge process, and an automatic enrichment step — built specifically to populate and maintain lookup data that conforms to that contract. CIM says what the fields should be called. The Asset and Identity framework is the part of Enterprise Security that actually keeps a real, current table of those fields and attaches it to your data.

[CONCEPT] The distinction matters because it's easy to conflate "my asset/identity data is CIM-compliant" (a field-naming question, Part 7's territory) with "my asset/identity data is current" (a maintenance question, this part's territory). A lookup table with every CIM-standard column named correctly and populated with data from eighteen months ago is fully CIM-compliant and still wrong. Nothing about compliant field naming tells Enterprise Security, or the analyst reading a notable, whether the `priority` value in a given row still describes the host it's attached to.

Table 17.1 — Core asset and identity fields. Use this table to decide what a new asset or identity source actually needs to populate before it's useful to Enterprise Security; `priority` is the single field the rest of this part returns to most, because it's the one that reaches a notable's urgency computation (§3).

Field	Framework	What It Captures	Typical Values
<code>ip</code> , <code>mac</code> , <code>dns</code> , <code>nt_host</code>	Asset	Identifiers Enterprise Security matches against a raw event's host-like fields	IP address, MAC address, DNS name, NetBIOS host name
<code>owner</code>	Asset	The person or team accountable for the asset	Free text, or a name matching an identity's <code>identity</code> field
<code>priority</code>	Asset, Identity	The criticality value that feeds a notable's urgency computation (Part 16)	<code>critical</code> , <code>high</code> , <code>medium</code> , <code>low</code> , <code>unknown</code>
<code>category</code>	Asset, Identity	A deployment-defined grouping label	Free text (<code>domain_controller</code> , <code>finance</code> , <code>contractor</code>)
<code>bunit</code>	Asset, Identity	The business unit the asset or identity belongs to	Free text, deployment-defined
<code>identity</code> , <code>nick</code>	Identity	Login IDs and aliases Enterprise Security matches against a raw event's user-like fields	Username, login alias
<code>email</code> , <code>phone</code>	Identity	Contact information surfaced to an analyst working a notable	Email address, phone number
<code>watchlist</code>	Identity	Flags a person Enterprise Security should treat as heightened-risk regardless of <code>priority</code>	Boolean
<code>startDate</code> , <code>endDate</code>	Identity	The validity window for the identity — used with the expiration settings in §4	Date

[PLATFORM ENGINEER] `priority`'s five values are worth naming precisely because `unknown` is not a placeholder for "not yet configured" — it is what Enterprise Security assigns to any raw event's host or user field that matches nothing in the merged asset or identity data at all. A host with genuinely no asset entry and a host with a stale, wrong asset entry produce two different failure shapes downstream: the first shows up as `unknown` priority, visibly under-contextualized; the second shows up as a confidently wrong `critical` or `low`, which looks like real, current context and isn't. §5 is built entirely around that second, more dangerous shape.

2. Assets and Identities — sources, the merge, and where the data actually lives

Assets and Identities — the two default KV Store collections everything else merges into

[PLATFORM ENGINEER] Enterprise Security ships with two default, hand-editable lists — **Assets** and **Identities** — reachable from the app's own asset-and-identity management page, and backed by the same KV Store mechanism Part 8 §4 covers generally: a MongoDB-backed collection, edited a row at a time through the UI or the REST API rather than replaced wholesale the way a CSV lookup is. For a small environment, hand-maintaining these two lists directly — adding a row per critical server, per VIP identity — is a legitimate, if labor-intensive, way to run the framework. For anything past a handful of entries, an organization instead configures one or more **additional sources**: a CSV lookup exported from a CMDB or HR system, or a separate KV Store collection an external script keeps current, each mapped to the same standard field set named in Table 17.1. Enterprise Security merges every configured source — the two defaults plus however many additional ones are configured — into the working asset and identity data that correlation searches and notables actually draw on, resolving conflicts between sources (two sources disagreeing about one host's **priority**) according to a precedence order set on the same management page.

Table 17.2 — Asset and identity source shapes. Use this table the same way Part 8's Table 8.1 is used for a general-purpose lookup: match the update pattern a given feed actually has to the source shape built for it, rather than defaulting to whichever shape is most familiar.

Source Shape	Update Pattern	Typical Feed	Expiration Available
Default Assets / Identities list	One row at a time, edited in the ES UI or via REST	A handful of manually tracked critical hosts or VIP identities	Configurable per source
Custom CSV lookup source	Whole-file replace	A scheduled CMDB or HR export dropped into the app's lookups/ directory	Configurable per source
Custom KV Store source	Per-document create/update/delete via REST or SPL	An automated feed from a CMDB or identity-provider API kept continuously current	Configurable per source

[PLATFORM ENGINEER] Every row in Table 17.2 is a maintained artifact in exactly the sense Part 8 §7 already argues for a general lookup table — a file or a collection with an owner, a review cadence, and a place in a version-controlled deployment pipeline, not a one-time import. Part 8's own Detection Autopsy (§3.1 of that part, "the scanner allowlist that outlived the scanner") names this framework specifically as the system that should already know when a host is decommissioned. §5 below is this part's answer to that forward reference — and the uncomfortable finding is that the framework knowing about a decommission and the framework's own source data actually reflecting it are still two different facts, exactly the way Part 8 draws that distinction for any other lookup.

Product Version Note Risk-based alerting, the risk index, and the Asset and Identity framework covered in this part are all licensed under Enterprise Security's *Essentials* edition, not gated to *Premier* — the Premier-only capabilities are User and Entity Behavior Analytics (UEBA), Splunk SOAR integration, and "Automated Threat Analysis." As of 2026-09-15, verified against Splunk's own Enterprise Security product page ([REFERENCES.md](#) entry [\[SPLUNK-ES-PRODUCT-PAGE\]](#); the same split Part 13 §2 documents for Section E generally). What would make this stale: an edition restructure that moves asset/identity correlation behind a higher tier, or a rename of either edition — confirm this against the specific license an environment is running before assuming the framework is available by default.

3. From a raw field to `dest_priority`: how enrichment actually reaches a notable

[DETECTION ENGINEER] Once a source's data is merged (§2), Enterprise Security applies asset and identity correlation automatically to any CIM-aliased host or user field a correlation search's results carry — `src`, `dest`, `dvc`, `src_user`, `user`, and the other Network Traffic and Authentication data-model fields Parts 5–7 already establish. The mechanism adds a matching set of fields for each role a given field played in the match, prefixed by that field's own name: a result carrying `dest` gets `dest_priority`, `dest_category`, `dest_bunit`, and `dest_owner` if `dest` matched an asset entry; a result carrying `user` gets `user_priority`, `user_category`, `user_bunit`, and `user_watchlist` if `user` matched an identity entry. A single event with both a matched `dest` and a matched `user` field carries both prefix families at once, independently — a notable can be enriched with asset context, identity context, or both, depending on which of its own fields the underlying event actually populated.

```
# CONCEPTUAL SAMPLE – illustrative field shape after asset/identity correlation runs against a
# result carrying dest and user; exact field availability depends on which sources (§2) actually
# have a matching entry, and this has not been validated against a live Enterprise Security
# instance (none exists in this book's evidence base).
dest=db-prod-07.internal
dest_priority=critical
dest_category=database
dest_bunit=finance
dest_owner=dba-team
user=svc_backup
user_priority=medium
user_category=service_account
user_watchlist=false
```

[DETECTION ENGINEER] The "Suspicious LSASS Memory Access - Non-Allowlisted Process" correlation search Part 14 §4 built around DET-26-01 gives a concrete case: its `action.risk.param._risk_object_1 = ComputerName` line makes `ComputerName` the risk object every match annotates. Once that same host's `ComputerName` value has a matching entry in the merged asset data, every notable and every risk-index event referencing it carries a `ComputerName`-derived priority alongside the risk score — a match against a host whose asset entry says `critical` reads differently to an analyst than the identical match against a

host whose asset entry says `low`, even though DET-26-01's own SPL body and the correlation search's severity setting are unchanged between the two.

[DETECTION ENGINEER] That `priority` value is also one of the two inputs — the correlation search's own configured severity is the other — to the severity-by-priority matrix Enterprise Security uses to compute a notable's `urgency` field. Part 16 owns that matrix and the analyst-facing consequences of a given urgency value in depth; this part's point is narrower: `priority` is not a cosmetic annotation sitting next to a notable, it is a direct, load-bearing input into the single field most SOCs sort Incident Review by. Getting it wrong doesn't just mislabel a notable — it mis-sorts the analyst's own queue.

```
flowchart TB
  subgraph SOURCES["Asset & identity sources (S2)"]
    DEF["Default Assets / Identities\nKV Store lists"]
    CSV["Custom CSV source\n(CMDB / HR export)"]
    KV["Custom KV Store source\n(automated feed)"]
  end
  end
  DEF --> MERGE["Merge by configured\nprecedence order"]
  CSV --> MERGE
  KV --> MERGE
  MERGE --> WORK["Working asset & identity data"]
  EVT["Correlation search result\n(dest / user / src fields)"] --> CORR["Asset & identity\ncorrelation (automatic)"]
  WORK --> CORR
  CORR -->| "match found" | ENR["Result enriched:\ndest_priority, user_priority, ..."]
  CORR -->| "no match" | UNK["priority = unknown"]
  ENR --> NOT["Notable event\n(urgency matrix, Part 16)"]
  ENR --> RISK["Risk object annotation\n(risk index, Part 15)"]
  UNK --> NOT
  UNK --> RISK
```

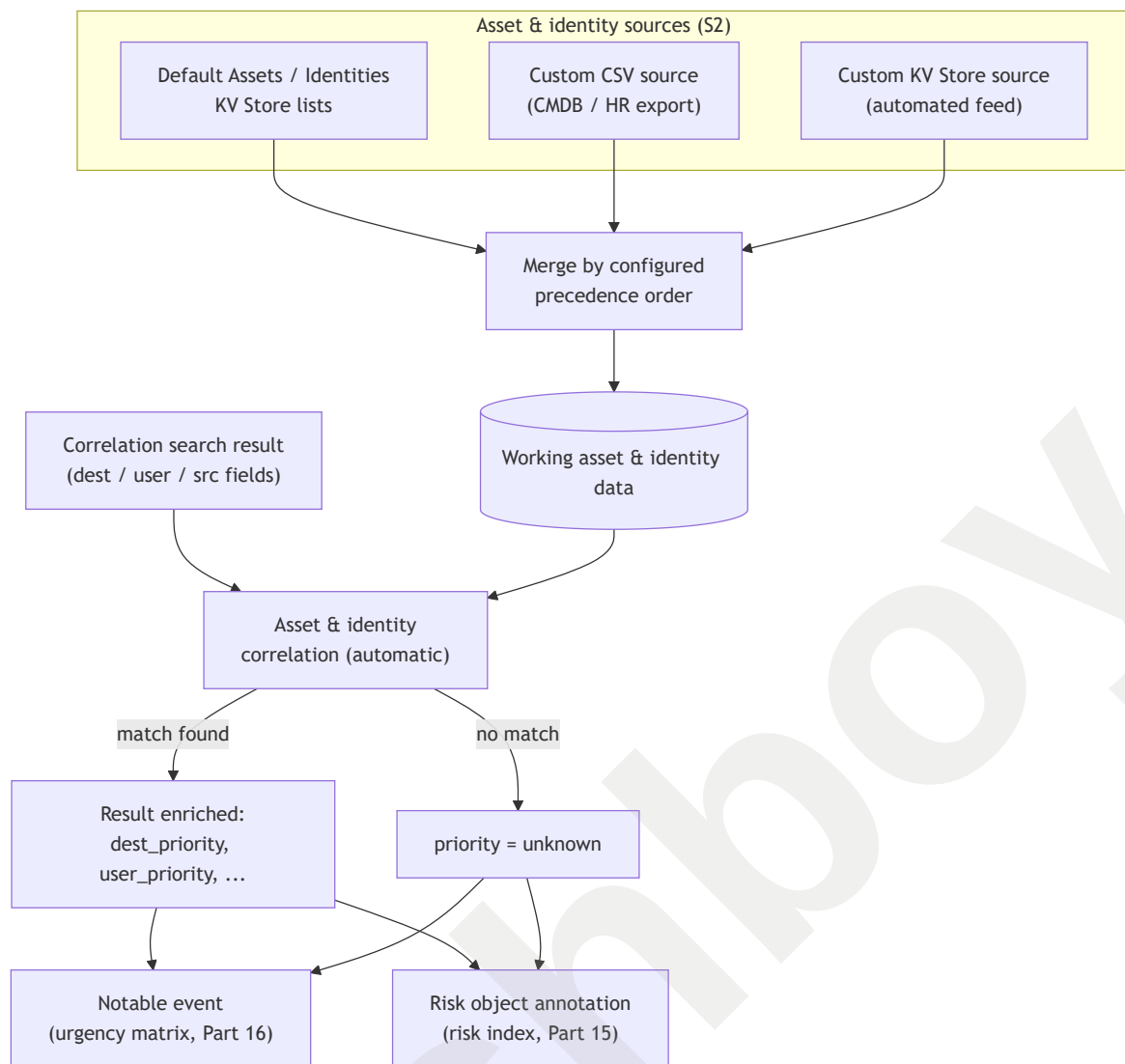


Figure 17.1 — Asset and identity data from source to a notable's urgency input. *CONCEPTUAL.*

Illustrates the merge of configured sources into working asset and identity data, the automatic correlation step that enriches a correlation search's own result fields, and the two paths — a real match or a fallback to `unknown` — that reach a notable event and a risk object. This is a diagram of documented expected mechanics, not a capture from a live Enterprise Security instance; none exists in this book's evidence base (STYLE-GUIDE.md §9.2).

4. Expiration settings, and what they only partly solve

[PLATFORM ENGINEER]

Each configured source (§2) can have an expiration window enabled — a duration after which an entry Enterprise Security hasn't seen refreshed in that source is dropped from the merged working data rather than retained indefinitely. This exists specifically to bound the risk Table 17.1's `startDate` / `endDate` fields hint at: an identity whose `endDate` has passed, or an asset entry a CMDB export stopped including because the underlying feed removed the row, ages out of the merged data instead of sitting there unchanged forever.

[PLATFORM ENGINEER]

Expiration solves exactly one shape of staleness: an entry that stops being *present* in its source. It does nothing for an entry that stays present but goes *wrong* — a CMDB export that still lists a host, still refreshes on schedule, and still says `priority=critical`, because nobody edited that one field when the host's actual role changed. A row that keeps refreshing on time looks, from the expiration mechanism's point of view, exactly like current, correct data. It is current. It is not correct. That gap is §5's whole subject.

Engineering Reality *Expiration windows are configured per source, not globally, and a common mistake is assuming a short expiration on one custom CSV source protects the whole merged dataset — it doesn't touch the default `Assets / Identities` lists (§2) or any other configured source at all. An organization running a fast-refreshing CMDB feed alongside a hand-maintained default list of "known critical servers" someone typed in eighteen months ago and never revisited has two very different staleness profiles sitting in the same merged working data, and only one of them is protected by an expiration window at all.*

5. The decommissioned-host failure mode: a wrong-priority alert, not a missing one

[PLATFORM ENGINEER]

Consider a host decommissioned during an infrastructure refresh and its hostname and IP reassigned six weeks later to a new, genuinely low-value development box — the same reassignment pattern Part 8 §3.1's Detection Autopsy already walked through for a scanner-exclusion lookup. That lookup's failure mode was silence: a correlation search kept suppressing a host it should have stopped suppressing, and the notable that should have fired never did. The Asset and Identity framework's failure mode, when its own source data goes stale the same way, looks nothing like silence.

Blind Spot *If the old asset entry is simply removed once a host is decommissioned — the seemingly-safe, conservative cleanup step — the reassigned host's next genuine match falls back to `priority=unknown` (§1). That's a real degradation (a critical-looking match now sorts lower in Incident Review than it should), but it's at least a visible one: an analyst who knows to distrust an `unknown`-priority notable on a host they'd expect to have asset coverage has a signal to act on. The more common failure is the opposite: the old entry is never removed, never updated, and never expires, because no expiration window is configured on the source that carries it (§4) or because the entry was hand-typed into the default `Assets` list years ago and nobody's process for decommissioning a host ever touches that list at all.*

False Positive Trap A stale `critical`-priority asset entry attached to a now-low-value host doesn't stop a correlation search from matching, and it doesn't stop a notable from firing — DET-26-01, deployed as the correlation search Part 14 §4 built, still fires exactly when its own SPL logic says it should. What changes is the `priority` value merged onto that notable, and therefore its `urgency` (Part 16). A notable against the reassigned host gets sorted, worked, and possibly escalated as if it were still hitting the decommissioned critical production system it used to be — full analyst attention spent on a development box's alert, at exactly the priority a genuinely critical host's alert should have gotten instead. The reverse direction is just as real and harder to notice: a genuinely critical host stood up under a hostname or IP that an old, stale entry still marks `Low` or `medium` gets systematically under-triaged, and nothing in Incident Review flags that the priority value driving that sort order is eighteen months out of date. Neither direction produces a missing alert an analyst would go looking for. Both produce a present, correctly-firing notable carrying the wrong weight — which an analyst has no reason to question unless they already suspect the asset data itself, not the detection, is the thing that's wrong.

[DETECTION ENGINEER] The fix mirrors Part 8 §3.1's own fix for the scanner allowlist, applied to whichever source (§2) actually carries the stale entry: tie asset-list maintenance to the same decommission and reassignment workflow that already exists for infrastructure change management, rather than treating "someone remembers to update Enterprise Security too" as a step with no owner and no trigger. Where the underlying source is a CMDB-fed CSV or KV Store feed, that often means confirming the CMDB's own decommission process actually removes or reclassifies the row — not just assuming an automated feed is current because it's automated — and, where the source is one of the hand-maintained default lists, assigning the same review-and-ownership discipline Part 8 §7 recommends for any manually maintained lookup, because a hand-typed `Assets` entry has exactly the same no-owner failure mode as a hand-typed CSV allowlist and none of the mechanical protections (expiration, an automated refresh) that a fed source at least has available to configure.

6. Threat hunter workflow: pivoting through asset and identity context

[THREAT HUNTER] Asset and identity data is as useful for deciding where to look next as it is for triaging what already fired. A hunter starting from a risk object (Part 15) with a moderate, sub-threshold accumulated score can pivot into that object's current asset or identity context — its `category`, its `bunit`, its `watchlist` flag — before deciding whether the object is worth escalating into an active hunt, the same way an analyst would use it to triage a notable, just earlier in the investigation and without a notable having fired yet.

Hunter's Note A risk object's `watchList` flag (identity) or an unusually specific `category` value (asset) is worth pivoting on directly, independent of accumulated score: a moderate-score risk object that also happens to be a `watchList`-flagged identity, or an asset tagged `category=domain_controller`, is a stronger hunt candidate than the same score against an ordinary workstation or a non-watchlisted account — the framework's own enrichment fields are a legitimate prioritization signal for a hunt queue, not only for Incident Review's own urgency sort. The caveat is §5's own finding: pivot on asset/identity context as a signal to look closer, not as a substitute for confirming the underlying host or identity role directly when a hunt is about to consume real analyst time — the same staleness risk that produces a wrong-priority notable produces a wrong-priority hunt lead.

7. SOC management: keeping asset and identity data current

SOC Management View The Asset and Identity framework's entire value proposition is that it turns "how important is this?" from a question every correlation search would otherwise have to answer for itself into a single, centrally maintained answer every correlation search and every notable inherits. That centralization is also what makes staleness expensive in a way it wouldn't be if criticality were scattered, redundantly, across individual detections: one stale asset entry doesn't just mis-sort one detection's output, it mis-sorts every correlation search's output that happens to touch that host or identity, silently, until someone corrects the one row responsible. Budget asset and identity data maintenance as a standing operational cost with a named owner and a review cadence — the same governance Part 8 §7 argues for a general lookup table, applied here to the single lookup dataset with the broadest blast radius in the whole Enterprise Security deployment — rather than a one-time onboarding task closed out when the framework was first configured.

[SOC MANAGEMENT]

In practice, that ownership question usually lands on whichever team already owns the CMDB or identity-provider data a source (§2) is fed from, not the detection-engineering team that builds correlation searches — the asset/identity data's accuracy is a data-quality problem inherited from an upstream system of record, not a Splunk configuration problem Enterprise Security itself can solve. A SOC that treats a wrong-priority notable as a detection-engineering bug to fix in the correlation search is fixing the wrong layer; the correlation search matched correctly, and the fix belongs in whichever source's data was wrong, with a feedback path back to that source's own owner when a stale entry is found — not a one-off manual correction inside Enterprise Security that the next feed refresh silently overwrites anyway.

Closing: where this hands off

[CONCEPT] Everything a correlation search (Part 14) or a risk-based alerting rule (Part 15) matches on is telemetry about an event. The Asset and Identity framework is the one piece of Enterprise Security whose entire job is telemetry about the *thing the event happened to* — and it inherits, in its own maintenance discipline, exactly the staleness risk Part 8 already named for lookup tables generally, at a scale where getting it wrong mis-prioritizes every notable and every risk object touching the affected host or identity, not just one detection's output. Part 16 picks this up from the analyst's side: what a wrong or stale **priority** value actually looks like inside Incident Review, and how an analyst working a notable can tell the difference between "this is genuinely low-urgency" and "this urgency value is wrong because the asset data behind it is stale" before that distinction costs triage time on the wrong alert.

Cross-references: DEH Part 26 (Splunk SPL) §5; this book's Parts 5–7 (the Common Information Model); Part 8 (Lookup Tables as Detection Infrastructure) §3.1, §4, §7; Part 13 (Enterprise Security Architecture, Editions, and the CIM Dependency) §2, §4; Part 14 (Correlation Searches) §4; Part 15 (Risk-Based Alerting); Part 16 (Notable Events and the Incident Review Workflow).

Section F — Dashboards, Investigation, and the Extended Ecosystem

neshboy

Part 18 — Dashboards and Visualizations for Security Operations

Why this part exists

Every part in Sections C through E of this book — lookups and macros (Parts 8–9), acceleration and performance (Parts 10–12), and Enterprise Security's correlation searches, risk scoring, notable events, and asset/identity enrichment (Parts 13–17) — ultimately produces something a person has to look at. A dashboard is the rendering layer sitting on top of all of it: a saved arrangement of panels, each panel bound to a search, each search paying the same concurrency and acceleration costs Parts 10 and 12 already cover, and each panel's visual design either helping or actively working against the SOC analyst trying to read it under time pressure. This part is operations layer, and it is deliberately narrower than it might sound: it covers the dashboard as a Splunk object — how a panel is actually built and bound to data, what a dashboard load actually costs the search head, and what design choices make a panel readable at a glance versus merely present — not the specific analyst-facing workflow of any one named dashboard. Part 16 already owns Incident Review, Enterprise Security's own notable-event triage dashboard, in depth; this part's panel-design guidance applies to Incident Review too, but Part 16 is where its urgency/severity fields and ownership workflow get taught. Part 19 owns the analyst's actual pivot path across several dashboards and views during an investigation; this part is where the container those pivots land on comes from.

This part does not teach SPL. Every panel in either dashboard technology this part covers is, underneath its visualization, an ordinary Splunk search — `stats`, `timechart`, `top`, whatever a panel's author chose — and DEH Part 26 already owns that syntax in full. Where a panel's search body matters below, this part states the one sentence of context needed and points to DEH Part 26 rather than re-deriving pipeline mechanics. This part also assumes Part 6's data model and Pivot coverage, Part 10's data model and report acceleration, and Part 12's search concurrency and scheduling mechanics as background it cites rather than repeats — a dashboard panel is, from the search head's point of view, just another search competing for the same finite resources Part 12 already named, and a slow or accelerated panel is the same mechanism Part 10 covers, applied to a visualization instead of an ad hoc query.

1. One dashboard shape, two technologies that produce it

[CONCEPT] Strip away the visual chrome and a Splunk dashboard is a small, specific thing: a saved view — a named knowledge object, stored and permissioned exactly the way a saved search, a macro (Part 9), or a lookup definition (Part 8) is stored, with the same private/app/global sharing model — that arranges one or more panels on a layout. Each panel pairs exactly two things: a search (inline SPL, a reference to a saved report, or a shared base search) and a visualization type (a table, a line chart, a single-value tile, a choropleth map) that renders that search's results. Some dashboards add input controls — a time-range picker, a dropdown populated from a lookup, a text field — that set tokens a panel's search can reference, turning a static view into something an analyst can filter without editing the underlying search.

That shape has been implemented by exactly two Splunk technologies, and this part's central practical job is naming both clearly enough that a reader can tell, from a dashboard's own source view, which one they're looking at before trying to edit it. **Simple XML** — also called Classic dashboards in Splunk's own current terminology — defines a dashboard as an XML document: a `<dashboard>` or `<form>` root element containing `<row>` and `<panel>` elements, each panel holding a visualization element (`<chart>`, `<table>`, `<single>`, and others) bound to a `<search>`. **Dashboard Studio** defines the same shape as a JSON document instead, edited primarily through a visual canvas builder rather than hand-written markup, with its own broader set of visualization types and two selectable layout modes — a freeform, pixel-positioned "Absolute" canvas and a responsive "Grid" canvas. Both are stored as the same kind of knowledge object, in the same part of an app's directory structure, subject to the same permission and promotion mechanics Part 9 already covers for macros; the difference that matters day to day is entirely in the source format, the editing surface, and the visualization/interaction feature set each one exposes — not in what a dashboard fundamentally is.

Figure 18.1 traces what actually happens between a dashboard loading in a browser and a panel rendering, for both an unaccelerated and an accelerated panel search, and where a drilldown token re-triggers the cycle.

```
flowchart TB
    subgraph SEARCHES["What a panel's search can be bound to"]
        RAW["Raw stats/search\nover unaccelerated events"]
        ACC["tstats against an\naccelerated data model\n(Part 10)"]
        BASE["Shared base search /\nData Source, read by\nmultiple panels"]
    end
    end
    LOAD["Dashboard opens,\nor auto-refresh fires,\nor a token changes"] --> DISPATCH["Each bound search\ndispatches as a job,\ncompeting for the\nconcurrency ceiling\n(Part 12 Section 1)"]
    DISPATCH --> RAW
    DISPATCH --> ACC
    DISPATCH --> BASE
    RAW --> RENDER1["full scan every load --\nslow, and slower as the\ntime range widens"| RENDER["Panel renders"]]
    ACC --> RENDER2["reads the pre-built\nsummary -- fast,\nbounded by summary range"| RENDER]
    BASE --> RENDER2["one job's results,\nshared across every\npanel that reads it"| RENDER2["Multiple panels render\nfrom one dispatch"]]
    RENDER --> DRILL["analyst clicks a row --\nsets a token"| DRILL["Drilldown token set"]]
    DRILL --> DISPATCH
```

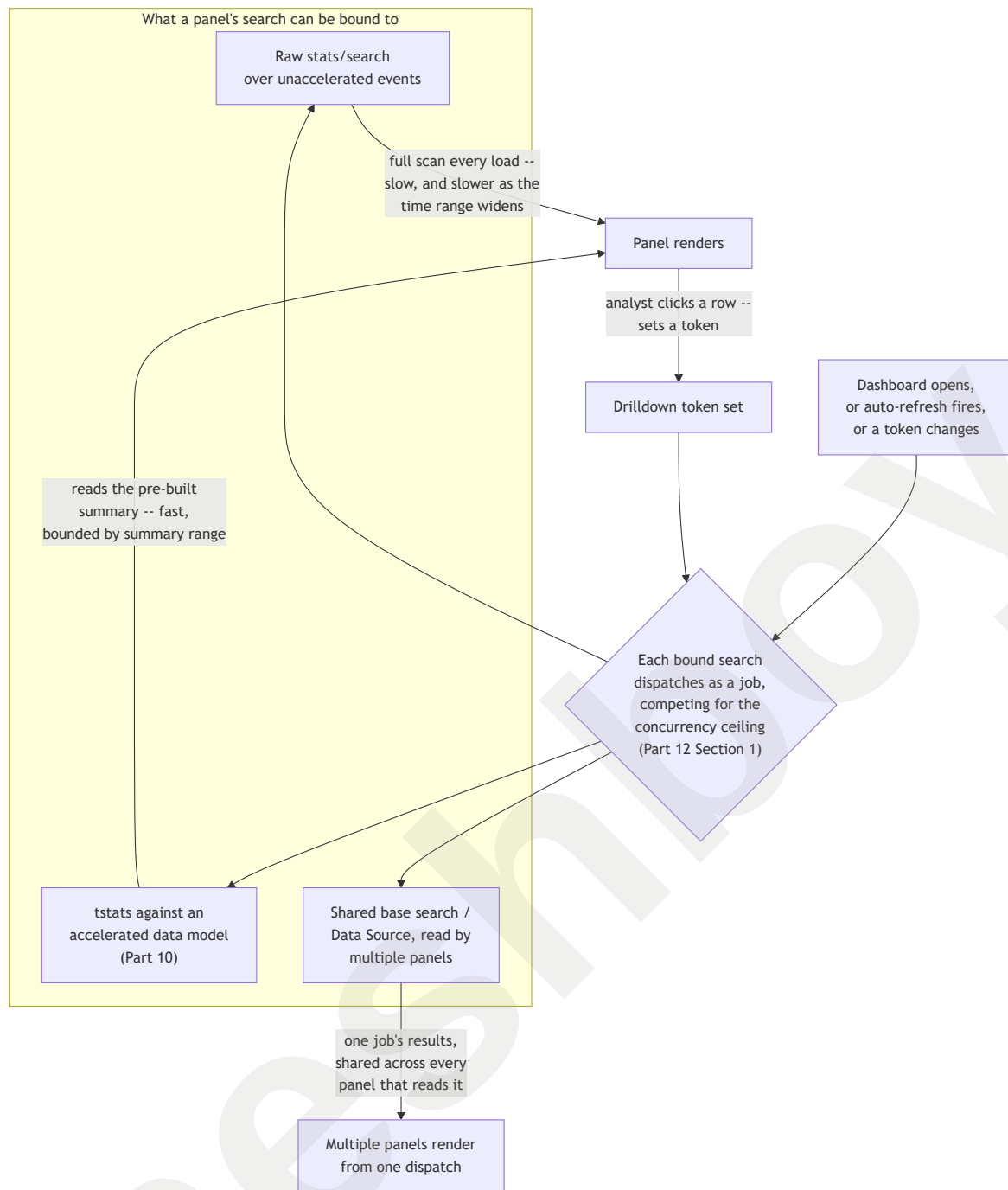


Figure 18.1 — From dashboard load to rendered panel, and the drilldown loop. *CONCEPTUAL*.

Illustrates the expected dispatch path a panel's search follows on load, refresh, or drilldown, and where an unaccelerated versus accelerated search binding and a shared base search change that path's cost. This is a sketch of documented expected mechanism, not a capture from a live Splunk search head or Job Inspector — none exists in this book's evidence base (STYLE-GUIDE.md §9.2).

2. Simple XML and Dashboard Studio: what's actually different

[PLATFORM ENGINEER]

The table below is the working reference for deciding which technology a given dashboard should use, or for orienting quickly inside one you didn't build. It is not exhaustive of either

technology's feature surface — it names the dimensions that actually change how you'd build or troubleshoot a panel.

Dimension	Simple XML (Classic)	Dashboard Studio
Definition format	XML view definition	JSON view definition
Primary editing surface	Drag-and-drop classic editor, or hand-edited XML source	Visual canvas builder (Absolute or Grid layout), or JSON source editor
Layout model	Fixed row-and-panel flow, top to bottom	Freeform "Absolute" canvas, or a responsive "Grid" canvas
Panel search binding	Inline <code><search></code> , a saved-report reference, or a shared base search read by later panels via <code><search base="..."></code> post-process chaining	A named data source (<code>ds</code>) definition, bindable to one or more visualizations
Drilldown/token model	<code><drilldown></code> and <code><change></code> blocks setting tokens consumed by other panels' searches	An equivalent token/interaction model, configured through the visual builder rather than hand-written condition blocks
Migration path	Native format; no conversion needed	Splunk provides a Classic-to-Studio conversion path; not every classic panel or custom visualization converts cleanly
Support status	Fully supported; existing Classic dashboards continue to run unmodified	Positioned by Splunk as the current recommended surface for building new dashboards

Product Version Note *Splunk has, over several recent major releases, positioned Dashboard Studio as the recommended experience for building new dashboards, while Classic (Simple XML) dashboards remain fully supported and continue to run without migration — this book could not verify, as of 2026-09-15, the exact release that made Dashboard Studio the default creation path in Splunk Web, or whether that default varies between Splunk Enterprise and Splunk Cloud Platform, against a reachable source: docs.splunk.com returned an HTTP 403 to this book's research tooling on every retrieval attempt made while drafting this part, and no product page or Splunkbase listing in [REFERENCES.md](#) covers dashboard-technology default behavior at this level of specificity. Treat "Dashboard Studio is the modern default, Classic is legacy-but-still-real" as directionally correct and treat any specific version number attached to that claim as unverified. What would make this stale: a further release narrowing or removing Classic dashboard creation from Splunk Web entirely, which Splunk has not done as of this writing but has not ruled out either — confirm against your installed version's own dashboard-creation menu before asserting which technology is the default for a specific environment.*

[PLATFORM ENGINEER]

The practical reading for a team maintaining an existing dashboard library: neither technology is going away on its own, and "which one is this dashboard" is the first question to answer before touching it, not an afterthought. A Classic dashboard's XML source is visible from its own edit menu; a Dashboard Studio dashboard's JSON source is visible the same way. Guessing wrong — editing a Classic dashboard's panel as though it exposed Dashboard Studio's data-source model, or vice

versa — produces confusing, technology-specific error messages that read like a broken dashboard rather than a wrong assumption about which format it's written in.

3. Panel searches, base searches, and what one dashboard load actually costs

[PLATFORM ENGINEER] A dashboard with ten panels, each bound to its own independent search, dispatches ten separate search jobs the moment it loads — and dispatches ten more on every auto-refresh cycle, and ten more for every analyst who opens the same dashboard in a separate browser tab. Part 12 §1 already established that a search head enforces a finite concurrency ceiling on how many searches can run at once; a dashboard is not exempt from that ceiling just because its searches are short and its panels look lightweight individually. A "SOC overview" dashboard left open on a wallboard, auto-refreshing every thirty seconds, with ten independently-searched panels, is a standing, recurring claim on the same concurrency budget Part 12 covers for scheduled correlation searches — and it competes for the same slots.

[PLATFORM ENGINEER] Simple XML's long-standing answer to that multiplication problem is the **base search**: one panel's `<search>` is given an `id`, and subsequent panels reference it with `<search base="...">` plus a post-process SPL fragment, rather than each declaring an independent search from raw events. Splunk dispatches the base search once; every panel chained to it as a post-process search runs its own additional aggregation, but against the base search's already-returned result rows, not against a fresh scan of the underlying index. That collapses ten independent jobs into one expensive job plus several cheap ones, which is the single highest-leverage change available to a dashboard author before touching acceleration at all.

```
<!-- CONCEPTUAL SAMPLE -- illustrative Simple XML fragment, not captured from a
      running dashboard. Shows one base search feeding two panels via post-process. -->
<panel>
  <table>
    <search id="base_auth_failures">
      <query>index=windows sourcetype=WinEventLog:Security EventCode=4625
| stats count by user, dest</query>
      <earliest>-24h</earliest>
    </search>
  </table>
</panel>
<panel>
  <chart>
    <search base="base_auth_failures">
      <query>| stats sum(count) as failures by dest | sort - failures | head 10</query>
    </search>
  </chart>
</panel>
```

[PLATFORM ENGINEER] Post-process search has a real limitation worth knowing before relying on it for every panel in a dense dashboard: the SPL available to a post-process fragment is restricted relative to what the base search itself can run, and a post-process search operates against a capped subset of the base search's returned rows rather than its full underlying event population — this book's own research pass could not confirm the current default row-cap value against a reachable source, so check the

relevant `limits.conf` stanza and your installed version's documentation directly before assuming a specific number, the same caution Part 10 §2.1 raises for `datamodels.conf`'s own undocumented-here defaults. Dashboard Studio's equivalent concept — a named data source referenced by more than one visualization — aims at the same problem from a different angle, sharing one dispatched job's results across every bound visualization; this book has not independently verified whether that sharing is implemented as the same kind of post-process chaining Classic dashboards use internally or as a different caching mechanism, and treats the practical outcome (fewer independent dispatches than one-search-per-panel) as the confirmed part, not the internal mechanism.

Engineering Reality *"The dashboard is slow" is almost never one problem. A dashboard with ten independently-searched panels can be slow because the search head is out of concurrency slots (Part 12 §1), because one specific panel's search is unfiltered and scanning far more raw data than it needs to (Part 12 §4's Job Inspector guidance applies to a panel's search exactly the way it applies to any other search job), or because nobody chained the panels into a shared base search and every refresh multiplies a fixable one-search cost by the panel count. Check the Job Inspector for the specific slow panel's own job — not the dashboard as a whole — before assuming the fix is acceleration, a base search, or a smaller time range; each of those three fixes a different bottleneck.*

4. Tokens and drilldowns: how one panel filters another

[SOC ANALYST] The interaction pattern that makes a dashboard more than a static report — click a row in one panel, and a second panel updates to show detail scoped to whatever was clicked — runs on tokens in both dashboard technologies. An input control (a dropdown, a time picker) or a drilldown action on a panel sets a named token to some value (a host, a user, a source IP); any other panel's search that references that token by name re-dispatches with that value substituted in, the moment the token changes. This is the mechanism behind the single most common SOC-dashboard layout: an overview panel showing many entities at once (top talkers, top failed-login users, top alerting hosts), with a detail panel below it that stays empty or shows an aggregate view until an analyst clicks one row in the overview, at which point the detail panel narrows to that one entity.

[SOC ANALYST] Part 17's asset and identity correlation is a natural token target: a drilldown from a notable-event-style overview panel into a token holding a specific host or user is exactly the pivot point where an analyst would next want that entity's asset criticality or identity context pulled in, whether that context arrives as a separate panel on the same dashboard or as a link into a dedicated asset-lookup view. Part 19 covers the fuller version of that pivot path across multiple views; this part's contribution is narrower — the token is the mechanism, and it's the same mechanism whether the next panel is a simple `stats` breakdown or a lookup-enriched detail view.

[DETECTION ENGINEER] Where a token's raw value needs transformation before a downstream panel's search can use it cleanly — stripping a domain suffix from a token holding a fully-qualified hostname, or building a wildcard pattern from a token holding a partial IP — that transformation is ordinary

`eval / rex` work inside the receiving panel's search, and DEH Part 26 owns that syntax; this part's job is only to flag that the transformation happens in the panel's SPL, not in some token-specific syntax unique to dashboards.

5. Accelerated searches: what actually keeps a panel from timing out at scale

[PLATFORM ENGINEER] Part 10 covers data model and report acceleration as a general pre-computation strategy; a dashboard is one of the two workloads (alongside Enterprise Security's own correlation searches) that benefits from it the most directly, because a dashboard's whole reason to exist is being opened and refreshed repeatedly by more than one person. A panel built as a plain `stats` search over three months of raw authentication events re-scans that entire three months, from `_raw`, every single time anyone opens the dashboard or the auto-refresh interval elapses. The same panel rewritten as a `tstats` search against the accelerated `Authentication` data model (the CIM data model covering login/logoff events across log sources, introduced in Part 5 and structured in Part 6) reads a pre-built summary instead — the exact mechanism Part 10 §2 describes — and that speed difference compounds every time the same dashboard is opened again, by the same analyst or a different one, in a way a single ad hoc query's speed difference never does.

[PLATFORM ENGINEER] That compounding is the specific reason acceleration is a dashboard-design decision, not just a query-optimization afterthought: a correlation search runs on its own fixed schedule regardless of whether anyone's looking, but a dashboard panel's cost is paid once per view, and a popular SOC-overview dashboard left open on several analysts' second monitors all shift can rack up far more repeated executions of the same expensive raw scan than a correlation search running once every five minutes ever would. Before building a dashboard panel meant to be viewed constantly throughout a shift, check whether the data model it needs is already accelerated (Part 10 §2), and if it isn't, weigh that acceleration's storage and backfill cost (Part 10 §2.1) against the alternative — an expensive raw search, paid over and over, by every viewer, every refresh, indefinitely.

Blind Spot Part 10 §4 names the summary-range blind spot: a `tstats` search against an accelerated data model returns silently incomplete results, with no error, for any time range extending past the model's configured summary coverage. On a dashboard, that failure mode has a specific, easy-to-miss shape: a trend panel with a time-range picker lets an analyst widen the window past the underlying data model's summary range with one click, and the panel simply renders a shorter, real trend that looks exactly as complete as a longer one would — a chart with six months of real, correctly-computed data looks the same as a chart that silently dropped everything before the summary range began. A dashboard is worse exposure for this blind spot than a one-off query, specifically because a dashboard's time-range picker invites exactly the widening that triggers it, repeatedly, from whichever analyst happens to be looking that day.

Detection Autopsy — the trend panel that quietly lost its own history

The rule: a SOC-overview dashboard's "failed authentication trend, last 12 months" panel, built as `tstats` against the accelerated `Authentication` data model, rendering a clean monthly line chart every time anyone opened the dashboard.

Why it shipped: the panel was built and validated while the data model's `acceleration.earliest_time` was configured for a full year of coverage, and the chart's own visual output — a smooth twelve-month line — gave no indication of how far back the underlying summary actually reached; it looked the same whether the summary covered twelve months or three.

How it failed: a platform-engineering change months later, made to cut acceleration storage cost (Part 10 §2.1's tradeoff, applied in the cost-cutting direction), shortened the data model's summary range to three months. Nobody updated the dashboard, because nothing about the dashboard referenced the summary range explicitly, and the panel kept rendering — now showing three months of real data as if it were still twelve, with the oldest nine months simply absent and no visual cue distinguishing "quiet quarter" from "never summarized."

The fix: a standing convention that any panel whose usefulness depends on a specific summary range states that dependency next to the panel — in its title or description text, not buried in `datamodels.conf` — so a later acceleration-range change is caught by a dashboard-review step, not discovered by an analyst wondering why a trend that used to show a full year now looks unusually flat before month four. This is a documented failure pattern consistent with the summary-range mechanics Part 10 §4 already establishes, not a specific cited incident — no real Splunk deployment backs this account in this book's own evidence base (STYLE-GUIDE.md §9.2).

6. Panel-design patterns for SOC consumption

[SOC ANALYST] Not every SOC dashboard is built for the same kind of looking. A **wallboard** — a dashboard displayed on a shared monitor, glanced at from across a room, never clicked — needs a small number of high-contrast, immediately legible panels: single-value tiles with color thresholds (green/amber/red against a defined severity band), simple trend lines, nothing that requires reading a table's fine print from six feet away. An **investigative dashboard** — one an analyst opens, sits with, and interacts with during active triage — can and should carry more density, more tables, and more drilldown-driven detail panels, because the reading pattern is close-up and deliberate rather than a passing glance. Building one dashboard trying to serve both purposes at once routinely serves neither: a wallboard cluttered with detail tables nobody at a distance can read, or an investigative dashboard stripped down to wallboard-style single-value tiles that force an analyst into raw search for anything specific.

Dimension	Wallboard (passive viewing)	Investigative dashboard (analyst-driven)
Panel density	Low — a handful of large, high-contrast panels	Higher — tables and detail panels an analyst reads up close
Interactivity	None assumed; no drilldowns designed to be clicked	Token-driven drilldowns (§4) as the primary navigation pattern
Refresh behavior	Short auto-refresh interval, but real-time search avoided per Part 12 §7's sustained-cost warning	Refreshed on demand or on a longer interval; real-time reserved for genuinely time-critical panels only
Visualization choice	Single-value tiles with color thresholds, simple trend lines	Tables, multi-series charts, and Trellis-split panels breaking one chart into a grid of per-entity small multiples
Primary failure mode if misdesigned	Panels nobody can read from a distance; color thresholds set arbitrarily rather than against a defined severity band	Overview panels with no drilldown path, forcing analysts into raw search for routine detail lookups

[SOC ANALYST] The color-threshold row deserves one specific caution: a single-value tile's red/amber/green boundary is a claim about what "bad" means for that specific metric, and setting it without reference to an actual severity definition — copying a default threshold, or picking round numbers that felt reasonable when the panel was built — produces a wallboard that looks alarming or looks calm for reasons that have nothing to do with the SOC's actual risk tolerance. Tie a threshold to something Part 16 already defines (a notable's urgency/severity banding) or to a documented SOC-specific baseline, not to whatever number made the panel look right in a screenshot during a demo.

[PLATFORM ENGINEER] Trellis layout — splitting one visualization into a grid of smaller multiples, one per value of a chosen split-by field, instead of overlaying every series on a single chart — is a long-standing Splunk visualization option well suited to exactly the "one panel per host" or "one panel per analyst queue" pattern an investigative dashboard often wants, without hand-building a separate panel and separate search for every entity. This book has not verified Trellis's current feature parity between Simple XML and Dashboard Studio in detail; treat it as available in some form in both, and confirm the specific configuration surface against whichever technology a given dashboard uses (§2 above) before assuming identical behavior across the two.

7. SOC Management View: dashboard sprawl as an unbudgeted cost

SOC Management View *A correlation search's cost is visible in one place — its own `savedsearches.conf` schedule, auditable and countable (Part 14). A dashboard's cost is not visible the same way, because it isn't paid on a schedule; it's paid every time someone opens a browser tab, and Splunk does not hand a SOC manager a single report titled "how many dashboard tabs are open right now, and what are they costing the search head." A SOC that has carefully tuned its correlation-search concurrency budget (Part 12 §1) can still watch that same budget erode from an unrelated direction — a popular, unaccelerated, frequently-refreshed dashboard that a dozen analysts each keep open all shift, none of whom individually did anything wrong. Budgeting for dashboards means budgeting for a workload with no fixed schedule and no simple headcount, which is a genuinely harder planning problem than budgeting for a content library of scheduled searches — treat "how many dashboards do we have, and which ones are accelerated versus raw" as a standing inventory question, not a one-time build decision, the same discipline Part 4 already asks for around ingest and licensing generally.*

[SOC MANAGEMENT] The practical lever available here is the same one named throughout this part: acceleration (§5) turns a dashboard's per-view cost from "pays for a raw scan every time" into "reads a bounded, shared summary," and a base search or shared data source (§3) turns a ten-panel dashboard's per-view cost from ten dispatches into one expensive dispatch plus several cheap ones. Neither fix is free — acceleration costs storage and backfill time (Part 10 §2.1), and a shared base search constrains what a chained panel's SPL can do (§3) — but both are cheaper, in the long run, than discovering a dashboard-driven concurrency problem the same way a skipped correlation search is usually discovered: after the fact, during an investigation into why something that should have been visible wasn't.

8. Dashboards as maintained knowledge objects

[PLATFORM ENGINEER] A dashboard is a knowledge object with the same permission model Part 9 describes for macros and Part 8 describes for lookups: an owner, a sharing level (private to that user, shared across one app, or shared globally across the instance), and the same promotion-across-environments question every other piece of detection or reporting content faces — a dashboard built and tested in a development search head has to be deliberately migrated to production, not assumed to appear there automatically because it works locally. Treat a dashboard the way Part 14 treats a correlation search: something with a lifecycle, not a one-time build.

[PLATFORM ENGINEER] The drift risk that follows is the same shape Part 9 already names for macro sprawl, applied to panels instead of shared logic: a dashboard panel referencing a saved report, a macro, or a lookup by name has no built-in mechanism that warns its owner when that referenced object is renamed, re-scoped, or deleted elsewhere. The panel doesn't error loudly — it returns an empty result or a Splunk error message an analyst reads as "the dashboard is broken today" rather than "someone renamed a macro three panels' worth of dependencies away, and this dashboard was never checked." A dashboard library of any real size benefits from the same discipline Part 9 recommends for

macros: know what depends on what, and review dependent dashboards before renaming or removing a shared object out from under them.

9. What this part hasn't verified

What Would Change My Mind This part's central technology-comparison claim — that Dashboard Studio is positioned as the current recommended path for new dashboards while Classic dashboards remain fully supported — is stated with real confidence, because it's consistent with the general direction Splunk's own product messaging has taken across recent releases. The specific version boundary, the exact feature-parity gaps between the two technologies (Trellis behavior, post-process versus data-source result-sharing internals, §3 and §6 above), and the actual concurrency and Job Inspector cost difference a real accelerated versus unaccelerated dashboard panel produces under real analyst load are all claims this part could not verify against a reachable, current source during drafting, and all rest on documented mechanism rather than a measured result — no Splunk deployment exists in this book's evidence base (STYLE-GUIDE.md §9.2). A real Splunk instance, with a real dashboard library, observed through its own Job Inspector and Monitoring Console under real multi-analyst viewing load over a real shift, is the specific missing evidence that would move this part's cost claims from "documented mechanism, reasoned through" to "measured and confirmed" — and it's exactly the kind of observation Part 20 names as outstanding across this book as a whole.

Cross-references: DEH Part 26 (Splunk SPL — panel search syntax this part assumes rather than re-teaches) · Part 5–6 (the Common Information Model and data models — the `Authentication` data model used in §5's example) · Part 8–9 (lookup tables and macros — the knowledge-object permission and drift model §8 extends to dashboards) · Part 10 (data model and report acceleration — the mechanism behind §5's panel-speed argument and the summary-range blind spot in §5's Detection Autopsy) · Part 12 (search performance and tuning at scale — the concurrency ceiling, Job Inspector, and real-time search cost §3, §6, and §7 all lean on directly) · Part 14 (correlation searches — the content-lifecycle discipline §8 mirrors for dashboards) · Part 16 (notable events and Incident Review — the flagship ES dashboard this part's panel-design guidance applies to without re-teaching its triage workflow) · Part 17 (asset and identity correlation — a natural drilldown target from §4's token mechanism) · Part 19 (Splunk-specific investigation workflows — the fuller multi-view pivot path this part's single-dashboard scope feeds into) · Part 20 (validation gaps and the path to real evidence).

Part 19 — Splunk-Specific Investigation Workflows

Why this part exists

[CONCEPT] Parts 14 through 17 each built one piece of Enterprise Security's investigation surface in isolation: how a correlation search turns a match into a notable or a risk-score contribution (Part 14), how risk-based alerting accumulates score against a risk object instead of firing one notable per match (Part 15), how an analyst works a notable inside Incident Review (Part 16), and how the Asset and Identity framework enriches that notable with host and user context (Part 17). None of those parts followed the actual path an analyst's mouse and keyboard take once a notable lands in their queue. This part follows that path — from the notable itself, into a risk object's contributing events if the notable is risk-based, out into a raw search against the underlying data model, sideways into asset and identity context, and, where the investigation grows past one notable, into a case container that holds all of it together. Nothing here is a new mechanism; everything here already exists in Parts 14–17. This part's contribution is the sequence, and the specific points in that sequence where Splunk's own tooling has renamed or relocated itself across recent Enterprise Security releases — a risk Part 13 §6 named in advance and handed forward to here.

This part does not re-teach SPL syntax (DEH Part 26 owns that), does not re-derive how a correlation search's adaptive response actions are configured (Part 14), does not re-explain risk-score accumulation mechanics (Part 15), and does not re-teach the Asset and Identity framework's own enrichment logic (Part 17) — each gets a one-clause pointer where the pivot path touches it. What follows assumes DET-26-01, deployed as the "Suspicious LSASS Memory Access - Non-Allowlisted Process" correlation search built in Part 14 §4, as the running example throughout, the same way Part 14 assumed DEH Part 23's DET-23-01 and DEH Part 26's DET-26-01 without renumbering either.

1. The pivot path, end to end

[SOC ANALYST] An analyst who opens a notable in Incident Review (the analyst-facing dashboard for working notable events, introduced in Part 16) is looking at one row: a title, an urgency and severity, an owner and status field, and a handful of fields the correlation search's own results carried into the notable. Everything past that row is a pivot — a deliberate action that leaves the notable and goes looking at something else the notable points to. Five pivots recur often enough across a real Enterprise Security deployment to name as a single path, even though not every investigation uses all five and a traditional per-event notable skips the second one entirely:

1. **Notable → drilldown search.** The analyst runs the search the correlation search's own author built for exactly this notable, populated with this notable's own field values (§2).
2. **Notable → contributing events**, if and only if the notable was generated by risk-based alerting rather than a traditional per-match correlation search — the analyst needs to see which of the risk object's several contributing events actually crossed the threshold, not just the aggregate score (§3).

3. **Drilldown or contributing-events result** → **raw search against the underlying data model**, when the notable's own fields aren't enough and the analyst needs to see everything else the same host, user, or process touched, via Pivot, ad hoc SPL, or a `tstats` sweep against an accelerated data model (§4).
4. **Any of the above** → **asset and identity context**, pulling in the host's criticality rating or the user's department and manager from the Asset and Identity framework (§5, Part 17's subject in depth).
5. **The whole investigation** → **a case container**, once more than one notable, more than one analyst, or more than one shift is involved and the investigation has outgrown a single Incident Review row (§6).

flowchart TB

```
N["Notable event\n(Incident Review, Part 16)"]
N -->|"drilldown_search"| D["Drilldown search\n(populated with notable's own fields)"]
N -->|"if RBA-generated"| R["Risk object\ncontributing events\n(risk index, Part 15)"]
D --> RAW["Raw search / Pivot / tstats sweep\nagainst the underlying data model\n(Part 6 §4, Part 10 §2)"]
R --> RAW
RAW --> AI["Asset and Identity context\n(Part 17)"]
D --> AI
R --> AI
N --> WB["Investigation Workbench\n(case container, §6)"]
RAW --> WB
AI --> WB
```

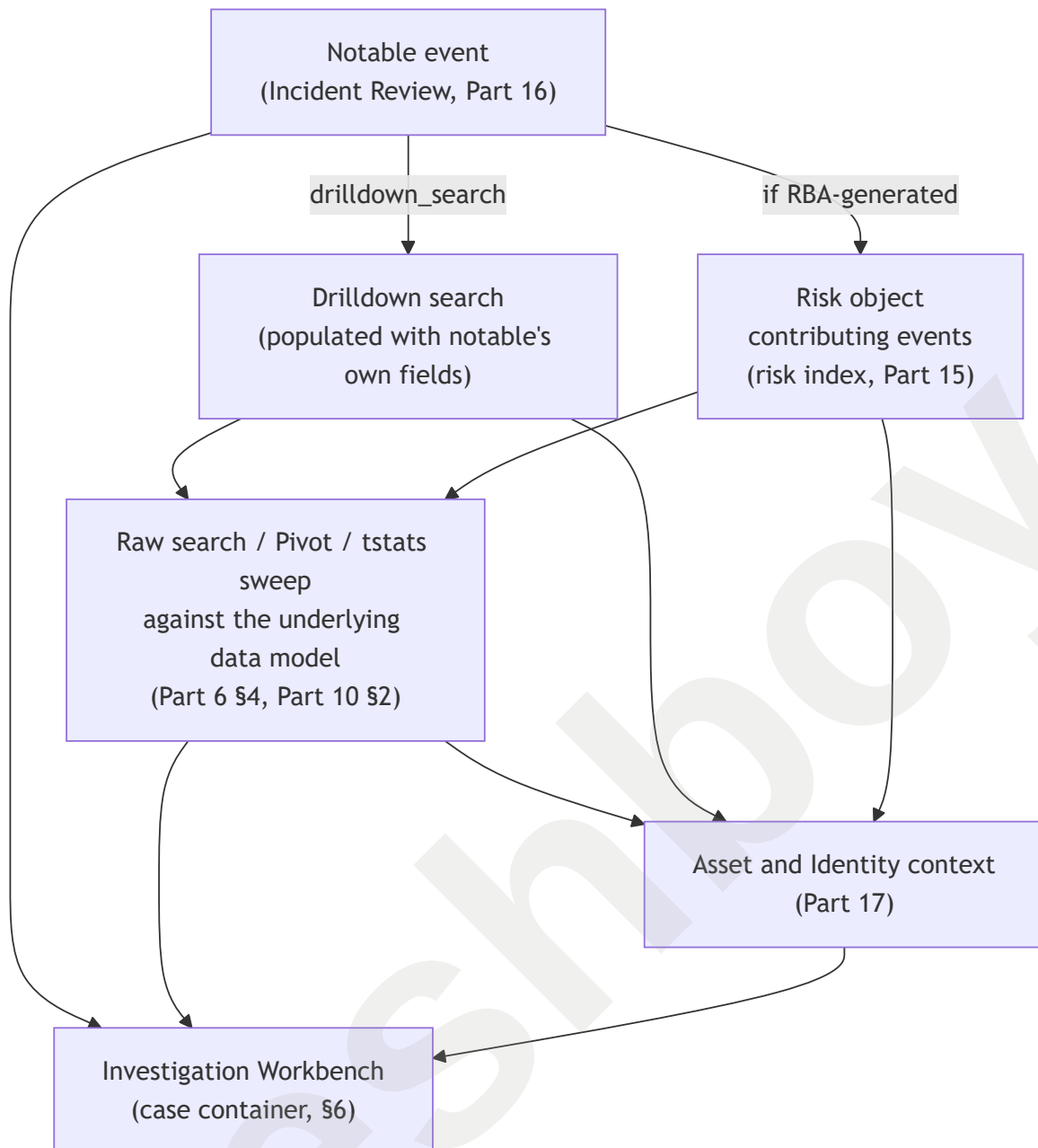


Figure 19.1 — The Enterprise Security investigation pivot path. *CONCEPTUAL.* Illustrates the sequence of pivots an analyst or threat hunter follows outward from a single notable event, through its drilldown search or risk-object contributing events, into raw search against the underlying CIM data model, sideways into asset and identity context, and — where the investigation spans more than one notable — into a case container. This is a structural diagram of documented expected navigation, not a capture from a live Enterprise Security instance; no such instance exists in this book's evidence base (see STYLE-GUIDE.md §9.2).

[THREAT HUNTER] The same five pivots run in reverse for a threat hunter who starts from a hypothesis instead of a notable — Pivot or an ad hoc `tstats` sweep first, a candidate notable-worthy pattern second, asset and identity context to decide whether it's worth escalating third. §4 covers that direction explicitly; the path itself doesn't change shape depending on which end you start from.

2. From a notable to its own drilldown search

action.notable.param.drilldown_search — the analyst's first click

[SOC ANALYST] Every notable in Incident Review carries a drilldown link, and what that link actually runs depends entirely on whether the correlation search that generated the notable defined one. Part 14 §4.1 already showed the concrete case: the "Suspicious LSASS Memory Access - Non-Allowlisted Process" correlation search's stanza sets `action.notable.param.drilldown_search` to a query that substitutes the notable's own `ComputerName` and `SourceImage` field values in place of the token syntax (`$ComputerName$`, `$SourceImage$`) the stanza defines them with. Clicking that notable's drilldown in Incident Review dispatches exactly that search, with those two tokens filled in from the specific notable's own fields — not a generic re-run of the correlation search's full query, and not a search the analyst has to write by hand.

[SOC ANALYST] When a correlation search's author never sets `action.notable.param.drilldown_search`, Enterprise Security falls back to a default drilldown built from the notable's own indexed fields and the correlation search's original time range — usable, but generic, and it re-runs the correlation search's full query rather than a query an analyst actually wants at triage time (which is almost always narrower: this host, this process, not every host the correlation search ever matches). A correlation search with no custom drilldown is not broken; it just hands the analyst a blunter first pivot than one that was deliberately authored, which is why Part 14 §4.1's example sets one explicitly rather than relying on the default.

Hunter's Note A drilldown search is scoped to convince an analyst the notable is real, not to bound the investigation. If the LSASS-access notable's drilldown comes back confirming the match, don't stop at the drilldown's own time window — widen `dispatch.earliest_time` well past the notable's own and check whether the same `SourceImage` touched any other `ComputerName` in that window before you close it as a single-host event. The drilldown search answers "did this notable's own claim actually happen"; it was never scoped to answer "is this contained to one host," and treating a confirmed drilldown as proof of scope is the single most common way a real lateral-movement pattern gets missed at the first pivot.

[PLATFORM ENGINEER] Mechanically, the drilldown search dispatches as an ordinary ad hoc search against whatever indexes and sourcetypes its own SPL names — it consumes a search-slot the same way any other search does (Part 12's concurrency-limit content applies to it identically), and it is not itself accelerated or otherwise privileged just because a notable launched it. A drilldown search written against raw `index=windows` data, rather than against the accelerated data model the correlation search itself queried, pays the same non-accelerated search cost Part 10 covers for any raw search over a wide time range — worth remembering before widening a drilldown's time range across a busy index and being surprised at how long it takes to return.

3. Contributing events: pivoting from a risk object to what actually fired

[DETECTION ENGINEER]

A traditional, notable-generating correlation search's notable is self-contained: the fields on the notable are the fields the search's own matching result row carried, and there is nothing further to look up to know what fired. A risk-based notable is different in a way Part 15 covers in depth and this section only needs the consequence of: the notable exists because a risk object's *accumulated* score crossed a threshold, and that accumulation is the sum of however many separate risk-index events — each written by whatever correlation search actually matched — landed against that risk object inside the accumulation window. The notable itself typically carries the risk object's identity and its total score, not the individual matches behind it. Finding out what actually fired is a second pivot, not something already sitting on the notable.

[SOC ANALYST]

That second pivot is a search against the `risk` index (the Enterprise Security index every risk-scoring adaptive response action writes to, introduced in Part 13 §4 and covered as a retained dataset in its own right in Part 15), filtered to the specific risk object named on the notable and a time window that covers the accumulation period the risk-based correlation search actually used — not an arbitrarily wide window, for the reason the Blind Spot box below names.

```
index=risk risk_object="WIN-DB01" earliest=-24h
| stats count, values(source) as contributing_searches, sum(risk_score) as total_risk_score
  by risk_object, risk_object_type
```

This query targets the same `risk` index Enterprise Security's own Risk Analysis dashboard reads from; its limitation is that a raw `index=risk` search over a wide window pays the unaccelerated search cost Part 10 §2 covers, where the dashboard's own panels are more likely built against an accelerated summary for exactly that reason — treat this query as the ad hoc, investigation-time version of what a dashboard panel does at scale, not a replacement for one.

Blind Spot A risk object accumulates score continuously, not only inside the specific window that happened to cross the threshold this particular time. Searching `index=risk` over a window wider than the accumulation period a risk-based correlation search actually used pulls in risk events from correlation searches that never contributed to this threshold crossing — a risk object that crossed its threshold on Tuesday from three specific matches can show a dozen unrelated risk events from the preceding week in a loosely-windowed search, and an analyst who reads all dozen as "contributing" to Tuesday's notable is misattributing cause to events the actual RBA correlation search's own window never counted. Confirm the accumulation window a given risk-based correlation search actually uses (Part 15's subject) before treating every risk-index hit in a convenient time range as evidence behind the specific notable in front of you.

[THREAT HUNTER]

The same `index=risk` search is a hunting entry point independent of any notable: a risk object with a rising score that hasn't yet crossed its threshold is, by construction, not a notable yet — searching `index=risk` directly for a specific host or user ahead of any notable existing is the only way to see that accumulation building, and it's the risk-based-alerting-specific version of the general principle that waiting for the alert is not the same discipline as hunting for the pattern the alert is built to eventually catch.

4. From a data model to raw search: Pivot, ad hoc SPL, and a tstats sweep

[THREAT HUNTER] Once a drilldown or a contributing-events search confirms a notable is worth pursuing further, the next pivot leaves both the notable and the risk index behind and goes straight at the underlying CIM data model (Part 6) the correlation search itself queried — the Authentication data model, the Endpoint data model, or whichever data model the analytic's own field selection targets. This part does not re-teach how a data model is structured or how Pivot builds a query against one; Part 6 §4 owns Pivot's own building blocks, and Part 10 §2 owns the `tstats` command that reads an accelerated data model's `tsidx` summary directly rather than touching raw events. What this section adds is only the investigation-specific framing: which of the three query paths — Pivot, ad hoc SPL against the raw data, or a `tstats` sweep against the accelerated summary — fits an analyst mid-investigation, and why the answer usually isn't the same one that fits the correlation search itself.

Query path	Fits when	Cost profile	Owning part
Pivot against the data model	Analyst doesn't write SPL directly, or needs a fast visual sweep across a dataset's own fields	Same cost as the SPL Pivot generates underneath — no shortcut, just a UI over it	Part 6 §4
Ad hoc SPL against raw indexed data	The field the analyst needs isn't in the data model yet, or the investigation needs <code>_raw</code> itself	Unaccelerated; scales with the raw event volume in the searched window	DEH Part 26 (syntax); Part 10 §2 (cost vs. acceleration)
<code>tstats</code> against an accelerated data model	The data model is already accelerated and the investigation only needs the model's own summarized fields, not <code>_raw</code>	Reads the <code>tsidx</code> summary directly; far cheaper than raw search over the same window, bounded by the summary's own range (Part 10 §4)	Part 10 §2, §4

[THREAT HUNTER] The table above is the decision this section supports: an analyst confirming a single notable's own claim usually reaches for Pivot or a narrow ad hoc search, because the investigation is bounded and the field it needs is already on the notable. A hunter sweeping the same data model across every host for the same pattern, with no single notable bounding the search, reaches for `tstats` instead — the same data, at the scale where the raw-search cost difference actually matters. Both are the same underlying data model; only the query path and the scope of the question change.

Detection Test This book has no live Enterprise Security deployment to run the check below against — it states what to run against your own instance, not a result already observed here.

Setup: The "Suspicious LSASS Memory Access - Non-Allowlisted Process" correlation search (Part 14 §4) deployed, with its target sourcetype tagged into whichever CIM data model your own Endpoint or Authentication mapping uses for process-access events. **Action:** From a confirmed notable, run a `tstats` sweep against that same accelerated data model for the same `SourceImage` across every `ComputerName` in the summary's range, rather than the single host the notable named. **Expected result:** A result count and field shape consistent with what the Job Inspector reports for a `tstats`-backed search against that data model — materially faster than the equivalent raw search over the same window — and, if the same `SourceImage` shows up against a second host the original notable never named, evidence the investigation's scope was wider than one notable suggested. If the sweep takes as long as a raw search, the data model likely isn't actually accelerated for the range being searched; check Part 10 §4's summary-range guidance before assuming the sweep itself is broken.

5. Asset and identity context along the way

[SOC ANALYST] Every pivot above eventually needs an answer to a question none of the data model's own fields carry: how much does this host or user matter, and who owns it. That's the Asset and Identity framework's job, covered in depth in Part 17 — this section names only where it enters the pivot path, not how its own enrichment lookups are built or maintained. In Incident Review itself, a notable's host and user fields typically carry an inline asset/identity context panel already populated from the framework's own lookups (asset criticality, category, and owner for a host; department, manager, and title for a user) without a separate search — the enrichment already happened at notable-creation time, via the same correlation-search machinery Part 17 covers. The pivot an analyst actually performs is usually a *verification* pivot: confirming the inline criticality rating is still current, not running a fresh lookup from scratch, because Part 17's own Blind Spot content covers exactly how that inline rating goes stale (a decommissioned host that never left the asset lookup, reassigned and carrying an old, wrong criticality rating into every notable that touches it afterward).

[SOC ANALYST] The practical consequence for this part's pivot path: asset and identity context is available earliest, not latest, in the sequence Figure 19.1 draws — it's already sitting on the notable before the analyst clicks anything, which is exactly why it's easy to skip verifying it and treat an inline criticality badge as settled fact rather than a lookup value with its own staleness risk. Part 17 owns the fix (a review cadence for the underlying asset/identity lookup data); this part's job is only to flag that "asset context is already there" and "asset context is current" are two different claims, and the pivot path above never actually forces an analyst to check the second one unless they deliberately do.

6. Building a case around more than one notable

Investigation Workbench — building a timeline and artifact set around a notable

[SOC ANALYST]

Once an investigation spans more than one notable — the LSASS-access notable plus a second notable from a different correlation search against the same host, discovered via the `tstats` sweep in §4 — Incident Review's own row-per-notable view stops being the right unit of work. Enterprise Security has, across a long span of releases, shipped a case-management surface, commonly referred to by its documented name Investigation Workbench, for exactly this situation: a container an analyst builds around a set of notables and other artifacts (raw search results, external indicators, free-text notes) with its own timeline of analyst actions, its own status and collaborator assignment, and its own boundary distinct from any single notable's own status field in Incident Review.

Product Version Note Enterprise Security has documented a dedicated case-management surface under the name Investigation Workbench across multiple past releases, supporting notable/artifact aggregation, a timeline view, and collaborator assignment on a container broader than one notable. This book's own research for this part could not freshly verify Investigation Workbench's current name or feature scope against a reachable source as of 2026-09-15: Splunk's own documentation domain (`docs.splunk.com`) was not reliably fetchable by this book's research tooling, per `STYLE-GUIDE.md` §9.4's disclosed fallback policy, and neither the current Enterprise Security product page nor the Enterprise Security Splunkbase listing — both retrieved successfully on 2026-09-15 — name Investigation Workbench, Incident Review, or Mission Control anywhere in their own marketing copy (`REFERENCES.md` entries `[SPLUNK-ES-PRODUCT-PAGE]`, `[ES-SPLUNKBASE]`); those two pages instead foreground Detection Studio, an "AI Assistant," risk-based alerting, SOAR, and UEBA as the current headline capabilities. That silence is evidence about what Splunk is marketing right now, not evidence that Investigation Workbench has been renamed or removed — treat the feature's continued existence as likely given its long documentation history, but confirm its current name and location against your own installed ES version's own in-product help before writing an internal runbook that names it.

[SOC ANALYST]

Artifacts added to an investigation — a captured asset-criticality value, a specific search result, a note pasted from a raw event — are a snapshot at the moment they were added, not a live query. The same staleness risk Part 17 names for an inline asset/identity badge in Incident Review (§5 above) applies again here, one layer further removed: a criticality rating pinned into an investigation's artifact list six weeks ago doesn't update itself when the underlying asset lookup changes, and a closed-then-reopened investigation can end up presenting an analyst with a stale snapshot next to fields elsewhere in the same UI that have since moved on. Treat an investigation's own artifacts as a dated record of what was true when captured, not a live view — the same discipline Part 8 asks of any maintained lookup, applied to a case file instead of a CSV.

7. The investigation surface's own naming churn

[SOC MANAGEMENT]

Part 13 §6 named this theme in advance from the correlation-search-authoring side: whatever a given release calls the surface for building a correlation search, the underlying `savedsearches.conf` stanza it produces is unchanged. The investigation side of the same theme matters

more in practice, because an analyst's actual workflow — which menu item to click, which surface a screenshot in last year's training material still points at — is far more sensitive to a renamed or relocated UI feature than a detection engineer's `.conf`-level view of the object underneath it. Two confirmed data points from this book's own research anchor that claim rather than leaving it as an assertion:

- Splunk's Enterprise Security product page and Splunkbase listing, both retrieved 2026-09-15, market "AI Assistant" as delivering "instant, tailored investigation guidance, simplified query creation, clear summaries, and automated reports" (REFERENCES.md entry [SPLUNK-ES-PRODUCT-PAGE]) — a capability squarely inside this part's own subject (query creation and investigation guidance) branded and marketed as new, sitting alongside, not replacing, the investigation surfaces this part describes.
- The same two pages name Detection Studio, AI Assistant, risk-based alerting, SOAR, and UEBA as the product's headline capabilities and do not mention Incident Review, Investigation Workbench, or a previously-marketed unified case-management surface once branded Mission Control anywhere in their current copy.

Product Version Note *Splunk has marketed an "AI Assistant" for Enterprise Security offering investigation guidance, query-building assistance, and automated summarization, positioned by Splunk's own product materials as available across both current editions (Essentials and Premier — see Part 13 §2's edition table). As of 2026-09-15, verified against Splunk's own Enterprise Security product page and Splunkbase listing (REFERENCES.md entries [SPLUNK-ES-PRODUCT-PAGE], [ES-SPLUNKBASE]). What would make this stale: an edition-gating change moving the AI Assistant behind Premier specifically, a rename, or a scope change from "guidance and summarization" to something Splunk markets as replacing rather than assisting the pivot path this part describes — none of which this book's evidence base can currently rule in or out for a future release.*

[SOC MANAGEMENT]

The staffing consequence is direct: a SOC that trains analysts once, against one release's menu layout and feature names, and never revisits that training is the specific failure mode this naming churn creates, independent of whether the underlying pivot path in Figure 19.1 has actually changed shape. Budget investigation-workflow training as a recurring cost tied to Enterprise Security's own upgrade cadence, not a one-time onboarding task — the mechanics in §1 through §6 above are stable across the churn; the menu labels and screenshots pointing analysts at them are not, and a training deck two major releases out of date is a real, if minor, source of the same kind of friction Part 12 names for a search-concurrency bottleneck: not a failure, just a cost nobody scheduled.

8. SOC management view: measuring pivot friction, not just notable volume

SOC Management View Every metric this book has named so far for Enterprise Security's own health — notable volume (Part 15), search concurrency (Part 12), CIM coverage (Parts 5–7) — measures the platform. None of them measure how long an analyst actually spends moving through the pivot path this part describes before reaching a disposition. Mean-time-to-triage figures that only start the clock at notable creation and stop it at status change miss every minute spent in a drilldown search, a contributing-events lookup, a `tstats` sweep, or an Investigation Workbench artifact review in between — and a SOC that only ever tunes correlation-search thresholds to manage analyst load is optimizing the input side of a process whose actual bottleneck, on plenty of real deployments, is the pivot path itself: how many clicks and separate searches it takes to go from "notable exists" to "disposition made." Instrumenting the pivot path's own friction — even something as coarse as counting how many separate searches an analyst runs per closed notable — is the specific, checkable measurement most SOC-management notable-volume metrics never capture.

[SOC MANAGEMENT] The practical lever available here doesn't require new tooling: Splunk's own audit logging already records every search an analyst dispatches, so "searches run per closed notable" is a query against data the platform is already collecting, not a new instrumentation project. Baseline it before, not after, a release that renames or relocates the investigation surfaces in §7 — a friction count taken against one release's menu layout doesn't compare cleanly to one taken after the surfaces analysts click through have moved, and treating the two counts as directly comparable without checking §7's naming churn first risks reading a UI change as a training win or a training failure it isn't.

Closing: where this hands off

[CONCEPT] This part traced one path through machinery Parts 14–17 each built independently: a notable's drilldown search, a risk object's contributing events, a raw or accelerated sweep against the data model underneath, asset and identity context riding along for most of it, and a case container once the investigation outgrows one notable. None of the five pivots in Figure 19.1 is new; what's new here is naming them as one sequence and flagging, honestly, where this book could and couldn't verify the current name of the surface an analyst actually clicks through to run each one. Part 20 closes the book by taking that same honesty inventory — what's verified, what's conceptual, what a real Enterprise Security deployment would be needed to confirm — across every part in this book at once, this part's Investigation Workbench and Mission Control uncertainty included.

Cross-references: DEH Part 26 (Splunk SPL); this book's Part 6 §4 (Pivot), Part 8 (Lookup Tables as Detection Infrastructure), Part 10 §2/§4 (`tstats` and the summary-range blind spot), Part 12 (Search Performance and Tuning at Scale), Part 13 §4/§6 (Enterprise Security Architecture and the naming-churn theme), Part 14 §4 (the "Suspicious LSASS Memory Access - Non-Allowlisted Process" correlation search), Part 15 (Risk-Based Alerting), Part 16 (Notable Events and the Incident Review Workflow), Part 17 (Asset and Identity Correlation), Part 20 (Validation Gaps and the Path to Real Evidence).

Part 20 — What This Book Doesn't Know Yet: Validation Gaps and the Path to Real Evidence

Why this part exists

[CONCEPT] Part 1 §3 made a promise this part exists to keep: "Part 20 closes the book by inventorying every one of those notes and marks in one place, organized by what specific observation would upgrade each claim." Nineteen parts later, that inventory is due. This part does not introduce a single new platform mechanism, correlation search, or dashboard pattern — every technical claim it needs was already made, once, in Parts 1–19, each carrying its own Product Version Note or evidence-class tag per `STYLE-GUIDE.md` §6.9 and §9. What this part adds is synthesis: pulling those scattered notes into one place, sorted by what kind of gap they actually represent, so a reader planning to act on this book's guidance can see in a single pass which claims are load-bearing documentation, which are this book's own labeled illustrations, and — for every one of them — what specific, checkable observation would move it from "documented" to "verified."

That structure mirrors DEH's own closing pattern. DEH's Part 40 (Detection Autopsy) and Part 43 (Detection Debt) both close out earlier material as synthesis rather than new subject matter; this part plays the same role here, made more load-bearing by this book's evidentiary position than either DEH capstone needed to be, because this book's gap isn't "some detections decay" — it's "no claim in this book has been checked against a running instance of the product it's about."

1. What this part does, and does not, do

[SOC MANAGEMENT] This part is a ledger, not a lab report. It does not run a `tstats` search, inspect a Job Inspector output, or observe a notable event fire, because none of those things has happened anywhere in this book's evidence base — Part 1 §3 stated that constraint as a structural fact, and this part inherits it rather than quietly working around it in its own closing pages. Concretely:

- It does not re-derive any platform mechanic already taught in Parts 2–19. If a claim needs restating to make sense in the ledger below, it gets one clause, not a paragraph — the same non-duplication discipline `STYLE-GUIDE.md` §0 holds this book to against DEH applies just as hard to this book citing itself.
- It does not mint new detection IDs, new correlation searches, or new CIM mappings. Where a gap references a specific artifact (DET-23-01 as a correlation search, the `Authentication` data model's field set), it points at the part that owns that artifact rather than reproducing it.
- It does not pretend a lab build-out closes every gap it names. §6 below is explicit about which gaps a modest, single-instance lab could plausibly close and which ones — multi-tenant licensing economics at real enterprise ingest volume, RBA's marketed alert-reduction figure at real SOC operating scale, search-head-cluster contention under real analyst load — would still be out of reach even with real infrastructure, because the lab this book could plausibly stand up is not the deployment size those specific claims describe.

[SOC MANAGEMENT]

One more scope note, because it's easy to misread a validation-gaps chapter as an apology: naming a gap honestly is not the same claim as "this book's guidance is unreliable." Splunk's own public documentation, product pages, and Splunkbase listings are real evidence — a reasonable, checkable basis for a platform-operations claim, just not the same kind of evidence as a captured `_internal` log line from a system that actually ran the thing. This part's job is keeping those two kinds of evidence from blurring together in a reader's memory six months after finishing the book, not discrediting the first kind for lacking the second.

2. The evidence-class ledger, Parts 1–19

[PLATFORM ENGINEER]

STYLE-GUIDE.md

§9.2 states this book's evidence-class default plainly: zero `CONTROLLED LAB EXAMPLE` and zero `REAL LAB EXAMPLE` tags anywhere in the book, because no Splunk instance exists in the author's lab to produce either. Every figure and every version-specific claim in Parts 1–19 therefore sits in one of two classes — `OFFICIAL REFERENCE` (traceable to a cited Splunk source in `REFERENCES.md`) or `CONCEPTUAL` (this book's own labeled illustration, making no claim of capture from a running system). Table 20.1 is not exhaustive of every sentence in nineteen parts — that would be the whole book restated — but it is representative of the specific gap pattern each part carries, one row per part, so a reader can locate the exact kind of unverified claim a given part is most exposed on before relying on it for a production decision.

Table 20.1 supports one specific decision: if a reader can stand up only a limited amount of real Splunk infrastructure, which part's claims does that infrastructure actually get to move off `CONCEPTUAL / OFFICIAL REFERENCE` and onto something checked.

Table 20.1 — Representative validation gap by part.

Part	Layer	Representative claim resting on documented, not observed, evidence	What would upgrade it
1	Bridging	The platform-layer/operations-layer split and the DEH non-duplication boundary	Not upgradable by a lab — this is a structural framing choice, not a platform-behavior claim
2	Platform	Indexer/search-head clustering topology and replication/search factor tradeoffs	A multi-node cluster under real query load; a single-instance lab cannot exercise this at all
3	Platform	Hot/warm/cold/frozen bucket rollover timing and index-level RBAC enforcement	A real index observed through at least one full bucket rollover cycle
4	Platform, SOC management	Current ingest vs. workload (SVC) pricing mechanics; license-usage-report threshold behavior (flagged unverified in Part 4's own What Would Change My Mind)	A real billing relationship's Settings > Licensing > Usage Report , observed across a rolling window
5	Platform	Authentication / Network Traffic data model field sets and CIM's tags.conf / eventtypes.conf mechanism	A real CIM Add-on compliance check run against a real sourcetype
6	Platform	Data model inheritance and Pivot's generated-SPL behavior	A real Pivot session compared against its own generated search
7	Platform	Field-aliasing and tag mapping surviving (or not) a TA upgrade	A real TA version bump observed against a previously-mapped source
8	Operations	CSV lookup vs. KV Store collection behavior under concurrent write load	A real lookup under simultaneous correlation-search and manual-edit access
9	Operations	Macro-argument substitution edge cases and macro-sprawl detection via search-log analysis	A real _internal search log queried for macro-expansion frequency
10	Platform	tsidx summary storage overhead and summary-range coverage gaps	An actual accelerated data model's on-disk summary size and a real tstats result count
11	Platform	Summary-index backfill mechanics and staleness after a skipped run	A real summary-indexing search observed through one missed and one backfilled run
12	Platform	limits.conf concurrency defaults, explicitly flagged illustrative in Part 12 itself (docs.splunk.com unreachable during research)	btool output from a real search head, or the current Admin Manual once reachable
13	Operations	ES Essentials/Premier edition gating and Detection Studio's current naming and scope	Re-check against Splunk's own product page at time of use; a real ES install's own About page
14	Operations	Correlation-search throttling behavior and ES Content Update's packaged-detection install flow	A real correlation search observed firing, throttling, and suppressing across a scheduled window
15	Operations	Splunk's own marketed "up to 90%" notable-volume reduction from risk-based alerting	A real SOC's before/after notable count over a comparable retention window — a vendor figure, not measurable from documentation alone

Part	Layer	Representative claim resting on documented, not observed, evidence	What would upgrade it
16	Operations	Urgency/severity computation and Incident Review's ownership/status workflow	A real notable worked end-to-end in a real Incident Review instance
17	Operations	Asset and Identity framework staleness after a host decommission	A real asset lookup observed against a real decommissioning event
18	Operations	Simple XML vs. Dashboard Studio's current default status for new dashboards	Re-check against the current product documentation at time of use; a real dashboard created in a current release
19	Operations	The exact pivot-path UI naming from notable to risk object to raw search across recent ES releases	A real analyst's click-path captured against a currently-installed ES version
20	Capstone	This ledger itself	Every row above, closed one at a time

Engineering Reality Table 20.1's pattern is not random. Every platform-layer row (2–3, 5–7, 10–12) names a mechanism-level default, a storage number, or a config stanza — exactly the kind of fact that changes least often release to release, but is also the kind of fact this book's research could verify least directly, because `docs.splunk.com` refused every retrieval attempt made during this book's research (recorded per part in each affected part's own `REFERENCES.md` note). Every operations-layer row (13–19) names an edition boundary, a UI surface, or a marketed figure — the kind of fact that changes most often release to release, and the reason this book's ninth callout exists at all. The platform layer's gap is "documentation was unreachable"; the operations layer's gap is "the product moved since the documentation was written." Those are different failure modes with different fixes, and conflating them in a single "not verified" shrug would hide which fix — a working docs mirror, or a re-check against a current release — actually closes a given gap.

3. Where the gaps cluster: three distinct failure modes, not one

[PLATFORM ENGINEER] Sorting Table 20.1 by cause rather than by part number surfaces three genuinely different reasons a claim in this book is unverified, and each one has a different remedy:

Unreachable-documentation gaps. Several parts — 12 most explicitly, but also portions of 3 and 10 — state plainly that `docs.splunk.com` returned an HTTP error on every attempt made during that part's research, and that a Splunkbase listing, product page, or established general knowledge of the mechanism was used instead where it could substitute. This is the gap category a working documentation mirror or a different research tool closes without needing any Splunk instance at all — it is a research-access problem, not an evidence-class problem, and conflating the two would misdirect effort toward standing up infrastructure to fix what a reachable copy of the Admin Manual would fix directly.

No-instance-exists gaps. This is the majority of the ledger, and the one Part 1 §3 named as this book's central structural fact: a `tstats` result count, a Job Inspector output, a real notable's contributing-event

list, a real bucket rollover — none of these can be produced by better documentation access, because they are not documentation claims. They are observations a running system produces, and the only remedy is a running system. §4 below is about what the smallest version of that system would need to look like.

Product-velocity gaps. Rows 4, 13, 15, 18, and 19 share a different shape: the underlying fact (an edition split, a UI surface's name, a marketed percentage) was genuinely, correctly checked against a live, cited source as of 2026-09-15 — but Splunk's own release cadence for Enterprise Security specifically makes that fact's shelf life short by design, not by research failure. A lab wouldn't fully fix this either, unless it stayed continuously current with every ES release — the actual remedy is the Product Version Note discipline itself, applied at the moment of use rather than at the moment of authoring: re-check the cited source before trusting the note, every time, not just once at publication.

Product Version Note *The single fact recurring most often across Table 20.1's operations-layer rows is Splunk Enterprise Security's current Essentials/Premier edition split — UEBA, SOAR integration, and "Automated Threat Analysis" gated to Premier; AI capabilities, SIEM, Threat Intelligence, Detection Studio, and Exposure Analytics shipping in both. As of 2026-09-15, verified against Splunk's own Enterprise Security product page and the Enterprise Security app's Splunkbase listing (ES 8.7.0, released 2026-09-02; [REFERENCES.md](#) entries [\[ES-PRODUCT-PAGE\]](#) and [\[ES-SPLUNKBASE\]](#)), and restated identically in Parts 1, 13, and 14 rather than re-derived each time. What would make this stale: Splunk restructuring the two-edition model itself, moving a capability across the Essentials/Premier boundary, or retiring either edition name — any of which would require updating all three of those parts and this row together, not just one.*

4. What a real Splunk lab would actually need to look like

[PLATFORM ENGINEER] The no-instance-exists category from §3 is the one this book can actually describe a concrete remedy for, because the author's home lab already has the two ingredients a minimal Splunk stand-up would need to point at: a honeynet with live indicator enrichment and a vulnerability-scanner platform, both producing real, continuously-generated log traffic today, documented separately from this book. Neither requires Splunk to exist for its own purposes — both would simply become data sources a Splunk instance ingests, the same way any other log source would.

A lab sized to close the largest number of Table 20.1's rows, without requiring a multi-node cluster this book has no plan to build, would need:

1. **One Splunk Enterprise instance**, non-clustered, licensed under a trial or developer license rather than a production entitlement — sufficient to exercise indexing, search, data models, and the Enterprise Security app, but not sufficient to validate Part 2's clustering claims or Part 4's real ingest-volume billing behavior at any meaningful scale.
2. **A universal forwarder on each of the honeynet and vulnerability-scanner hosts**, sending real telemetry into two raw indexes — closing the "no-instance-exists" half of Parts 3, 5–7, and 10–

12's gaps directly, because those parts' claims are about mechanism behavior, not about volume or cluster topology.

3. **CIM field mapping work against those two real sources** — the literal implementation task Part 7 already describes in the abstract, now with a real sourcetype to map instead of an illustrative one, closing Part 5–7's compliance-check gaps specifically.
4. **The Enterprise Security app, installed against that mapped data**, with one or two correlation searches (a natural candidate: DET-26-01's SPL body, already carried through DEH Parts 24–29 and this book's own Part 14 §4, wrapped exactly the way Part 14 describes) — closing Part 14's scheduling/throttling gaps and Part 16's Incident Review gaps for whatever volume of real notables that traffic actually produces.

Product Version Note *This section assumes Splunk currently offers a trial or developer licensing path suitable for a single, non-production instance of the kind described above. The exact current terms of that licensing path — duration, ingest ceiling, which features are entitled under it — were not independently re-verified for this part; Splunk has changed its no-cost/low-cost licensing offerings before (Splunk Free's own deprecation for new customers is public history). Confirm current trial/developer license terms against Splunk's own site before planning a build-out on the strength of this section alone.*

Figure 20.1 — Path from documented claim to verified claim. CONCEPTUAL. Illustrates the two evidence streams feeding this book's current claims (Splunk's own public documentation and this book's labeled illustrative diagrams), the hypothetical single-instance lab described above that does not currently exist, and which parts' specific gap categories that lab would move from documented to observed. This is a sketch of a proposed evidence path, not a capture of any system that has been built.

flowchart TD

```
subgraph SRC["Current evidence sources"]
    DOCS["Splunk public docs,\nproduct pages, Splunkbase\n(OFFICIAL REFERENCE)"]
    SKETCH["This book's own\nillustrative diagrams\n(CONCEPTUAL)"]
end
```

```
BOOK["Parts 1-19's claims,\nas published"]
```

```
DOCS --> BOOK
```

```
SKETCH --> BOOK
```

```
BOOK -->|"gap named in\nTable 20.1"| GAP{"No-instance-exists\ngap?"}
```

```
GAP -->|no - unreachable docs\nor product-velocity gap| REMEDY1["Re-check cited source\nor find a reachable mirror\n(no lab needed)"]
```

```
GAP -->|yes| LAB["Hypothetical single-instance\nSplunk lab (not built)"]
```

```
subgraph LABBUILD["Inside the hypothetical lab"]
```

```
    UF1["Forwarder on honeynet host"]
```

```
    UF2["Forwarder on vuln-scanner host"]
```

```
    RAWIDX[("Raw indexes:\nhoneynet + vulnscan events")]
```

```
    CIMMAP["CIM field mapping\n(tags.conf / eventtypes.conf)"]
```

```
    DM["Accelerated data model\n+ tstats"]
```

```
    ES["Enterprise Security:\ncorrelation search + notable"]
```

```
end
```

```
LAB --> UF1 --> RAWIDX
```

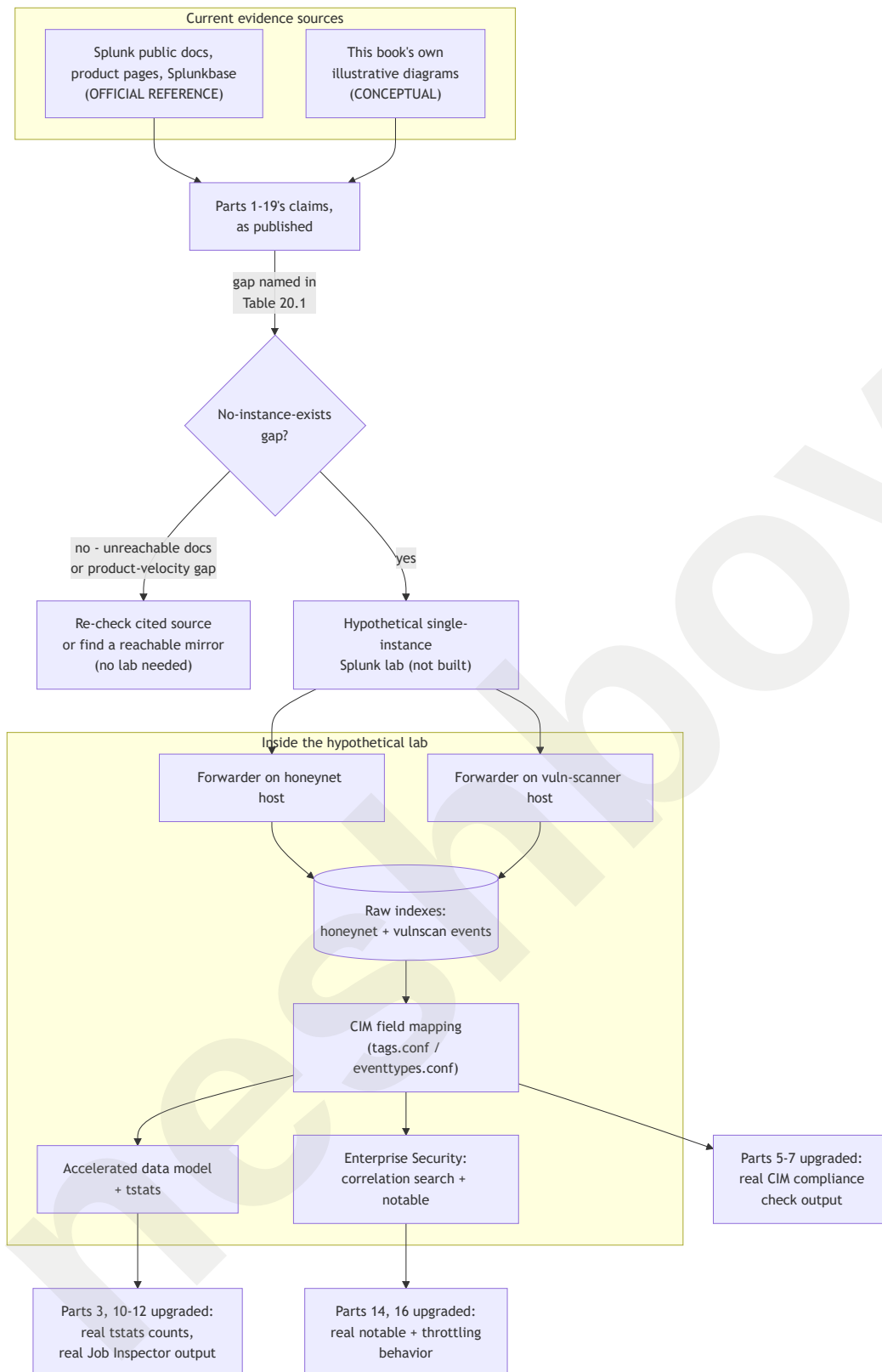
```
LAB --> UF2 --> RAWIDX
```

```
RAWIDX --> CIMMAP
```

```
CIMMAP --> DM --> UPGRADE1["Parts 3, 10-12 upgraded:\nreal tstats counts,\nreal Job Inspector output"]
```

```
CIMMAP --> UPGRADE2["Parts 5-7 upgraded:\nreal CIM compliance\ncheck output"]
```

```
CIMMAP --> ES --> UPGRADE3["Parts 14, 16 upgraded:\nreal notable + throttling\nbehavior"]
```



5. What that lab would upgrade first, and what it still would not touch

[SOC MANAGEMENT] Not every row in Table 20.1 is equally cheap to close, and a reader deciding whether standing up the lab in §4 is worth the effort needs the two ends of that range side by side, not

just the optimistic end.

Table 20.2 — Lab build-out payoff, ranked.

Priority	Gap closed	Effort relative to the \$4 build	Still unresolved after
1	Parts 5–7: real CIM-compliance check output against two real sources	Low — the mapping work itself, once forwarders are running	Compliance behavior for sourcetypes this lab's two sources don't represent
2	Parts 10–12: real <code>tstats</code> counts, real Job Inspector output, real skipped-search log entries	Low — falls out of running any search at all against real data	Concurrency behavior under real multi-analyst, multi-search-head load
3	Part 14: real correlation-search scheduling and throttling behavior	Medium — requires the ES app installed and configured	Behavior of ES Content Update's packaged detections specifically (Part 14 §5), not authored ones
4	Part 16: real notable urgency/severity computation and Incident Review workflow	Medium — depends on Priority 3 actually producing notables	Multi-analyst ownership/handoff dynamics at real SOC staffing levels
5	Part 15: risk-based alerting mechanics on a real risk index	Medium-high — needs sustained traffic over time, not a single test event	The marketed 90% notable-reduction figure itself, which needs a real SOC's operating history to compare against, not a lab's synthetic volume
—	Part 2: clustering, replication/search factor behavior	Not addressed by this build at all	Everything — this gap needs a multi-node deployment this lab does not include
—	Part 4: real ingest/workload billing thresholds	Not addressed by this build at all	A real commercial licensing relationship, which a trial/developer license (\$4's Product Version Note) does not represent

SOC Management View *The honest business case for the \$4 lab is narrower than "this closes the book's evidence gap." It closes the mechanism-level gaps in Table 20.2's top rows cheaply, because the author's honeynet and vulnerability-scanner platform already generate the raw traffic those mechanisms need and this book's Part 20 is the only reason that traffic isn't already flowing into an index somewhere. It does not touch the two rows a real production decision is most likely to actually hinge on — clustering behavior under load, and real licensing economics — because those aren't gaps a single instance closes at any size. A reader deciding whether to build this lab should weigh it as "worth it for the mechanism claims, not sufficient for the scale claims," not as an all-or-nothing bet.*

6. What this part cannot close, even in principle

[SOC MANAGEMENT] Two categories of claim in this book would remain unverifiable even after a lab far larger than \$4's minimal build:

- **Splunk's own marketed figures**, most prominently risk-based alerting's "up to 90%" notable-volume reduction (Part 15) and Enterprise Security's positioning language generally. These are Splunk's claims about Splunk's own product, made across an unknown population of real customer deployments this book has no access to and no standing to reproduce. A real lab could show that RBA reduced *this book's own* synthetic notable count by some measured amount — but that number would describe one small lab's traffic pattern, not a claim about the figure Splunk markets, and presenting it as confirmation of Splunk's percentage would be exactly the kind of overclaim `STYLE-GUIDE.md` §1.3's Example 1 already warns against.
- **Organizational-scale claims** — Part 4's real licensing-threshold behavior at production ingest volume, Part 17's asset/identity staleness at real headcount and real decommissioning cadence, Part 2's clustering behavior under real concurrent analyst load. These describe emergent behavior of a scale this book's evidence base cannot manufacture by standing up one more virtual machine, no matter how faithfully that machine mirrors Splunk's own architecture.

What Would Change My Mind Every gap in Table 20.1 that this part marks "no-instance-exists" would move toward "verified" the same way: a real Splunk instance, actually stood up, actually ingesting the honeynet's and vulnerability-scanner's own real traffic, actually observed over a real retention window rather than a single test run. That is a specific, checkable, and — per §4 — plausibly buildable answer, not a deflection repeated twenty times for lack of a better one. What it would not do is retroactively upgrade this book's `CONCEPTUAL` figures to `CONTROLLED LAB EXAMPLE` on their own terms; per `STYLE-GUIDE.md` §9.2, that upgrade happens claim by claim, dated and disclosed, the next time each affected part is actually revised against real observed output — and even then, Table 20.2's bottom rows and this section's two categories would still need a scale of deployment this book's evidence base has no plan to reach. The honest current answer, unchanged from Part 1 §3, is that this instance does not exist as of this part's `last_validated` date.

7. Reading the rest of this book with this ledger in hand

[SOC ANALYST] For a reader using this book operationally rather than as a straight read-through, the practical habit this part exists to instill is small: before treating any specific version number, edition boundary, percentage, or default-setting claim in Parts 1–19 as current, find that claim's Product Version Note, note the date and source it cites, and re-check that source if the gap between that date and today is more than a release cycle or two. That habit costs a few minutes per claim and is the entire point of tagging these notes distinctly from ordinary prose in the first place — a grep for "Product Version Note" across this book's chapters, the same way `STYLE-GUIDE.md` §6.9 intends, is a faster and more reliable validation pass than re-reading each part in full. Table 20.1 is that grep's output, pre-sorted; this part's only real deliverable is having done that sorting once so twenty separate readers don't each have to do it themselves.

Cross-references: Part 1 §3 (the evidentiary-honesty constraint this part closes out) · Part 3 (bucket lifecycle) · Part 4 (licensing and ingest economics) · Parts 5–7 (the Common Information Model) · Parts 10–12 (acceleration, summary indexing, search performance) · Part 13 (Enterprise Security architecture and editions) · Part 14 (correlation searches) · Part 15 (risk-based alerting) · Part 16 (notable events and Incident Review) · Part 17 (asset and identity correlation) · Parts 18–19 (dashboards and investigation workflows) · DEH Part 23 (Query Language Strategy) · DEH Part 26 (Splunk SPL) · [STYLE-GUIDE.md](#) §6.9 and §9 (Product Version Note and evidence-classification rules this part applies throughout).

neshbov